

Quartus® Prime Pro Edition User Guide

Timing Analyzer

Updated for Quartus® Prime Design Suite: **25.3**

This document is part of a collection - [Quartus® Prime Pro Edition User Guides - Combined PDF link](#)

Answers to Top FAQs:

- Q What are the basic concepts of timing analysis?**
A [Timing Analysis Basic Concepts](#) on page 4
- Q When do I apply timing constraints?**
A [Using Constraints throughout Design Flow](#) on page 27
- Q How do I run timing analysis?**
A [Run the Timing Analyzer](#) on page 34
- Q Where are the timing-critical paths in my design?**
A [Report Timing By Source Files](#) on page 140
- Q What are the recommended initial constraints?**
A [Recommended Initial SDC Constraints](#) on page 50
- Q How do I constrain CDC buses?**
A [Constraining CDC Paths](#) on page 93
- Q How can I debug timing issues?**
A [Troubleshooting Common Timing Issues](#) on page 187
- Q Do you have training on timing analysis?**
A [Technical Training: Timing Analysis](#)

Contents

1. Timing Analysis Introduction.....	4
1.1. Timing Analysis Basic Concepts.....	4
1.1.1. Timing Path and Clock Analysis.....	5
1.1.2. Clock Setup Analysis.....	9
1.1.3. Clock Hold Analysis.....	10
1.1.4. Recovery and Removal Analysis.....	11
1.1.5. Multicycle Path Analysis.....	12
1.1.6. Metastability Analysis.....	16
1.1.7. Timing Pessimism.....	17
1.1.8. Clock-As-Data Analysis.....	18
1.1.9. Multicorner Timing Analysis.....	19
1.1.10. Time Borrowing.....	20
1.2. Timing Analysis Overview Document Revision History.....	25
2. Using the Quartus Prime Timing Analyzer.....	26
2.1. Using Timing Constraints throughout the Design Flow.....	27
2.2. Timing Analysis Flow.....	28
2.2.1. Step 1: Specify General Timing Analyzer Settings.....	29
2.2.2. Step 2: Specify Timing Constraints.....	31
2.2.3. Step 3: Run the Timing Analyzer.....	34
2.2.4. Step 4: Analyze Timing Reports.....	40
2.3. Applying Timing Constraints.....	50
2.3.1. Recommended Initial Conventional SDC Constraints.....	50
2.3.2. Example Circuit and Conventional SDC File.....	55
2.3.3. SDC File Precedence.....	56
2.3.4. Iteratively Modifying Constraints.....	57
2.3.5. Applying Entity-Bound Timing Constraints.....	58
2.3.6. Constraining Design Partition Ports.....	75
2.3.7. Using Fitter Overconstraints.....	76
2.4. Timing Constraint Descriptions.....	78
2.4.1. Clock Constraints.....	79
2.4.2. I/O Constraints.....	94
2.4.3. Delay and Skew Constraints.....	98
2.4.4. Timing Exception Constraints.....	102
2.4.5. Delay Annotation.....	131
2.5. Timing Report Descriptions.....	132
2.5.1. Report Fmax Summary.....	133
2.5.2. Report Timing.....	134
2.5.3. Report Timing By Source Files.....	140
2.5.4. Report Data Delay.....	141
2.5.5. Report Net Delay.....	141
2.5.6. Report Clocks and Clock Network.....	141
2.5.7. Report Clock Transfers.....	144
2.5.8. Report Metastability.....	145
2.5.9. Report CDC Viewer	146
2.5.10. Report Asynchronous CDC.....	148
2.5.11. Report Logic Depth.....	152

2.5.12. Report Neighbor Paths.....	154
2.5.13. Report Register Spread.....	155
2.5.14. Report Route Net of Interest.....	160
2.5.15. Report Retiming Restrictions.....	162
2.5.16. Report Register Statistics.....	162
2.5.17. Report Pipelining Information.....	164
2.5.18. Report Time Borrowing Data.....	167
2.5.19. Report Exceptions and Exceptions Reachability.....	168
2.5.20. Report Bottlenecks.....	169
2.5.21. Check Timing.....	170
2.5.22. Report SDC.....	173
2.5.23. Design Closure Summary.....	173
2.6. Scripting Timing Analysis.....	174
2.6.1. The quartus_sta Executable.....	175
2.6.2. The quartus_staw Executable.....	176
2.6.3. Collection Commands.....	177
2.7. Using the Quartus Prime Timing Analyzer Document Revision History.....	181
2.8. Quartus Prime Pro Edition User Guide: Timing Analyzer Archive.....	186
3. Troubleshooting Common Timing Issues.....	187
3.1. Troubleshooting Asynchronous Clock Domain Crossing (CDC) Timing Issues.....	187
3.1.1. CDC Timing Overview.....	187
3.1.2. Identifying CDC Timing Issues Using Design Assistant.....	188
3.1.3. Identifying CDC Timing Issues Using Timing Reports.....	189
3.1.4. Debug CDC Example 1—Incorrect SDC Definition.....	199
3.1.5. Debug CDC Example 2—Additional Logic in the Crossing.....	208
3.1.6. Debug CDC Example 3—CDC Depending on Two Simultaneous Clock Domains.....	212
3.2. Troubleshooting Reset Domain Crossing (RDC) Timing Issues.....	215
3.2.1. RDC Timing Overview.....	215
3.2.2. Debug RDC Example—Asynchronous Reset Is Not Synchronized.....	216
3.3. Troubleshooting Common Timing Issues Revision History.....	221
A. Quartus Prime Pro Edition User Guides.....	222

1. Timing Analysis Introduction

Comprehensive timing analysis of your design allows you to validate circuit performance, identify timing violations, and drive the Fitter's placement of logic to meet your timing goals. The Quartus® Prime Timing Analyzer uses industry-standard constraint and analysis methodology to report on all data required times, data arrival times, and clock arrival times for all register-to-register, I/O, and asynchronous reset paths in your design.

The Timing Analyzer verifies that your design meets all required timing relationships to correctly function, and confirms actual signal arrival times against the constraints that you specify. This user guide provides an introduction to basic timing analysis concepts, along with step-by-step instructions for using the Quartus Prime Timing Analyzer.

1.1. Timing Analysis Basic Concepts

This user guide introduces the following concepts to describe timing analysis:

Table 1. Timing Analyzer Terminology

Term	Definition
Arrival time	The Timing Analyzer calculates the data and clock arrival time versus the required time at register pins.
Cell	Device resource that contains look-up tables (LUT), registers, digital signal processing (DSP) blocks, memory blocks, or I/O elements. In Stratix® series devices, the LUTs and registers are contained in logic elements (LE) modeled as cells.
Clock	Named signal representing clock domains inside or outside of your design.
Clock-as-data analysis	More accurate timing analysis for complex paths that includes any phase shift associated with a PLL for the clock path, and considers any related phase shift for the data path.
Clock hold time	Minimum time interval that a signal must be stable on the input pin that feeds a data input or clock enable, after an active transition on the clock input.
Clock launch and latch edge	The launch edge is the clock edge that sends data out of a register or other sequential element, and acts as a source for the data transfer. The latch edge is the active clock edge that captures data at the data port of a register or other sequential element, acting as a destination for the data transfer.
Clock pessimism	Clock pessimism refers to use of the maximum (rather than minimum) delay variation associated with common clock paths during static timing analysis.
Clock setup time	Minimum time interval between the assertion of a signal at a data input, and the assertion of a low-to-high transition on the clock input.
Maximum or minimum delay constraint	A constraint that specifies timing path analysis with a non-default setup or hold relationship.
Net	A collection of two or more interconnected components.
<i>continued...</i>	

Term	Definition
Node	Represents a wire carrying a signal that travels between different logical components in the design. Most basic timing netlist unit. Used to represent ports, pins, and registers.
Pin	Inputs or outputs of cells.
Port	Top-level module inputs or outputs; for example, a device pin.
Metastability	Metastability problems can occur when a signal transfers between circuitry in unrelated or asynchronous clock domains. The Timing Analyzer analyzes the potential for metastability in your design and can calculate the MTBF for synchronization register chains.
Multicorner analysis	Timing analysis of slow and fast timing corners to verify your design under a variety of voltage, process, and temperature operating conditions.
Multicycle paths	A data path that requires a non-default number of clock cycles for proper analysis.
Recovery and removal time	Recovery time is the minimum length of time for the deassertion of an asynchronous control signal relative to the next clock edge. Removal time is the minimum length of time the deassertion of an asynchronous control signal must be stable after the active clock edge.
Timing netlist	A Compiler-generated list of your design's synthesized nodes and connections. The Timing Analyzer requires this netlist to perform timing analysis.
Timing path	The wire connection (net) between any two sequential design nodes, such as the output of a register to the input of another register.

1.1.1. Timing Path and Clock Analysis

The Timing Analyzer measures the timing performance for all timing paths identified in your design. Prior to running full timing analysis, you can run post-synthesis early timing analysis to obtain an early view of the design core timing. For post-fit timing analysis, the Timing Analyzer requires a timing netlist that describes your design's nodes and connections for analysis and uses routing delays between core blocks represented by average interconnect delays. The post-synthesis timing delays reflect the delays of each type of connected core block.

Post-fit timing analysis also determines clock relationships for all register-to-register transfers in your design by analyzing the clock setup and hold relationship between the launch edge and latch edge of the clock.

1.1.1.1. The Timing Netlist

The Timing Analyzer uses the timing netlist data to determine the data and clock arrival time versus required time for all timing paths in the design. Post-synthesis timing analysis utilizes a timing netlist that incorporates core blocks (including their internal logic) and peripheral blocks (excluding their internal details). The Timing Analyzer estimates pre-synthesis routing delays by using an average interconnect model that the Analysis & Elaboration compilation stage generates. You can generate the post-fit timing netlist in the Timing Analyzer any time after running the Fitter.

The following figures illustrate division of a simple design schematic into timing netlist delays.

Figure 1. Simple Design Schematic

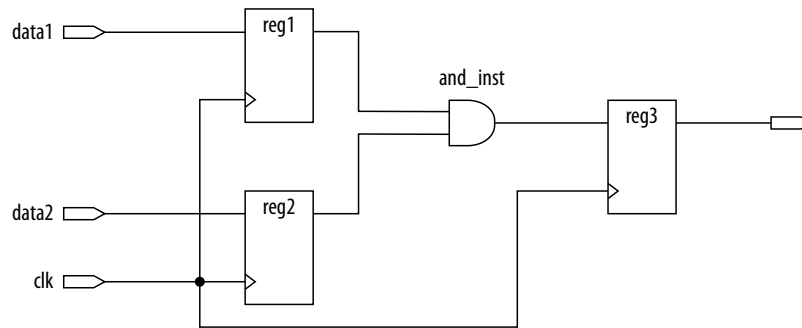
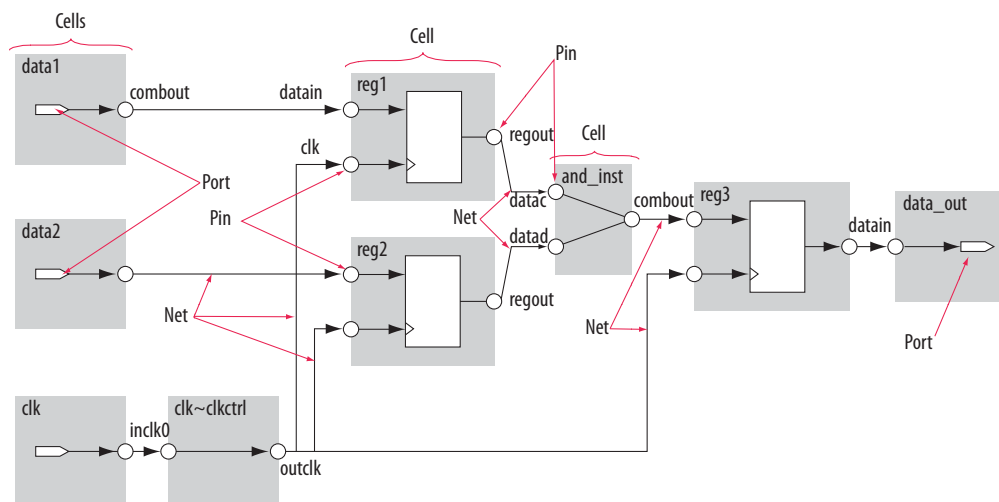


Figure 2. Division of Elements into Timing Netlist Delays



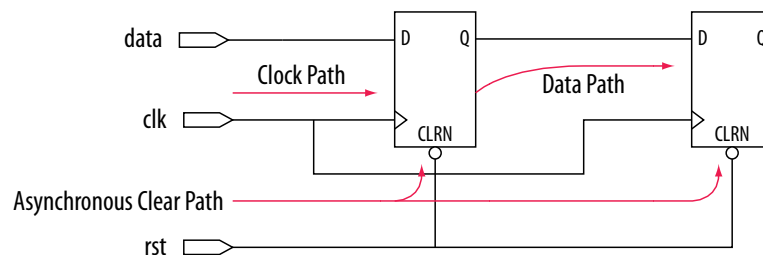
1.1.1.2. Timing Paths

Timing paths connect two design nodes, such as the output of a register to the input of another register.

Understanding the types of timing paths is important to timing closure and optimization. The Timing Analyzer recognizes and analyzes the following timing paths:

- **Edge paths**—connections from ports-to-pins, from pins-to-pins, and from pins-to-ports.
- **Clock paths**—connections from device ports or internally generated clock pins to the clock pin of a register.
- **Data paths**—connections from a port or the data output pin of a sequential element to a port or the data input pin of another sequential element.
- **Asynchronous paths**—connections from a port or asynchronous pins of another sequential element such as an asynchronous reset or asynchronous clear.

Figure 3. Path Types Commonly Analyzed by the Timing Analyzer



In addition to identifying various paths in a design, the Timing Analyzer analyzes clock characteristics to compute the worst-case requirement between any two registers in a single register-to-register path. You must constrain all clocks in your design before analyzing clock characteristics.

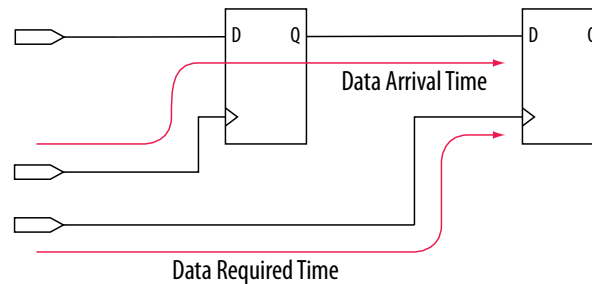
1.1.1.3. Data and Clock Arrival Times

After the Timing Analyzer identifies the path type, the Timing Analyzer can report data and clock arrival times at register pins.

The Timing Analyzer calculates data arrival time by adding the launch edge time to the delay from the clock source to the clock pin of the source register, the micro clock-to-output delay (μt_{CO}) of the source register, and the delay from the source register's data output (Q) to the destination register's data input (D).

The Timing Analyzer calculates data required time by adding the latch edge time to the sum of all delays between the clock port and the clock pin of the destination register. It includes any clock port buffer delays and subtracts the micro setup time (μt_{SU}) (or adds the micro hold time) of the destination register. Where the μt_{SU} is the intrinsic setup time of an internal register in the FPGA.

Figure 4. Data Arrival and Data Required Times



The basic calculations for data arrival and data required times including the launch and latch edges.

Figure 5. Data Arrival and Data Required Time Equations

$$\begin{aligned} \text{Data Arrival Time} &= \text{Launch Edge} + \text{Source Clock Delay} + \mu t_{co} + \text{Register-to-Register Delay} \\ \text{Data Required Time} &= \text{Latch Edge} + \text{Destination Clock Delay} - \mu t_{su} \end{aligned}$$

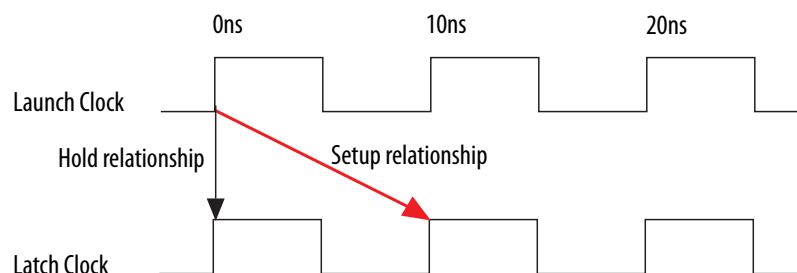
1.1.1.4. Launch and Latch Edges

All timing analysis requires the presence of one or more clock signals. The Timing Analyzer determines clock relationships for all register-to-register transfers in your design by analyzing the clock setup and hold relationship between the launch edge and latch edge of the clock.

The launch edge of the clock signal is the clock edge that sends data out of a register or other sequential element, and acts as a source for the data transfer. The latch edge is the active clock edge that captures data at the data port of a register or other sequential element, acting as a destination for the data transfer.

Figure 6. Setup and Hold Relationship for Launch and Latch Edges 10ns Apart

In this example, the launch edge sends the data from register *reg1* at 0 ns, and the register *reg2* captures the data when triggered by the latch edge at 10 ns. The data arrives at the destination register before the next latch edge.



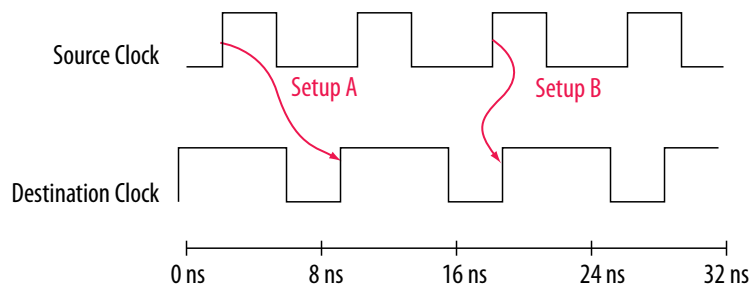
You must define all clocks in your design by assigning a clock constraint to each clock source node. These clock constraints provide the structure required for repeatable data relationships. If you do not constrain the clocks in your design, the Quartus Prime software analyzes all clocks as 1 GHz clocks to maximize timing based Fitter effort. To ensure realistic slack values, you must constrain all clocks in your design with real values.

1.1.2. Clock Setup Analysis

To perform a clock setup check, the Timing Analyzer determines a setup relationship by analyzing each launch and latch edge for each register-to-register path.

For each latch edge at the destination register, the Timing Analyzer uses the closest previous clock edge at the source register as the launch edge. The following figure shows two setup relationships, setup A and setup B. For the latch edge at 10 ns, the closest clock that acts as a launch edge is at 3 ns and has the setup A label. For the latch edge at 20 ns, the closest clock that acts as a launch edge is 19 ns and has the setup B label. The Timing Analyzer analyzes the most restrictive setup relationship, in this case setup B; if that relationship meets the design requirement, then setup A meets the requirement by default.

Figure 7. Setup Check



The Timing Analyzer reports the result of clock setup checks as slack values. Slack is the margin by which a circuit meets or does not meet the timing requirement. Positive slack indicates the margin by the circuit meets the requirement. Negative slack indicates the margin by which the circuit does not meet the requirement.

Figure 8. Clock Setup Slack for Internal Register-to-Register Paths

$$\begin{aligned}\text{Clock Setup Slack} &= \text{Data Required Time} - \text{Data Arrival Time} \\ \text{Data Arrival Time} &= \text{Launch Edge} + \text{Clock Network Delay to Source Register} + \mu t_{co} + \text{Register-to-Register Delay} \\ \text{Data Required Time} &= \text{Latch Edge} + \text{Clock Network Delay to Destination Register} - \mu t_{su} - \text{Setup Uncertainty}\end{aligned}$$

The Timing Analyzer performs setup checks using the maximum delay when calculating data arrival time, and minimum delay when calculating data required time. Some of the spread between maximum arrival path delays and minimum required path delays may be recoverable with path pessimism removal, as [Timing Pessimism](#) on page 17 describes.

Figure 9. Clock Setup Slack from Input Port to Internal Register

$$\begin{aligned}\text{Clock Setup Slack} &= \text{Data Required Time} - \text{Data Arrival Time} \\ \text{Data Arrival Time} &= \text{Launch Edge} + \text{Clock Network Delay} + \text{Input Maximum Delay} + \text{Port-to-Register Delay} \\ \text{Data Required Time} &= \text{Latch Edge} + \text{Clock Network Delay to Destination Register} - \mu t_{su} - \text{Setup Uncertainty}\end{aligned}$$

Figure 10. Clock Setup Slack from Internal Register to Output Port

$$\begin{aligned} \text{Clock Setup Slack} &= \text{Data Required Time} - \text{Data Arrival Time} \\ \text{Data Required Time} &= \text{Latch Edge} + \text{Clock Network Delay to Output Port} - \text{Output Maximum Delay} \\ \text{Data Arrival Time} &= \text{Launch Edge} + \text{Clock Network Delay to Source Register} + \mu t_{co} + \text{Register-to-Port Delay} \end{aligned}$$

1.1.3. Clock Hold Analysis

To perform a clock hold check, the Timing Analyzer determines a hold relationship for each possible setup relationship that exists for all source and destination register pairs. The Timing Analyzer checks all adjacent clock edges from all setup relationships to determine the hold relationships.

The Timing Analyzer performs two hold checks for each setup relationship. The first hold check determines that the data launched by the current launch edge is not captured by the previous latch edge. The second hold check determines that the data launched by the next launch edge is not captured by the current latch edge. From the possible hold relationships, the Timing Analyzer selects the hold relationship that is the most restrictive. The most restrictive hold relationship is the hold relationship with the smallest difference between the latch and launch edges and determines the minimum allowable delay for the register-to-register path. In the following example, the Timing Analyzer selects hold check A2 as the most restrictive hold relationship of two setup relationships, setup A and setup B, and their respective hold checks.

Figure 11. Setup and Hold Check Relationships

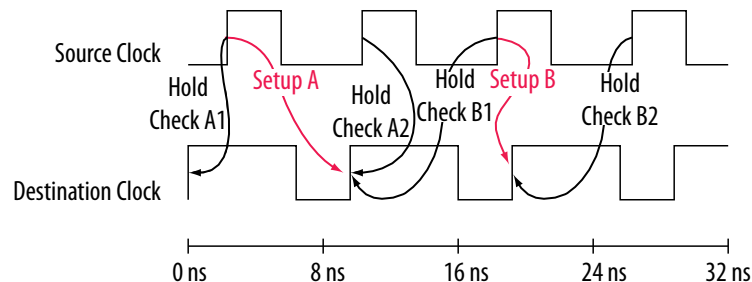


Figure 12. Clock Hold Slack for Internal Register-to-Register Paths

$$\begin{aligned} \text{Clock Hold Slack} &= \text{Data Arrival Time} - \text{Data Required Time} \\ \text{Data Arrival Time} &= \text{Launch Edge} + \text{Clock Network Delay to Source Register} + \mu t_{co} + \text{Register-to-Register Delay} \\ \text{Data Required Time} &= \text{Latch Edge} + \text{Clock Network Delay to Destination Register} + \mu t_{h} + \text{Hold Uncertainty} \end{aligned}$$

The Timing Analyzer performs hold checks using the minimum delay when calculating data arrival time, and maximum delay when calculating data required time.

Figure 13. Clock Hold Slack Calculation from Input Port to Internal Register

$$\begin{aligned} \text{Clock Hold Slack} &= \text{Data Arrival Time} - \text{Data Required Time} \\ \text{Data Arrival Time} &= \text{Launch Edge} + \text{Clock Network Delay} + \text{Input Minimum Delay} + \text{Pin-to-Register Delay} \\ \text{Data Required Time} &= \text{Latch Edge} + \text{Clock Network Delay to Destination Register} + \mu t_{h} \end{aligned}$$

Figure 14. Clock Hold Slack Calculation from Internal Register to Output Port

$$\begin{aligned}\text{Clock Hold Slack} &= \text{Data Arrival Time} - \text{Data Required Time} \\ \text{Data Arrival Time} &= \text{Launch Edge} + \text{Clock Network Delay to Source Register} + \mu t_{co} + \text{Register-to-Pin Delay} \\ \text{Data Required Time} &= \text{Latch Edge} + \text{Clock Network Delay} - \text{Output Minimum Delay}\end{aligned}$$

1.1.4. Recovery and Removal Analysis

Recovery time is the minimum length of time for the deassertion of an asynchronous control signal relative to the next clock edge.

For example, signals such as `clear` and `preset` must be stable before the next active clock edge. The recovery slack calculation is similar to the clock setup slack calculation, but the calculation applies to asynchronous control signals.

Figure 15. Recovery Slack Calculation if the Asynchronous Control Signal is Registered

$$\begin{aligned}\text{Recovery Slack Time} &= \text{Data Required Time} - \text{Data Arrival Time} \\ \text{Data Required Time} &= \text{Latch Edge} + \text{Clock Network Delay to Destination Register} - \mu t_{su} \\ \text{Data Arrival Time} &= \text{Launch Edge} + \text{Clock Network Delay to Source Register} + \mu t_{co} + \text{Register-to-Register Delay}\end{aligned}$$

Figure 16. Recovery Slack Calculation if the Asynchronous Control Signal is not Registered

$$\begin{aligned}\text{Recovery Slack Time} &= \text{Data Required Time} - \text{Data Arrival Time} \\ \text{Data Required Time} &= \text{Latch Edge} + \text{Clock Network Delay to Destination Register} - \mu t_{su} \\ \text{Data Arrival Time} &= \text{Launch Edge} + \text{Clock Network Delay} + \text{Input Maximum Delay} + \text{Port-to-Register Delay}\end{aligned}$$

Note: If the asynchronous reset signal is from a device I/O port, you must create an input delay constraint for the asynchronous reset port for the Timing Analyzer to perform recovery analysis on the path.

Removal time is the minimum length of time the deassertion of an asynchronous control signal must be stable after the active clock edge. The Timing Analyzer removal slack calculation is similar to the clock hold slack calculation, but the calculation applies to asynchronous control signals.

Figure 17. Removal Slack Calculation if the Asynchronous Control Signal is Registered

$$\begin{aligned}\text{Removal Slack Time} &= \text{Data Arrival Time} - \text{Data Required Time} \\ \text{Data Arrival Time} &= \text{Launch Edge} + \text{Clock Network Delay to Source Register} + \mu t_{co} \text{ of Source Register} + \text{Register-to-Register Delay} \\ \text{Data Required Time} &= \text{Latch Edge} + \text{Clock Network Delay to Destination Register} + \mu t_{H}\end{aligned}$$

Figure 18. Removal Slack Calculation if the Asynchronous Control Signal is not Registered

$$\begin{aligned}\text{Removal Slack Time} &= \text{Data Arrival Time} - \text{Data Required Time} \\ \text{Data Arrival Time} &= \text{Launch Edge} + \text{Clock Network Delay} + \text{Input Minimum Delay of Pin} + \text{Minimum Pin-to-Register Delay} \\ \text{Data Required Time} &= \text{Latch Edge} + \text{Clock Network Delay to Destination Register} + \mu t_{H}\end{aligned}$$

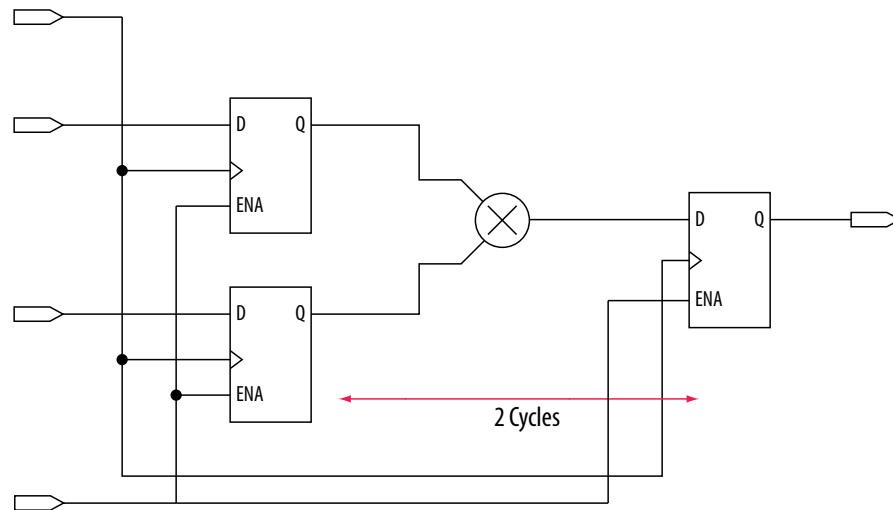
If the asynchronous reset signal is from a device pin, you must create an input delay constraint to the asynchronous reset pin for the Timing Analyzer to perform removal analysis on the path.

1.1.5. Multicycle Path Analysis

Multicycle paths are data paths that require an exception to the default setup or hold relationship, for proper analysis. For example, a register that requires data capture on every second or third rising clock edge (multicycle exception), rather than requiring capture on every clock edge (default analysis).

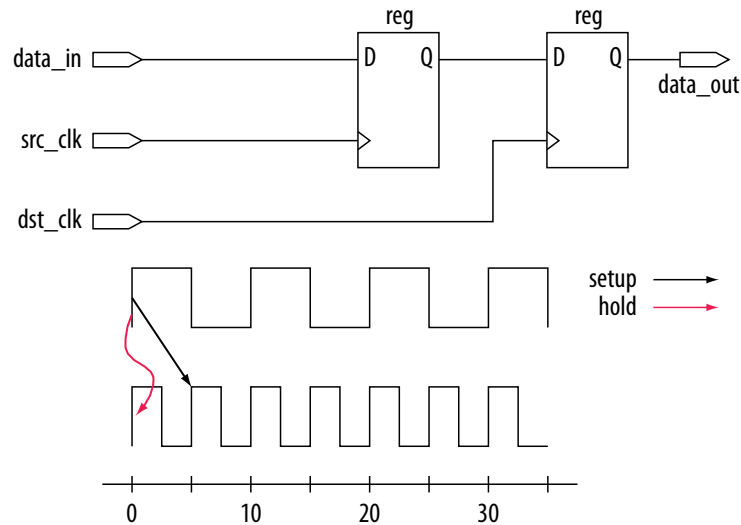
A multicycle path occurs between the input registers of a multiplier, and an output register with a destination that latches data on every other clock edge.

Figure 19. Multicycle Path



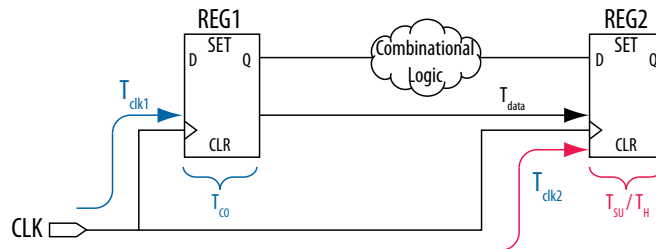
A register-to-register path is for the default setup and hold relationship. Also, for the respective timing diagrams for the source and destination clocks and the default setup and hold relationships, when the source clock, `src_clk`, has a period of 10 ns and the destination clock, `dst_clk`, has a period of 5 ns. The default setup relationship is 5 ns; the default hold relationship is 0 ns.

Figure 20. Register-to-Register Path and Default Setup and Hold Timing Diagram



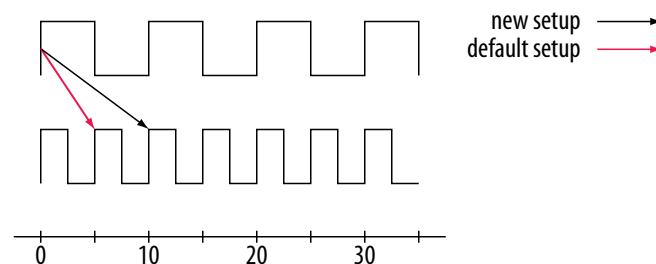
To accommodate the system requirements, you can modify the default setup and hold relationships by specifying a multicycle timing constraint to a register-to-register path.

Figure 21. Register-to-Register Path



The exception has a multicycle setup assignment of two to use the second occurring latch edge; in this example, to 10 ns from the default value of 5 ns.

Figure 22. Modified Setup Diagram



1.1.5.1. Multicycle Clock Hold

The number of clock periods between the clock launch edge and the latch edge defines the setup relationship.

By default, the Timing Analyzer performs a single-cycle path analysis. When analyzing a path, the Timing Analyzer performs two hold checks. The first hold check determines that the data that launches from the current launch edge is not captured by the previous latch edge. The second hold check determines that the data that launches from the next launch edge is not captured by the current latch edge. The Timing Analyzer reports only the most restrictive hold check. The Timing Analyzer calculates the hold check by comparing launch and latch edges.

Figure 23. Hold Check

The Timing Analyzer uses the following calculation to determine the hold check.

$$\text{hold check 1} = \text{current launch edge} - \text{previous latch edge}$$

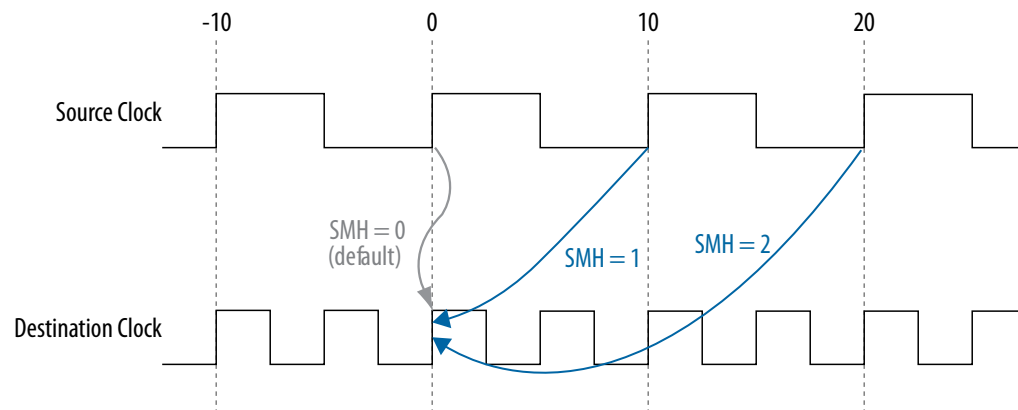
$$\text{hold check 2} = \text{next launch edge} - \text{current latch edge}$$

Tip:

If a hold check overlaps a setup check, the hold check is ignored.

A start multicycle hold assignment modifies the launch edge of the source clock by moving the launch edge the number of clock periods you specify to the right of the default launch edge. The following figure shows various values of the start multicycle hold (SMH) assignment and the resulting launch edge.

Figure 24. Start Multicycle Hold Values



An end multicycle hold assignment modifies the latch edge of the destination clock by moving the latch edge the specific number of clock periods to the left of the default latch edge. The following figure shows various values of the end multicycle hold (EMH) assignment and the resulting latch edge.

Figure 25. End Multicycle Hold Values

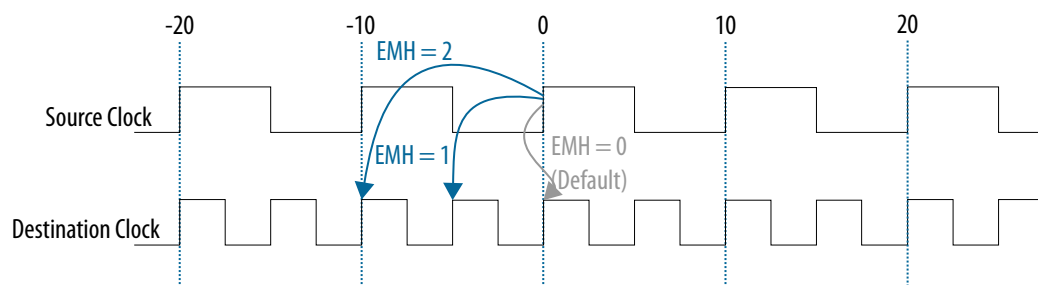
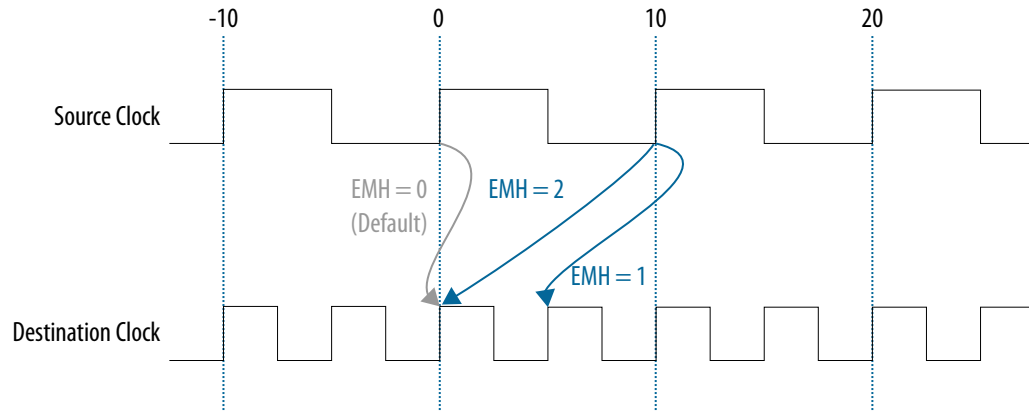


Figure 26. End Multicycle Hold Values the Timing Analyzer Reports

The following shows the hold relationship the Timing Analyzer reports for the negative hold relationship:

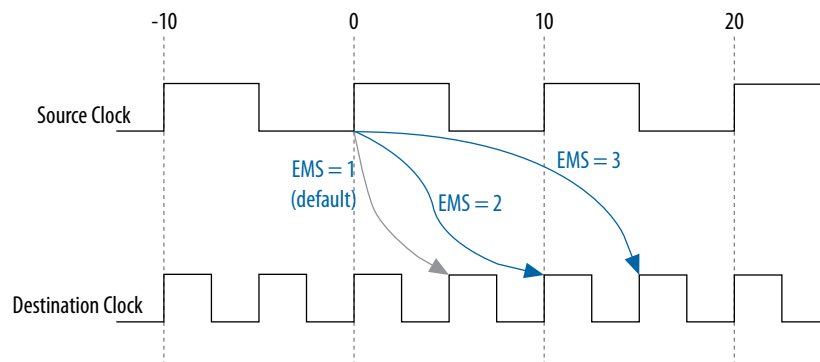


1.1.5.2. Multicycle Clock Setup

The setup relationship is the number of clock periods between the latch edge and the launch edge. By default, the Timing Analyzer performs a single-cycle path analysis, which results in the setup relationship being equal to one clock period (latch edge – launch edge). Applying a multicycle setup assignment, adjusts the setup relationship by the multicycle setup value. The adjustment value may be negative.

An end multicycle setup assignment modifies the latch edge of the destination clock by moving the latch edge the specified number of clock periods to the right of the determined default latch edge. The following figure shows various values of the end multicycle setup (EMS) assignment and the resulting latch edge.

Figure 27. End Multicycle Setup Values



A start multicycle setup assignment modifies the launch edge of the source clock by moving the launch edge the specified number of clock periods to the left of the determined default launch edge. A start multicycle setup (SMS) assignment with various values can result in a specific launch edge.

Figure 28. Start Multicycle Setup Values

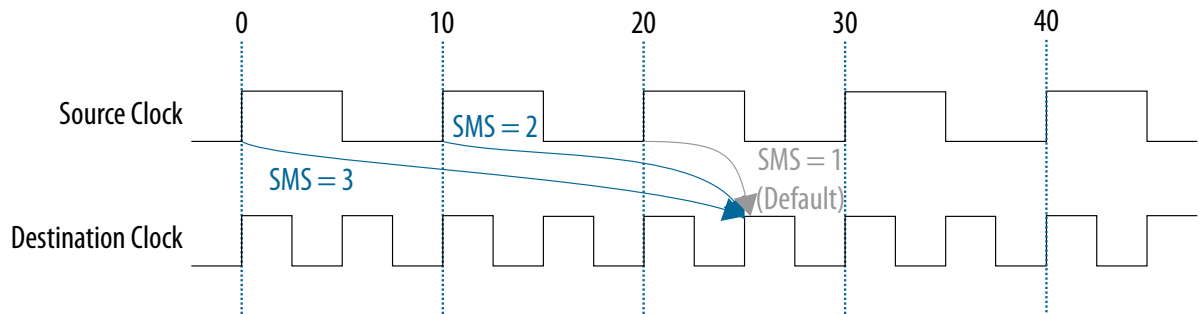
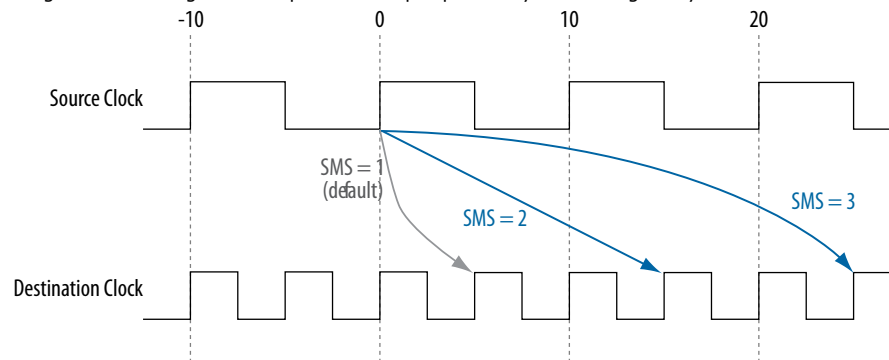


Figure 29. Start Multicycle Setup Values Reported by the Timing Analyzer

The following shows the negative setup relationship reported by the Timing Analyzer.



1.1.6. Metastability Analysis

Metastability problems can occur when a signal transfers between circuitry in unrelated or asynchronous clock domains because the signal does not meet setup and hold time requirements.

To minimize the failures due to metastability, circuit designers typically use a sequence of registers, also known as a synchronization register chain, or synchronizer, in the destination clock domain to resynchronize the data signals to the new clock domain.

The mean time between failures (MTBF) is an estimate of the average time between instances of failure due to metastability.

The Timing Analyzer analyzes the potential for metastability in your design and can calculate the MTBF for synchronization register chains. The Timing Analyzer then estimates the MTBF of the entire design from the synchronization chains the design contains.

In addition to reporting synchronization register chains found in the design, the Quartus Prime software also protects these registers from optimizations that might negatively impact MTBF, such as register duplication and logic retiming. The Quartus Prime software can also optimize the MTBF of your design if the MTBF is too low.

Related Information

- [Report Metastability](#) on page 145

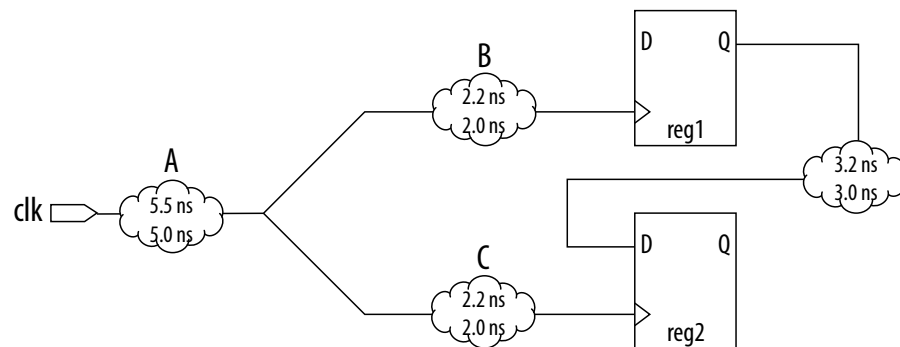
- [Step 1: Specify General Timing Analyzer Settings](#) on page 29
- [Troubleshooting Clock Domain Crossing \(CDC\) Timing Issues](#) on page 187

1.1.7. Timing Pessimism

Common clock path pessimism removal accounts for the minimum and maximum delay variation associated with common clock paths during static timing analysis by adding the difference between the maximum and minimum delay value of the common clock path to the appropriate slack equation.

Minimum and maximum delay variation can occur when timing analysis uses two different delay values for the same clock path. For example, in a simple setup analysis, the maximum clock path delay to the source register determines the data arrival time. The minimum clock path delay to the destination register determines the data required time. However, if the clock path to the source register and to the destination register share a common clock path, both the maximum delay and the minimum delay model the common clock path during timing analysis. The use of both the minimum delay and maximum delay results in an overly pessimistic analysis since two different delay values, the maximum and minimum delays, cannot be used to model the same clock path.

Figure 30. Typical Register to Register Path



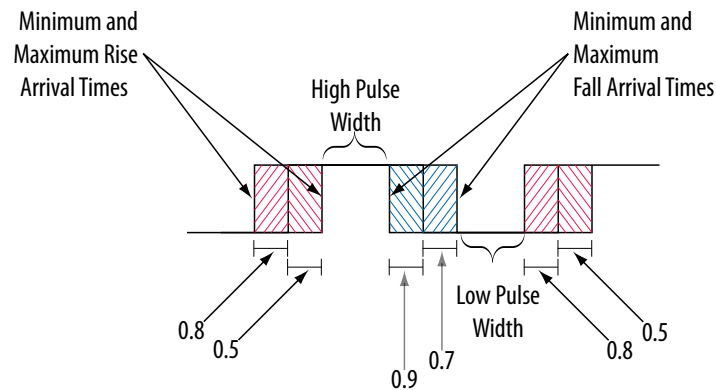
Segment A is the common clock path between *reg1* and *reg2*. The minimum delay is 5.0 ns; the maximum delay is 5.5 ns. The difference between the maximum and minimum delay value equals the common clock path pessimism removal value; in this case, the common clock path pessimism is 0.5 ns. The Timing Analyzer adds the common clock path pessimism removal value to the appropriate slack equation to determine overall slack. Therefore, if the setup slack for the register-to-register path in the example equals 0.7 ns without common clock path pessimism removal, the slack is 1.2 ns with common clock path pessimism removal.

You can also use common clock path pessimism removal to determine the minimum pulse width of a register. A clock signal must meet a register's minimum pulse width requirement for recognition by the register. A minimum high time defines the minimum pulse width for a positive-edge triggered register. A minimum low time defines the minimum pulse width for a negative-edge triggered register.

Clock pulses that violate the minimum pulse width of a register prevent data from latching at the data pin of the register. To calculate the slack of the minimum pulse width, the Timing Analyzer subtracts the required minimum pulse width time from the actual minimum pulse width time. The Timing Analyzer determines the actual

minimum pulse width time by the clock requirement you specify for the clock that feeds the clock port of the register. The Timing Analyzer determines the required minimum pulse width time by the maximum rise, minimum rise, maximum fall, and minimum fall times.

Figure 31. Required Minimum Pulse Width time for the High and Low Pulse



With common clock path pessimism, the minimum pulse width slack can increase by the smallest value of either the maximum rise time minus the minimum rise time, or the maximum fall time minus the minimum fall time. In the example, the slack value can increase by 0.2 ns, which is the smallest value between 0.3 ns (0.8 ns – 0.5 ns) and 0.2 ns (0.9 ns – 0.7 ns).

1.1.8. Clock-As-Data Analysis

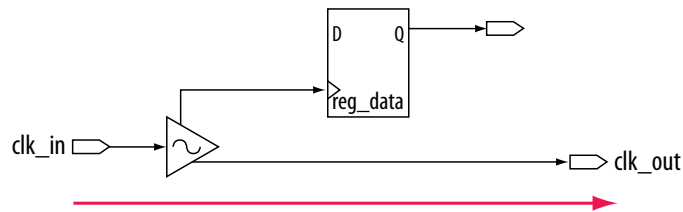
The majority of FPGA designs contain simple connections between any two nodes, known as either a data path or a clock path.

A data path is a connection between the output of a synchronous element to the input of another synchronous element.

A clock is a connection to the clock pin of a synchronous element. However, for more complex FPGA designs, such as designs that use source-synchronous interfaces, this simplified view is no longer sufficient. The Timing Analyzer performs clock-as-data analysis in circuits with elements such as clock dividers and DDR source-synchronous outputs.

You can treat the connection between the input clock port and output clock port as a clock path or a data path. [Simplified Source Synchronous Output](#) shows a design where the path from port `clk_in` to port `clk_out` is both a clock and a data path. The clock path is from the port `clk_in` to the register `reg_data` clock pin. The data path is from port `clk_in` to the port `clk_out`.

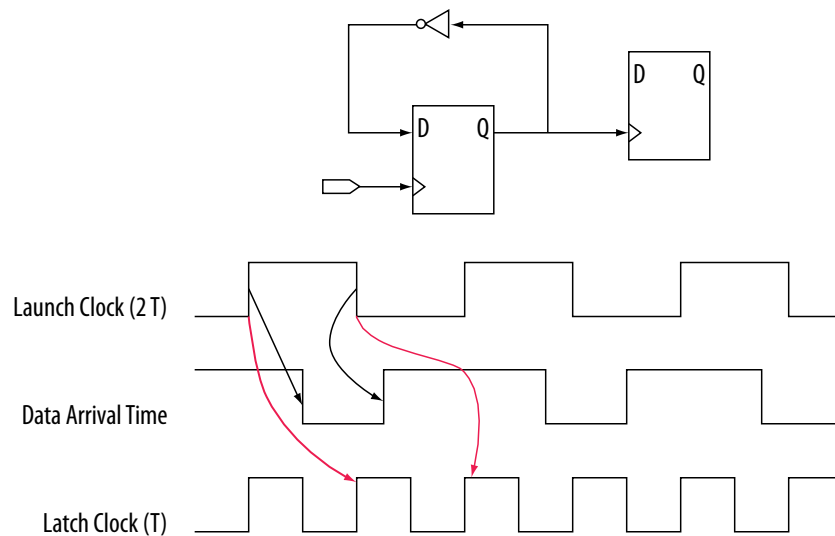
Figure 32. Simplified Source Synchronous Output



With clock-as-data analysis, the Timing Analyzer provides a more accurate analysis of the path based on user constraints. In the clock path analysis, the Timing Analyzer includes any phase shift associated with the phase-locked loop (PLL). For the data path analysis, the Timing Analyzer includes any phase shift associated with the PLL, rather than ignoring the phase shift.

The clock-as-data analysis also applies to internally generated clock dividers. In the following figure, the waveforms are for the inverter feedback path, analyzed during timing analysis. The output of the divider register determines the launch time, and the clock port of the register determines the latch time.

Figure 33. Clock Divider



1.1.9. Multicorner Timing Analysis

You can direct the Timing Analyzer to perform multicorner timing analysis to verify your design under different voltage, process, and temperature operating conditions.

To ensure that no violations occur under different timing conditions (models) during device operation, you must perform static timing analysis under all available operating conditions.

You specify the operating conditions in the Timing Analyzer prior to running analysis.

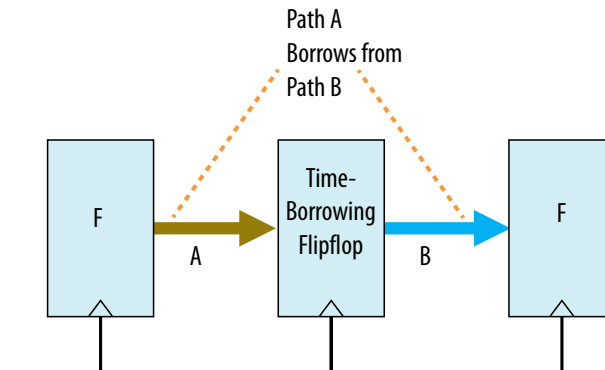
Related Information

[Setting the Operating Conditions for Timing Analysis](#) on page 38

1.1.10. Time Borrowing

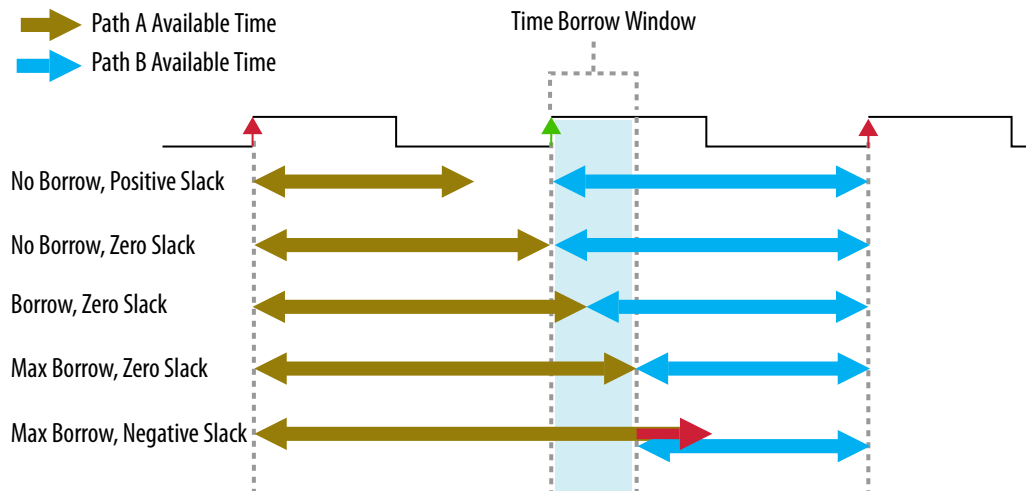
Time borrowing can improve performance by enabling the path ending at a time-borrowing flipflop or latch to "borrow" time from the next path in the register pipeline. The borrowed time subtracts from the next path, resulting in the same cumulative timing. In this way, time borrowing can shift slack to more critical parts of the design. Without time borrowing, the Timing Analyzer analyzes each path independently and normally assumes exactly one clock cycle for each transfer.

Figure 34. Time Borrowing Example



Some of the flipflops in Stratix 10 and Arria® 10 devices allow time borrowing. The exact size of the available time borrow window depends on hardware settings. The Fitter (Finalize) stage automatically configures the appropriate borrow window for each time-borrowing flipflop, based on hardware restrictions and the available hold slack.⁽¹⁾

Figure 35. Time Borrowing with Various Slack Conditions and Borrow Values



⁽¹⁾ In Partial Reconfiguration designs, additional restrictions may apply to time-borrowing window size.

Altera FPGA devices generally support only a few borrow window sizes. For example, Stratix 10 devices support narrow, medium, and wide. Typically, groups of several flipflops must share the same setting. The actual borrowed amount is completely flexible within a given borrow window. The Timing Analyzer calculates the borrowed amount separately for each operating condition, clock, and signal rise and fall edge. Selecting a wider borrow window reduces hold slack. The Compiler only selects wider settings if hold slack allows. Furthermore, if the Compiler determines that a narrower window is sufficient for a given group of registers (based on the optimal time borrowing solution), the Compiler uses the narrower window, even if there is sufficient hold slack for a wider window.

For a given borrow window size, the exact size of the borrow window may depend on the register input (for example, `d` or `sclr`), the edge of the incoming signal (rising or falling), the device speed grade, and operating conditions.

You can enable automatic implementation of time borrowing without making any RTL changes. Once enabled, the Fitter automatically configures the window size. The Fitter also determines the optimal time borrow amount within the available borrow window for any design registers that the Fitter places in time-borrowing flipflops.

Proper timing analysis of designs that contain level-sensitive latches typically requires time borrowing. However, the automatic Fitter time borrowing optimizations do not apply for level-sensitive latches, as [Time Borrowing with Latches](#) describes in detail.

Related Information

- [Enabling Time Borrowing Optimization](#) on page 23
- [Report Time Borrowing Data](#) on page 167

1.1.10.1. Time Borrowing Limitations

Time borrowing optimization, which occurs in the Fitter (Finalize) stage, cannot occur for the following registers. For these registers, the time borrowing optimization is zero, and the maximum operating frequency in the **Fmax Summary** reports include zero time borrowing:

- Any register that is the source of a cross-clock transfer
- Any register that is the source of a `set_max_skew` or `set_max_delay` assignment
- Any register in a clock domain with one or more level-sensitive latch

Furthermore, registers that are destinations of cross-clock transfers, `set_max_skew` or `set_max_delay` constraints do not have borrowing values that are optimized for such transfers (but may still have non-zero borrowing from other transfers).

If any such registers are on the critical timing path, you can possibly report better performance by enabling Dynamic time borrowing mode, which reports time borrowing for all borrow-capable registers. Dynamic time borrowing mode can then provide a more accurate (less pessimistic) analysis, but only at a specific set of clock frequencies that you specify in the `.sdc`.

To view time borrowing results for such registers:

1. Set the clock frequency in the .sdc file to a value higher than the clock frequency that **Fmax Summary** reports.
2. Reset your design and read the SDC file again in the Timing Analyzer.
3. Run the `update_timing_netlist -dynamic_borrow` command.
4. View the results in **Slack Summary** reports (**Reports > Slack** to determine if timing passes. The **Fmax Summary** report does not reflect any gains from dynamic borrowing.

Note: Time borrowing is similar to the beneficial clock skew technique, whereby you delay the clock to a given register, giving more time to the incoming paths at the expense of the outgoing paths. However, time borrowing and beneficial clock skew have the following important differences:

- Time borrowing offers more flexibility than clock skew in distributing setup slack. Skewing is typically limited to fixed increments. You can use borrowing to shift any amount of slack (however small) from one side of the register to the other, as long as the amounts fit within the available borrow window. Furthermore, borrow amounts calculate separately for each operating condition, making it possible to shift the optimum amount of slack for each operating condition. This type of shifting is not possible with skewing.
- You can use beneficial clock skew to increase Hold slack on outgoing paths from the register where the clock is skewed. Time borrowing does not offer this benefit.

1.1.10.2. Time Borrowing with Latches

The Quartus Prime Timing Analyzer treats level-sensitive latches similar to registers. The Timing Analyzer treats the latch enable pin as a clock pin, while modifying the clock relationship appropriately.

You can run **Reports > Constraint Diagnostics > Check Timing** in the Timing Analyzer **Tasks** pane to view a list of the level-sensitive latches in your design.

Implementation of latch time borrowing requires that you enable Dynamic borrowing mode (`update_timing_netlist -dynamic_borrow`). Otherwise, the Timing Analyzer calculates zero time borrowing for latches. In Dynamic mode, the Timing Analyzer simply reports the amount of time borrowing that would physically happen in the circuit, given the clock frequencies you specify in SDC constraints, and does not actually optimize borrowing in any way.

For latches, the setup relationship is to the opening edge of the latch, which allows time borrowing. The hold relationship is to the closing edge of the latch. For example, a path from a positive register to another positive register has a default setup clock relationship of one clock period. A path from a positive register to a positive (open-high), level-sensitive latch has a default setup clock relationship of zero clock periods, plus any time borrow value.

The Timing Analyzer treats paths to and from a latch as two separate paths. For example, in a positive register--> positive latch--> negative register transfer, the Timing Analyzer does not analyze the overall register-->register transfer, even though you expect the latch to be transparent for the entire duration of the transfer. The Timing Analyzer analyzes and reports the paths to and from the latch separately.

The Timing Analyzer automatically computes the maximum amount of time borrowing available for each latch. Typically, the maximum amount of time borrowing available is roughly equivalent to half the clock period. The exact amount of time borrowing available is based on:

- The timing of opening and closing latch edges
- Physical latch implementation (closing-edge μt_{SU} of the latch)
- Clock uncertainty and other effects

The time borrowing never exceeds the maximum borrow value. However, you can specify a smaller maximum borrow time with the `set_max_time_borrow` SDC constraint. For example:

```
#Borrow at most 3ns at all "lat*" latches:  
set_max_time_borrow 3 [get_registers lat*]
```

Specifying a clock with a negative borrow window can result in negative maximum borrowable time, which is equivalent to a minimum pulse width violation. For example, this condition can occur if half the clock period is smaller than the closing-edge μt_{SU} of the latch. If such a violation occurs, a warning indicates that the design cannot pass timing.

Note: Whether you use time borrowing or not, do not rely on the timing analysis **Fmax Summary** report for any clock domains with latches. The **Fmax Summary** values for such clock domains include no borrowing, and are therefore significantly pessimistic.

1.1.10.3. Enabling Time Borrowing Optimization

During any High or Superior **Performance** compilation, the Compiler automatically computes and stores **Optimal** time borrow values for Stratix 10 and Arria 10 designs during the Finalize stage. By default, the subsequent timing analysis results reflect the **Optimal** borrow values from the Finalize stage.

Follow these steps to enable time borrowing for supported devices:

1. Click **Assignments > Settings > Compiler Settings > Optimization Mode**. Select any high or superior **Performance** setting.
2. Run the Fitter and Timing Analyzer, as [Step 3: Run the Timing Analyzer](#) on page 34 describes.
3. To generate reports showing time borrowing data, click **Reports > Timing Slack > Report Timing**. Time borrowing data appears on the critical path for a given clock domain, as [Report Time Borrowing Data](#) describes.

Figure 36. Performance Compiler Optimization Mode Settings

Compiler Settings

Specify high-level optimization settings for the Compiler (including design synthesis and fitting).

☒ Enable Intermediate Fitter Snapshots

Optimization Mode Register Optimization Incremental Compile

☒ **Balanced (normal flow)**

Performance:

☐ High performance effort

☐ High performance with maximum placement effort

☐ Superior performance (adds synthesis optimizations for speed)

☐ Superior performance with maximum placement effort

- To specify time borrowing optimization without changing the Compiler **Optimization Mode**, specify the following assignment in the project .qsf:

```
set_global_assignment -name ENABLE_TIME_BORROWING_OPTIMIZATION <ON|OFF>
```

- To manually specify the time borrow mode during timing analysis, run one of the following `update_timing_netlist` command options:

Table 2. Time Borrowing Modes

Time Borrowing Mode	Command Option	Default Mode For
Optimal —timing analysis includes optimal time borrow values from the Finalize stage. You can optionally add the <code>recompute_borrow</code> option to <code>update_timing_netlist</code> to recompute the borrow amounts, but not the borrow window sizes.	<code>update_timing_netlist</code>	High and Superior performance compilations for Stratix 10 and Arria 10 designs.
Dynamic —timing analysis reports the time borrowing that would physically occur on the device, with respect to your SDC constraints, without any optimization. That is, timing analysis applies as much borrowing as necessary to fix all negative slack. Timing analysis assumes maximum possible borrowing for any timing path where the maximum amount of time borrowing is insufficient to eliminate all negative slack. Only mode that allows borrowing for level-sensitive latches.	<code>update_timing_netlist -dynamic_borrow</code> <code>update_timing_netlist -loop_aware_dynamic_borrow</code>	None
Zero —timing analysis uses zero time borrowing.	<code>update_timing_netlist -no_borrow</code>	Unsupported devices, or any Compiler Optimization mode other than a Performance mode.

Note: Dynamic mode cannot yield the optimal results with overconstrained clocks, as overconstrained clocks result in excessive negative slack on almost every path. This condition causes use of maximum time borrowing everywhere, which is unlikely to be optimal. When using Partial Reconfiguration, if you compile the base design with time borrowing enabled, compile the implementation design(s) with time borrowing enabled. Otherwise, time borrowing amounts in the base design reset to zero, and the design may not pass timing. If this condition occurs, you can use the `update_timing_netlist -recompute_borrow` command to restore time borrowing amounts throughout the design.

Related Information

[Time Borrowing](#) on page 20

1.2. Timing Analysis Overview Document Revision History

Document Version	Quartus Prime Version	Changes
2024.09.30	24.3	Updated <i>What's New In This Version</i> topic for new SDC report tabs, missing collection commands, and deprecation of the -blackbox argument.
2024.04.01	24.1	Updated <i>What's New In This Version</i> topic for DNI, SDC-on-RTL, and locating from report to source file.
2023.04.03	23.1	Updated <i>What's New In This Version</i> topic for changes to entity-bound SDCs.
2022.03.28	22.1	- Added Top FAQ navigation to cover page. - Added new <i>What's New In This Version</i> topic for changes to Report Register Statistics report.
2020.09.28	20.3	Revised setup arrow direction in multiple timing diagrams for consistency.
2020.04.13	20.1	Added "Time Borrowing" section.
2019.09.30	19.3	Updated "MultiCorner Timing Analysis" code example and stated limitation for operating conditions.
2019.07.15	19.2	Updated "MultiCorner Analysis" for SmartVID timing models.
2018.09.24	18.1	Minor text enhancements for clarity and style.

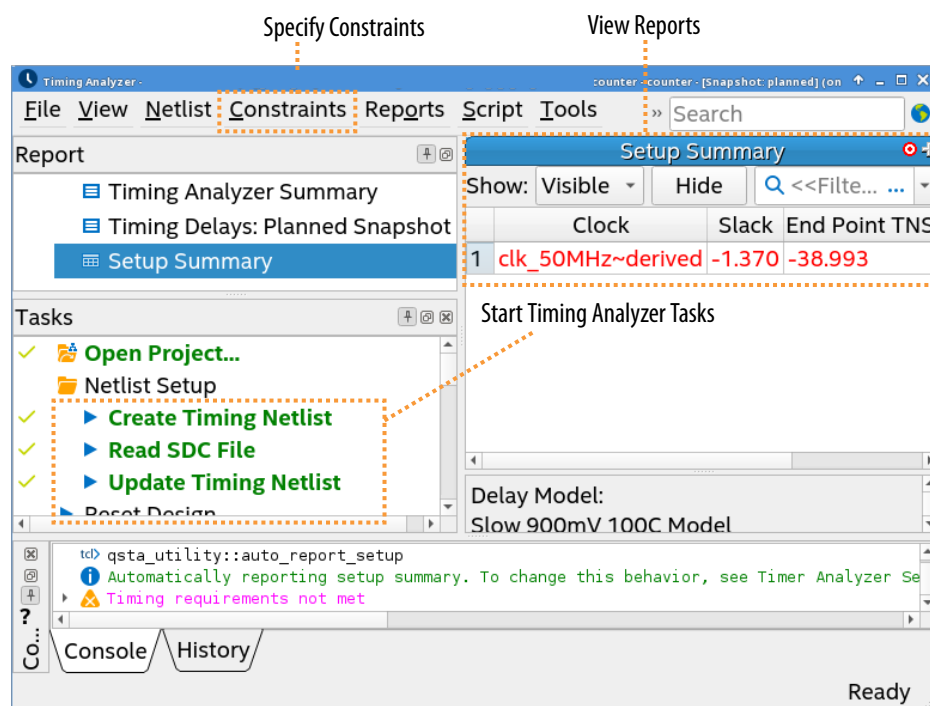
Table 3. Document Revision History

Date	Version	Changes
2016.10.31	16.1	Implemented Intel rebranding.
2016.05.02	16.0	Corrected typo in Fig 6-14: Clock Hold Slack Calculation from Internal Register to Output Port
2015.11.02	15.1	Changed instances of <i>Quartus II</i> to <i>Quartus Prime</i> .
2014.12.15	14.1	Moved Multicycle Clock Setup Check and Hold Check Analysis section from the Timing Analyzer chapter.
June 2014	14.0	Updated format
June 2012	12.0	Added social networking icons, minor text updates
November 2011	11.1	Initial release.

2. Using the Quartus Prime Timing Analyzer

The Quartus Prime Timing Analyzer is a powerful ASIC-style timing analysis tool that validates the timing performance of all logic in your design using an industry-standard constraint, analysis, and reporting methodology. Use the Timing Analyzer GUI or command-line interface to constrain, analyze, and report results for all timing paths in your design.

Figure 37. Quartus Prime Timing Analyzer GUI



Related Information

- [Timing Analyzer Quick-Start Tutorial: Quartus Prime Pro Edition](#)
- [Altera FPGA Technical Training](#)

2.1. Using Timing Constraints throughout the Design Flow

To ensure accurate timing analysis, it is essential to define proper timing constraints that specify your design's clock frequency requirements, timing exceptions, and I/O timing specifications for comparison with actual conditions.

You specify the timing constraints in various Synopsys Design Constraint (SDC) files that you add to the project. You can define SDC files and run timing analysis at two key stages of the design compilation flow:

- After running **Analysis & Synthesis**—you can run Early Timing Analysis based on the synthesized design and initial SDC-on-RTL constraints you specify. If you're starting a new design, you can use SDC-on-RTL constraint methodology to learn benefits of timing analysis after synthesis. If you've already partially completed an existing design, it is best to use conventional SDC constraints.
- After running the **Fitter**—you can run post-fit timing analysis that accounts for actual path delays and the conventional SDC constraints you specify.

Post-Synthesis Early Timing Analysis Constraints

After running **Analysis & Synthesis**, you can run post-synthesis Early Timing Analysis based on the synthesized design and initial constraints that you define with SDC-on-RTL (`.rtl.sdc`) or a synthesis-only conventional (`.sdc`). SDC-on-RTL allows you to define constraints using the same names in your design RTL, ensuring that your timing constraints names align closely with the RTL node names in the elaborated netlist.

Early timing analysis uses the initial SDC-on-RTL constraints that you specify to perform post-synthesis static timing analysis without needing to run the Fitter. The Compiler reads the constraints during Analysis & Elaboration, and then applies the SDC-on-RTL constraints for all downstream stages of compilation. For more details, refer to [Specifying SDC-on-RTL Timing Constraints](#) for step by step instructions.

As an alternative to SDC-on-RTL, you can define a conventional synthesis-only `.sdc` that applies the constraints only for design synthesis, as [Specifying Synthesis-Only SDC Timing Constraints](#) describes.

Post-Fit Timing Analysis Constraints

After running the Fitter's Plan, Place Route, Fitter (Finalize) stage, you can run post-fit timing analysis that accounts for actual path delays based on the Planned, Placed, or Routed design with constraints that you define in conventional SDC (`.sdc`) files. This post-fit timing analysis provides the most precise control over timing constraints.

You can define a conventional `.sdc` file directly in the Timing Analyzer GUI, or use the SDC file templates available using **Edit > Insert Template**. You can alternatively define an `.sdc` file in any text editor and then integrate it into your project. Refer to [Specifying Conventional SDC Timing Constraints](#).

The [SDC File Types Supported](#) table summarizes the differences between the various SDC file types and when the Quartus Prime software uses them.

Table 4. Supported SDC File Types

	SDC-on-RTL	Synthesis-Only SDC	Conventional SDC
Stage where constraints are read	Analysis & Elaboration	Synthesis	Fitter, Signoff
Stage where constraints are processed	Synthesis through Fitter	Synthesis only	Fitter, Signoff
QSF assignment	RTL_SDC_FILE (supports entities)	SDC_FILE SDC_ENTITY_FILE - read_during_post_syn_and_post_fit_ timing_analysis	SDC_FILE
		SDC_FILE SDC_ENTITY_FILE - read_during_post_syn_and_not_post_ fit_timing_analysis	
Syntax supported	Tcl with SDC 2.1 commands	Tcl with Quartus Prime SDC commands	Tcl with Quartus Prime SDC commands
SDC 2.1-compliant	Yes	No	No
Target type	RTL	Quartus Prime timing graph	Quartus Prime timing graph
Hierarchical targets	Yes	No	No
Buried timing nodes (used by IP)	No	Core fabric only. Such nodes do not exist for the periphery in post-synthesis timing analysis.	Yes
STA command to load constraints	Executes the read_sdc or import_sdc command in any snapshot.	Executes the read_sdc command only during static timing analysis on the synthesized snapshot.	Executes the read_sdc command during static timing analysis on any fitter snapshot (plan, place, route, retime). Not loaded during synthesis.

Related Information

- [Applying Timing Constraints](#) on page 50
- [Using Entity-Bound SDC Files](#) on page 65
- [Using Entity-Based SDC-on-RTL Constraints](#) on page 58
- [Creating Constraints in SDC-on-RTL SDC Files, Quartus Prime Pro Edition User Guide: Design Compilation](#)

2.2. Timing Analysis Flow

The following describes high-level steps in the timing analysis flow. This section describes each step in detail:

- [Step 1: Specify General Timing Analyzer Settings](#) on page 29
- [Step 2: Specify Timing Constraints](#) on page 31
- [Step 3: Run the Timing Analyzer](#) on page 34
- [Step 4: Analyze Timing Reports](#) on page 40

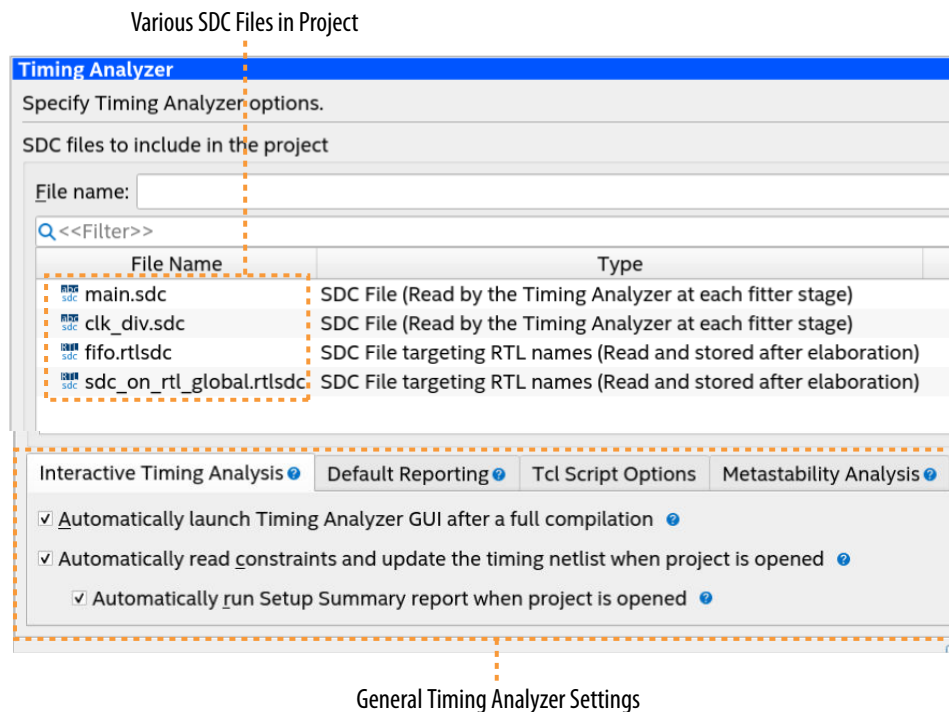
2.2.1. Step 1: Specify General Timing Analyzer Settings

Before running timing analysis, you must open an Quartus Prime project. You can then consider and specify general settings for timing analysis, as well as other project-wide Compiler settings that impact the timing analysis results.

To specify general Timing Analyzer settings, follow these steps:

1. To open an existing project, click **File** ► **Open Project**.
2. Click **Assignments** ► **Settings** ► **Timing Analyzer** to open the **Timing Analyzer** settings.

Figure 38. Timing Analyzer Page of Settings Dialog Box



3. In the **Timing Analyzer** page, specify one or more of the following general Timing Analyzer general settings:

Table 5. Timing Analyzer General Settings

Setting	Description
SDC files to include in the project	Specifies the name and processing order of timing constraint files in the project, such as conventional .sdc files and SDC-on-RTL (.rtlsdc) files. For more details, refer to Using Timing Constraints Effectively and Step 2: Specify Timing Constraints .
Interactive Timing Analysis	Specify options for automatically running timing analysis, reading constraints, and generating reports automatically. Turn on or off:

continued...

Setting	Description
	• Automatically launch Timing Analyzer GUI after a full compilation (default, on) • Automatically read constraints and update the timing netlist when project is opened in Timing Analyzer (default, on) • Automatically run setup summary report when project is opened in Timing Analyzer (default, on)
Default Reporting	Specify options to automatically Report worst-case paths during compilation (default, on). Specify the Paths reported per clock domain (default, 10), and whether to Show routing (default, off) in reports.
Tcl Script Options	Tcl Script File name specifies the file name for a custom timing analysis script. You can specify whether to Run default timing analysis before running custom script .
Metastability Analysis	Specifies how the Timing Analyzer identifies registers as being part of a synchronization register chain for metastability analysis.

4. Consider and specify project-wide Compiler settings that impact timing analysis:

Table 6. Compiler Settings Impacting Timing Analysis

Setting	Description	Location
Enable multicorner support for Timing Analyzer and EDA Netlist Writer (default, on)	Directs the Timing Analyzer to perform multicorner timing analysis by default, which analyzes the design against best-case and worst-case operating conditions.	Assignments > Settings > Compilation Process Settings
Optimization Mode (default, Balanced)	Specifies the focus of Compiler optimization efforts during synthesis and fitting. Specify a Balanced strategy, or optimize for Performance , Area , Power , Routability , or Compile Time .	Assignments > Settings > Compiler Settings
SDC Constraint Protection (default, off)	Verifies .sdc constraints in register merging. This option helps to maintain the validity of .sdc constraints through compilation.	Assignments > Settings > Compiler Settings > Advanced Settings (Synthesis)
Synchronization Register Chain Length (default, 3)	Specifies the maximum number of registers in a row that the Compiler considers as a synchronization chain. The Compiler considers these registers for metastability analysis. The Compiler prevents optimizations of these registers, such as retiming. When gate-level retiming is enabled, the Compiler does not remove these registers.	Assignments > Settings > Compiler Settings > Advanced Settings (Synthesis)
Optimize Design for Metastability (default, on)	This setting improves the reliability of the design by increasing its Mean Time Between Failures (MTBF). The Fitter increases the output setup slacks of synchronizer registers in the design. This slack can exponentially increase the design MTBF. This option only applies when using the Timing Analyzer for timing-driven compilation. Use the Timing Analyzer <code>report_metastability</code> command to review the synchronizers detected in your design and to produce MTBF estimates.	Assignments > Settings > Compiler Settings > Advanced Settings (Fitter)

2.2.2. Step 2: Specify Timing Constraints

You can specify timing constraints for your Quartus Prime project throughout the design flow, as [Using Timing Constraints Effectively throughout the Design Flow](#) describes. This section describes how to specify SDC-on-RTL timing constraints, conventional SDC timing constraints, and synthesis-only SDC timing constraints.

2.2.2.1. Specifying SDC-on-RTL Timing Constraints

To specify SDC-on-RTL timing constraints for post-synthesis Early Timing Analysis, follow these steps:

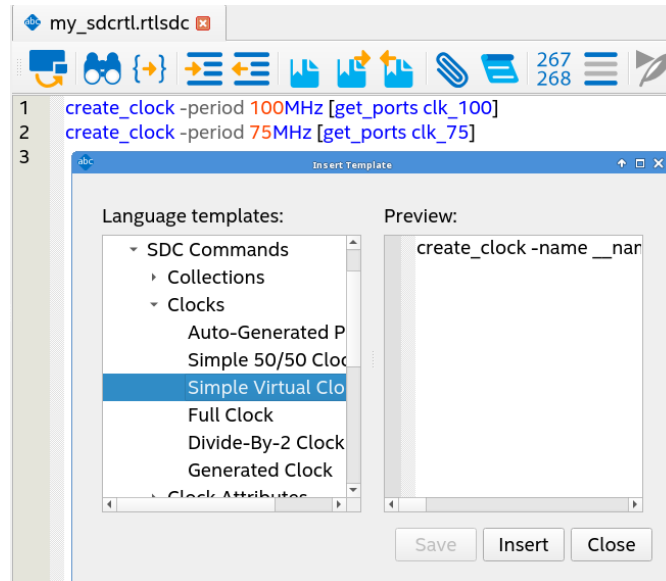
1. In the Quartus Prime software, click **File > New** and then select the **SDC File Targeting RTL Names** (.rtlsdc) file type. The new file opens in the Text Editor.

You can specify any extension for SDC-on-RTL constraints, but this document always uses the file extension .rtlsdc to distinguish from conventional SDC files.

2. In the Text Editor, define SDC-on-RTL constraints. You can click **Edit > Insert Template > Timing Analyzer** to insert available SDC templates. Alternatively, use any other text editor to enter the constraints and save as .rtlsdc file type.

Note: SDC-on-RTL constraints support a subset of conventional SDC commands. The syntax and arguments of SDC-on-RTL constraints aligns with the SDC 2.1 standard. The Quartus Prime software may support more than just the SDC 2.1 commands because of some Quartus Prime software-specific arguments.

Figure 39. Inserting SDC Templates



Note: The `-comment` argument in SDC-on-RTL allows you to add a constraint comment. This comment does not appear in timing analysis reports.

3. Save the .rtlsdc file in the Quartus Prime Text Editor, turning on the **Add File to Project** option.

Note: You can add any SDC file to the project at any time by clicking **Assignments > Settings > Timing Analyzer**.

4. Run post-synthesis Early Timing Analysis, as [Running Post-Synthesis Early Timing Analysis](#) describes.

Note: As an alternative to SDC-on-RTL constraints, you can consider initially specifying synthesis-only constraints that apply only to the Analysis & Synthesis stage of compilation, as [Specifying Synthesis-Only SDC Timing Constraints](#) describes.

Related Information

[Using Entity-Based SDC-on-RTL Constraints](#) on page 58

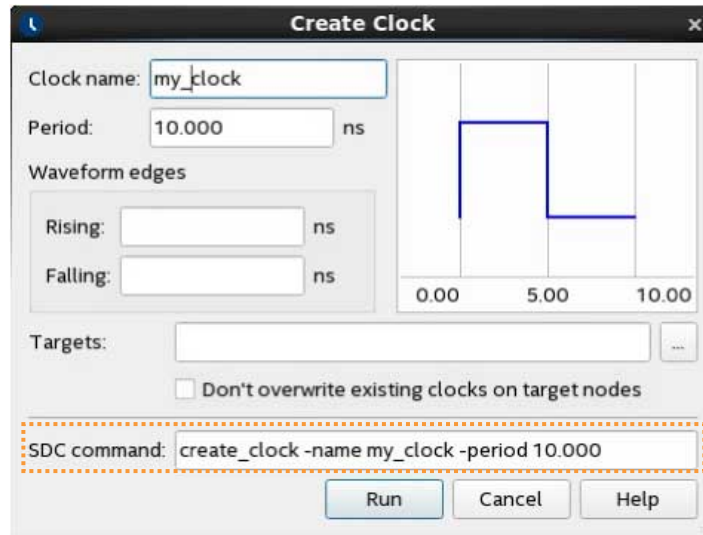
2.2.2.2. Specifying Conventional SDC Timing Constraints

To specify conventional timing constraints for post-fit timing analysis in the Timing Analyzer GUI, follow these steps:

Note: As an alternative to the Timing Analyzer GUI, you can specify conventional SDC constraints directly in a `.sdc` text file and add it to the project.

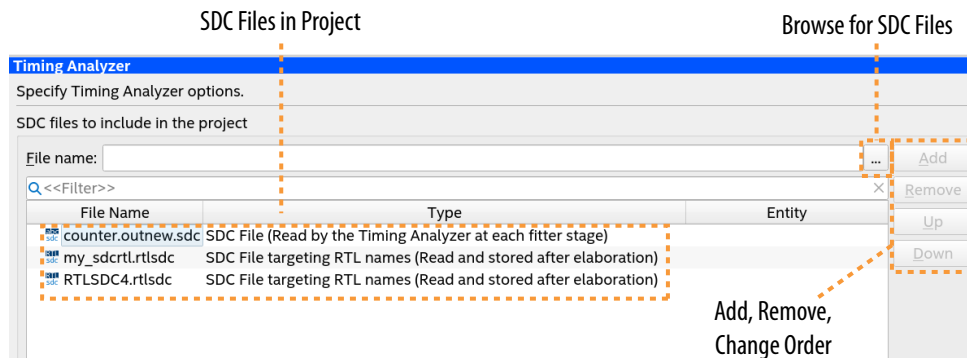
1. In the Timing Analyzer GUI, click the **Constraints** menu and then click the constraint that you want to define. You can start by adding the [Recommended Initial Conventional SDC Constraints](#), and then iteratively modify the constraints and reanalyze the timing results.
2. In the GUI, specify values for the constraint and click **OK**. The **SDC command** field echoes the corresponding SDC command that applies as you enter the constraint value in the GUI.

Figure 40. Create Clock Dialog Defines Clock Constraints



3. When done entering constraints, click **Constraints** ► **Write SDC File** to save the constraints to an .sdc file.
4. To add the conventional .sdc file to your project, click **Assignments** ► **Settings** ► **Timing Analyzer**.

Figure 41. Adding an SDC to the Project



5. Run post-fit timing analysis, as [Running Post-Fit Timing Analysis](#) describes.

Related Information

[Using Entity-Bound SDC Files](#) on page 65

2.2.2.3. Specifying Synthesis-Only SDC Timing Constraints

You use this synthesis-only SDCs if you want to create custom SDC's that apply just for post-synthesis timing analysis, but you do not want the constraints to apply to other downstream stages of the compilation flow. If you want the constraints to persist post-synthesis, you can use SDC-on-RTL constraints, as [Specifying SDC-on-RTL Timing Constraints](#) describes.

Follow these steps to define a conventional .sdc file that applies only to the Analysis & Synthesis stage of compilation.

1. Create a conventional .sdc file that contains only the timing constraints for the Analysis & Synthesis stage of compilation, as [Specifying Conventional SDC Timing Constraints](#) describes.
2. To apply the conventional .sdc to the project for synthesis-only, add the following assignment to the project:

```
set_global_assignment -name SDC_ENTITY_FILE <file>.sdc /
    -entity <name> -read_during_post_syn_and_not_post_fit_timing_analysis
```

3. To run design synthesis and apply the constraints to the timing netlist, click **Analysis & Synthesis** on the Compilation Dashboard.
4. Click the **Open Timing Analyzer** icon next to **Analysis & Synthesis** on the Compilation Dashboard. The synthesis-only constraints now apply to only the static timing analysis of this synthesized snapshot.
5. Analyze the results of Early Timing Analysis, as [Step 4: Analyze Timing Reports](#) on page 40 describes.

2.2.3. Step 3: Run the Timing Analyzer

The Timing Analyzer generates reports that you can review to determine the performance of your design compared against your timing constraints. You can run timing analysis at two key stages of the design compilation flow:

- After running **Analysis & Synthesis**—you can run Early Timing Analysis based on the synthesized design and initial SDC-on-RTL constraints you specify.
- After running the **Fitter**—you can run post-fit timing analysis that accounts for actual path delays and the conventional SDC constraints you specify.

2.2.3.1. Running Post-Synthesis Early Timing Analysis

Running the Early Timing Analysis stage of compilation provides a preliminary view of your design's core timing. Before running Early Timing Analysis, you must setup your design RTL, the Quartus Prime project, specify timing constraints, and run the Compiler through the Analysis & Synthesis stage.

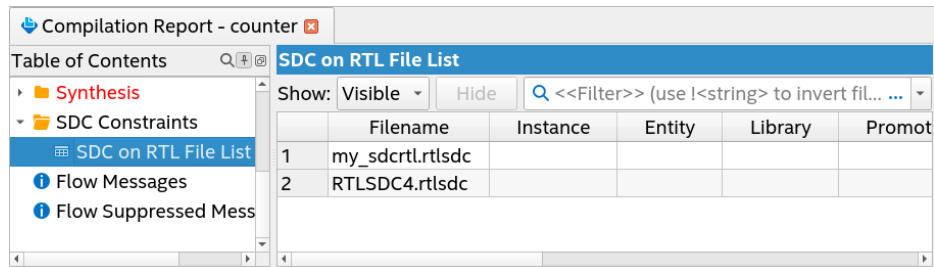
To run post-synthesis Early Timing Analysis, follow these steps:

1. Specify SDC-on-RTL timing constraints for Early Timing Analysis, as [Specifying SDC-on-RTL Timing Constraints](#) describes.

Note: As an alternative to SDC-on-RTL, you can define a synthesis-only .sdc, as [Specifying Synthesis-Only SDC Timing Constraints](#) describes.

2. On the Compilation Dashboard, click **Analysis & Elaboration**. Analysis & Elaboration processes all SDC-on-RTL, applying the constraints to the design netlist. During Analysis & Elaboration, messages confirm that the Compiler appropriately applies each .rtl.sdc file according to its assigned module.
3. To review the implementation of constraints when Elaboration & Analysis completes, click the **Open Compilation Reports** icon next to **Elaboration & Analysis** in the Compilation Dashboard.
4. Click the **SDC Constraints ► SDC-on-RTL File List** report to view all SDC-on-RTL files in the current project.

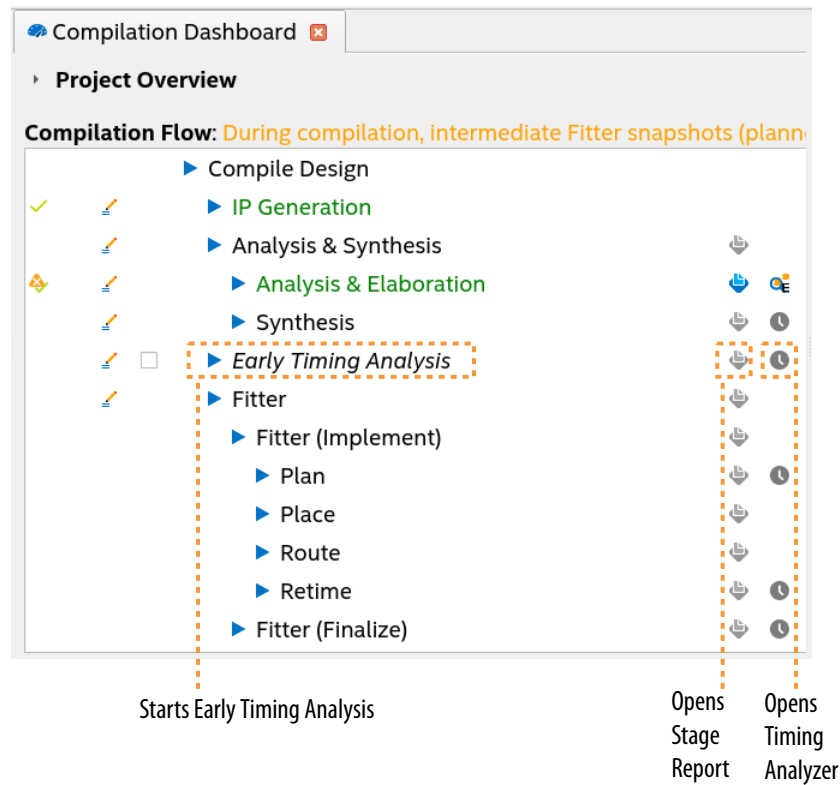
Figure 42. SDC-on-RTL File List Report



	Filename	Instance	Entity	Library	Promot
1	my_sdctrl.rtlsdc				
2	RTLSDC4.rtlsdc				

5. Click **Analysis & Synthesis** on the Compilation Dashboard. Analysis & Synthesis transforms the elaborated netlist into a node netlist for device resource mapping and generates a simplified device delay model that excludes precise Fitter-generated timing delays. This simplified delay model provides an early overview of the design delays based on block types that connect to a net. Analysis & Synthesis propagates SDC-on-RTL constraints to subsequent compilation stages, thereby applying to all subsequent Timing Analyzer runs.
6. To run Early Timing Analysis and view the results, double-click **Early Timing Analysis** on the Compilation Dashboard. The Compiler runs Analysis & Synthesis and then initializes the Timing Analyzer.

Figure 43. Running Early Timing Analysis from Compilation Dashboard



7. When Analysis & Synthesis completes, click the **Open Timing Analyzer** icon next to **Early Timing Analysis** on the Compilation Dashboard. The Timing Analyzer opens with the updated timing netlist loaded automatically.
8. In the Timing Analyzer reports, view the preliminary timing report data measured against your SDC-on-RTL constraints, such as the **Setup Summary**, **Create Generated Clocks**, and **Set False Path** reports. Refer to [Step 4: Analyze Timing Reports](#).

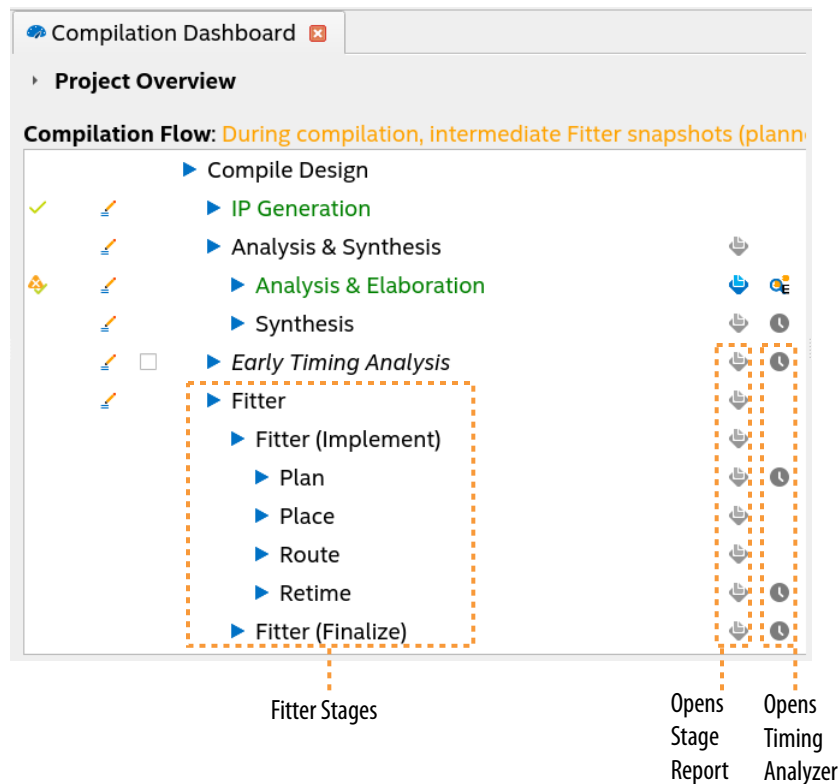
2.2.3.2. Running Post-Fit Timing Analysis

Before running post-fit timing analysis, you must run the Fitter to apply conventional SDC constraints to the post-fit timing netlist. The Fitter then attempts to place the logic of your design to adhere to the timing constraints you specify. After running the Fitter, the Timing Analyzer generates reports detailing the margin (slack) by which your design either meets or fails each constraint. The post-fit timing netlist accounts for actual path delays.

To run post-fit timing analysis, follow these steps:

1. Specify conventional SDC timing constraints for post-fit timing analysis, as [Specifying Conventional SDC Timing Constraints](#) describes.
2. On the Compilation Dashboard, run any stage of the **Fitter (Plan, Place, Route, Retime, Fitter (Finalize))** or run a full compilation.

Figure 44. Fitter Stages in Compilation Dashboard



3. When the Fitter completes, click the Timing Analyzer icon next to the completed stage in the Compilation Dashboard. The Setup Summary report opens by default in the Timing Analyzer.
4. Review the timing reports. To generate additional timing reports for analysis, click the **Reports** menu, and then click one of the submenu items to generate that report, as [Step 4: Analyze Timing Reports](#) describes.

Figure 45. Setup Summary Report

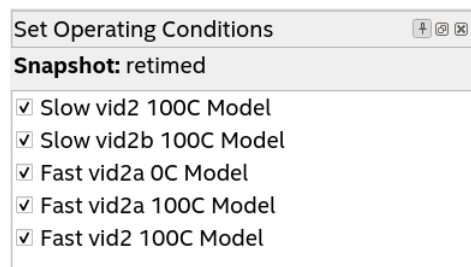
	Clock	Slack	End Point TNS	Worst-Case Operating Conditions
1	clk	38.402	0.000	Slow 900mV 100C Model

- To run timing analysis under different operation conditions, click **Set Operating Conditions** on the **Tasks** pane and specify options, as [Setting the Operating Conditions for Timing Analysis](#) describes. By default, the Timing Analyzer generates reports for all supported operating conditions.
- If you specify any settings or constraints that impact timing analysis, click **Update Timing Netlist** on the **Tasks** pane to apply the new constraints to the timing netlist.

2.2.3.2.1. Setting the Operating Conditions for Timing Analysis

Click **View > Timing Corners** in the Timing Analyzer to specify the operating conditions for the timing analysis under different power and temperature ranges. The available operating conditions vary by device and speed grade. The various operating conditions that you can select represent the different "timing corners" in a multi-corner timing analysis.

Figure 46. Set Operating Conditions Panel



The Timing Analyzer displays the selectable operating conditions that are appropriate for your device in the **Set Operating Conditions** panel, according to the following timing model name conventions:

<process in speed grade> <voltage/vid> <temperature>

The following examples illustrate this naming convention:

- **Fast 900mV 40C Model**
 - **Fast**—timing model for the fastest process within a speed grade.
 - **900mV**—nominal 900mV timing model.
 - **40C**—low temperature (40 Celsius) timing model.

- **Slow vid<n> 100C Model**
 - **Slow**—timing model for the slowest process within a speed grade.
 - **vid**—timing model for analysis with SmartVID.
 - **<n>**—the device speed grade of the timing model.
 - **100C**—high temperature (100 Celsius) timing model.

Select one or more voltage and temperature combinations and double-click **Report Timing...** under **Timing Slack** in the **Tasks** pane to configure the generation of timing analysis reports for that model. The generated report shows the worst-case timing path slack across all operating conditions that you specify. After generating the report for that model, the report shows the worst-case timing path slacks across all operating conditions that you select.

You can use the following context menu options to generate or regenerate reports in the **Report** window:

- **Regenerate**—regenerates the report you select.
- **Regenerate All Out of Date**—regenerates all reports that are outdated because of SDC changes since the last generation.
- **Delete All Out of Date**—removes all outdated report data.

As an alternative to the GUI, you can run the `set_operating_conditions` command with the `-model`, `-speed`, `-temperature`, and `-voltage` options to specify the operating conditions. When running `set_operating_conditions`, you must only specify valid operating conditions for the current device. Run the `get_available_operating_conditions` command to return a list of all valid operating conditions for the current device.

The following example sets the operating condition to the slow timing model, with a voltage of 900 mV, and temperature of 100° C:

```
set_operating_conditions -model slow -temperature 100 -voltage 900
```

Related Information

[Multicorner Timing Analysis](#) on page 19

2.2.3.2.2. Promoting Critical Warnings to Errors

You can promote critical warnings to errors so that the Timing Analyzer halts on receiving the critical warnings (as it does for all errors). All critical warnings support promotion to error. However, you can only promote the message IDs of open projects.

1. In the **Message** dialog box, right-click on the critical warning you want to promote to an error.
2. Click **Message Promotion > Promote Critical Message ID to Error**
3. To clear all promotions, click **Message Promotion > Clear All Message Promotions**
4. Alternatively, promote or demote a critical warning in the `.qsf` with the following:

```
set_global_assignment -name PROMOTE_WARNING_TO_ERROR
<ID_Number>
```

2.2.4. Step 4: Analyze Timing Reports

During analysis, the Timing Analyzer examines the timing paths in the design, calculates the propagation delay along each path, checks for timing constraint violations, and reports timing results as positive slack or negative slack. Negative slack indicates a timing violation. Positive slack indicates that timing requirements are met.

The Timing Analyzer provides very fine-grained reporting and analysis capabilities to identify and correct violations along timing paths. Generate timing reports to view how to best optimize the critical paths in your design. Many timing reports are searchable using the Filter bar, allowing you to find the incremental delays for cells, or interconnect, or just find a specific node of interest.

If you modify, remove, or add constraints, you must re-run timing analysis to see the latest results. This iterative process helps resolve timing violations in your design.

Figure 47. Timing Analyzer Shows Failing Paths in Red

Filters Report Data

	Clock	Slack	End Point TNS	Worst-Case Operating Conditions
1	n/a	-2.925	-2.925	Slow 900mV 100C Model
2	clock	-1.594	-9.686	Slow 900mV 100C Model

Reports that indicate failing timing performance appear in red text, and reports that pass appear in black text. A gold question mark icon indicates reports that are outdated due to SDC changes since generation. Regenerate these reports to show the latest data.

The following sections describe how to generate various timing reports for analysis.

Related Information

[Timing Report Descriptions](#) on page 132

2.2.4.1. Cross-Probing with Design Assistant

The Quartus Prime Design Assistant can automatically report any violations against a standard set of recommended design guidelines during stages of compilation. You can specify which rules you want the Design Assistant to check in your design, and customize the severity levels, thus eliminating or waiving rule checks that are not important for your design.

When you run Design Assistant during compilation, Design Assistant utilizes the in-flow (transient) data that generates during compilation to check for rule violations.

In certain designs, you may see differences in slack values between the Design Assistant and Timing Analyzer for certain paths at different stages of the compilation process. These slack value differences often occur because various timing constraints can be used at each compilation stage. IP commonly uses different timing constraints

to help make improvements during the compilation process. Because of this fact, the Design Assistant may first show negative slack, while the Timing Analyzer subsequently shows positive slack once the plan is fully built.

Also, during the first rounds of the retiming process, the Design Assistant may identify paths with negative slack. You can fix these paths in later rounds, but initial reports may show the paths that are not working. It is important to understand that these types of situations do not necessarily indicate a timing problem with the design. For the most accurate verification of timing performance, always refer to the Timing Analyzer report from the **Fitter (Finalize)** stage.

When you run Design Assistant in analysis mode from the Timing Analyzer, Design Assistant performs design rule checks using the static compilation snapshot data that you load.

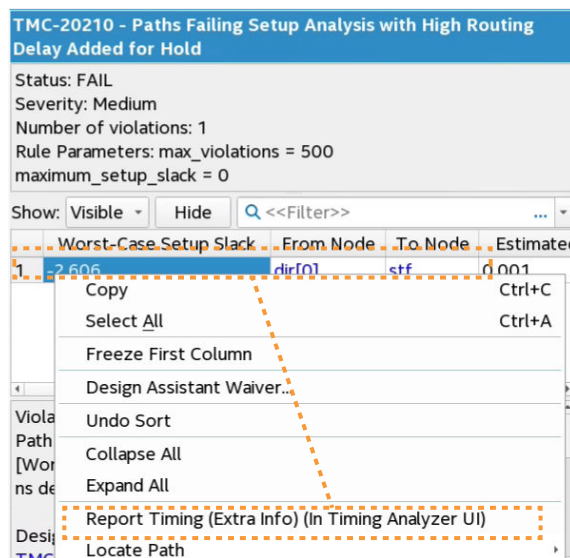
Some Design Assistant rule violations allow cross-probing into the related timing analysis data. Cross-probing can help you to more quickly identify the root cause and resolve any Design Assistant rule violations. For example, for a path with a setup analysis violation, you can cross-probe into the Timing Analyzer to identify the edge that has delay added for hold time.

Note: You must run the Compiler through at least the Plan stage before you can cross-probe to Timing Analyzer.

2.2.4.1.1. Cross-Probing from Design Assistant to Timing Analyzer

Some Design Assistant rule violations allow cross-probing into Timing Analyzer. For example, for a path that Design Assistant flags with a setup analysis violation due to delay added for hold, you can cross-probe into the Timing Analyzer to view more information on the affected path and edge.

Figure 48. Cross Probing from Design Assistant Rule TMC-20210 Violations to Timing Analyzer



Follow these steps to cross-probe from such Design Assistant rule violations to the Timing Analyzer:

1. Compile the design through at least the Compiler's Plan stage.
2. Locate a rule violation in the Design Assistant folder of the Compilation Report.
3. Right-click the rule violation to display any **Report Timing** commands available for the violation.
4. Click the **Report Timing** command. The Timing Analyzer opens and reports the timing data for the violation path. **Report Timing (Extra Info)** includes Estimated Delay Added for Hold and Route Stage Congestion Impact extra data.

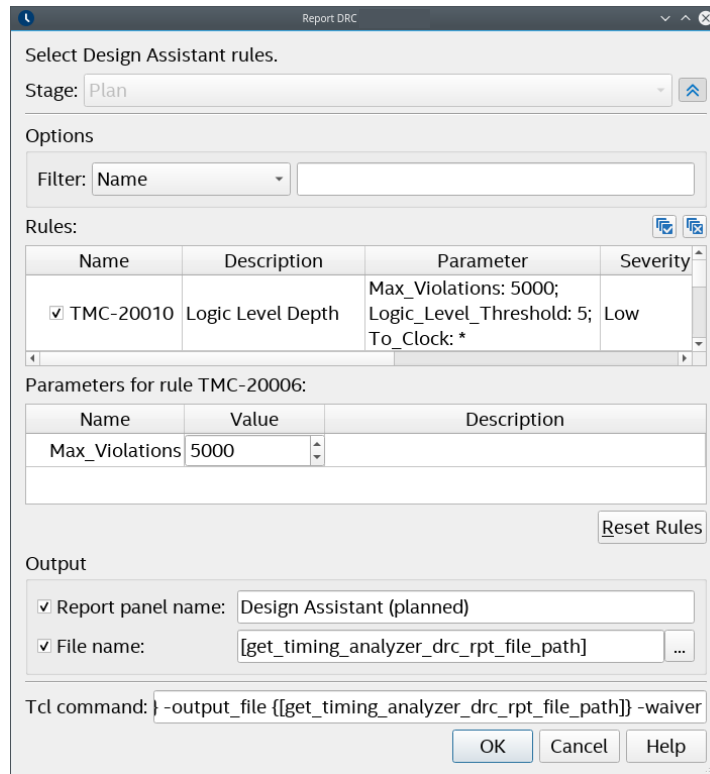
2.2.4.2. Launching Design Assistant from Timing Analyzer

You can run Design Assistant directly from the Timing Analyzer to assist when optimizing timing paths and other timing conditions. When you launch Design Assistant from the Timing Analyzer, Design Assistant only checks rules that relate to timing analysis.

Follow these steps to run the Design Assistant from the Timing Analyzer:

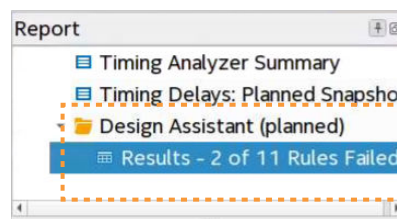
1. Compile the design through at least the Compiler's Plan stage.
2. Open the Timing Analyzer for the Compiler stage from the Compilation Dashboard.
3. In the Timing Analyzer, click **Reports > Design Assistant > Report DRC**. The **Report DRC** (design rule check) dialog box opens.
4. Under **Rules**, disable any rules that are not important to your analysis by removing the check mark.
5. Consider whether to adjust rule parameter values in the **Parameters** field.

Figure 49. Report DRC (Design Rule Check) Dialog Box



6. Confirm the **Report panel name** and optionally specify an output **File name**.
7. Click **Run**. The Results reports generate and appear in the **Report** pane, as well as the main Compilation Report.

Figure 50. Design Assistant Reports in Timing Analyzer Report Pane

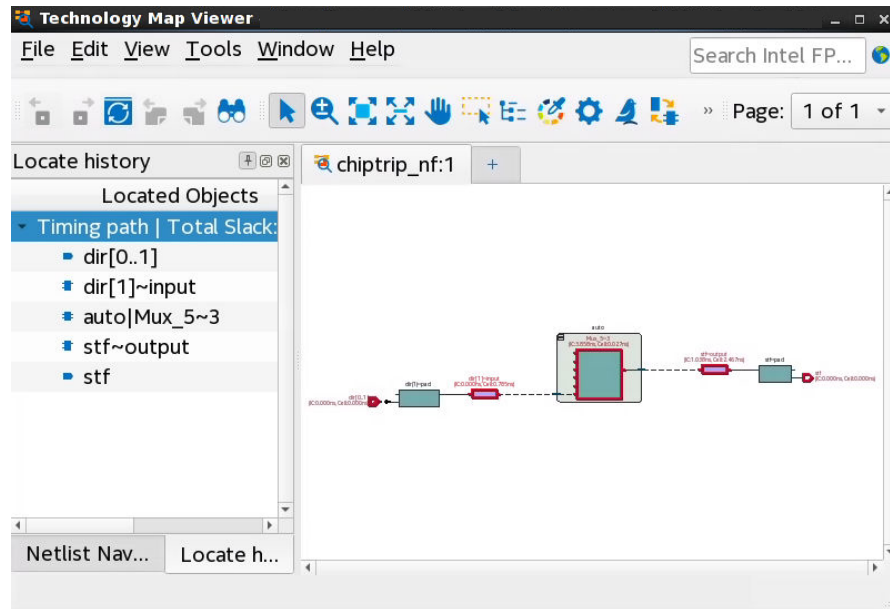


2.2.4.3. Locating Timing Paths in Other Tools

You can locate from paths and elements in the Timing Analyzer to other visualization tools in the Quartus Prime software, such as the Chip Planner, Technology Map Viewer, or Resource Property Viewer.

You can right-click most paths or node names in the Timing Analyzer reports and click the **Locate Node** or **Locate Path** commands. Use these commands in the Timing Analyzer GUI or the `locate` command in the Tcl console to locate to that node in other Quartus Prime tools.

Figure 51. Locate Path from Timing Analyzer to Technology Map Viewer



The following examples show how to locate the ten paths with the worst timing slack from Timing Analyzer to the Technology Map Viewer, and locate all ports matching data* in the Chip Planner.

Example 1. Locating from the Timing Analyzer

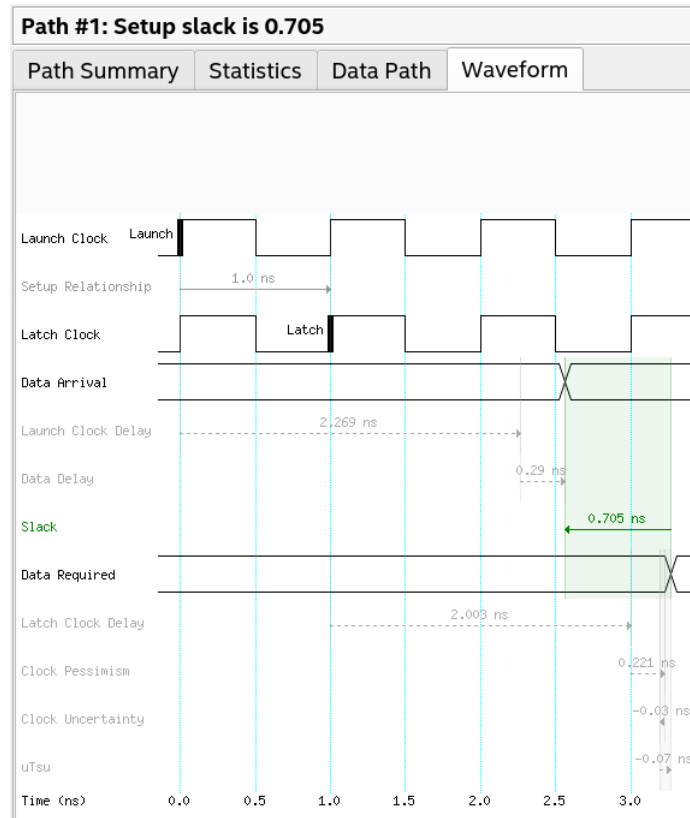
```
# Locate in the Technology Map Viewer the ten paths with the worst slack
locate [get_timing_paths -npaths 10] -tmv
# locate all ports that begin with data in the Chip Planner
locate [get_ports data*] -chip
```

2.2.4.4. Correlating Constraints to the Timing Report

Understanding how timing constraints and violations appear in the timing analysis reports is critical to understanding the results. The following examples show how specific constraints impact the timing analysis reports. Most timing constraints only affect the clock launch and latch edges. Specifically, `create_clock` and `create_generated_clock` create clocks with default relationships. However, the `set_multicycle_path` exception modifies the default setup and hold relationships. The `set_max_delay` and `set_min_delay` constraints are low-level overrides that explicitly indicate the maximum and minimum delays for the launch and latch edges.

The following figures show the results of running **Report Timing** on a particular path. You can view the incremental delays on the **Data Path** and **Waveform** tabs after running **Report Timing**. The **Waveform** tab allows you to visually reference the **Data Path** data, as well as the original .sdc constraints. You can use the **Waveform** tab to easily see how and where the constraints apply.

Figure 52. Report Timing (Waveform Tab)



In the following example, the design includes a clock driving the source and destination registers with a period of 10 ns. This results in a setup relationship of 10 ns (launch edge = 0 ns, latch edge = 10 ns) and hold relationship of 0 ns (launch edge = 0 ns, latch edge = 0 ns) from the command:

```
create_clock -name clocktwo -period 10.000 [get_ports {clk2}]
```

Figure 53. Setup Relationship 10ns

Path #1: Setup slack is 6.429

Path Summary Statistics Data Path Waveform Extra Fitter Information							
Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	
1	0.000	0.000				launch edge time	
2	4.578	4.578				clock path	
1	0.000	0.000				source latency	
2	0.000	0.000			1	PIN_H13	clk2
Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	
1	10.000	10.000				latch edge time	
2	13.876	3.876				clock path	
1	10.000	0.000				source latency	
2	10.000	0.000			1	PIN_H13	clk2
3	10.000	0.000	RR	IC	1	IOIBUF_X56_Y81_N1	clk2~input[i]

Figure 54. Hold Relationship 0ns

Path #1: Hold slack is 0.468

Path Summary Statistics Data Path Waveform Extra Fitter Information							
Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	
1	0.000	0.000				launch edge time	
2	4.397	4.397				clock path	
1	0.000	0.000				source latency	
2	0.000	0.000			1	PIN_N16	clk1
3	0.000	0.000	RR	IC	1	IOIBUF_Y80_V35_N44	clk1~input[i]
Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	
1	0.000	0.000				latch edge time	
2	4.539	4.539				clock path	
1	0.000	0.000				source latency	
2	0.000	0.000			1	PIN_N16	clk1
3	0.000	0.000	RR	IC	1	IOIBUF_Y80_V35_N44	clk1~input[i]

Adding `set_multicycle_path` constraints adds multicycles to relax the setup relationship, or open the window, making the setup relationship 20 ns while maintaining the hold relationship at 0 ns:

```
set_multicycle_path -from clocktwo -to clocktwo -setup -end 2
set_multicycle_path -from clocktwo -to clocktwo -hold -end 1
```

Figure 55. Setup Relationship 20ns

Path #1: Setup slack is 16.429							
Path Summary Statistics Data Path Waveform Extra Fitter Information							
Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	
1	0.000	0.000				launch edge time	
2	4.578	4.578				clock path	
1	0.000	0.000				source latency	
2	0.000	0.000			1	PIN_H13	clk2
!!!							
Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	
1	20.000	20.000				latch edge time	
2	23.876	3.876				clock path	
1	20.000	0.000				source latency	
2	20.000	0.000			1	PIN_H13	clk2
3	20.000	0.000	RR	IC	1	IOIBUF_X56_Y81_N1	clk2~input[i]
!!!							

Adding the following `set_max_delay` constraints explicitly overrides the setup relationship:

```
set_max_delay -from [get_registers {regA}] -to \
    [get_registers {regB}] 15
```

Note that the only thing changing for these different constraints are the launch edge time and latch edge times for setup and hold analysis. Every other line item comes from delays inside the FPGA and are static for a given fit. View these reports to analyze how your constraints affect the timing reports.

Figure 56. Using `set_max_delay`

Path #1: Setup slack is 11.429							
Path Summary Statistics Data Path Waveform Extra Fitter Information							
Data Arrival Path							
	Total	Incr	RF	Type	Fanout	Location	
1	0.000	0.000				launch edge time	
2	4.578	4.578				clock path	
1	0.000	0.000				source latency	
2	0.000	0.000			1	PIN_H13	clk2
!!!							
Data Required Path							
	Total	Incr	RF	Type	Fanout	Location	
1	15.000	15.000				latch edge time	
2	18.876	3.876				clock path	
1	15.000	0.000				source latency	
2	15.000	0.000			1	PIN_H13	clk2
3	15.000	0.000	RR	IC	1	IOIBUF_X56_Y81_N1	clk2~input[i]
!!!							

For I/O, you must add `set_input_delay` and `set_output_delay` constraints, as the following example shows. These constraints describe delays on signals from outside of the FPGA design that connect to the design's I/O ports.

```
create_clock -period 10 [get_ports clk]
# Clock used by the transfer, clock relationship is 10ns

# Setup constraints
set_output_delay -clock clk -max 1.2 [get_ports out]
# Subtracted from Data Required Path as oExt
set_max_delay -from [get_registers B] 12
# Sets latch edge time

# Hold constraints
set_output_delay -clock clk -min 2.3 [get_ports out]
# Subtracted from Data Required Path as oExt
set_min_delay -from [get_registers B] 8
# Sets latch edge time
```

The values of these constraints are the delays of the external signals between an external register and a port on the design. The `-clock` argument to the `set_input_delay` and `set_output_delay` specifies the clock domain that the external signal belongs to, or rather, the clock domain of the external register connected to the I/O port. The `-min` and `-max` options specify the worst-case or best-case delay; not specifying either option causes the worst- and best-case delays to be equal. I/O delays display as **iExt** or **oExt** in the **Type** column, as the following example reports shows.

Figure 57. Setup Slack Path Report and Waveforms for a Reg-To-Output Same-Clock Transfer

Path #1: Setup slack is 3.027								
Path Summary		Statistics	Data Path		Waveform			
Data Arrival Path								
	Total	Incr	RF	Type	Fanout	Location	HS/LP	Element
1	0.000	0.000						launch edge time
2	0.000	0.000		borrow				time borrowed
3	4.088	4.088						clock path
1	0.000	0.000						source latency
2	0.000	0.000			1	PIN_L3		clk
3	0.000	0.000	RR	IC	1	IOIBUF_X102_Y34_N47		clk-input i
4	0.675	0.675	RR	CELL	1	IOIBUF_X102_Y34_N47		clk-input o
5	0.856	0.181	RR	CELL	1	IOIBUF_X102_Y34_N47		clk-input~io_48_lvds_tile_edge/lockin[3]
6	0.856	0.000	RR	IC	2	CLKCTRL_3B_G_I23		clk-inputCLKENA0 inclck
7	1.476	0.620	RR	CELL	2	CLKCTRL_3B_G_I23		clk-inputCLKENA0 outclk
8	4.088	2.612	RR	IC	1	FF_X101_Y7_N25	High Speed	B clk
9	4.088	0.000	RR	CELL	1	FF_X101_Y7_N25	High Speed	B
4	7.743	3.655						data path
1	4.322	0.234	FF	uTco	1	FF_X101_Y7_N25		B q
2	4.450	0.128	FF	CELL	1	FF_X101_Y7_N25	High Speed	B~la_lab/laboutt[16]
3	5.012	0.562	FF	IC	1	IOOBUF_X102_Y12_N18	High Speed	out~output i
4	7.743	2.731	FF	CELL	1	IOOBUF_X102_Y12_N18		out~output o
5	7.743	0.000	FF	CELL	0	PIN_AC6		out
Data Required Path								
	Total	Incr	RF	Type	Fanout	Location	HS/LP	Element
1	12.000	12.000						latch edge time
2	12.000	0.000						clock path
1	12.000	0.000	R					clock network delay
3	11.970	-0.030						clock uncertainty
4	10.770	-1.200	F	oExt	0	PIN_AC6		out

Figure 58. Hold Slack Path Report and Waveforms for a Reg-To-Output Same-Clock Transfer

Path #1: Hold slack is -2.323 (VIOLATED)								
Path Summary		Statistics	Data Path		Waveform			
Data Arrival Path								
	Total	Incr	RF	Type	Fanout	Location	HS/LP	Element
1	0.000	0.000						launch edge time
2	0.000	0.000		borrow				time borrowed
3	2.029	2.029						clock path
1	0.000	0.000						source latency
2	0.000	0.000			1	PIN_L3		clk
3	0.000	0.000	RR	IC	1	IOIBUF_X102_Y34_N47		clk~input i
4	0.313	0.313	RR	CELL	1	IOIBUF_X102_Y34_N47		clk~input o
5	0.375	0.062	RR	CELL	1	IOIBUF_X102_Y34_N47		clk~input~io_48_lvds_tile_edge/ioclk[3]
6	0.375	0.000	RR	IC	2	CLKCTRL_3B_G_I23		clk~inputCLKENAO inclk
7	0.612	0.237	RR	CELL	2	CLKCTRL_3B_G_I23		clk~inputCLKENAO outclk
8	2.029	1.417	RR	IC	1	FF_X101_Y7_N25	High Speed	B clk
9	2.029	0.000	RR	CELL	1	FF_X101_Y7_N25	High Speed	B
4	3.407	1.378						data path
1	2.125	0.096	RR	uTco	1	FF_X101_Y7_N25		B q
2	2.187	0.062	RR	CELL	1	FF_X101_Y7_N25	High Speed	B~la_lab/laboutt[16]
3	2.370	0.183	RR	IC	1	IOOBUF_X102_Y12_N18	High Speed	out~output i
4	3.407	1.037	RR	CELL	1	IOOBUF_X102_Y12_N18		out~output o
5	3.407	0.000	RR	CELL	0	PIN_AC6		out
Data Required Path								
	Total	Incr	RF	Type	Fanout	Location	HS/LP	Element
1	8.000	8.000						latch edge time
2	8.000	0.000						clock path
1	8.000	0.000	R					clock network delay
3	8.030	0.030						clock uncertainty
4	5.730	-2.300	R	oExt	0	PIN_AC6		out

A clock relationship, which is the difference between the launching and latching clock edge of a transfer, is determined by the clock waveform, multicycle constraints, and minimum and maximum delay constraints. The Timing Analyzer also adds the value of `set_output_delay` as an **oExt** value. For outputs, this value is part of the **Data Required Path**, since this is the external part of the analysis. The setup report subtracts the `-max` value, making the setup relationship harder to meet, since the **Data Arrival Path** delay must be shorter than the **Data Required Path** delay. The Timing Analyzer also subtracts the `-min` value. This subtraction is why a negative number causes more restrictive hold timing. The **Data Arrival Path** delay must be longer than the **Data Required Path** delay.

Related Information

- [Running Post-Synthesis Early Timing Analysis](#) on page 34
- [Running Post-Fit Timing Analysis](#) on page 37
- [Scripting Timing Analysis](#) on page 174
- [Relaxing Setup with Multicycle \(`set\_multicycle\_path`\)](#) on page 107
- [Creating Virtual Clocks](#) on page 81
- [I/O Constraints](#) on page 94

2.3. Applying Timing Constraints

This section provides examples and describes how to correctly apply SDC timing constraints that guide design synthesis, Fitter placement, and produce accurate timing analysis results under various circumstances. You can define a set of initial timing constraints, and then iteratively modify those constraints as the design progresses. The Quartus Prime software checks the correctness of SDC constraints to prevent basic errors.

Early in the design cycle, you can use SDC-on-RTL constraints to target analysis of RTL nodes. This analysis provides a stable reference for constraints that can remain unchanged in subsequent compilation stages, such as clock definitions. Establishing a set of SDC-on-RTL constraints enables their propagation and application throughout the entire design cycle. Concurrently, you can create a conventional .sdc file for analysis of the remaining design elements, providing flexibility for iterative constraint adjustments as the design evolves.

This section also outlines the proper application of recommended conventional SDC timing constraints. Conventional SDC constraints guide Fitter placement via .sdc files, offering alternative approaches to achieve precise control over constraints throughout the design flow.

Related Information

- [Using Timing Constraints throughout the Design Flow](#) on page 27
- [Applying Entity-Bound Timing Constraints](#) on page 58
- [Recommended Initial Conventional SDC Constraints](#) on page 50

2.3.1. Recommended Initial Conventional SDC Constraints

Include the following basic SDC constraints in your conventional .sdc file. The following example shows application of the recommended conventional SDC constraints for a simple dual-clock design:

```
create_clock -period 20.00 -name adc_clk [get_ports adc_clk]
create_clock -period 8.00 -name sys_clk [get_ports sys_clk]

derive_pll_clocks
derive_clock_uncertainty
```

Note:

Only Arria 10 and Cyclone® 10 GX devices support the **Derive PLL Clocks** (derive_pll_clocks) constraint. For all other supported devices, the Timing Analyzer automatically derives PLL clocks from constraints bound to the related IP.

[Create Clock \(create_clock\)](#) on page 51

[Derive PLL Clocks \(derive_pll_clocks\)](#) on page 51

[Derive Clock Uncertainty \(derive_clock_uncertainty\)](#) on page 52

[Set Clock Groups \(set_clock_groups\)](#) on page 53

2.3.1.1. Create Clock (create_clock)

The **Create Clock** (`create_clock`) constraint allows you to define the properties and requirements for a clock in the design. You must define clock constraints to determine the performance of your design and constrain the external clocks coming into the FPGA. You can enter the constraints in the Timing Analyzer GUI, or in the `.sdc` file directly.

You specify the **Clock name** (`-name`), clock **Period** (`-period`), rising and falling **Waveform edge** values (`-waveform`), and the target signal(s) to which the constraints apply.

The following command creates the `sys_clk` clock with an 8ns period, and applies the clock to the `fpga_clk` port.:

```
create_clock -name sys_clk -period 8.0 \  
[get_ports fpga_clk]
```

Note: Tcl and `.sdc` files are case-sensitive. Ensure that references to pins, ports, or nodes match the case of names in your design.

By default, the `sys_clk` example clock has a rising edge at time 0 ns, a 50% duty cycle, and a falling edge at time 4 ns. If you require a different duty cycle, or to represent an offset, specify the `-waveform` option.

Typically, you name a clock with the same name as the port you assign. In the example above, the following constraint accomplishes this:

```
create_clock -name fpga_clk -period 8.0 [get_ports fpga_clk]
```

There are now two unique objects called `fpga_clk`, a port in your design and a clock applied to that port.

Note: In Tcl syntax, square brackets execute the command inside them. `[get_ports fpga_clk]` executes a command that finds and returns a collection of all ports in the design that match `fpga_clk`.

Warning: Constraints that you define in the Timing Analyzer apply directly to the timing database, but do not automatically transfer to the `.sdc` file. Click **Write SDC File** on the Timing Analyzer **Tasks** pane to preserve constraints changes from the GUI in an `.sdc` file.

Related Information

[Creating Base Clocks](#) on page 79

2.3.1.2. Derive PLL Clocks (derive_pll_clocks)

The **Derive PLL Clocks** (`derive_pll_clocks`) constraint automatically creates clocks for each output of any PLL in your design.

Note: Only Arria 10 and Cyclone 10 GX devices support the **Derive PLL Clocks** (`derive_pll_clocks`) constraint. For all other supported devices, the Timing Analyzer automatically derives PLL clocks from constraints bound to the related IP.

The constraint can generate multiple clocks for each output clock pin if the PLL is using clock switchover: one clock for the `inclk[0]` input clock pin, and one clock for the `inclk[1]` input clock pin. Specify the **Create base clocks** (`create_base_clocks`) option to create base clocks on the inputs of the PLLs by default. By default the clock name is the same as the output clock pin name, or specify the **Use net name as clock name** (`use_net_name`) option to use the net name.

When you create PLLs, you must define the configuration of each PLL output. This definition allows the Timing Analyzer to automatically constrain the PLLs with the `derive_pll_clocks` command. This command also constrains transceiver clocks and adds multicycles between LVDS SERDES and user logic.

The `derive_pll_clocks` command prints an Info message to show each generated clock the command creates.

As an alternative to `derive_pll_clocks` you can copy-and-paste each `create_generated_clock` assignment into the `.sdc` file. However, if you subsequently modify the PLL setting, you must also change the generated clock constraint in the `.sdc` file. Examples of this type of change include modifying an existing output clock, adding a new PLL output, or making a change to the PLL's hierarchy. Use of `derive_pll_clocks` eliminates this requirement.

Related Information

- [Creating Base Clocks](#) on page 79
- [Deriving PLL Clocks](#) on page 85

2.3.1.3. Derive Clock Uncertainty (`derive_clock_uncertainty`)

The **Derive Clock Uncertainty** (`derive_clock_uncertainty`) constraint applies setup and hold clock uncertainty for clock-to-clock transfers in the design. This uncertainty represents characteristics like PLL jitter, clock tree jitter, and other factors of uncertainty.

You can enable the **Add clock uncertainty assignment** (`-add`) to add clock uncertainty values from any **Set Clock Uncertainty** (`set_clock_uncertainty`) constraint. You can **Overwrite existing clock uncertainty assignments** (`-overwrite`) any `set_clock_uncertainty` constraints.

```
create_clock -period 10.0 -name fpga_sys_clk [get_ports fpga_sys_clk] \
  derive_clock_uncertainty -add -overwrite
```

The Timing Analyzer generates an information message if you omit `derive_clock_uncertainty` from the `.sdc` file.

Related Information

[Accounting for Clock Effect Characteristics](#) on page 91

2.3.1.4. Set Clock Groups (set_clock_groups)

The **Set Clock Groups** (set_clock_groups) constraint allows you to specify which clocks in the design are unrelated. By default, the Timing Analyzer assumes that all clocks with a common base or parent clock are related, and that all transfers between those clock domains are valid for timing analysis. You can exclude transfers between specific clock domains from timing analysis by cutting clock groups.

Conversely, clocks without a common parent or base clock are always unrelated, but timing analysis includes the transfers between such clocks, unless those clocks are in different clock groups (or if all of their paths are cut with false path constraints).

You define the clock signals to include in each Group (-group), then specify the relationship between different groups, and then specify whether the groups are **Logically exclusive** (-logically_exclusive), **Physically exclusive** (-physically_exclusive), or **Asynchronous** (-asynchronous) from one another.

```
set_clock_groups -asynchronous -group {<clock1>...<clockn>} ... \
    -group {<clocka>...<clockn>}
```

- -logically_exclusive—defines clocks that are logically exclusive and not active at the same time, such as multiplexed clocks
- -physically_exclusive—defines clocks that cannot be physically on the device at the same time.
- -asynchronous—defines completely unrelated clocks that have different ideal clock sources. This flag means the clocks are both switching, but not in a way that can synchronously pass data.

For example, if there are paths between an 8ns clock and 10ns clock, even if the clocks are completely asynchronous, the Timing Analyzer attempts to meet a 2ns setup relationship between these clocks, unless you specify that they are not related.

The following shows example constraints for a clock mux with two inputs, with multi-rate clocks, and the appropriate combinations of logical and physical exclusivity:

```
# First profile
create_clock -name clk_a1 -period 10 [get_ports clk_a]
create_clock -name clk_b1 -period 20 [get_ports clk_b]

# Second profile
create_clock -name clk_a2 -period 100 [get_ports clk_a] -add
create_clock -name clk_b2 -period 200 [get_ports clk_b] -add

# Mark base clocks as asynchronous to each other
set_clock_groups -asynchronous -group {clk_a?} -group {clk_b?}

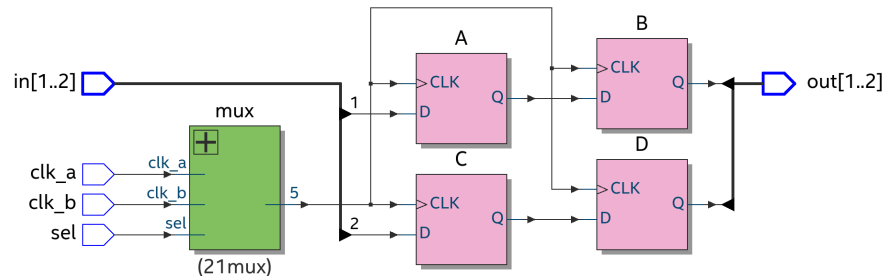
# Define muxed clocks for each profile
set muxout [get_pins -compatibility_mode {mux*|combout}]
foreach profile {1 2} {
    set mux_clk_a "mux_clk_a${profile}"
    set mux_clk_b "mux_clk_b${profile}"

    create_generated_clock -name $mux_clk_a -source [get_ports clk_a] \
        -master_clock "clk_a${profile}" $muxout -add
    create_generated_clock -name $mux_clk_b -source [get_ports clk_b] \
        -master_clock "clk_b${profile}" $muxout -add

    # Mark each muxed clock as logically exclusive to each other
    set_clock_groups -logically_exclusive -group $mux_clk_a \
        -group $mux_clk_b
}
```

```
# Mark profile clocks as physically exclusive to each other
# (Do this after defining the derived clocks so they get cut too)
set_clock_groups -physically_exclusive -group {*clk_?1} \
    -group {*clk_?2}
```

Figure 59. Example Constraints Design Topology



Although the **Set Clock Groups** dialog box only permits two clock groups, you can specify an unlimited number of `-group {<group of clocks>}` options in the `.sdc` file. If you omit an unrelated clock from the assignment, the Timing Analyzer acts conservatively and analyzes that clock in context with all other domains to which the clock connects.

The Timing Analyzer does not currently analyze crosstalk explicitly. Instead, the timing models use extra guard bands to account for any potential crosstalk-induced delays. The Timing Analyzer treats the `-asynchronous` and `-exclusive` options the same for crosstalk-related analysis, but treats asynchronous and exclusive clock groups differently for things like max skew reporting and synchronizer detection.

A clock cannot be within multiple groups (`-group`) in a single assignment; however, you can have multiple `set_clock_groups` assignments.

Another way to cut timing between clocks is to use `set_false_path`. To cut timing between `sys_clk` and `dsp_clk`, you can use:

```
set_false_path -from [get_clocks sys_clk] -to [get_clocks dsp_clk]
```

```
set_false_path -from [get_clocks dsp_clk] -to [sys_clk]
```

This technique is effective if there are only a few clocks, but can become unmanageable with a large number of constraints. In a simple design with three PLLs that have multiple outputs, the `set_clock_groups` command can cut timing between clocks with less than ten lines, but the `set_false_path` command may require more than 50 lines.

Related Information

- [Creating Generated Clocks \(create_generated_clock\)](#) on page 83
- [Relaxing Setup with Multicycle \(set_multicycle_path\)](#) on page 107
- [Accounting for a Phase Shift \(-phase\)](#) on page 109

2.3.2. Example Circuit and Conventional SDC File

The following .sdc file demonstrates constraining a dual-clock, phase-locked loop (PLL) example that illustrates. and other common synchronous design elements.

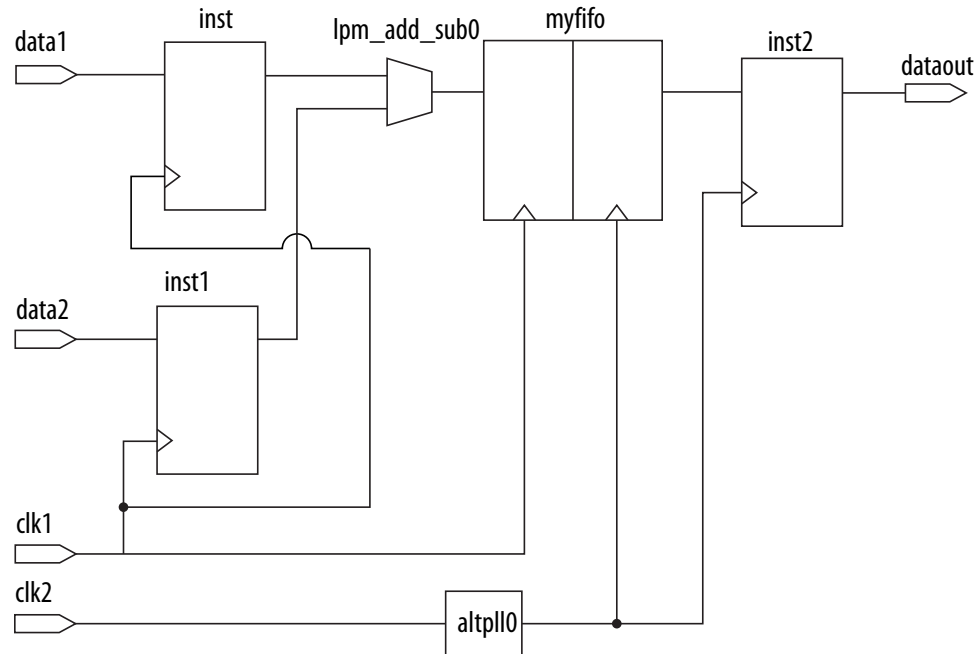
Example 2. Conventional .sdc Constraints Example

```
# Create clock constraints
create_clock -name clockone -period 10.000Ns [get_ports {clk1}]
create_clock -name clocktwo -period 10.000Ns [get_ports {clk2}]
# Create virtual clocks for input and output delay constraints
create_clock -name clockone_ext -period 10.000Ns
create_clock -name clocktwo_ext -period 10.000Ns
# derive PLL clocks to create the altp1l0| clock referenced later
derive_pll_clocks
# derive clock uncertainty
derive_clock_uncertainty
# Specify that clockone and clocktwo are unrelated by assigning
# them to separate asynchronous groups
set_clock_groups \
  -asynchronous \
  -group {clockone} \
  -group {clocktwo altp1l0|altp1l_component|auto_generated|p1l1|clk[0]}
# set input and output delays
set_input_delay -clock { clockone_ext } -max 4 [get_ports {data1}]
set_input_delay -clock { clockone_ext } -min -1 [get_ports {data1}]
set_input_delay -clock { clockone_ext } -max 4 [get_ports {data2}]
set_input_delay -clock { clockone_ext } -min -1 [get_ports {data2}]
set_output_delay -clock { clocktwo_ext } -max 6 [get_ports {dataout}]
set_output_delay -clock { clocktwo_ext } -min -3 [get_ports {dataout}]
```

The conventional .sdc file contains the following constraints that you typically include for most designs:

- Definitions of `clockone` and `clocktwo` as base clocks, and assignment of those constraints to nodes in the design.
- Definitions of `clockone_ext` and `clocktwo_ext` as virtual clocks, which represent clocks driving external devices interfacing with the FPGA.
- Automated derivation of generated clocks on PLL outputs.
- Derivation of clock uncertainty.
- Specification of two clock groups, the first containing `clockone` and its related clocks, the second containing `clocktwo` and the output of the PLL. This specification overrides the default analysis of all clocks in the design as related to each other.
- Specification of input and output delays for the design.

Figure 60. Dual-Clock Design Constraint Example



Related Information

[Asynchronous Clock Groups \(-asynchronous\)](#) on page 89

2.3.3. SDC File Precedence

To ensure proper integration into the compilation flow, you must add any SDC-on-RTL and conventional SDC files to your project, as [Step 1: Specifying General Timing Analyzer Settings](#) describes. Alternatively, you can add files to your project by modifying the assignments in the project `.qsf` file directly.

The Compiler processes conventional SDC files in the order listed in the `.qsf`. You can add, remove, or change the processing order of `.sdc` files using **Assignments > Settings > Timing Analyzer**, or by modifying the `.qsf` directly.

Note: SDC-on-RTL files take precedence and the Compiler always processes `.rtl.sdc` files before conventional `.sdc` files that target the timing netlist, regardless of order in **Assignments > Settings > Timing Analyzer**.

When using the `read_sdc` command at the command line without any arguments, the Compiler reads constraints in the following sequence:

1. Initially, the Compiler reads any SDC-on-RTL constraints.
2. Next, the Compiler reads any synthesis-only constraints that apply to only the synthesis stage.
3. Next, the Compiler reads any conventional SDC constraints. For conventional SDC constraints, the following order applies:

- a. First, the Compiler processes constraints embedded in HDL files.
- b. Finally, the Compiler processes `.sdc` files based on file order.

2.3.4. Iteratively Modifying Constraints

You initially establish SDC-on-RTL constraints during the Analysis & Elaboration stage of the compilation flow. Making iterative changes to SDC-on-RTL constraints may require you to rerun Analysis & Elaboration multiple times to apply revised constraints to the netlist.

It is best to designate constraints that remain constant across compilation stages as SDC-on-RTL constraints. Subsequently, you can iteratively modify and reanalyze the constraints in the rest of your design using conventional constraint files.

To iteratively modify constraints, follow these steps

1. Click **Tools > Timing Analyzer**.
2. Generate the reports you want to analyze. Double-click **Report All Summaries** under **Macros** to generate setup, hold, recovery, and removal summaries, summaries for supported reports, and a list of all the defined clocks in the design. These summaries cover all paths you constrain in your design. Whenever modifying or correcting constraints, generate the **Constraint Diagnostic** reports to identify unconstrained parts of your design, or ignored constraints.
3. Analyze the results in the reports. When done modifying constraints, rerun the reports to find any unexpected results. For example, a cross-domain path might indicate that you forgot to cut a transfer by including a clock in a clock group.
4. Create or edit the appropriate constraints in your `.sdc` file and save the file.
5. Double-click **Reset Design** in the **Tasks** pane. This removes all constraints from your design. Removing all constraints from your design allows rereading the SDC files, including your changes.
6. Regenerate the reports you want to analyze.
7. Reanalyze the results.
8. Repeat steps 4-7 as necessary.

Using this approach, timing analysis runs with updated constraints, preserving the existing logic placement. The Fitter relies on the original constraints for design place-and-route, while the Timing Analyzer incorporates the newly applied constraints. If any timing issues arise in relation to the updated constraints, rerun the Fitter stage of compilation. Furthermore, for enhanced control over your design, consider converting select refined constraints to the SDC-on-RTL approach, as [Specifying SDC-on-RTL Timing Constraints](#) describes.

Related Information

[Relaxing Setup with Multicycle \(set_multicycle_path\)](#) on page 107

2.3.5. Applying Entity-Bound Timing Constraints

Entity-bound timing constraints enable meticulous control of timing constraints by allowing you to confine (bind) a constraint set to a specific design entity or a group of entities. You can define entity-based SDC-on-RTL constraints that enable early timing analysis after running only Analysis & Elaboration. You can similarly use conventional SDCs for post-fit timing analysis.

Related Information

- [Using Entity-Based SDC-on-RTL Constraints](#) on page 58
- [Using Entity-Bound SDC Files](#) on page 65

2.3.5.1. Using Entity-Based SDC-on-RTL Constraints

A typical design includes a combination of third-party IPs and actively evolving RTL components. You can use entity-based SDC-on-RTL constraints to define precise timing constraints at module boundaries, helping to ensure the seamless integration of IP and constraints. Entity-based SDC-on-RTL constraints allow IP authors to encapsulate the SDC constraints for their IP.

Conventional SDC timing constraints generally apply globally throughout a design, rather than to specific entities. However, proper encapsulation of IP SDCs allows you to use the IP without encountering unexpected SDC leaks. Entity binding prefixes filters with the full path name of each IP, effectively limiting the scope of the SDC constraints. This entity binding effectively prevents any SDC leaks and any potential impact on design paths with a matching name.

IP authors can optimize these constraints for post-synthesis Early Timing Analysis within the context IP instantiation in the design hierarchy. Even if an IP author does not know where the IPs are instantiated yet, the constraints remain effective. This approach allows IP authors to implement SDC constraints without requiring detailed information about the eventual placement of the IP within the design hierarchy.

The Compiler reads entity-based SDC-on-RTL constraints in designs and IP cores during Analysis & Elaboration. The Compiler preserves the constraints in a low-level entity database. The Compiler processes these constraints in the SDC read-in order and applies the constraints to the hierarchical netlist objects during compilation.

QSF Assignment Syntax

```
set_instance_assignment -name RTL_SDC_FILE <sdc_file_name> \
    -entity <entity_name> [-no_sdc_promotion]
```

Where:

Argument	Description
RTL_SDC_FILE	Specifies the SDC-on-RTL file name.
-entity	Specifies an entity-based assignment. The SDC file applies to each instance of the design entity. The instance hierarchy path implicitly applies to the <code>pattern</code> argument search for <code>dni::get_*</code> (<code>get_cells</code> , <code>get_pins</code> , <code>get_ports</code> , and <code>get_nets</code>) commands.
[-no_sdc_promotion]	An optional argument that requires the <code>-entity</code> flag. For entity-based constraints, the <code>[-no_sdc_promotion]</code> argument removes the default behavior of the instance hierarchy path being the default implicit with the netlist search commands. This
continued...	

Argument	Description
	argument specifies which individual command to apply the instance hierarchy path in the netlist object search. Use the <code>get_entity_current_instance</code> Tcl command to obtain the current instance hierarchy path of the entity. For example: <pre>set_false_path -from [get_pins [get_entity_current_instance] ff_src clk] \ -to [get_pins [get_entity_current_instance] ff_dst d]</pre>

2.3.5.1.1. Targeting Constraints to Module Inputs and Outputs

SDC-on-RTL allows you to define constraints at module boundaries, even if some internal connections within the modules or IP remain partially unknown. It is best to apply SDC-on-RTL constraints at the module boundaries, specifically at the input and output boundaries of each module.

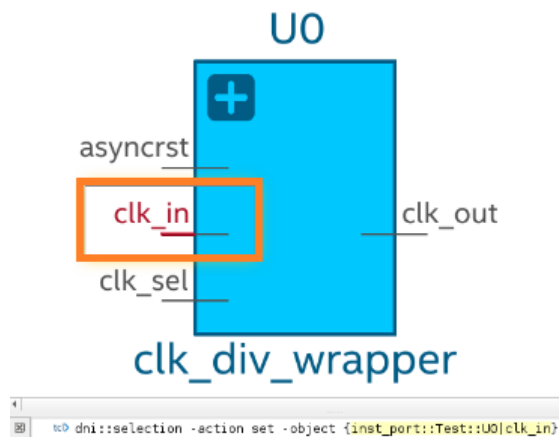
When targeting your timing constraints to the inputs and outputs of a module, you can target the following different element types, depending on your circumstances:

inst_port—these elements are retrieved in collections due to applying the `get_pins` filter. They target inputs and outputs of modules in a manner similar to addressing pins on registers and LUTs.

```
# inside
get_pins {clk_in} clk_dic.rtlcdc
```

Note: Use `get_pins` for constraints that expect pins as targets.

Figure 61. Targeting the Instance Port



2.3.5.1.2. Entity Based SDC-on-RTL Constraint Scope

The entity-based SDC-on-RTL approach offers diverse scoping possibilities for determining the constraint's scope of influence.

Table 7. Entity-based Constraint Scope

Constraint Scope Type	Features	To Enable Instance-based Scoping
Automatic	- Configurable only by .qsf assignment. - Under automatic scoping, constraints apply to every instance of the assigned entity across the project. - Each result from any get command (for example, get_pins, get_ports, and so on) in the SDC file is prepended with the instance's path. The Timing Analyzer evaluates the options of the get command in the context of the entity's instance hierarchy as the current instance. - All get commands are confined to the target elements within the bound instance associated with the SDC-on-RTL file.	Use the following arguments: <pre>name RTL_SDC_FILE <sdc_on_rtl_file_name> -entity <entity_name> -library <library_name></pre>
Manual	- Configurable only by .qsf assignment. - The -no_sdc_promotion setting disables automatic scoping, necessitating full hierarchical path for targeting nodes. - The get_entity_current_instance command returns the top-level path to the current entity's instance, allowing you to merge filters targeting elements in the current instance with commands addressing those beyond entity boundaries. - Allows targeting nodes beyond entity boundaries.	Use the following arguments: <pre>-name RTL_SDC_FILE <sdc_on_rtl_file_name> -entity <entity_name> -library <library_name> -no_sdc_promotion</pre> Prepend each collection filter with get_entity_current_instance to target nodes within the entity boundaries. For example: <pre>get_pins [get_entity_current_instance] reg[*] q</pre>

When you define entity-bound SDC files, the software applies the constraints using automatic scoping, unless the -no_sdc_promotion argument is present.

Automatic scoping involves prepending filters with the instance's path. To provide clarity, the following table illustrates how paths are interpreted in various Tcl commands due to the automatic scoping of constraints:

Table 8. Automatic Scope of Constraints

Constraint Example	Auto-Scope Constraint Interpretation for Instance X Y
set_false_path -from [get_pins reg_a clk]	set_false_path -from [get_pins X Y reg_a clk]
set_false_path -from [get_pins reg_a clk] -to [get_pins reg_b d]	set_false_path -from [get_pins X Y reg_a clk] -to [get_pins X Y reg_b d]
set_false_path -from [get_clocks clk_1] -to [get_clocks clk_2]	set_false_path -from [get_clocks clk_1] -to [get_clocks clk_2]
set_max_delay -from [get_ports in] -to [get_pins reg_a d] 2.0	set_max_delay -from [get_ports in] -to [get_pins X Y reg_a d] 2.0
get_ports *	get_ports X Y *
continued...	

Constraint Example	Auto-Scope Constraint Interpretation for Instance X Y
<i>Note:</i> In a design where you apply constraints at the entity level, <code>get_ports</code> enables you to target pins at the periphery of a module after Analysis & Elaboration. Once the timing netlist undergoes propagation and is subsequently flattened, the port pins of entities might transition to a different node type, influenced by their respective connections. This alteration can impact your originally intended scope of the entity-bound constraint definition, as the nature of the port's connectivity dictates its final node representation within the flattened hierarchy.	
<code>get_clocks *</code>	<code>get_clocks *</code>
<code>get_ports a</code>	<code>get_ports X Y a</code>
<code>get_clocks a</code>	<code>get_clocks a</code>

When automatic scoping is disabled through QSF assignments, including the use of the `-no_sdc_promotion` argument, you must manually prepend the top-level path to achieve the same behavior as automatic scoping. To simplify this process, use the `-get_entity_current_instance` command that returns the top-level path of the current instance. The following table illustrates how paths are interpreted when you use the `-get_entity_current_instance` command to add the top-level path to certain Tcl commands:

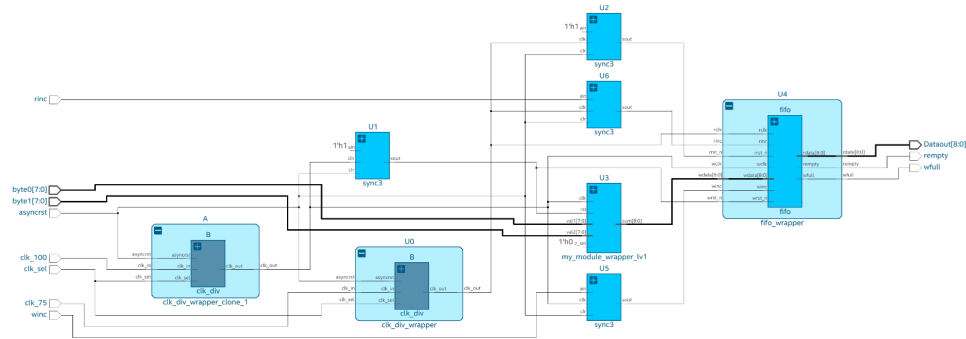
Table 9. Manual Scope of Constraints

Constraint Example	Manual Scope Constraint Interpretation
<pre>set_false_path -from [get_entity_current_instance]reg_a clk -to [get_entity_current_instance]reg_b d</pre>	<pre>set_false_path -from i1 inner reg_a clk -to i1 inner reg_b d set_false_path -from i2 inner reg_a clk -to i2 inner reg_b d set_false_path -from i3 reg_a clk -to i3 reg_b d</pre>
<pre>create_generated_clock -divide_by 2 -source \ [get_ports inclk] -name \ [get_entity_current_instance]divclk \ [get_entity_current_instance]div set_multicycle_path -from \ [get_entity_current_instance] a \ -to [get_entity_current_instance] b 2</pre>	<pre># Evaluated for instances i1 and i2 create_generated_clock -divide_by 2 -source \ [get_ports inclk] -name i1_divclk i1 div set_multicycle_path -from i1 a -to i1 b 2 \ create_generated_clock -divide_by 2 -source \ [get_ports inclk] -name i2_divclk \ i2 div set_multicycle_path -from i2 a \ -to i2 b 2 \</pre>

2.3.5.1.3. Automatic Scope Example for SDC-on-RTL

This example illustrates how to employ entity-based SDC-on-RTL constraints with automatic scope in your design. The following example uses two instances of `clk_div` and an additional `fifo` instance to illustrate how to apply this automatic scope approach:

Figure 62. Entity-Based SDC-on-RTL Design Example



1. Apply global SDC-on-RTL constraints to the design:

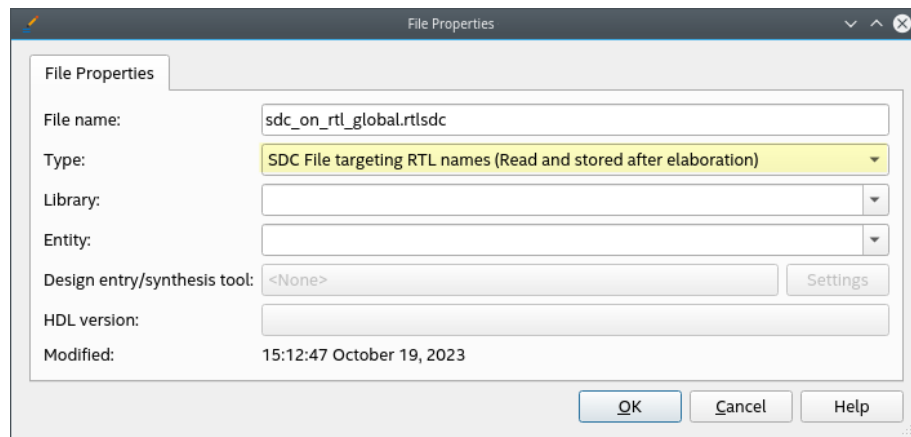
```
# sdc_on_rtl_global.rtlsdc

create_clock -period 100MHz [get_ports clk_100]
create_clock -period 75MHz [get_ports clk_75]
```

2. Use the **File Properties** dialog to assign this .rtlsdc file as the **SDC File Targeting RTL names** or use the following .qsf assignment:

```
set_global_assignment -name RTL_SDC_FILE <filename>
```

Figure 63. File Properties Dialog



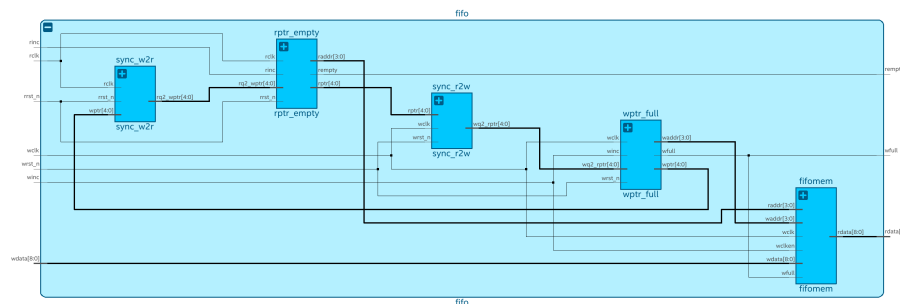
3. Define the following contents of the first entity based .rtlsdc file that is confined to the clk_div modules. In this case, the inner content of the module is unknown, but you can describe the known multiplexed clock behavior of the output in the .rtlsdc:

```
# clk_div.rtlsdc
set current_instance [get_entity_current_instance];

create_generated_clock -name ${current_instance}_clk_mux_2 -source
[get_ports clk_in] -divide_by 2 [get_ports clk_out]
create_generated_clock -name ${current_instance}_clk_mux_1 -source
[get_ports clk_in] [get_ports clk_out] -add
```

In contrast, you can still define constraints using the following entity-bound approach for the `fifo` module that for which inner logic is already known. This approach can be advantageous, especially when multiple instances of the same module share identical constraints.

Figure 64. FIFO Module



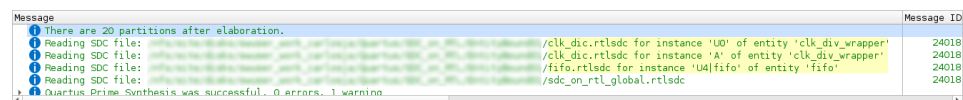
```
set_false_path -from [get_pins wptr_full|wptr[*]|clk] -to [get_pins sync_w2r|
rql_wptr[*]|d]
set_false_path -from [get_pins rptr_empty|rbin[4]|clk] -to [get_pins
sync_r2w|wql_rptr[4]|d]
set_false_path -from [get_pins rptr_empty|rptr[*]|clk] -to [get_pins
sync_r2w|wql_rptr[*]|d]
set_false_path -from [get_pins wptr_full|wbin[4]|clk] -to [get_pins sync_w2r|
rql_wptr[4]|d]
```

The following .qsf assignments add the `clk_div.rtlsdc` and `fifo.rtlsdc` files to the project, and define the file behavior as entity-based SDC-on-RTL:

```
set_global_assignment -name RTL_SDC_FILE clk_div.rtlsdc -entity clk_div_wrapper -
library clk_div_wrapper
set_global_assignment -name RTL_SDC_FILE fifo.rtlsdc -entity fifo -library fifo
```

This assignment reduces the scope of each .rtlsdc file to the entities that match the assigned name. During Analysis & Elaboration, messages confirm that the Compiler appropriately applies each .rtlsdc file according to its assigned module.

Figure 65. Analysis & Elaboration Messages Confirm .rtlsdc Files Applied



You can further validate the correct application of each .rtlsdc file in the SDC File List report. The SDC File List report contains a comprehensive list, delineating each SDC file read, the file's assigned instance, and the file uses the SDC-on-RTL approach.

Figure 66. SDC File List Report

SDC File List							
Show:	Visible	Hide	Q <<Filter>> (use !<string> to invert filter)				
	SDC File Path	Instance	Entity	Library	Promoted	Status	SDC on RTL
1	sd_on_rtl_global.rtlsdc				No	OK	Yes
2	clk_dic.rtlsdc	U0	clk_div_wrapper	clk_div_wrapper	Yes	OK	Yes
3	clk_dic.rtlsdc	A	clk_div_wrapper	clk_div_wrapper	Yes	OK	Yes
4	fifo.rtlsdc	U4 fifo	fifo	fifo	Yes	OK	Yes

You can further confirm the correct application of constraints according to the constraint target and purpose in the Create Generated Clocks and Set False Paths reports.

Figure 67. Create Generated Clock Report

Create Generated Clock							
Show:	Visible	Hide	Q <<Filter>> (use !<string> to invert filter)				
	SDC Command	Name	Source	Divide By	Master Clock	Targets	Add Clock
1	create_generated_clock	U0_clk_mux_2	[get_pins {clk_75}]	2	clk_75	[get_pins {U0 B clk_out input}]	-add
2	create_generated_clock	U0_clk_mux_1	[get_pins {clk_75}]		clk_75	[get_pins {U0 B clk_out input}]	-add
3	create_generated_clock	A_clk_mux_2	[get_pins {clk_100}]	2	clk_100	[get_pins {A B clk_out input}]	-add
4	create_generated_clock	A_clk_mux_1	[get_pins {clk_100}]		clk_100	[get_pins {A B clk_out input}]	-add

Figure 68. Set False Path Report

Set False Path				
Show:	Visible	Hide	Q <<Filter>> (use !<string> to invert filter)	
	SDC Command	From	To	Location
1	set_false_path	[get_pins {U4 fifo wptr_full wptr...k U4 fifo wptr_full wptr[2] clk}]	[get_pins {U4 fifo sync_w2 r1...4 fifo sync_w2 r1_wptr[2] d}]	fifo.rtlsdc:1
2	set_false_path	[get_pins {U4 fifo rptr_empty rbin[4] clk}]	[get_pins {U4 fifo sync_r2w wq1_rptr[4] d}]	fifo.rtlsdc:2
3	set_false_path	[get_pins {U4 fifo rptr_empty rpt... U4 fifo rptr_empty rptr[2] clk}]	[get_pins {U4 fifo sync_r2w wq1...4 fifo sync_r2w wq1_rptr[2] d}]	fifo.rtlsdc:3
4	set_false_path	[get_pins {U4 fifo wptr_full wbin[4] clk}]	[get_pins {U4 fifo sync_w2 r1...4 fifo sync_w2 r1_wptr[4] d}]	fifo.rtlsdc:4
5	set_false_path	[get_pins {clk_sel input clk_sel}]		sd_on_rtl...Lrtlsdc:8
6	set_false_path	[get_ports {asynrst}]		sd_on_rtl...Lrtlsdc:9
7	set_false_path	[get_ports {rinc}]		sd_on_rtl...Lrtlsdc:10
8	set_false_path	[get_ports {winc}]		sd_on_rtl...Lrtlsdc:11

2.3.5.1.4. Manual Scope Example for SDC-on-RTL

You can change the scope of entity-based SDC-on-RTL files to manual by including the `-no_sdc_promotion` parameter in the `RTL_SDC_FILE` file definition. This parameter prevents the Compiler from prepending each collection filter with the full path of the current instance.

For example, to change to manual the scoping of the `clk_dic.rtlsdc` file associated with the `clk_div_wrapper` entity, add the `-no_sdc_promotion` parameter as follows:

```
set_global_assignment -name RTL_SDC_FILE clk_dic.rtlsdc -entity clk_div_wrapper \
-library clk_div_wrapper -no_sdc_promotion
```

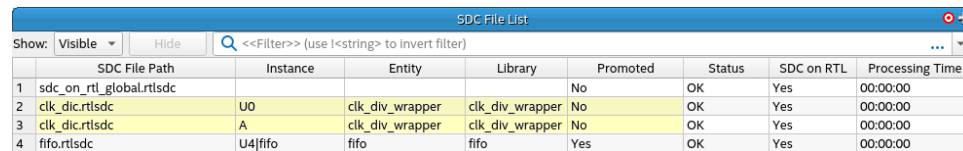
To scope your collection filters precisely, use the `get_entity_current_instance` command. Specifying the top-level path to the present instance streamlines the process of scoping filters directed towards elements within instance boundaries.

```
set current_instance [get_entity_current_instance]
create_generated_clock -name ${current_instance}_clk_mux_2 -source \
[get_ports $current_instance|clk_in] -divide_by 2 [get_ports $current_instance|
clk_out]

create_generated_clock -name ${current_instance}_clk_mux_1 -source \
[get_ports $current_instance|clk_in] [get_ports $current_instance|clk_out] -add
```

When the automatic scope is disabled for a designated entity-based SDC-on-RTL file, the SDC File List report indicates this change by displaying "No" in the promoted column.

Figure 69. SDC File List Report



	SDC File Path	Instance	Entity	Library	Promoted	Status	SDC on RTL	Processing Time
1	sdcl_on_rtl_global.rtlcdc				No	OK	Yes	00:00:00
2	clk_dlc.rtlcdc	U0	clk_dlv_wrapper	clk_dlv_wrapper	No	OK	Yes	00:00:00
3	clk_dlc.rtlcdc	A	clk_dlv_wrapper	clk_dlv_wrapper	No	OK	Yes	00:00:00
4	fifo.rtlcdc	U4lfifo	fifo	fifo	Yes	OK	Yes	00:00:00

2.3.5.2. Using Entity-Bound SDC Files

Throughout the design flow, most timing constraints specified in a Synopsys Design Constraints (SDC) file have a global scope across your project. However, if you want to associate a distinct set of constraints with a specific design entity, you can use the `SDC_ENTITY_FILE` assignment to assign SDC files to particular entity modules within your project.

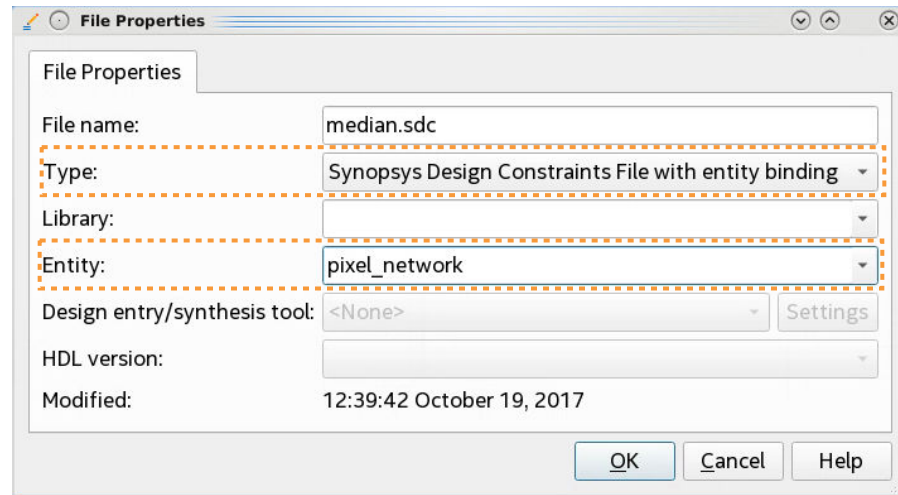
Entity-bound SDC files significantly enhance the precision of timing constraints by allowing you to target constraints exclusively to specific entities where they are required. This approach bypasses the inadvertent ramifications of global constraints, which could encompass more targets than intended. As a result, you gain greater control over the precise locations within your design where these constraints take effect.

In addition to enhanced constraint precision, entity-bound SDC files also provide portability of the constraints. When you export a partition that contains entity-bound SDC constraints, you can optionally include these constraints in the exported file using the **Include entity-bound SDC files for the selected partition** option in the **Export Design Partition** dialog box. Alternatively, you can specify this with the `include_sdc_entity_in_partition` argument via a Tcl command.

To associate a specific SDC file with an entity using the entity-bound SDC file approach within the Quartus Prime Pro Edition software, follow these steps:

1. Generate a new SDC file and include it in your project by clicking **Project ► Add/Remove files in project**.
2. Navigate to the files list and select the newly created SDC file.
3. Click the **Properties** button.
4. In the **Type** drop-down list, select **Synopsys Design Constraints File with entity binding**.

Figure 70. Entity Binding



5. In the **Entity** drop-down list, identify the entity you intend to bind to the SDC file.
6. Click **OK** to save the changes.

Alternatively, you can define the association between a specific SDC file and an entity by using the following assignment in the .qsf file:

QSF Assignment Syntax:

```
set_global_assignment -entity <entity_name> -name SDC_ENTITY_FILE
<sdc_file_name> \
-library <library_name> [-no_sdc_promotion] [-no_auto_inst_discovery]
```

Where:

Argument	Description
-entity <entity_name>	Mandatory argument. It defines the entity you want to bind to the SDC file.
-name SDC_ENTITY_FILE <sdc_file_name>	Specifies the SDC file name. The file's name is relative to the project path. In terms of scoping, the default setup for SDC_ENTITY_FILE is automatic constraint scoping. Automatic scoping means that each result from any get command (for example, get_pins, get_ports, and so on) in the SDC file is prepend with the instance's path. This confines all get commands to target items solely within the bound instance associated with the SDC file. You can deactivate this configuration (which facilitates instance-level targeting) using the -no_sdc_promotion argument available through SDC commands. With this choice, you can handle the responsibility of manually scoping your collections either by explicitly providing the top-level path to the current instance or using the get_current_instance command that delivers the top-level path to the current instance. Such an approach proves invaluable when combining local commands with those that necessitate targeting global, top-level paths or objects outside the instance associated with the SDC file.
-library <library_name>	Indicates a library for the referenced entity. If you choose not to specify a library, the Quartus Prime Pro Edition software automatically defaults to the altera_work library.
-no_sdc_promotion -no_auto_inst_discovery	Converts any entity-bound SDC into a global SDC file read just once for the entire compilation, making it particularly fitting for bundling an SDC with an entity designated for export as a qdb file. This configuration still permits SDC's collection filters to specify global top-level paths in your get commands.

2.3.5.2.1. Entity-Bound SDC Constraint Scope

The entity-bound SDC approach offers diverse scoping possibilities for your constraints, each dictating the extent of their influence.

Table 10. Entity-bound Constraint Scope

Constraint Scope Type	Features	To Enable Instance-bound Scoping
Automatic	- Default mode applied to entity-bound SDC files defined in the Quartus Prime Pro Edition GUI. - Under automatic scoping, constraints apply to every instance of the assigned entity across the project. - Each result from any <code>get</code> command (for example, <code>get_pins</code>, <code>get_ports</code>, and so on) in the SDC file is prepended with the instance's path. - All <code>get</code> commands are confined to the target elements within the bound instance associated with the SDC file.	Default mode for <code>SDC_ENTITY_FILE</code> . No additional steps required.
Manual	- Configurable only by <code>.qsf</code> assignment. - The <code>-no_sdc_promotion</code> setting disables automatic scoping, necessitating full hierarchical path for targeting nodes. - Allows the flexibility to target nodes beyond entity boundaries. - The <code>get_current_instance</code> command specifies the top-level path to the current instance, allowing you to merge filters targeting elements in the current instance with commands addressing those beyond entity boundaries.	Use <code>-no_sdc_promotion</code> . Append each collection filter with <code>get_current_instance</code> to target nodes within the entity boundaries. For example: <pre>get_registers [get_current_instance] reg[*]</pre>
Disabled	- Configurable only by <code>.qsf</code> assignment. <code>-no_sdc_promotion</code> and the <code>-no_auto_inst_discovery</code> arguments together disable scoping, treating an entity-bound SDC as a global SDC file. The SDC file is read only once for the entire compilation instead of processing repeatedly for each instance it is linked to. - This mode is ideal when bundling an SDC with an entity destined for export as a <code>qdb</code> file, while preserving the capability for SDC's collection filters to specify global, top-level paths in their <code>get</code> commands.	Use <code>-no_sdc_promotion</code> and <code>-no_auto_inst_discovery</code> arguments.

When you define entity-bound SDC files either through the GUI or via `.qsf` assignments (excluding the `-no_sdc_promotion` and `-no_auto_inst_discovery` arguments), the constraints use automatic scoping. Automatic scoping involves

prepending filters with the instance's path. To provide clarity, the following table illustrates how paths are interpreted in various Tcl commands due to the automatic scoping of constraints:

Table 11. Automatic Scope of Constraints

Constraint Example	Auto-Scope Constraint Interpretation for Instance X Y
<code>set_false_path -from [get_keepers a]</code>	<code>set_false_path -from [get_keepers X Y a]</code>
<code>set_false_path -from [get_registers a] -to [get_registers b]</code>	<code>set_false_path -from [get_registers X Y a] -to [get_registers X Y b]</code>
<code>set_false_path -from [get_clocks clk_1] -to [get_clocks clk_2]</code>	<code>set_false_path -from [get_clocks clk_1] -to [get_clocks clk_2]</code>
<code>set_max_delay -from [get_ports in] -to [get_registers A] 2.0</code>	<code>set_max_delay -from [get_ports in] -to [get_registers X Y A] 2.0</code>
<code>get_ports *</code> <i>Note: get_ports in conventional entity SDC's always refers to top level ports, and cannot be used to refer to boundary ports on an entity, as boundary ports do not exist in a flattened netlist.</i>	<code>get_ports *</code>
<code>get_clocks *</code>	<code>get_clocks *</code>
<code>get_ports a</code>	<code>get_ports a</code>
<code>get_clocks a</code>	<code>get_clocks a</code>

When you disable automatic scoping through .qsf assignments, including the use of the `-no_sdc_promotion` argument, you must manually prepend the top-level path to achieve the same behavior as automatic scoping. To simplify this process, use the `-get_current_instance` command to return the top-level path of the current instance. The following table illustrates how paths are interpreted when the `-get_current_instance` command is employed to add the top-level path to certain Tcl commands:

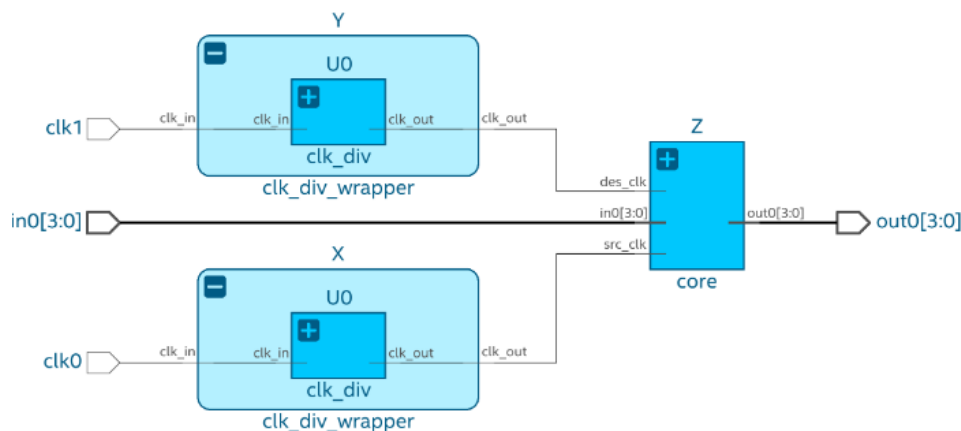
Table 12. Manual Scope of Constraints

Constraint Example	Manual Scope Constraint Interpretation
<code>set_false_path -from [get_current_instance] d\ -to [get_current_instance] e</code>	<code>set_false_path -from i1 inner d -to i1 inner e</code> <code>set_false_path -from i2 inner d -to i2 inner e</code> <code>set_false_path -from i3 d -to i3 e</code>
<code>create_generated_clock -divide_by 2 -source \ [get_ports inclk] -name \ [get_current_instance] divclk \ [get_current_instance] div</code> <code>set_multicycle_path -from [get_current_instance] a \ -to [get_current_instance] b 2</code>	<code>create_generated_clock -divide_by 2 -source \ [get_ports inclk] -name "i1_divclk" i1 div</code> <code>set_multicycle_path -from i1 a -to i1 b 2 \</code> <code>create_generated_clock -divide_by 2 -source \ [get_ports inclk] -name "i2_divclk" i2 div</code> <code>set_multicycle_path -from i2 a -to i2 b 2</code>

2.3.5.2.2. Automatic Scope Entity-bound Constraint Example

In the following design, two instances of the `clk_div` entity are constrained using the entity-bound SDC file approach. After running the Fitter's Plan stage and the timing netlist becomes available, follow the steps below to associate an SDC file with a particular entity in your design:

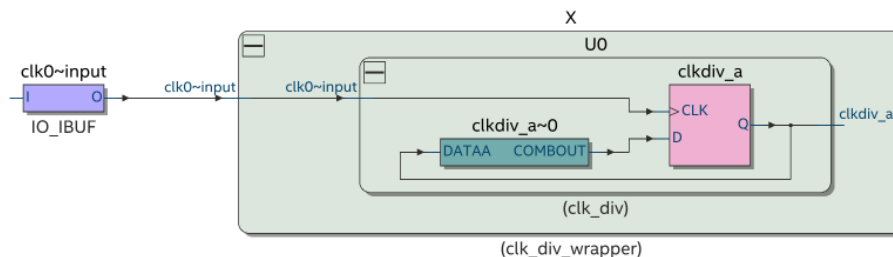
Figure 71. Automatic Scope Example



1. Create a non-entity-bound SDC file that defines constraints targeted by a global scope and add it to the project. For this design, two clocks (`clk0` and `clk1`) are defined.

```
# global.sdc
create_clock -period 100MHz -name clk_100 [get_ports clk0]
create_clock -period 75MHz -name clk_75 [get_ports clk1]
```

2. Create a second SDC file to constrain your target entity and add it to the project. This file follows the entity-bound approach and associates with the `clk_div` entity. Consequently, you define constraints as if this entity were at the top-level hierarchy, with path names relative to the entity. For example, the `get_pins clkdiv_a|q` command does not require the `X|U0` hierarchy. In this example, the `get_current_instance` Tcl command generates a unique name for each clock.



In this specific case, the `.sdc` file creates a new clock on the output of the module, and the `get_current_instance` Tcl command generates a unique name for each clock.

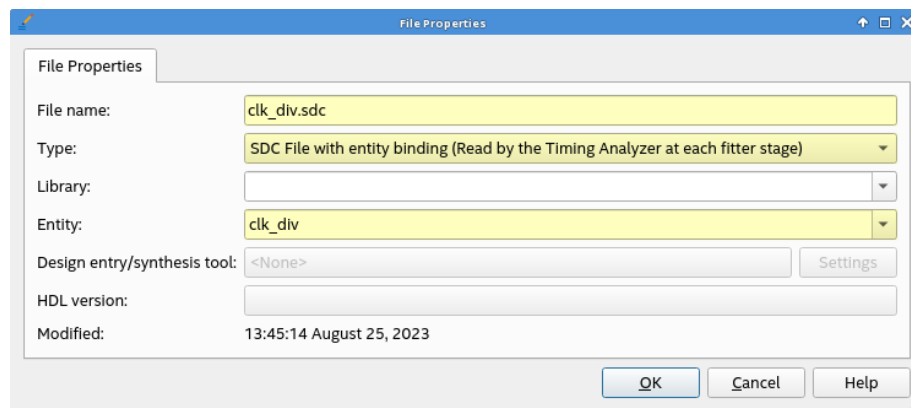
```
# clk_div.sdc
set unique_clock_name "[get_current_instance]_clkout"
create_generated_clock -divide_by 2 -source [get_pins clkdiv_a|clk] -name
$unique_clock_name [get_pins clkdiv_a|q]
```

3. Open the **File Properties** dialog for the .sdc file associated with the clk_div entity from the files list in the left-hand **Project Tasks** pane.
 - a. Select the **SDC File with entity binding (Read by the Timing Analyzer at each fitter stage)** option in the **Type** list.
 - b. In the **Entity** list, select **clk_div**.

This step applies automatic scoping to the entity-bound file, where all paths in filter commands are prepended with the top-level path of the current hierarchy. For instance, the command `get_registers clkdiv_a` is dynamically transformed into its fully hierarchical counterpart, such as `get_registers X|U0|clkdiv_a`.

Note: You must first complete Analysis & Synthesis to populate the list of entities.

Figure 72. File Properties Dialog



Alternatively, you can use the following .qsf assignment to set the entity binding:

```
set_global_assignment -name SDC_ENTITY_FILE clk_div.sdc -entity clk_div
```

4. Recompile the project to apply the changes. This results in the corresponding .sdc file being effectively bound to the entity in the automatic scope mode.
5. Verify the implementation of the entity-bound property by reviewing the **SDC File List** report in the Timing Analyzer. This report provides a comprehensive list of the applied SDC files for the design. For entity-bound SDC files, the report includes the associated instance, entity name, library, and the status of automatic scoping:

Figure 73. SDC File List Report in the Timing Analyzer

SDC File List									
Show:	Visible	Hide	Q <<Filter>> (use <string> to invert filter)						
	SDC File Path	Instance	Entity	Library	Promoted	Status	Read at	Processing Time	SDC on RTL
1	main.sdc				No	OK	Fri Aug 25 18:30:09 2023	00:00:00	No
2	clk_div.sdc	X U0	clk_div	altera_work	Yes	OK	Fri Aug 25 18:30:09 2023	00:00:00	No
3	clk_div.sdc	Y U0	clk_div	altera_work	Yes	OK	Fri Aug 25 18:30:09 2023	00:00:00	No

6. Determine the correct application of each constraint according to the intended purpose. For example, examine the generated clocks and cross-reference with the clock hierarchy to determine if the constraints are successfully applied, as shown in the following images:

Figure 74. Create Generated Clock Window in the Timing Analyzer

Create Generated Clock						
Show:	Visible	Hide	Q <<Filter>> (use !<string> to invert filter)			
	SDC Command	Name	Source	Master Clock	Targets	Location
1	create_generated_clock	X U0_clkout	[get_pins {X U0 clkdiv_a clk}]	clk_100	[get_pins {X U0 clkdiv_a q}]	clk_div.sdc:2
2	create_generated_clock	Y U0_clkout	[get_pins {Y U0 clkdiv_a clk}]	clk_75	[get_pins {Y U0 clkdiv_a q}]	clk_div.sdc:2

Figure 75. Clock Hierarchy Summary Window in the Timing Analyzer

Clock Hierarchy Summary									
	Q <<Filter>> (use !<string> to invert filter)								
	Clock Name	Type	Master	Source	Targets	Period	Frequency	Rise	Fall
1	clk_75	Base			{ clk1 }	13.333	75.0 MHz	0.000	6.666
1	Y U0_clkout	Generated	clk_75	Y U0 clkdiv_a clk	{ Y U0 clkdiv_a q }	26.666	37.5 MHz	0.000	13.333
2	clk_100	Base			{ clk0 }	10.000	100.0 MHz	0.000	5.000
1	X U0_clkout	Generated	clk_100	X U0 clkdiv_a clk	{ X U0 clkdiv_a q }	20.000	50.0 MHz	0.000	10.000

2.3.5.2.3. Manual Scope Entity-bound Constraint Example

To modify the scope of the entity-bound SDC file to manual, for example, to target elements outside the entity, utilize .qsf assignments. By including the `-no_sdc_promotion` parameter in the entity-bound SDC file definition, you can prevent the Compiler from prepending each collection filter with the full path of the current instance. For example, to change to manual scope, the .sdc file associated with the `clk_div` entity in the automatic scope example above, add the `-no_sdc_promotion` parameter as follows:

```
set_global_assignment -name SDC_ENTITY_FILE clk_div.sdc -entity clk_div -no_sdc_promotion
```

To scope your collection filters precisely, use the `get_current_instance` command. By specifying the top-level path to the present instance, `get_current_instance` streamlines the process of scoping filters directed towards elements confined within instance boundaries.

```
# clk_div.sdc
set_current_entity_instance [get_current_instance]
set_unique_clock_name "${current_entity_instance}_clkout"
create_generated_clock -divide_by 2 -source [get_pins clkdiv_a|clk] \
-name $unique_clock_name [get_pins ${current_entity_instance}|clkdiv_a|q]
```

If you disable the automatic scope for a designated entity-bound SDC file, the **SDC File List** report within the Timing Analyzer indicates this change by displaying "No" in the promoted column:

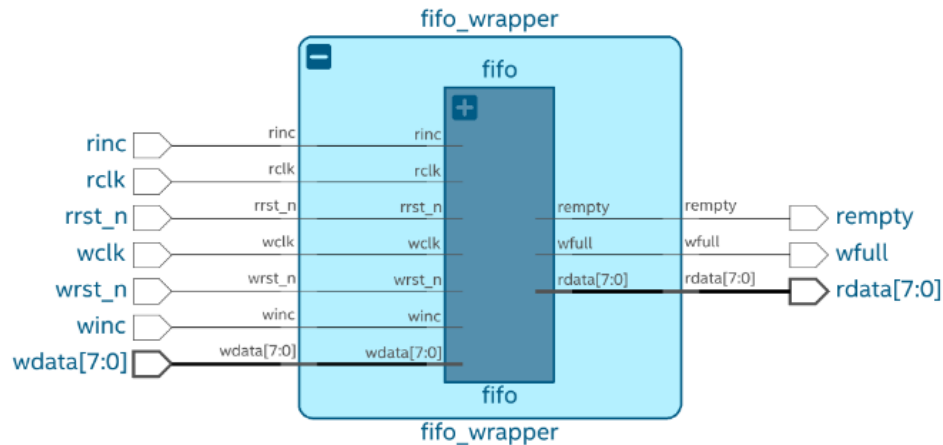
Figure 76. SDC File List Report in the Timing Analyzer

SDC File List								
Show:	Visible	Hide	Q <<Filter>> (use !<string> to invert filter)					
	SDC File Path	Instance	Entity	Library	Promoted	Status	Read at	Processing Time
1	main.sdc				No	OK	Fri Aug 25 17:40:25 2023	00:00:00
2	clk_div.sdc	X U0	clk_div	altera_work	No	OK	Fri Aug 25 17:40:25 2023	00:00:00
3	clk_div.sdc	Y U0	clk_div	altera_work	No	OK	Fri Aug 25 17:40:25 2023	00:00:00

2.3.5.2.4. Exporting a Design Partition with Entity-bound Constraints

The following example illustrates exporting a partition that includes entity bound constraints for use in another project. This example uses the `fifo` entity.

Figure 77. Entity-Bound Constraints Design Example



Perform these steps to export a design partition with entity-bound constraints:

1. After applying the constraints to the `fifo` entity, click **Fitter** on the Compilation Dashboard to run the Fitter. The Messages window report when the Fitter is complete.
2. Click **Assignments** ► **Design Partitions Window** and define **Default Type** design partition for the `fifo` entity in the **Assignments View** tab of the **Design Partition** dialog box.

Figure 78. Assignments View of the Design Partition Window

Assignments View		Compilation View			
Partition Name	Hierarchy Path	Type	Preservation Level	Empty	Partition Database File
<<new>>					
root_partition					
fifo	fifo_wrapper fifo	Default	Not Set	No	

3. Specify the entity-bound SDC **File name** and **Type** to establish the binding between the `fifo` entity and the `.sdc` file.

Figure 79. File Properties Dialog

File Properties

File name:

fifo.sdc

Type:

SDC File with entity binding (Read by the Timing Analyzer at each fitter stage)

Library:

Entity:

fifo

Design entry/synthesis tool:

<None>

Settings

HDL version:

Modified:

12:18:58 September 1, 2023

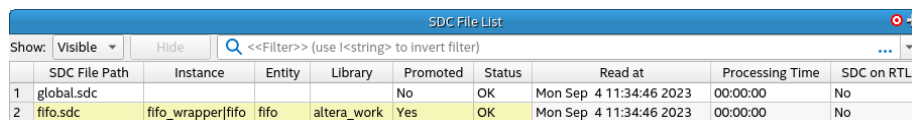
OK

Cancel

Help

4. Click **Compile Design** on the Compilation Dashboard to run a full compilation and apply the changes to your project. The Timing Analyzer opens by default following a successful full compilation.
5. Confirm the correct association of the `.sdc` file with the `fifo` entity by reviewing the **SDC File List** report in the Timing Analyzer.

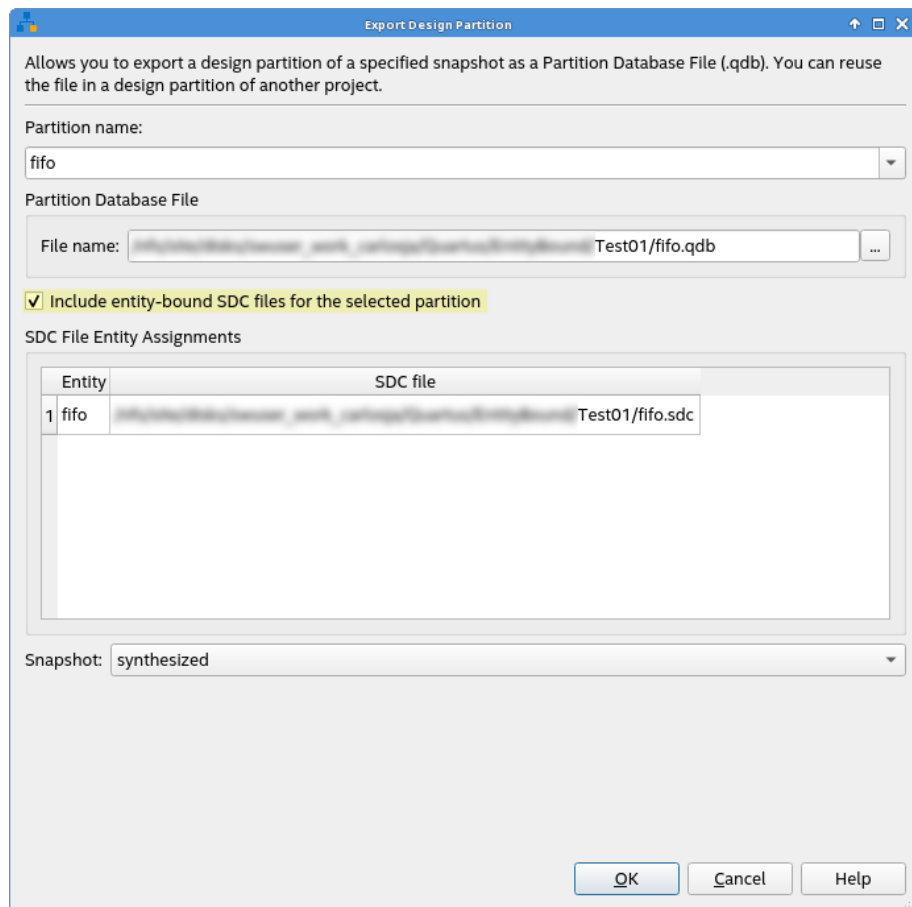
Figure 80. SDC File List Report in the Timing Analyzer



	SDC File Path	Instance	Entity	Library	Promoted	Status	Read at	Processing Time	SDC on RTL
1	global.sdc				No	OK	Mon Sep 4 11:34:46 2023	00:00:00	No
2	fifo.sdc	fifo_wrapper/fifo	fifo	altera_work	Yes	OK	Mon Sep 4 11:34:46 2023	00:00:00	No

6. Click **Project > Export Design Partition** and specify the following options:
 - a. In the **Partition name** list, select the partition to export.
 - b. In **Partition Database File**, specify the partition database file.
 - c. Turn on the **Include entity-bound SDC files for the selected partition** option for entity-bound SDC files linked to the chosen partitions.
 - d. For **Snapshot**, specify **Synthesized** or **Final**.

Figure 81. Export Design Partition Dialog



Allows you to export a design partition of a specified snapshot as a Partition Database File (.qdb). You can reuse the file in a design partition of another project.

Partition name:

Partition Database File

File name:

☒ Include entity-bound SDC files for the selected partition

SDC File Entity Assignments

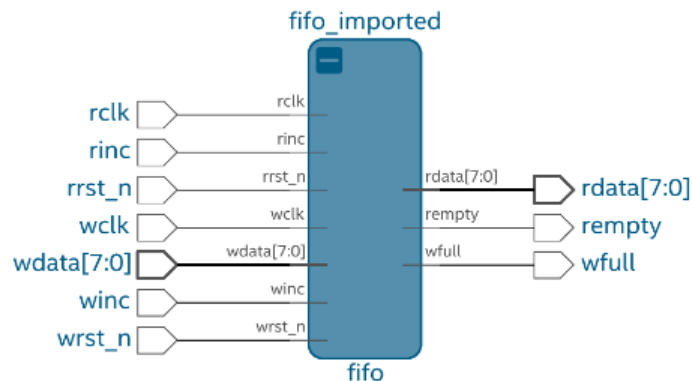
Entity	SDC file
1 fifo	Test01/fifo.sdc

Snapshot:

2.3.5.2.5. Importing a Design Partition with Entity-bound Constraints

Importing a partition with entity-bound SDC files requires that you define a black box wrapper for the entity in your design. This wrapper declares a partition in which the data from the imported partition is utilized. Consider the following example with an entity named `fifo_imported`:

Figure 82. Example of an Entity Named `fifo_imported`



To import a design partition with entity-bound constraints, perform these steps:

1. To run design synthesis, click **Analysis & Synthesis** on the Compilation Dashboard.
2. To create a new partition within the wrapper entity, click **Assignments > Design Partitions Window** and specify the following options on the **Assignments View** tab:
 - a. Specify the **Partition Name** and the **Hierarchy Path** of the entity instance.
 - b. Specify **Default** for the partition **Type**.
 - c. Specify **synthesized** in the **Preservation Level** column.
 - d. Specify the `.qdb` file from the previous project as the **Partition Database File**

Figure 83. Assignments View of the Design Partition Dialog

Assignments View		Compilation View			
Partition Name	Hierarchy Path	Type	Preservation Level	Empty	Partition Database File
<<new>>					
root_partition					
fifo_imported	fifo_imported	Default	synthesized	No	../Test01/fifo.qdb

3. To run a full compilation, click **Compile Design** on the Compilation Dashboard. The Timing Analyzer appears automatically following successful compilation.
4. Verify the correct application of the entity-bound SDC file assignment in the **SDC File List** report.

Figure 84. SDC File List Report in the Timing Analyzer

SDC File List									
Show:	Visible	Hide	Q <<Filter>> (use !<string> to invert filter)						
	SDC File Path	Instance	Entity	Library	Promoted	Status	Read at	Processing Time	SDC on RTL
1	fifo.sdc	fifo_imported	fifo	altera_work	Yes	OK	Mon Sep 4 11:57:38 2023	00:00:00	No
2	main.sdc				No	OK	Mon Sep 4 11:57:38 2023	00:00:00	No

2.3.6. Constraining Design Partition Ports

You can assign clock definitions and SDC exceptions to design partition ports. The block-based design and partial reconfiguration design flows require the use of design partitions.

Note: Beginning in Quartus Prime Pro Edition software version 24.3, support for the `blackbox` argument is removed. You could previously use the `blackbox` to perform delay budgeting on I/Os at the boundary of a design partition. Refer to [Deprecation of the blackbox Argument](#).

The Compiler represents design partition ports in your timing netlist as combinational nodes with persistent names that the Compiler cannot optimize away. You can safely refer to these ports as clock sources or `-through` points in SDC constraints. You can also use design partition port names as `-to` and `-from` points in the `report_path` command.

If a port on `partition_a` has the name `clk_divide`, then the SDC constraint is:

```
create_generated_clock -source clock -divide_by 2 \
    top|partition_a|clk_divide
```

If a set of ports on `partition_b` has the name `data_input[0..7]`, then the SDC constraint is:

```
set_multicycle_path -from top|partition_a|data_reg* \
    -through top|partition_b|data_input* 2
```

You can use multiple `-through` clauses. This technique allows you to specify paths that go through output ports of one design partition, and then through the input ports of another, downstream design partition.

To add constraints to partition ports:

1. Run Analysis & Synthesis or run full compilation on a design containing design partitions.
2. To open the RTL Viewer and locate the partition ports of interest, click **Tools > Netlist Viewers > RTL Viewer**.
3. Using the same names as the **RTL Viewer**, add clock and other SDC constraints to the `.sdc` file for your project. You can use wildcards to refer to more than one port.
4. Recompile the design to apply the new definitions and constraints.

Aside from block-based and PR flows, this technique also aids in emulation of ASICs using FPGAs. In this type of design, clock networks often span multiple hierarchies of partitions. Typically, designers remove the clock-dividing circuitry

from the netlist, since they cannot easily emulate this circuitry on Altera FPGAs. For such clock networks, this technique allows you to define different versions of the clock signal in places where the circuitry is removed.

You must design and place your partitions strategically, and then define the appropriate ports on these partitions. Ensure that your ports and partitions coincide with the part of the clock network which contains the special circuitry. You can manually edit the emulated ASIC netlist to annotate appropriate clock definitions and clock relationships. You can also use this technique in any projects where arbitrary locations on paths require constrained timing or defined clock sources.

Related Information

- [Output Constraints \(set_output_delay\)](#) on page 97
- [Input Constraints \(set_input_delay\)](#) on page 96
- [Deprecation of the -blackbox Argument](#) on page 95

2.3.6.1. Timing Analysis of Imported Compilation Results

You can preserve the compilation results for your design as a version-compatible Quartus database file (.qdb) that you can open in a later version of the Quartus Prime software without compatibility issues.

When you import and open the .qdb in a later version of software, you can run timing analysis on the imported compilation results without re-running the Compiler.

2.3.7. Using Fitter Overconstraints

Fitter overconstraints are timing constraints that you adjust to overcome modeling inaccuracies, mis-correlation, or other deficiencies in logic optimization. You can overconstrain setup and hold paths in the Fitter to enable more aggressive timing optimization of specific paths.

One typically writes overconstraints that apply only during the Fitter, with a conditional statement that checks the name of the executable that is reading the .sdc file. The following example shows a conditional statement that applies an overconstraint during the Fitter.

Note:

Fitter overconstraints generally require Fitter targets. For this reason, you use conventional SDCs (non SDC-on-RTL) for specification. SDC-on-RTL constraints do not support targeting specific Compiler stages.

```
# Example Fitter overconstraint targeting specific nodes
if { $::TimingAnalyzerInfo(nameofexecutable) eq "quartus_fit" } {
    set_max_delay -from ${my_src_regs} -to ${my_dst_regs} 1ns
}
```

One typically applies overconstraints because a particular path (or set of paths) requires more than the default optimization effort from the Fitter. Therefore, overconstraints typically specify particular node or register names in the path or paths that require extra optimization.

Other Overconstraint Combinations

You can apply timing constraints to specific stages of the Compiler, in addition to the Fitter as described above. The following table shows different combinations of compile stages and the conditional expression that applies those constraints during the stage or stages.

Table 13. Overconstraint Combinations

Compile Stages	Tcl Condition	Notes
Fitter	<code>if { \$::TimingAnalyzerInfo(nameofexecutable) eq "quartus_fit" }</code>	Applies constraints during entire Fitter stage. Use for fitter overconstraints with devices that do not support Hyperflex [®] architecture.
Signoff timing	<code>if { \$::TimingAnalyzerInfo(nameofexecutable) eq "quartus_sta" }</code>	Applies constraints during timing analysis. This condition is uncommon. Typically you apply timing analysis constraints during the entire compile flow. If you apply overconstraints with an expression conditioned on the name of an executable, ensure that is the only applicable executable where the constraints should apply. For example, to apply during signoff timing and power analysis, check the executable name against <code>quartus_sta</code> and <code>quartus_pow</code> .
Plan, Place, and Route	<code>if { ! [is_post_route] }</code>	Applies constraints during the Plan, Place, and Route stages in devices that support the Hyperflex architecture. Use for fitter overconstraints.
Retime (HyperRetimer), Finalize, Signoff timing	<code>if { [is_post_route] }</code>	Applies constraints after the Route stage completes in devices that support the Hyperflex architecture. This combination of stages is uncommon. Typically you apply timing analysis constraints during the entire compile flow.
Retime (HyperRetimer) and Finalize	<code>if { [is_post_route] && \$::TimingAnalyzerInfo(nameofexecutable) eq "quartus_fit" }</code>	Applies constraints during Retiming and Finalize stages in devices that support the Hyperflex architecture. This combination of stages is uncommon.

Overconstraints for Hyperflex Architecture Designs

For devices that support the Hyperflex architecture, such as Stratix 10 and Agilex[™] FPGA portfolio devices, timing constraints that apply to particular node or register names prevent the Hyper-Retimer from optimizing paths containing those nodes or registers.

Because the Hyper-Retimer is component of the Fitter, an overconstraint that conditionally applies during the Fitter is counter-productive during the Hyper-Retimer portion of the Fitter. The overconstraint actually prevents the optimization instead of focusing the optimization effectively.

When designing for the Hyperflex architecture, use the `is_post_route` Tcl function form of conditional statement to apply the overconstraint during placement and routing but not during the Hyper-Retimer. `is_post_route` allows you to apply overconstraints and adjust slack for stages of the Fitter, such as Plan, Place, and Route. `is_post_route` also allows post-route retiming via the Hyper-Retimer without affecting sign-off timing analysis.

```
# Example Fitter overconstraint targeting specific nodes (allows for post-route retiming)
if { ! [is_post_route] } {
    set_max_delay -from ${my_src_regs} -to ${my_dst_regs} 1ns
}
```

Overconstraints for Designs that Target All Other Device Families

You can assign Fitter overconstraints that check the name of the current executable, (either `quartus_fit` or `quartus_sta`) to apply different constraints for Fitter optimization and sign-off timing analysis.

```
set fit_flow 0
if { $::TimingAnalyzerInfo(nameofexecutable) eq "quartus_fit" } {
    set fit_flow 1
}
if {$fit_flow} {
    # Example Fitter overconstraint targeting specific nodes (restricts retiming)
    set_max_delay -from ${my_src_regs} -to ${my_dst_regs} 1ns
}
```

2.4. Timing Constraint Descriptions

This section provides examples and describes how to correctly apply SDC timing constraints that guide design synthesis, Fitter placement, and produce accurate timing analysis results.

You can define a set of initial timing constraints, and then iteratively modify those constraints as the design progresses.

Early in the design cycle, you can use SDC-on-RTL constraints to target analysis of RTL nodes. This analysis provides a stable reference for constraints that can remain unchanged in subsequent compilation stages, such as clock definitions. Establishing a set of SDC-on-RTL constraints enables their propagation and application throughout the entire design cycle. Concurrently, you can create a conventional `.sdc` file for analysis of the remaining design elements, providing flexibility for iterative constraint adjustments as the design evolves.

This section also outlines the proper application of recommended conventional SDC timing constraints. Conventional SDC constraints guide Fitter placement via `.sdc` files, offering alternative approaches to achieve precise control over constraints throughout the design flow.

2.4.1. Clock Constraints

You must define all clocks and any associated clock characteristics, such as uncertainty, latency or skew. The Timing Analyzer supports `.sdc` commands that accommodate various clocking schemes, such as:

- Base clocks
- Virtual clocks
- Multifrequency clocks
- Generated clocks
- Multiplexed clocks

You can verify correct implementation of clock constraints by using **Report Clocks** (`report_clocks`) to generate clock timing reports. You can use **Check Timing** (`check_timing`) to report problems with a variety of timing constraints, such as missing clocks.

If you have multiplexed clocks in your design, ensure you specify appropriate clock groups for the output clocks from the multiplexer, as the examples show in [Exclusive Clock Groups](#).

Related Information

- [Report Clocks](#) on page 141
- [Check Timing](#) on page 170

2.4.1.1. Creating Base Clocks

Base clocks are the primary input clocks to the device. The **Create Clock** (`create_clock`) constraint allows you to define the properties and requirements for base clocks in the design. Unlike clocks that are generated in the device (such as an on-chip PLL), base clocks are generated by off-chip oscillators or forwarded from an external device. Define base clocks at the top of your `.sdc` file, because generated clocks and other constraints often reference base clocks. The Timing Analyzer ignores any constraints that reference an undefined clock.

The following examples show common use of the `create_clock` constraint:

`create_clock` Command

The following specifies a 100 MHz requirement on a `clk_sys` input clock port:

```
create_clock -period 100Mhz -name clk_sys [get_ports clk_sys]
```

100 MHz Shifted by 90 Degrees Clock Creation

The following creates a 10 ns clock, with a 50% duty cycle, that is phase shifted by 90 degrees, and applies to port `clk_sys`. This type of clock definition commonly refers to source synchronous, double-rate data that is center-aligned with respect to the clock.

```
create_clock -period 10ns -waveform { 2.5 7.5 } [get_ports clk_sys]
```

Two Oscillators Driving the Same Clock Port

You can apply multiple clocks to the same target with the `-add` option. For example, to specify that you can drive the same clock input at two different frequencies, enter the following commands in your `.sdc` file:

```
create_clock -period 10ns -name clk_100 [get_ports clk_sys]
create_clock -period 5ns -name clk_200 [get_ports clk_sys] -add
```

Although uncommon to define more than two base clocks for a port, you can define as many as are appropriate for your design, making sure you specify `-add` for all clocks after the first.

Creating Multifrequency Clocks

You must create a multifrequency clock if your design has more than one clock source feeding a single clock node. The additional clock may act as a low-power clock, with a lower frequency than the primary clock. If your design uses multifrequency clocks, use the `set_clock_groups` command to define clocks that are physically exclusive (that is, clocks that are not physically present at the same time).

Use the `create_clock` command with the `-add` option to create multiple clocks on a clock node. You can create a 10 ns clock applied to clock port `clk`, and then add an additional 15 ns clock to the same clock port. The Timing Analyzer analyzes both clocks.

```
create_clock -period 10ns -name clock_primary -waveform { 0 5 } \
[get_ports clk]
create_clock -period 15ns -name clock_secondary -waveform { 0 7.5 } \
[get_ports clk] -add
```

You can verify correct implementation of clock constraints by using **Report Clocks** (`report_clocks`) to generate clock timing reports. You can use **Check Timing** (`check_timing`) to report problems with a variety of timing constraints, such as missing clocks.

Related Information

- [Accounting for Clock Effect Characteristics](#) on page 91
- [Report Clocks](#) on page 141
- [Check Timing](#) on page 170

2.4.1.1.1. Automatic Clock Detection and Constraint Creation

Use the `derive_clocks` command to automatically create base clocks in your design. The `derive_clocks` command is equivalent to using the `create_clock` command for each register or port feeding the clock pin of a register. The `derive_clocks` command creates clock constraints on ports or registers to ensure every register in your design has a clock constraints, and it applies one period to all base clocks in your design.

The following command specifies a base clock with a 100 MHz requirement for unconstrained base clock nodes.

```
derive_clocks -period 10
```

The `derive_clocks` command names the automatically created base clocks according to the name of the register or port that is the target of each clock. Automatically derived clocks have the suffix "`~derived`". You can choose another suffix to append with the `-suffix` option for the `derive_clocks` command.

Caution: If your design has more than a single clock, the `derive_clocks` command constrains all the clocks to the same specified frequency. To achieve a realistic analysis of your design's timing requirements, do not use `derive_clocks` command for final timing sign-off. Instead, use `create_clock` and `create_generated_clock` commands to make individual clock constraints for all clocks in your design.

If you want to create some base clocks automatically, use the `-create_base_clocks` option to `derive_pll_clocks`. With this option, the `derive_pll_clocks` command automatically creates base clocks for each PLL, based on the input frequency information that you specify when you generate the PLL. This feature works for simple port-to-PLL connections. Base clocks do not automatically generate for complex PLL connectivity, such as cascaded PLLs. You can also use the command `derive_pll_clocks -create_base_clocks` to create the input clocks for all PLL inputs automatically.

2.4.1.2. Creating Virtual Clocks

A virtual clock is a clock without a real source in the design, or a clock that does not interact directly with the design. You can use virtual clocks in I/O constraints to represent clocks that drive external devices connected to the FPGA.

To create virtual clocks, use the `create_clock` constraint with no value for the `<targets>` option.

This following example defines a 100 MHz virtual clock because the command includes no `<targets>`.

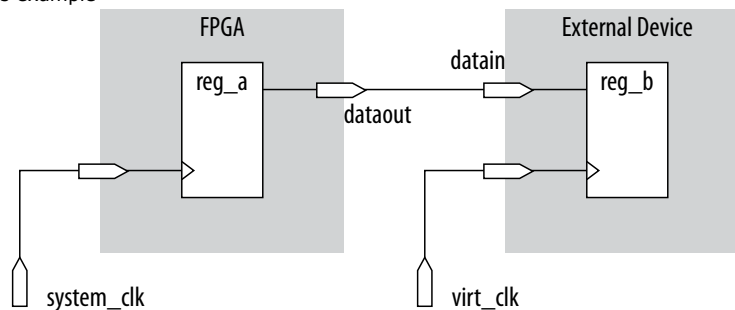
```
create_clock -period 10 -name my_virt_clk
```

I/O Constraints with Virtual Clocks

You can use a base clock to constrain the circuit in the FPGA and a virtual clock to represent the clock driving the external device.

Figure 85. Virtual Clock Board Topology

The figure shows the base clock (`system_clk`), virtual clock (`virt_clk`), and output delay for the virtual clock constraints example



The following example creates the 10 ns `virt_clk` virtual clock, with a 50% duty cycle, with the first rising edge occurring at 0 ns. The virtual clock can then become the clock source for an output delay constraint.

Example 3. Virtual Clock Constraints

```
#create base clock for the design
create_clock -period 5 [get_ports system_clk]
#create the virtual clock for the external register
create_clock -period 10 -name virt_clk
#set the output delay referencing the virtual clock
set_output_delay -clock virt_clk -max 1.5 [get_ports dataout]
set_output_delay -clock virt_clk -min 0.0 [get_ports dataout]
```

You can verify correct implementation of clock constraints by using **Report Clocks** (`report_clocks`) to generate clock timing reports. You can use **Check Timing** (`check_timing`) to report problems with a variety of timing constraints, such as the number of unreferenced virtual clocks without constraint.

Related Information

- [Report Clocks](#) on page 141
- [Check Timing](#) on page 170

2.4.1.2.1. Specifying I/O Interface Uncertainty

Virtual clocks are recommended for I/O constraints because they most accurately represent the clocking topology of the design. An additional benefit is that you can specify different uncertainty values on clocks that interface with external I/O ports and clocks that feed register-to-register paths inside the FPGA.

2.4.1.2.2. I/O Interface Clock Uncertainty Example

To specify I/O interface uncertainty, you must create a virtual clock and constrain the input and output ports with the `set_input_delay` and `set_output_delay` commands that reference the virtual clock.

When the `set_input_delay` or `set_output_delay` commands reference a clock port or PLL output, the virtual clock allows the `derive_clock_uncertainty` command to apply separate clock uncertainties for internal clock transfers and I/O interface clock transfers.

Create the virtual clock with the same properties as the original clock that is driving the I/O port, as the following example shows:

Example 4. SDC Commands to Constrain the I/O Interface

```
# Create the base clock for the clock port
create_clock -period 10 -name clk_in [get_ports clk_in]
# Create a virtual clock with the same properties of the base clock
# driving the source register
create_clock -period 10 -name virt_clk_in
# Create the input delay referencing the virtual clock and not the base
# clock
# DO NOT use set_input_delay -clock clk_in <delay value>
# [get_ports data_in]
set_input_delay -clock virt_clk_in <delay value> [get_ports data_in]
```

2.4.1.3. Creating Generated Clocks (create_generated_clock)

The **Create Generate Clock** (`create_generated_clock`) constraint allows you to define the properties and constraints of an internally generated clock in the design. You specify the **Clock name** (`-name`), the **Source** node (`-source`) from which clock derives, and the **Relationship to the source** properties. Define generated clocks for any node that modifies the properties of a clock signal, including modifying the phase, frequency, offset, or duty cycle.

You apply generated clocks most commonly on the outputs of PLLs, on register clock dividers, clock muxes, and clocks forwarded to other devices from an FPGA output port, such as source synchronous and memory interfaces. In the `.sdc` file, enter generated clocks after the base clocks definitions. Generated clocks automatically account for all clock delays and clock latency to the generated clock target.

The `-source` option specifies the name of a node in the clock path that you use as reference for your generated clock. The source of the generated clock must be a node in your design netlist, and not the name of a clock you previously define. You can use any node name on the clock path between the input clock pin of the target of the generated clock and the target node of its reference clock as the source node.

Specify the input clock pin of the target node as the source of your new generated clock. By accepting a node as the generated clock's source clock, the generated clock constraint decouples from the source clock. If you change the source clock for the generated clock and the source node is the same, you do not have to edit the generated clock constraint.

If you have multiple base clocks feeding a node that is the source for a generated clock, you must define multiple generated clocks. You associate each generated clock with one base clock using the `-master_clock` option in each generated clock statement. In some cases, generated clocks generate with combinational logic.

Depending on how your clock-modifying logic synthesizes, the source or target node can change from one compilation to the next. If the name changes after you write the generated clock constraint, the Compiler ignores the generated clock because that target name no longer exists in the design. To avoid this problem, use a synthesis attribute or synthesis assignment to retain the final combinational node name of the clock-modifying logic. Then use the kept name in your generated clock constraint.

Figure 86. Example of clock-as-data

Setup: clk

Command Info		Summary of Paths						
	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	9.166	toggle_reg q	toggle_reg	toggle_clk	clk	10.000	-0.158	0.593
2	9.171	toggle_reg q	toggle_reg	toggle_clk	clk	10.000	-0.158	0.588

Path #1: Setup slack is 9.166

Path Summary		Statistics	Data Path	Waveform	Extra Fitter Information			
	Property	Value						
1	From Node	toggle_reg q						
2	To Node	toggle_reg						
3	Launch Clock	toggle_clk (INVERTED)						
4	Latch Clock	clk						
5	Data Arrival Time	12.515						
6	Data Required Time	21.681						
7	Slack	9.166						

When you create a generated clock on a node that ultimately feeds the data input of a register, this creates a special case of "clock-as-data." The Timing Analyzer treats clock-as-data differently. For example, if you use clock-as-data with DDR, you must consider both the rise and the fall of this clock, and the Timing Analyzer reports both rise and fall. With clock-as-data, the Compiler treats the **From Node** as the target of the generated clock, and the **Launch Clock** as the generated clock.

In [Example of clock-as-data](#), the first path is from `toggle_clk (INVERTED)` to `clk`, and the second path is from `toggle_clk` to `clk`. The slack in both cases is slightly different due to the difference in rise and fall times along the path. The **Data Delay** column reports the ~5 ps difference. Only the path with the lowest slack value requires consideration. The Timing Analyzer only reports the worst-case path between the two (rise and fall). In this example, if you do not define the generated clock on the register output, then timing analysis reports only one path with the lowest slack value.

You can use the `derive_pll_clocks` command to automatically generate clocks for all PLL clock outputs. The properties of the generated clocks on the PLL outputs match the properties you define for the PLL.

You can verify correct implementation of clock constraints by using **Report Clocks** (`report_clocks`) to generate clock timing reports. You can use **Check Timing** (`check_timing`) to report problems with a variety of timing constraints, such as the number of generated clocks that are invalid.

Related Information

- [Deriving PLL Clocks](#) on page 85
- [Report Clocks](#) on page 141
- [Check Timing](#) on page 170
- [create_generated_clock](#)
- [derive_pll_clocks](#)

2.4.1.3.1. Clock Divider Example (-divide_by)

A common form of generated clock is the divide-by-two register clock divider. The following example constraint creates a half-rate clock on the divide-by-two register.

```
create_clock -period 10ns -name clk_sys [get_ports clk_sys]
create_generated_clock -name clk_div_2 -divide_by 2 -source \
    [get_ports clk_sys] [get_pins reg|q]
```

To specify the clock pin of the register as the clock source:

```
create_clock -period 10ns -name clk_sys [get_ports clk_sys]
create_generated_clock -name clk_div_2 -divide_by 2 -source \
    [get_pins reg|clk] [get_pins reg|q]
```


Figure 87. Clock Divider

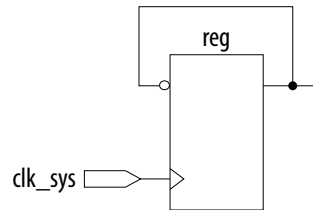
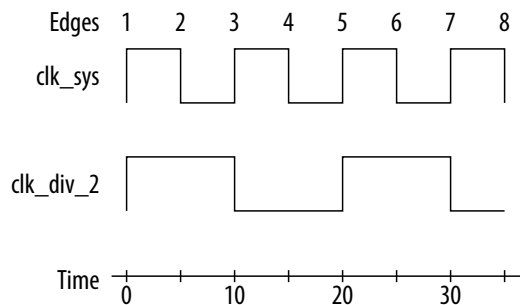


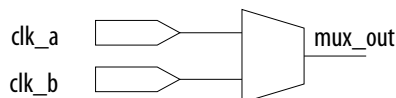
Figure 88. Clock Divider Waveform



2.4.1.3.2. Clock Multiplexer Example

The output of a clock multiplexer (mux) is a form of generated clock. Each input clock requires one generated clock on the output. The following .sdc example also includes the `set_clock_groups` command to indicate that the two generated clocks can never be active simultaneously in the design. Therefore, the Timing Analyzer does not analyze cross-domain paths between the generated clocks on the output of the clock mux.

Figure 89. Clock Mux



```
create_clock -name clock_a -period 10 [get_ports clk_a]
create_clock -name clock_b -period 10 [get_ports clk_b]
create_generated_clock -name clock_a_mux -source [get_ports clk_a] \
[get_pins clk_mux|mux_out]
create_generated_clock -name clock_b_mux -source [get_ports clk_b] \
[get_pins clk_mux|mux_out] -add
set_clock_groups -logically_exclusive -group clock_a_mux -group clock_b_mux
```

2.4.1.4. Deriving PLL Clocks

The **Derive PLL Clocks** (`derive_pll_clocks`) constraint automatically creates clocks for each output of any PLL in your design. `derive_pll_clocks` detects your current PLL settings and automatically creates generated clocks on the outputs of every PLL by calling the `create_generated_clock` command.

Note: Only Arria 10 and Cyclone 10 GX devices support the **Derive PLL Clocks** (`derive_pll_clocks`) constraint. For all other supported devices, the Timing Analyzer automatically derives PLL clocks from constraints bound to the related IP.

Create Base Clock for PLL Input Clock Ports

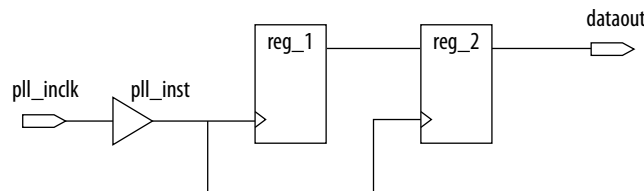
If your design contains transceivers, LVDS transmitters, or LVDS receivers, use the `derive_pll_clocks` to constrain this logic in your design and create timing exceptions for those blocks.

```
create_clock -period 10.0 -name fpga_sys_clk [get_ports fpga_sys_clk]
derive_pll_clocks
```

Include the `derive_pll_clocks` command in your `.sdc` file after any `create_clock` command. Each time the Timing Analyzer reads the `.sdc` file, the appropriate generated clock is created for each PLL output clock pin. If a clock exists on a PLL output before running `derive_pll_clocks`, the pre-existing clock has precedence, and an auto-generated clock is not created for that PLL output.

The following shows a simple PLL design with a register-to-register path:

Figure 90. Simple PLL Design



The Timing Analyzer generates messages like the following example when you use the `derive_pll_clocks` command to constrain the PLL.

Example 5. derive_pll_clocks Command Messages

```
Info:
Info: Deriving PLL Clocks:
Info: create_generated_clock -source pll_inst|altpll_component|pll|inclk[0] -
      divide_by 2 -name
      pll_inst|altpll_component|pll|clk[0] pll_inst|altpll_component|pll|clk[0]
Info:
```

The input clock pin of the PLL is the node `pll_inst|altpll_component|pll|inclk[0]` which is the `-source` option. The name of the output clock of the PLL is the PLL output clock node, `pll_inst|altpll_component|pll|clk[0]`.

If the PLL is in clock switchover mode, multiple clocks generate for the output clock of the PLL; one for the primary input clock (for example, `inclk[0]`), and one for the secondary input clock (for example, `inclk[1]`). Create exclusive clock groups for the primary and secondary output clocks since they are not active simultaneously.

You can verify correct implementation of clock constraints by using **Report Clocks** (`report_clocks`) to generate clock timing reports. You can use **Check Timing** (`check_timing`) to report problems with a variety of timing constraints, such as the number of instances where clocks that are assigned to a PLL do not correspond properly with the PLL settings you define in design files.

Related Information

- [Creating Clock Groups \(set_clock_groups\)](#) on page 87

- [Report Clocks](#) on page 141
- [Check Timing](#) on page 170

2.4.1.5. Creating Clock Groups (`set_clock_groups`)

The **Set Clock Groups** (`set_clock_groups`) constraint allows you to specify which clocks in the design are unrelated.

The `set_clock_groups` command allows you to cut timing between unrelated clocks in different groups. The Timing Analyzer performs the same analysis regardless of whether you specify `-exclusive` or `-asynchronous` groups. You define a clock group with the `-group` option. The Timing Analyzer excludes the timing paths between clocks for each of the separate groups.

The following tables show the impact of `set_clock_groups`.

Table 14. `set_clock_groups -group A`

Destination\Source	A	B	C	D
A	Analyzed	Cut	Cut	Cut
B	Cut	Analyzed	Analyzed	Analyzed
C	Cut	Analyzed	Analyzed	Analyzed
D	Cut	Analyzed	Analyzed	Analyzed

Table 15. `set_clock_groups -group {A B}`

Destination\Source	A	B	C	D
A	Analyzed	Analyzed	Cut	Cut
B	Analyzed	Analyzed	Cut	Cut
C	Cut	Cut	Analyzed	Analyzed
D	Cut	Cut	Analyzed	Analyzed

Table 16. `set_clock_groups -group A -group B`

Destination\Source	A	B	C	D
A	Analyzed	Cut	Analyzed	Analyzed
B	Cut	Analyzed	Analyzed	Analyzed
C	Analyzed	Analyzed	Analyzed	Analyzed
D	Analyzed	Analyzed	Analyzed	Analyzed

Table 17. `set_clock_groups -group {A C} -group {B D}`

Destination\Source	A	B	C	D
A	Analyzed	Cut	Analyzed	Cut
B	Cut	Analyzed	Cut	Analyzed
C	Analyzed	Cut	Analyzed	Cut
D	Cut	Analyzed	Cut	Analyzed

Table 18. `set_clock_groups -group {A C D}`

Destination\Source	A	B	C	D
A	Analyzed	Cut	Analyzed	Analyzed
B	Cut	Analyzed	Cut	Cut
C	Analyzed	Cut	Analyzed	Analyzed
D	Analyzed	Cut	Analyzed	Analyzed

You can verify correct implementation of clock constraints by using **Report Clocks** (`report_clocks`) to generate clock timing reports. You can use **Check Timing** (`check_timing`) to report problems with a variety of timing constraints, such as missing clocks.

Related Information

- [Timing Exception Precedence](#) on page 102
- [Report Clocks](#) on page 141
- [Check Timing](#) on page 170
- [set_clock_groups Command, Quartus Prime Help](#)

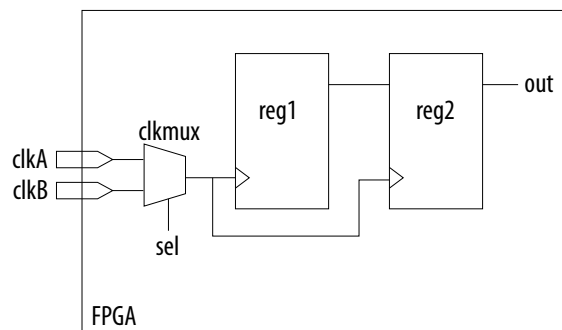
2.4.1.5.1. Exclusive Clock Groups (`-logically_exclusive` or `-physically_exclusive`)

You can use the `logically_exclusive` option to declare that two clocks are physically active simultaneously, but the two clocks are not actively used at the same time (that is, the clocks are logically mutually exclusive). The `physically_exclusive` option declares clocks that cannot be physically on the device at the same time.

If you define multiple clocks for the same node, you can use clock group assignments with the `logically_exclusive` option to declare clocks as mutually exclusive. This technique can be useful for multiplexed clocks.

For example, consider an input port that is clocked by either a 100-MHz or 125-MHz clock. You can use the `logically_exclusive` option to declare that the clocks are mutually exclusive and eliminate clock transfers between the 100-MHz and 125-MHz clocks, as the following diagrams and example SDC constraints show:

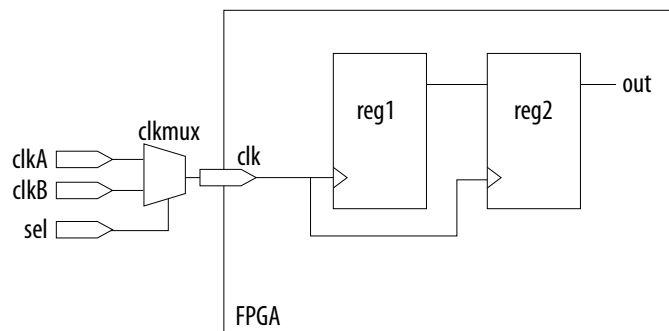
Figure 91. Synchronous Path with Clock Mux Internal to FPGA



Example SDC Constraints for Internal Clock Mux

```
# Create a clock on each port
create_clock -name clk_100 -period 10 [get_ports clkA]
create_clock -name clk_125 -period 8 [get_ports clkB]
# Create derived clocks on the output of the mux
create_generated_clock -name mux_100 -source [get_ports clkA] \
    [get_pins clkmux|combout]
create_generated_clock -name mux_125 -source [get_ports clkB] \
    [get_pins clkmux|combout] -add
# Set the two clocks as exclusive clocks
set_clock_groups -logically_exclusive -group {mux_100} -group {mux_125}
```

Figure 92. Synchronous Path with Clock Mux External to FPGA



Example SDC Constraints for External Clock Mux

```
# Create virtual clocks for the external primary clocks
create_clock -period 10 -name clkA
create_clock -period 20 -name clkB
# Create derived clocks on the port clk
create_generated_clock -name mux_100 -master_clock clkA [get_ports clk]
create_generated_clock -name mux_125 -master_clock clkB [get_ports clk] -add
# Assume no clock network latency between the external clock sources & the \
    clock mux output
set_clock_latency -source 0 [get_clocks {mux_100 mux_125}]
# Set the two clocks as exclusive clocks
set_clock_groups -physically_exclusive -group mux_100 -group mux_125
```

2.4.1.5.2. Asynchronous Clock Groups (-asynchronous)

Use the `-asynchronous` option to create asynchronous clock groups. You can use asynchronous clock groups to break the timing relationship when data transfers through a FIFO between clocks running at different rates.

2.4.1.5.3. set_clock_groups Constraint Tips

When you use `derive_pll_clocks` to create clocks, it can be time consuming to determine all the clock names to include in `set_clock_groups` constraints. However, you can use the following technique to somewhat automate clock constraint creation, even if you do not know all of the clock names.

1. Create a basic .sdc file that contains the [Recommended Initial Conventional SDC Constraints](#), except omit the `set_clock_groups` constraint for now.
2. To add the .sdc to the project, click **Assignments > Settings > Timing Analyzer**. Specify the .sdc file under **SDC files to include in the project**.
3. To open the Timing Analyzer, click **Tools > Timing Analyzer**.
4. In the **Task** pane, double-click **Report Clocks**. The Timing Analyzer reads your .sdc, applies the constraints (including `derive_pll_clocks`), and reports all the clocks.
5. From the Clocks Summary report, copy all the clock names that appear in the first column. The report lists the clock names in the correct format for recognition in the Timing Analyzer.
6. Open .sdc file and paste the clock names into the file, one clock name per line.
7. Format the list of clock names into the `set_clock_groups` command by cutting and pasting clock names into appropriate groups. Next, paste the following template into the .sdc file:

```
set_clock_groups -asynchronous -group { \
} \
-group { \
} \
-group { \
} \
-group { \
}
```

8. Cut and paste the clock names into groups to define their relationship, adding or removing groups, as necessary. Format the groups to make the code readable.

Note: This command can be difficult to read on a single line. You can use the Tcl line continuation character "\" to make this more readable. Place a space after the last character, and then place the "\" character at the end of the line. Be careful not to include any spaces after the escape character. Otherwise, the space becomes the escape character, rather than the end-of-line character.

```
set_clock_groups -asynchronous \
-group {adc_clk \
    the_adc_pll|altpll_component_autogenerated|pll|clk[0] \
    the_adc_pll|altpll_component_autogenerated|pll|clk[1] \
    the_adc_pll|altpll_component_autogenerated|pll|clk[2] \
} \
-group {sys_clk \
    the_system_pll|altpll_component_autogenerated|pll|clk[0] \
    the_system_pll|altpll_component_autogenerated|pll|clk[1] \
} \
-group {the_system_pll|altpll_component_autogenerated|pll|clk[2] \
}
```

Note: The last group has a PLL output `system_pll|..|clk[2]` while the input clock and other PLL outputs are in different groups. If you use PLLs, and the input clock frequency does not relate to the frequency of the PLL's outputs, you must treat the PLLs asynchronously. Typically the outputs of a PLL are related and are in the same group, but this is not a requirement.

For designs with complex clocking, creating clock groups can be an iterative process. For example, a design with two DDR3 cores and high-speed transceivers can have thirty or more clocks. In such cases, you start by adding the clocks that you manually create. Timing Analyzer assumes that the clocks not appearing in the clock groups command relate to every clock and conservatively groups the known clocks. If the design still has failing paths between unrelated clock domains, you can add the new clock domains, as necessary. In this case, a large number of the clocks are not in the `set_clock_groups` command, because they are either cut in the `.sdc` file for the IP (such as the `.sdc` files that the DDR3 cores generate), or they connect only to related clock domains.

For many designs, that is all that's necessary to constrain the IP.

Related Information

[Multicycle Paths](#) on page 106

2.4.1.6. Accounting for Clock Effect Characteristics

The clocks you create with the Timing Analyzer are ideal clocks that do not account for any board effects. You can account for clock effect characteristics with clock latency and clock uncertainty constraints.

You can verify correct implementation of clock constraints by using **Report Clocks** (`report_clocks`) to generate clock timing reports. You can use **Check Timing** (`check_timing`) to report problems with a variety of timing constraints, such as missing clocks.

Related Information

- [Report Clocks](#) on page 141
- [Check Timing](#) on page 170

2.4.1.6.1. Set Clock Latency (`set_clock_latency`)

The **Set Clock Latency** (`set_clock_latency`) constraint allows you to specify additional delay (that is, latency) in a clock network. This delay value represents the external delay from a virtual (or ideal) clock through the longest **Late** (`-late`) or shortest **Early** (`-early`) path, with reference to the **Rise** (`-rise`) or **Fall** (`-fall`) of the clock transition.

When calculating setup analysis, the Timing Analyzer uses the late clock latency for the data arrival path and the early clock latency for the clock arrival path. When calculating hold analysis, the Timing Analyzer uses the early clock latency for the data arrival time and the late clock latency for the clock arrival time.

There are two forms of clock latency: clock source latency, and clock network latency. Source latency is the propagation delay from the origin of the clock to the clock definition point (for example, a clock port). Network latency is the propagation delay from a clock definition point to a register's clock pin. The total latency at a register's clock pin is the sum of the source and network latencies in the clock path.

To specify source latency to any clock ports in your design, use the `set_clock_latency` command.

Note: The Timing Analyzer automatically computes network latencies. Therefore, you can only characterize source latency with the `set_clock_latency` command using the `-source` option.

Related Information

[set_clock_latency Command, Quartus Prime Help](#)

2.4.1.6.2. Clock Uncertainty

By default, the Timing Analyzer creates clocks that are ideal and have perfect edges. To mimic clock-level effects like jitter, you can add uncertainty to those clock edges. The Timing Analyzer automatically calculates appropriate setup and hold uncertainties and applies those uncertainties to all clock transfers in your design, even if you do not include the `derive_clock_uncertainty` command in your `.sdc` file. Setup and hold uncertainties are a critical part of constraining your design correctly.

The Timing Analyzer subtracts setup uncertainty from the data required time for each applicable path and adds the hold uncertainty to the data required time for each applicable path. This slightly reduces the setup and hold slack on each path.

The Timing Analyzer accounts for uncertainty clock effects for three types of clock-to-clock transfers: intraclock transfers, interclock transfers, and I/O interface clock transfers.

- Intraclock transfers occur when the register-to-register transfer takes place in the device and the source and destination clocks come from the same PLL output pin or clock port.
- Interlock transfers occur when a register-to-register transfer takes place in the core of the device and the source and destination clocks come from a different PLL output pin or clock port.
- I/O interface clock transfers occur when data transfers from an I/O port to the core of the device or from the core of the device to the I/O port.

To manually specify clock uncertainty, use the `set_clock_uncertainty` command. You can specify the uncertainty separately for setup and hold. You can also specify separate values for rising and falling clock transitions. You can override the value that the `derive_clock_uncertainty` command automatically applies.

The `derive_clock_uncertainty` command accounts for PLL clock jitter, if the clock jitter on the input to a PLL is within the input jitter specification for PLL's in the target device. If the input clock jitter for the PLL exceeds the specification, add additional uncertainty to your PLL output clocks to account for excess jitter with the `set_clock_uncertainty -add` command. Refer to the device handbook for your device for jitter specifications.

You can also use `set_clock_uncertainty -add` to account for peak-to-peak jitter from a board when the jitter exceeds the jitter specification for that device. In this case you add uncertainty to both setup and hold equal to 1/2 the jitter value:

```
set_clock_uncertainty -setup -to <clock name> \
    -setup -add <p2p jitter/2>
```

```
set_clock_uncertainty -hold -enable_same_physical_edge -to <clock name> \
    -add <p2p jitter/2>
```


There is a complex set of precedence rules for how the Timing Analyzer applies values from `derive_clock_uncertainty` and `set_clock_uncertainty`, which depend on the order of commands and options in your `.sdc` files. The Help topics below contain complete descriptions of these rules. These precedence rules are easier to implement if you follow these recommendations:

- To assign your own clock uncertainty values to any clock transfers, put your `set_clock_uncertainty` exceptions after the `derive_clock_uncertainty` command in the `.sdc` file.
- When you use the `-add` option for `set_clock_uncertainty`, the value you specify is additive to the `derive_clock_uncertainty` value. If you do not specify `-add`, the value you specify replaces the value from `derive_clock_uncertainty`.

2.4.1.7. Constraining CDC Paths

It is essential to apply timing constraints to the multibit clock domain crossing (CDC) paths in your design. You can use the following constraints to constrain CDC paths:

Attention: As of Quartus Prime Pro Edition software version 21.3, the `set_false_path` constraint does not override the `set_max_skew` constraint. You can now apply the `set_false_path` and `set_max_skew` constraints on the same path without override.

Table 19. CDC Path Constraints

Constraints	Description
<code>set_false_path</code> <code>set_clock_groups -asynchronous</code>	Both constraints prevent the Compiler from optimizing slack between asynchronous domain crossings. <code>set_clock_groups</code> is the most aggressive constraint. • Clock-based false paths are less aggressive because these constraints only cut timing on the <code>from_clock</code> to <code>to_clock</code> order specified. • Clock-based false paths are unlike clock groups that cut the path in both directions. • Path-based false paths are the most specific constraint because they cut only on the specified <code>from</code> and <code>to</code> nodes.
<code>set_max_skew</code>	Sets a bound on the allowable skew between different bus bits. • Check the timing of your clock domain crossing by running the Report Max Skew Summary command. • The actual skew requirements depends on your design characteristics, and how you handle the clock domain crossing.
<code>set_net_delay -max</code> <code>set_data_delay</code>	Sets a bound on the allowable datapath delay on any bit of a bus transfer. • <code>set_net_delay</code> is for constraining individual clock edges and nets. Run the Report Net Delay Summary command to report data for this constraint. • <code>set_data_delay</code> is for constraining entire paths. Run the Report Data Delay command to report data for this constraint.

The following shows example constraints for a clock domain crossing between `data_a` in clock domain `clk_a`, and `data_b` in clock domain `clk_b`:

```
create_clock -name clk_a -period 4.000 [get_ports {clk_a}]
create_clock -name clk_b -period 4.500 [get_ports {clk_b}]
set_clock_groups -asynchronous -group [get_clocks {clk_a}] -group \
    [get_clocks {clk_b}]
set_net_delay -from [get_registers {data_a[*]}] -to [get_registers \
```

```
{data_b[*]]} -max -get_value_from_clock_period \
dst_clock_period -value multiplier 0.8
set_max_skew -from [get_keepers {data_a[*]]} -to [get_keepers \
{data_b[*]]} -get_skew_value_from_clock_period min_clock_period \
-skew_value_multiplier 0.8
```

The following examples show applying `set_false_path` for a design that contains a DCFIFO block to avoid timing failures in the synchronization registers. These examples are for constraining single-bit synchronizer CDC paths:

- For paths crossing from the write into the read domain, apply a false path assignment between registers `delayed_wrptr_g` and `rs_dgrp`:

```
set_false_path -from [get_registers {*dcfifo*delayed_wrptr_g[*]]} \
-to [get_registers {*dcfifo*rs_dgrp*}]
```

- For paths crossing from the read into the write domain, apply a false path assignment between registers `rdptr_g` and `ws_dgrp`:

```
set_false_path -from [get_registers {*dcfifo*rdptr_g[*]]} \
-to [get_registers {*dcfifo*ws_dgrp*}]
```

You can verify correct implementation of clock constraints in the following CDC related reports:

Related Information

- [Report Clock Transfers](#) on page 144
- [Report CDC Viewer](#) on page 146
- [Report Asynchronous CDC](#) on page 148
- [Troubleshooting Clock Domain Crossing \(CDC\) Timing Issues](#) on page 187

2.4.2. I/O Constraints

The Timing Analyzer reviews setup and hold relationships for designs in which an external source interacts with a register internal to the design. The Timing Analyzer supports input and output external delay modeling with the `set_input_delay` and `set_output_delay` commands. You can specify the clock and minimum and maximum arrival times relative to the clock.

Specify internal and external timing requirements before you fully analyze a design. With external timing requirements specified, the Timing Analyzer verifies the I/O interface, or periphery of the device, against any system specification.

You can use the **Check Timing** (`check_timing`) command to report problems with a variety of timing constraints, such as the number of input ports that are not clocks that have no input delay constraint.

Related Information

[Check Timing](#) on page 170

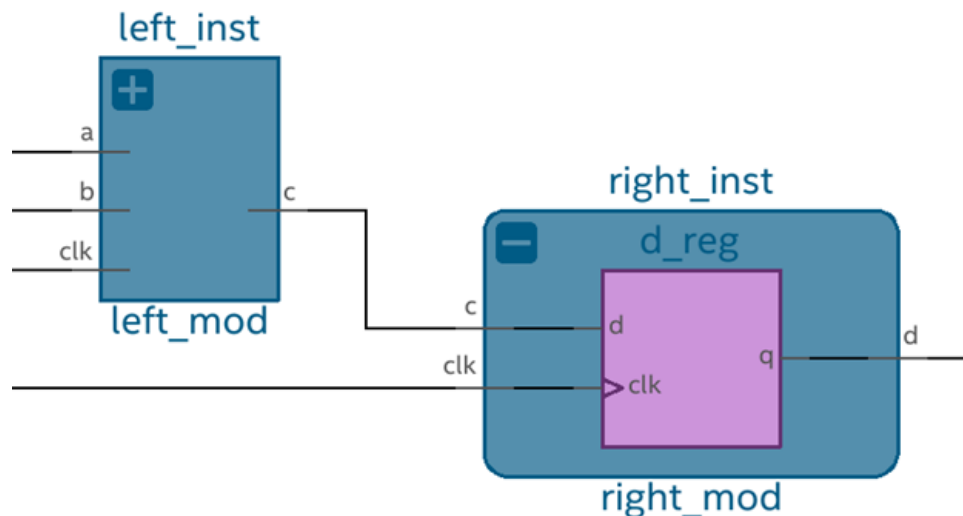
2.4.2.1. Deprecation of the blackbox Argument

Beginning in Quartus Prime Pro Edition version 24.1, support for the `blackbox` argument to the `set_input_delay` and `set_output_delay` constraints is deprecated. Beginning in Quartus Prime Pro Edition software version 24.3, support for the `blackbox` argument is removed. You could previously use the `blackbox` argument to perform delay budgeting on I/Os at the boundary of a design partition.

Going forward, you can use `set_max_delay` constraints to perform delay budgeting on I/Os at the boundary of a design partition, rather than using the `blackbox` argument. The following example shows how to convert a `set_output_delay -blackbox` constraint to a `set_max_delay` constraint.

[Sample Design Hierarchy with Design Partitions](#) shows a sample design hierarchy, where `left_inst` and `right_inst` are partitions, and the `c` pins of each partition are the partition boundaries.

Figure 93. Sample Design Hierarchy with Design Partitions



The module for `right_inst` is not yet available to the developer, and is represented by a single register, `d_reg`. The clock has a period of 4 ns.

To ensure that the path between `left_inst` and `right_inst` can pass timing after the `right_inst` module is available, the `left_inst` partition must not use the entire 4 ns available for the data transfer.

In previous versions of the Quartus Prime Pro Edition software, you could use a set of constraints like the following:

```
set budget 2.5
set ref_pin [get_pins right_inst|d_reg|clk]
set_output_delay -clock clk -blackbox -reference_pin $ref_pin $budget [get_pins
right_inst|c]
```

The example constraints set a delay budget of 2.5ns to `right_inst`, leaving a delay budget of 1.5ns for the `left_inst` partition. However, these example constraints require that you also specify the correct reference pin to accurately measure the impact of clock skew.

Beginning in Quartus Prime Pro Edition version 24.3, as an alternative to the delay budgeting approach, use the following technique. This technique is simpler, and automatically includes clock skew impact without requiring you to specify a reference pin.

```
set period 4
set budget 2.5
set_max_delay [expr {$period - $budget}] -through [get_pins right_inst|c]
```

2.4.2.2. Input Constraints (set_input_delay)

Input constraints specify delays for all external signals feeding the FPGA. Specify input requirements for all input ports in your design.

```
set_input_delay -clock { clock } -clock_fall -fall -max 20 foo
```

Use the **Set Input Delay** (`set_input_delay`) constraint to specify external input delay requirements. Specify the **Clock name** (`-clock`) to reference the virtual or actual clock. You can specify a clock to allow the Timing Analyzer to correctly derive clock uncertainties for interclock and intraclock transfers. The clock defines the launching clock for the input port. The Timing Analyzer automatically determines the latching clock inside the device that captures the input data, because all clocks in the device are defined.

Figure 94. Input Delay Diagram

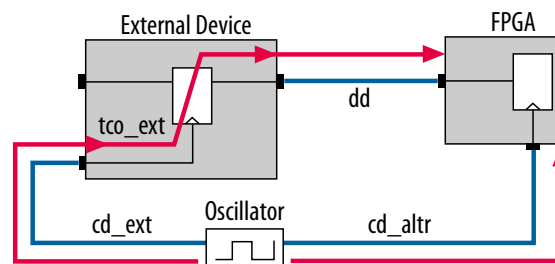


Figure 95. Input Delay Calculation

$$\begin{aligned} \text{input delay}_{\text{MAX}} &= (\text{cd_ext}_{\text{MAX}} - \text{cd_altr}_{\text{MIN}}) + \text{tco_ext}_{\text{MAX}} + \text{dd}_{\text{MAX}} \\ \text{input delay}_{\text{MIN}} &= (\text{cd_ext}_{\text{MIN}} - \text{cd_altr}_{\text{MAX}}) + \text{tco_ext}_{\text{MIN}} + \text{dd}_{\text{MIN}} \end{aligned}$$

Note: Beginning in Quartus Prime Pro Edition software version 24.3, support for the `blackbox` argument is removed. You could previously use the `blackbox` argument to perform delay budgeting on I/Os at the boundary of a design partition. Refer to [Deprecation of the blackbox Argument](#).

You can use the **Check Timing** (`check_timing`) command to report problems with a variety of timing constraints, such as the number of input ports that are not clocks that have no input delay constraint.

Related Information

[Check Timing](#) on page 170

2.4.2.3. Output Constraints (`set_output_delay`)

Output constraints specify all external delays from the device for all output ports in your design.

```
set_output_delay -clock { clock } -clock_fall -rise -max 2 foo
```

Use the **Set Output Delay** (`set_output_delay`) constraint to specify external output delay requirements. Specify the **Clock name** (`-clock`) to reference the virtual or actual clock. When specifying a clock, the clock defines the latching clock for the output port. The Timing Analyzer automatically determines the launching clock inside the device that launches the output data, because all clocks in the device are defined. The following figure is an example of an output delay referencing a virtual clock.

Figure 96. Output Delay Diagram

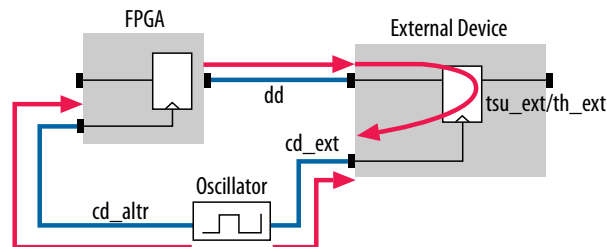


Figure 97. Output Delay Calculation

$$\begin{aligned}\text{output delay}_{\text{MAX}} &= dd_{\text{MAX}} + tsu_ext + (cd_altr_{\text{MAX}} - cd_ext_{\text{MIN}}) \\ \text{output delay}_{\text{MIN}} &= (dd_{\text{MIN}} - th_ext + (cd_altr_{\text{MIN}} - cd_ext_{\text{MAX}}))\end{aligned}$$

Note: Beginning in Quartus Prime Pro Edition software version 24.3, support for the `blackbox` argument is removed. You could previously use the `blackbox` argument to perform delay budgeting on I/Os at the boundary of a design partition. Refer to [Deprecation of the blackbox Argument](#).

You can use the **Check Timing** (`check_timing`) command to report problems with a variety of timing constraints, such as the number of output ports that have no output delay constraint.

Related Information

- [Check Timing](#) on page 170
- [set_input_delay Command, Quartus Prime Help](#)
- [set_output_delay Command, Quartus Prime Help](#)

2.4.3. Delay and Skew Constraints

You can specify skew and delays to model external device timing and board timing parameters.

2.4.3.1. Advanced I/O Timing and Board Trace Model Delay

The Timing Analyzer can use advanced I/O timing and board trace model constraints to model I/O buffer delays in your design.

If you change any advanced I/O timing settings or board trace model assignments, recompile your design before you analyze timing, or use the `-force_dat` option to force delay annotation when you create a timing netlist.

Example 6. Forcing Delay Annotation

```
create_timing_netlist -force_dat
```

2.4.3.2. Maximum Skew (set_max_skew)

The **Set Max Skew** (`set_max_skew`) constraint specifies the maximum allowable skew between the sets of registers or ports you specify. In order to constrain skew across multiple paths, you must constrain all such paths within a single `set_max_skew` constraint.

```
set_max_skew -from_clock { clock } -to_clock { * } -from foo -to blat 2
```

The `set_max_delay`, `set_min_delay`, `set_multicycle_path`, and `set_false_path` constraints do not affect the `set_max_skew` timing constraint. However, the `set_clock_groups` constraint does impact the `set_max_skew` constraint.

Note: Exclusive clock groups (set with `set_clock_groups -exclusive`) override `set_max_skew` constraints.

The Timing Analyzer does not compare two paths for skew if their clocks are exclusive to each other. However, the Timing Analyzer does analyze for skew paths whose clocks are asynchronous.

Table 20. set_max_skew Options

Arguments	Description
<code>-h -help</code>	Short help.
<code>-long_help</code>	Long help with examples and possible return values.
<code>-fall_from_clock <names></code>	Valid source clocks (Tcl matches string patterns). Analysis only considers paths from falling clock edges.
<code>-fall_to_clock <names></code>	Valid destination clocks (Tcl matches string patterns). Analysis only considers paths from falling clock edges.
<code>-from <names>^(2)</code>	Valid sources (Tcl matches string patterns).
<i>continued...</i>	

Arguments	Description
<code>-from_clock <names></code>	Valid source clocks (Tcl matches string patterns).
<code>-get_skew_value_from_clock_period <src_clock_period dst_clock_period min_clock_period></code>	Option to interpret skew constraint as a multiple of the clock period.
<code>-rise_from_clock <names></code>	Valid source clocks (Tcl matches string patterns). Analysis only considers paths from rising clock edges.
<code>-rise_to_clock <names></code>	Valid destination clocks (Tcl matches string patterns). Analysis only considers paths to rising clock edges.
<code>-skew_value_multiplier <multiplier></code>	Value by which the clock period multiplies to compute skew requirement.
<code>-to <names>⁽²⁾</code>	Valid destinations (Tcl matches string patterns)
<code>-to_clock <names></code>	Valid destination clocks (Tcl matches string patterns).
<code><skew></code>	The value of the skew you require.

Applying maximum skew constraints between clocks applies the constraint from all register or ports driven by the clock you specify (with the `-from` option) to all registers or ports driven by the clock you specify (with the `-to` option).

Maximum skew analysis can include data arrival times, clock arrival times, register micro parameters, clock uncertainty, on-die variation, and clock pessimism removal. Among these, the Fitter only disables clock pessimism removal by default.

Use `-get_skew_value_from_clock_period` to set the skew as a multiple of the launching or latching clock period, or whichever of the two has a smaller period. If you use this option, set `-skew_value_multiplier`, and you may not set the positional skew option. If more than one clock clocks the set of skew paths, Timing Analyzer uses the clock with smallest period to compute the skew constraint.

Click **Report Max Skew...** (`report_max_skew`) to view the max skew analysis. Since skew occurs between two or more paths, no results display if the `-from/-from_clock` and `-to/-to_clock` filters satisfy less than two paths.

Related Information

- [Timing Exception Precedence](#) on page 102
- [report_max_skew Command, Quartus Prime Help](#)

2.4.3.3. Net Delay (`set_net_delay`)

Use the `set_net_delay` command to set the net delays and perform minimum or maximum timing analysis across nets. A net delay constraint is invalid if there is a combinational cell between the From and To nodes.

⁽²⁾ Legal values for the `-from` and `-to` options are collections of clocks, registers, ports, pins, cells or partitions in a design.

The `-from` and `-to` options can be string patterns or pin, port, register, or net collections. When you use pin or net collection, include output pins or nets in the collection.

```
set_net_delay -from reg_a -to reg_c -max 20
```

Table 21. `set_net_delay` Options

Arguments	Description
<code>-h</code> <code>-help</code>	Short help.
<code>-long_help</code>	Long help with examples and possible return values.
<code>-from <names>⁽³⁾</code>	Valid source pins, ports, registers or nets (Tcl matches string patterns).
<code>-get_value_from_clock_period</code> <code><src_clock_period> <dst_clock_period> </code> <code><min_clock_period> <max_clock_period></code>	Option to interpret net delay constraint as a multiple of the clock period.
<code>-max</code>	Specifies maximum delay.
<code>-min</code>	Specifies minimum delay.
<code>-to <names>⁽⁴⁾</code>	Valid destination pins, ports, registers or nets (Tcl matches string patterns).
<code>-value_multiplier <multiplier></code>	Value by which the clock period multiplies to compute net delay requirement.
<code><delay></code>	Delay value.

If you use the `-min` option, the Timing Analyzer calculates slack by determining the minimum delay on the edge. If you use `-max` option, the Timing Analyzer calculates slack by determining the maximum edge delay.

Use `-get_value_from_clock_period` to set the net delay requirement as a multiple of the launching or latching clock period, or whichever of the two has a smaller or larger period. If you use this option, you must not set the positional delay option. If more than one clock clocks the set of nets, the Timing Analyzer uses the net with the smallest period to compute the constraint for a `-max` constraint, and the largest period for a `-min` constraint. If no clocks are clocking the endpoints of the net (that is, if the endpoints of the nets are not registers or constraint ports), the Timing Analyzer ignores the net delay constraint.

Related Information

- [Timing Exception Precedence](#) on page 102
- [report_net_delay Command, Quartus Prime Help](#)

⁽³⁾ If option is a wildcard (`"*"`) character, all the output pins and registers on timing netlist become valid source points.

⁽⁴⁾ If no option, or if option is a wildcard (`"*"`) character, all the output pins and registers on timing netlist become valid destination points.

2.4.3.4. Data Delay (set_data_delay)

The Set Data Delay (`set_data_delay`) constraint specifies a maximum datapath delay exception for the path you specify.

The maximum delay analysis includes t_{CO} of the launching register, and t_{SU} of the latching register. By default, the maximum delay analysis does not include the clock arrival times at the launching or latching register.

Note: Exclusive clock groups (set with `set_clock_groups -exclusive`) override `set_data_delay` constraints. Unlike paths between asynchronous clocks, paths between exclusive clocks are entirely inactive, and therefore are not suitable for timing requirement analysis. False paths do not override `set_data_delay` constraints.

Table 22. `set_data_delay` Options

Arguments	Description
<code>-h</code> <code>-help</code>	Short help.
<code>-long_help</code>	Long help with examples and possible return values.
<code>-add_latch_clock</code>	Include the latch clock path in timing analysis.
<code>-add_launch_clock</code>	Include the launch clock path in timing analysis.
<code>-allow_destination_borrowing</code>	Allow time borrowing at the destination.
<code>-fall_from <names></code>	Valid source clocks (string patterns are matched using Tcl string matching).
<code>-fall_to <names></code>	Valid destination clocks (string patterns are matched using Tcl string matching).
<code>-from <names></code>	Valid sources (string patterns are matched using Tcl string matching).
<code>-get_value_from_clock_period</code> <code><src_clock_period/dst_clock_period/</code> <code>min_clock_period/max_clock_period></code>	Compute constraint as a multiple of the clock period.
<code>-no_synchronizer</code>	Prevent this data delay from triggering a synchronizer.
<code>-rise_from <names></code>	Valid source clocks (string patterns are matched using Tcl string matching).
<code>-rise_to <names></code>	Valid destination clocks (string patterns are matched using Tcl string matching).
<code>-through <names></code>	Valid through nodes (string patterns are matched using Tcl string matching).
<code>-to <names></code>	Valid destinations (string patterns are matched using Tcl string matching).
<code>-value_multiplier <multiplier></code>	Value by which the clock period should be multiplied to compute requirement.
<code><value></code>	Time value.

A common use for the `set_data_delay` constraint is to specify an upper bound on the allowable delay on a false path in a cross-domain transfer. When you specify `set_false_path` path, the Fitter does not have to meet a timing requirement on that path. Consequently, the Fitter is free to place the endpoints of the path very far

apart, and to route the path in a convoluted way. Although a convoluted route does not often occur, a false path allows the Fitter to make that selection. When you use `set_false_path` to cut asynchronous transfers, you often want to keep the path delay down to reduce system latency. A `set_data_delay` constraint is one way to specify an upper bound for the path delay.

The `set_net_delay` constraint is also appropriate in many of the same situations. You can use `set_net_delay` to set an upper bound for a path that you specify as false. The `set_data_delay` and `set_net_delay` constraints differ in their behavior when there is combinational logic on the path:

- A register-to-register `set_data_delay` constraint applies to the path regardless of whether it has combinational logic.
- A register-to-register `set_net_delay` constraint does not apply if there is combinational logic on the path.

Generally, you should not use combinational logic on paths that transfer data between clock domains. Therefore, `set_data_delay` and `set_net_delay` constraints are both appropriate. However, when there is logic on paths between clock domains, use the `set_data_delay` constraint to constrain the paths.

2.4.4. Timing Exception Constraints

Timing exception assignments allow you to modify (or provide exception to) the default timing analysis behavior to account for your specific design conditions.

You can specify the following timing constraints that modify the default timing analysis behavior:

- Set False Path
- Set Multicycle Path
- Set Minimum Delay
- Set Maximum Delay

Verify correct implementation of timing exception assignments by using the **Report Exceptions** (`report_exceptions`) command to report all exceptions to default timing analysis conditions.

Related Information

[Report Exceptions](#) on page 168

2.4.4.1. Timing Exception Precedence

If the same clock or node names occur in multiple timing exceptions, the Timing Analyzer observes the following order of timing exception precedence:

1. **Set False Path** (`set_false_path`) is the first priority. False paths and clock groups have identical priority, except when you use the `-latency_insensitive` or `-no_synchronizer` options with a false path exception. With either option, the false path has priority over a clock group.
2. **Set Clock Groups** (`set_clock_groups`) is the second priority.
3. **Set Minimum Delay** (`set_min_delay`) and **Set Maximum Delay** (`set_max_delay`) are the third priority.
4. **Set Multicycle Path** (`set_multicycle_path`) is the fourth priority.

The false path timing exception has the highest precedence. Within each category, assignments to individual nodes have precedence over assignments to clocks. For exceptions of the same type:

1. `-from <node>` is the first priority.
2. `-to <node>` is the second priority.
3. `-thru <node>` is the third priority.
4. `-from <clock>` is the fourth priority.
5. `-to <clock>` is the fifth priority.

An asterisk wildcard (*) for any of these options applies the same precedence as not specifying the option at all. For example, `-from a -to *` is treated identically to `-from a` with regards precedence.

Precedence example

1. `set_max_delay 1 -from x -to y`
2. `set_max_delay 2 -from x`
3. `set_max_delay 3 -to y`

The first exception has higher priority than either of the other two, since the first exception specifies a `-from` (while #3 does not) and specifies a `-to` (while #2 does not). In the absence of the first exception, the second exception has higher priority than the third, since the second exception specifies a `-from`, which the third does not. Finally, the remaining order of precedence for additional exceptions is order-dependent, such that the assignments most recently created overwrite, or partially overwrite, earlier assignments.

The `set_net_delay`, `set_max_skew`, and `set_data_delay` constraints analyze independently of minimum or maximum delays, or multicycle path constraints.

- The `set_net_delay` exception applies regardless of the existence of a `set_false_path` exception, or `set_clock_groups` exception, or other path-based constraint or exception. It is a net-based exception, and net-based and path-based exceptions are applied independently of each other.
- The `set_max_skew` exception applies on paths cut by an asynchronous clock group, and regardless of any `set_false_path` exception. Exclusive clock groups override max skew exceptions, because paths between exclusive clocks are entirely inactive and should not be analyzed for timing or skew requirements. This precedence allows you to define more targeted constraints on asynchronous CDC bus transfers.
- The `set_data_delay` exception specifies a maximum datapath delay exception for a given path. Exclusive clock groups override data delay exceptions, because paths between exclusive clocks are entirely inactive and should not be analyzed for timing or data delay requirements. Asynchronous clock groups do not override data delay exceptions. False path exceptions override data delay exceptions in the Quartus Prime Pro software version 21.2 and earlier. Beginning in version 21.3, false path exceptions do not override data delay exceptions. This change in precedence allows you to write more targeted constraints on asynchronous CDC bus transfers.

Verify correct implementation of timing exception assignments by using the **Report Exceptions** (`report_exceptions`) command to report all exceptions to default timing analysis conditions.

Related Information

- [False Paths \(`set\_false\_path`\)](#) on page 104
- [Creating Clock Groups \(`set\_clock\_groups`\)](#) on page 87
- [Constraining CDC Paths](#) on page 93
- [Creating Clock Groups \(`set\_clock\_groups`\)](#) on page 87
- [Maximum Skew \(`set\_max\_skew`\)](#) on page 98
- [Minimum and Maximum Delays](#) on page 105
- [Net Delay \(`set\_net\_delay`\)](#) on page 99
- [Report Data Delay](#) on page 141
- [Report Exceptions](#) on page 168

2.4.4.2. False Paths (`set_false_path`)

The **Set False Path** (`set_false_path`) constraint allows you to exclude a path from timing analysis, such as test logic or any other path not relevant to the circuit's operation. You can specify the source (`-from`), common through elements (`-thru`), and destination (`-to`) elements of that path.

The following SDC command makes false path exceptions from all registers starting with A, to all registers starting with B:

```
set_false_path -from [get_pins A*] -to [get_pins B*]
```

You can specify either a point-to-point or clock-to-clock path as a false path. A false path's `-from` and `-to` targets can be either nodes or clocks. However, the `-thru` targets can only be combinational nodes. For example, you can specify a false path for a static configuration register that writes once during power-up initialization, but does not change state again.

Although signals from static configuration registers often cross clock domains, you may not want to make false path exceptions to a clock-to-clock path, because some data may transfer across clock domains. However, you can selectively make false path exceptions from the static configuration register to all endpoints.

The Timing Analyzer assumes all clocks are related unless you specify otherwise. Use clock groups to more efficiently make false path exceptions between clocks, rather than writing multiple `set_false_path` exceptions between each clock transfer you want to eliminate.

Verify correct implementation of timing exception assignments by using the **Report Exceptions** (`report_exceptions`) command to report all exceptions to default timing analysis conditions.

Related Information

- [Timing Exception Precedence](#) on page 102
- [Creating Clock Groups \(`set\_clock\_groups`\)](#) on page 87
- [Constraining CDC Paths](#) on page 93
- [Report Exceptions](#) on page 168
- [set_false_path Command, Quartus Prime Help](#)

2.4.4.3. Minimum and Maximum Delays

To specify an absolute minimum or maximum delay for a path, use the **Set Minimum Delay** (`set_min_delay`) or the **Set Maximum Delay** (`set_max_delay`) constraints, respectively. Specifying minimum and maximum delay directly overwrites existing setup and hold relationships with the minimum and maximum values.

Use the `set_max_delay` and `set_min_delay` constraints for asynchronous signals that do not have a specific clock relationship in your design, but require a minimum and maximum path delay. You can create minimum and maximum delay exceptions for port-to-port paths through the device without a register stage in the path. If you use minimum and maximum delay exceptions to constrain the path delay, specify both the minimum and maximum delay of the path; do not constrain only the minimum or maximum value.

If the source or destination node is clocked, the Timing Analyzer takes into account the clock paths, allowing more or less delay on the data path. If the source or destination node has an input or output delay, the minimum or maximum delay check also includes that delay.

If you specify a minimum or maximum delay between timing nodes, the delay applies only to the path between the two nodes. If you specify a minimum or maximum delay for a clock, the delay applies to all paths where the clock clocks the source node or destination node.

You can create a minimum or maximum delay exception for an output port that does not have an output delay constraint. You cannot report timing for the paths that relate to the output port; however, the Timing Analyzer reports any slack for the path in the setup summary and hold summary reports. Because there is no clock that relates to the output port, the Timing Analyzer reports no clock for timing paths of the output port.

Note: To report timing with clock filters for output paths with minimum and maximum delay constraints, you can set the output delay for the output port with a value of zero. You can use an existing clock from the design or a virtual clock as the clock reference.

Verify correct implementation of timing exception assignments by using the **Report Exceptions** (`report_exceptions`) command to report all exceptions to default timing analysis conditions.

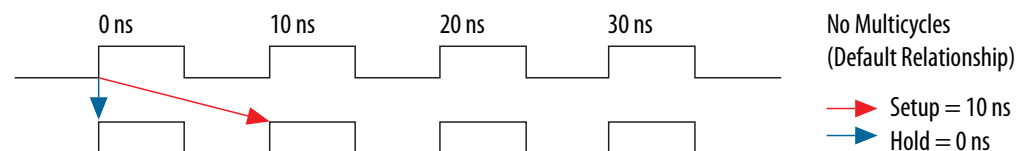
Related Information

- [Timing Exception Precedence](#) on page 102
- [Report Exceptions](#) on page 168
- [set_max_delay Command, Quartus Prime Help](#)
- [set_min_delay Command, Quartus Prime Help](#)

2.4.4.4. Multicycle Paths

By default, the Timing Analyzer performs a single-cycle analysis, which is the most restrictive type of analysis. When analyzing a path without a multicycle constraint, the Timing Analyzer determines the setup launch and latch edge times by identifying the closest two active edges in the respective waveforms.

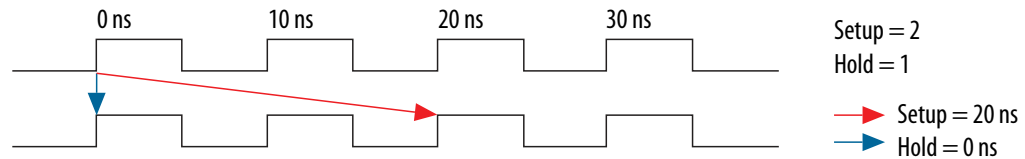
Figure 98. Default Setup and Hold Relationship (No Multicycle)



For hold time analysis, the timing analyzer analyzes the path for two timing conditions for every possible setup relationship, not just the worst-case setup relationship. Therefore, the hold launch and latch times can be unrelated to the setup launch and latch edges.

A multicycle constraint adjusts this default setup or hold relationship by the number of clock cycles you specify, based on the source (`-start`) or destination (`-end`) clock. A setup multicycle constraint of 2 extends the worst-case setup latch edge by one destination clock period. If you do not specify `-start` and `-end` values, the default constraint is `-end`.

Figure 99. Setup and Hold Relationship with Multicycle = 2



Hold multicycle constraints derive from the default hold position (the default value is 0). An end hold multicycle constraint of 1 effectively subtracts one destination clock period from the default hold latch edge.

When the objects are timing nodes, the multicycle constraint only applies to the path between the two nodes. When an object is a clock, the multicycle constraint applies to all paths where the source node (`-from`) or destination node (`-to`) is clocked by the clock. When you adjust a setup relationship with a multicycle constraint, the default hold relationship adjusts automatically.

You can use timing constraints to modify either the launch or latch edge times that the Timing Analyzer uses to determine a setup relationship or hold relationship.

Table 23. Multicycle Constraints

Command	Modification
<code>set_multicycle_path -setup -end <value></code>	Latch edge time of the setup relationship.
<code>set_multicycle_path -setup -start <value></code>	Launch edge time of the setup relationship.
<code>set_multicycle_path -hold -end <value></code>	Latch edge time of the hold relationship.
<code>set_multicycle_path -hold -start <value></code>	Launch edge time of the hold relationship.

Verify correct implementation of timing exception assignments by using the **Report Exceptions** (`report_exceptions`) command to report all exceptions to default timing analysis conditions.

Related Information

[Report Exceptions](#) on page 168

2.4.4.4.1. Common Multicycle Applications

Multicycle exceptions adjust the timing requirements for a register-to-register path, allowing the Fitter to optimally place and route a design. Two common multicycle applications are relaxing setup to allow a slower data transfer rate, and altering the setup to account for a phase shift.

2.4.4.4.2. Relaxing Setup with Multicycle (`set_multicycle_path`)

You can use a multicycle exception when the data transfer rate is slower than the clock cycle. Relaxing the setup relationship increases the window when timing analysis accepts data as valid.

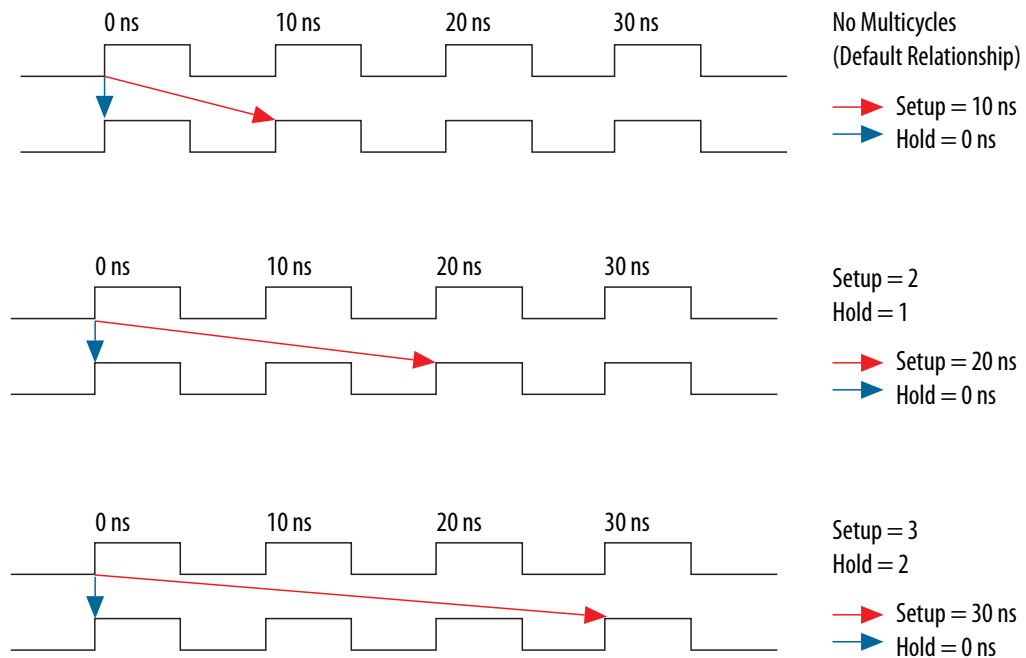
In the following example, the source clock has a period of 10 ns, but the clock enable signal controls a group of latching registers, so the registers only enable every other cycle. The 10 ns clock feeds registers, so the Timing Analyzer reports a setup of 10 ns and a hold of 0 ns. However, the data is transferring every other cycle, so the Timing Analyzer must analyze the relationships as if the clock is operating at 20 ns. The result is a setup of 20 ns, while the hold remains 0 ns, thus extending the window for data recognition.

The following pair of multicycle assignments relax the setup relationship by specifying the `-setup` value of N and the `-hold` value as N-1. You must specify the hold relationship with a `-hold` assignment to prevent a positive hold requirement.

Constraint to Relax Setup and Maintain Hold

```
set_multicycle_path -setup -from src_reg* -to dst_reg* 2
set_multicycle_path -hold -from src_reg* -to dst_reg* 1
```

Figure 100. Multicycle Setup Relationships



You can extend this pattern to create larger setup relationships to ease timing closure requirements. A common use for this exception is when writing to asynchronous RAM across an I/O interface. The delay between address, data, and a write enable may be several cycles. A multicycle exception to I/O ports allows extra time for the address and data to resolve before the enable occurs.

The following constraint relaxes the setup by three cycles:

Three Cycle I/O Interface Constraint

```
set_multicycle_path -setup -to [get_ports {SRAM_ADD[*] SRAM_DATA[*]}] 3
set_multicycle_path -hold -to [get_ports {SRAM_ADD[*] SRAM_DATA[*]}] 2
```


2.4.4.4.3. Accounting for a Phase Shift (-phase)

In the following example, the design contains a PLL that performs a phase-shift on a clock whose domain exchanges data with domains that do not experience the phase shift. This occurs when the destination clock phase-shifts forward, and the source clock does not shift. The default setup relationship becomes that phase-shift, thus shifting the window when data is valid.

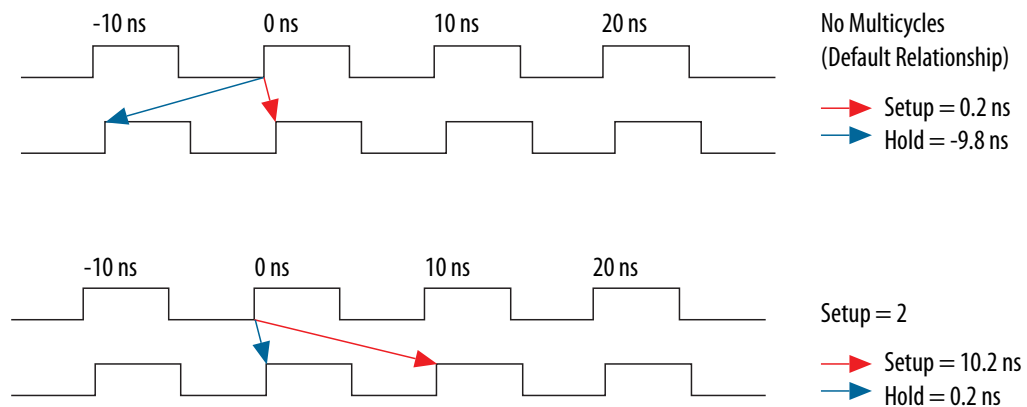
For example, the following code phase-shifts one output of a PLL forward by a small amount, in this case 0.2 ns.

Cross Domain Phase-Shift

```
create_generated_clock -source pll|inclk[0] -name pll|clk[0] pll|clk[0]
create_generated_clock -source pll|inclk[0] -name pll|clk[1] -phase 30 pll|clk[1]
```

The default setup relationship for this phase-shift is 0.2 ns, shown in Figure A, creating a scenario where the hold relationship is negative, which makes achieving timing closure nearly impossible.

Figure 101. Phase-Shifted Setup and Hold



The following constraint allows the data to transfer to the following edge:

```
set_multicycle_path -setup -from [get_clocks clk_a] -to [get_clocks clk_b] 2
```

The hold relationship derives from the setup relationship, making a multicycle hold constraint unnecessary.

Related Information

- [Same Frequency Clocks with Destination Clock Offset](#) on page 118
- [set_multicycle_path Command, Quartus Prime Help](#)

2.4.4.5. Multicycle Exception Examples

The examples in this section illustrate how the multicycle exceptions affect the default setup and hold analysis in the Timing Analyzer. The multicycle exceptions apply to a simple register-to-register circuit. Both the source and destination clocks are set to 10 ns.

Verify correct implementation of timing exception assignments by using the **Report Exceptions** (`report_exceptions`) command to report all exceptions to default timing analysis conditions.

Related Information

[Report Exceptions](#) on page 168

2.4.4.5.1. Default Multicycle Analysis

By default, the Timing Analyzer performs a single-cycle analysis to determine the setup and hold checks. Also, by default, the Timing Analyzer sets the end multicycle setup assignment value to one and the end multicycle hold assignment value to zero.

The source and the destination timing waveform for the source register and destination register, respectively where HC1 and HC2 are hold checks 1 and 2 and SC is the setup check.

Figure 102. Default Timing Diagram

The timing waveforms show the source and destination registers of a data transfer. HC1 and HC2 are the hold checks that Timing Analyzer performs. SC is the setup check that Timing Analyzer performs.

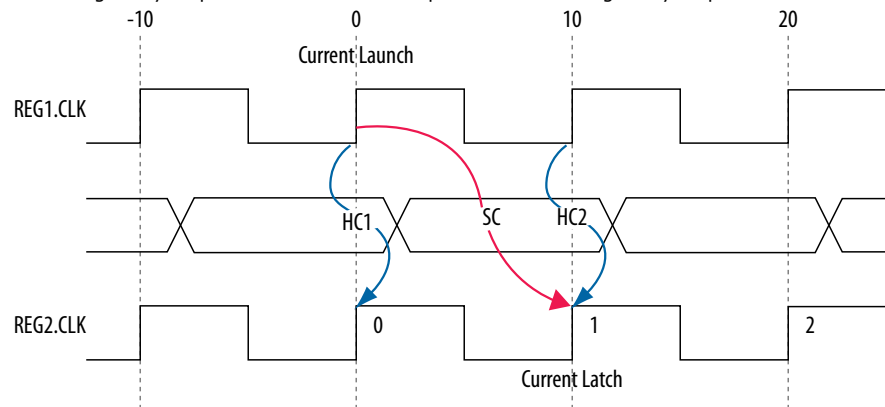


Figure 103. Setup Check Calculation

$$\begin{aligned}
 \text{setup check} &= \text{current latch edge} - \text{closest previous launch edge} \\
 &= 10 \text{ ns} - 0 \text{ ns} \\
 &= 10 \text{ ns}
 \end{aligned}$$

The most restrictive default single-cycle setup relationship, with an implied end multicycle setup assignment of one, is 10 ns.

Figure 104. Default Setup Report

Path #1: Setup slack is 8.226					
Path Summary		Statistics	Data Path	Waveform	
Data Arrival Path					
	Total	Incr	RF	Type	Element
1	0.000	0.000			launch edge time
2	0.000	0.000		borrow	time borrowed
3	2.603	2.603			clock path
1	2.603	2.603	R		clock network delay
2	2.603	0.000			src
4	3.770	1.167			data path
1	2.832	0.229	RR	uTco	src q
2	2.991	0.159	RR	CELL	src~la_lab/laboutb[16]
3	3.770	0.779	RR	IC	dst d
4	3.770	0.000	RR	CELL	dst
Data Required Path					
	Total	Incr	RF	Type	Element
1	10.000	10.000			latch edge time
2	10.000	0.000		borrow	time borrowed
3	11.983	1.983			clock path
1	11.983	1.983	R		clock network delay
4	11.953	-0.030			clock uncertainty
5	11.996	0.043		uTsu	dst

Figure 105. Hold Check Calculation

The figure shows the hold timing report with the launch and latch edge times highlighted.

$$\begin{aligned}
 \text{hold check 1} &= \text{current launch edge} - \text{previous latch edge} \\
 &= 0 \text{ ns} - 0 \text{ ns} \\
 &= 0 \text{ ns}
 \end{aligned}$$

$$\begin{aligned}
 \text{hold check 2} &= \text{next launch edge} - \text{current latch edge} \\
 &= 10 \text{ ns} - 10 \text{ ns} \\
 &= 0 \text{ ns}
 \end{aligned}$$

The most restrictive default single-cycle hold relationship, with an implied end multicyle hold assignment of zero, is 0ns.

Figure 106. Default Hold Report

The figure shows the hold timing report with the launch and latch edge times highlighted.

Path #1: Hold slack is 0.230					
Path Summary		Statistics	Data Path		Waveform
Data Arrival Path					
	Total	Incr	RF	Type	Element
1	0.000	0.000			launch edge time
2	0.000	0.000		borrow	time borrowed
3	▼ 1.012	1.012			clock path
1	1.012	1.012	R		clock network delay
2	1.012	0.000			src
4	▼ 1.529	0.517			data path
1	1.113	0.101	RR	uTco	src[q
2	1.175	0.062	RR	CELL	src~la_lab/laboutb[16]
3	1.529	0.354	RR	IC	dst[d
4	1.529	0.000	RR	CELL	dst
Data Required Path					
	Total	Incr	RF	Type	Element
1	0.000	0.000			latch edge time
2	0.000	0.000		borrow	time borrowed
3	▼ 1.134	1.134			clock path
1	1.134	1.134	R		clock network delay
4	1.164	0.030			clock uncertainty
5	1.299	0.135		uTh	dst

2.4.4.5.2. End Multicycle Setup = 2 and End Multicycle Hold = 0

In this example, the end multicycle setup assignment value is two, and the end multicycle hold assignment value is zero.

Multicycle Constraint

```
set_multicycle_path -from [get_clocks clk_src] -to [get_clocks clk_dst] \
    -setup -end 2
```

Note: The Timing Analyzer does not require an end multicycle hold value because the default end multicycle hold value is zero.

In this example, the setup relationship relaxes by a full clock period by moving the latch edge to the next latch edge. The hold analysis is does not change from the default settings. The following shows the setup timing diagram for the analysis that the Timing Analyzer performs. The latch edge is a clock cycle later than in the default single-cycle analysis.

Figure 107. Setup Timing Diagram

The figure shows the setup timing diagram for the analysis that the Timing Analyzer performs. Without the multicycle constraint the latching edge is edge 1. However, with the multicycle constraint the latching edge is edge 2.

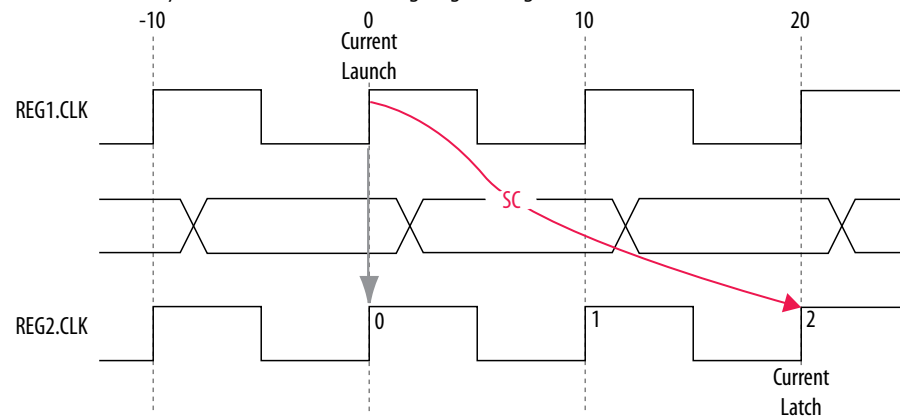


Figure 108. Setup Check Calculation

$$\begin{aligned}
 \text{setup check} &= \text{current latch edge} - \text{closest previous launch edge} \\
 &= 20 \text{ ns} - 0 \text{ ns} \\
 &= 20 \text{ ns}
 \end{aligned}$$

The most restrictive setup relationship with an end multicycle setup assignment of two is 20 ns. The following shows the setup report in the Timing Analyzer and highlights the launch and latch edges.

Figure 109. Setup Report with Setup Multicycle Exception

Path #1: Setup slack is 18.226					
Path Summary Statistics Data Path Waveform					
Data Arrival Path					
	Total	Incr	RF	Type	Element
1	0.000	0.000			launch edge time
2	0.000	0.000		borrow	time borrowed
3	2.603	2.603			clock path
1	2.603	2.603	R		clock network delay
2	2.603	0.000			src
4	3.770	1.167			data path
1	2.832	0.229	RR	uTco	src q
2	2.991	0.159	RR	CELL	src~la_lab/laboutb[16]
3	3.770	0.779	RR	IC	dst d
4	3.770	0.000	RR	CELL	dst
Data Required Path					
	Total	Incr	RF	Type	Element
1	20.000	20.000			latch edge time
2	20.000	0.000		borrow	time borrowed
3	21.983	1.983			clock path
1	21.983	1.983	R		clock network delay
4	21.953	-0.030			clock uncertainty
5	21.996	0.043		uTsu	dst

Because the multicycle hold latch and launch edges are the same as the results of hold analysis with the default settings, the multicycle hold analysis in this example is equivalent to the single-cycle hold analysis. The hold checks are relative to the setup check. Normally, the Timing Analyzer performs hold checks on every possible setup check, not only on the most restrictive setup check edges.

Figure 110. Hold Timing Diagram

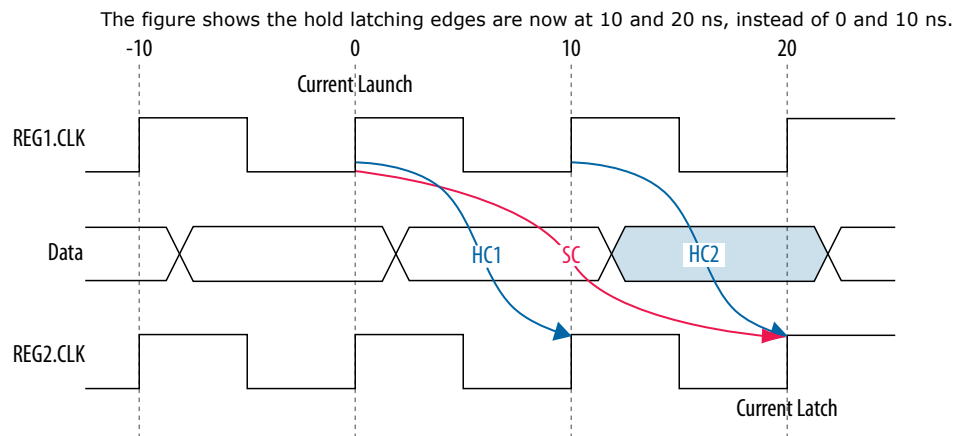


Figure 111. Hold Report with Setup Multicycle Exception

Path #1: Hold slack is -9.770 (VIOLATED)					
Path Summary Statistics Data Path Waveform					
Data Arrival Path					
	Total	Incr	RF	Type	Element
1	0.000	0.000			launch edge time
2	0.000	0.000		borrow	time borrowed
3	1.012	1.012			clock path
1	1.012	1.012	R		clock network delay
2	1.012	0.000			src
4	1.529	0.517			data path
1	1.113	0.101	RR	uTco	src q
2	1.175	0.062	RR	CELL	src~la_lab/laboutb[16]
3	1.529	0.354	RR	IC	dst d
4	1.529	0.000	RR	CELL	dst
Data Required Path					
	Total	Incr	RF	Type	Element
1	10.000	10.000			latch edge time
2	10.000	0.000		borrow	time borrowed
3	11.134	1.134			clock path
1	11.134	1.134	R		clock network delay
4	11.164	0.030			clock uncertainty
5	11.299	0.135		uTh	dst

Figure 112. Hold Check Calculation

hold check 1 = current launch edge – previous latch edge
 = 0 ns – 10 ns
 = -10 ns

hold check 2 = next launch edge – current latch edge
 = 10 ns – 20 ns
 = -10 ns

2.4.4.5.3. End Multicycle Setup = 2 and End Multicycle Hold = 1

In this example, the end multicycle setup assignment value is two, and the end multicycle hold assignment value is one.

Multicycle Constraint

```
set_multicycle_path -from [get_clocks clk_src] -to [get_clocks clk_dst] \
  -setup -end 2
set_multicycle_path -from [get_clocks clk_src] -to
  \[get_clocks clk_dst] -hold -end 1
```

In this example, the setup relationship relaxes by one clock period by moving the latching edge to the right of the default latching edge by 1 clock period. The hold relationship relaxes by one clock period by moving the latch edges to the left of the default latching edges by one.

The following shows the setup timing diagram for the analysis that the Timing Analyzer performs:

Figure 113. Setup Timing Diagram

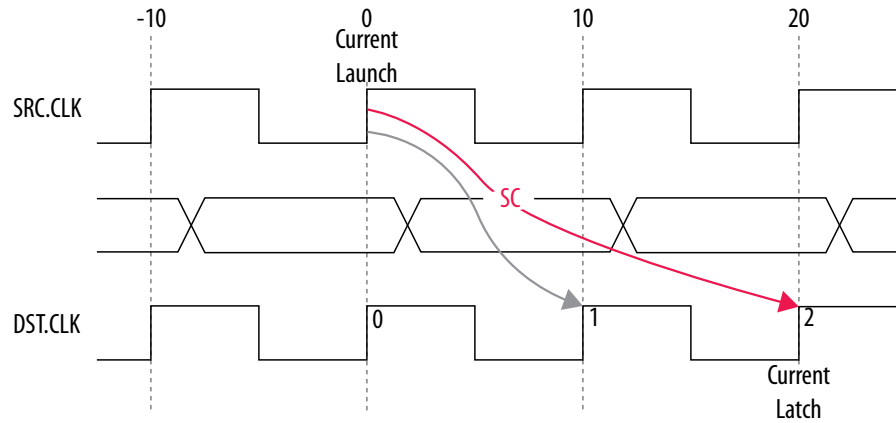


Figure 114. Setup Check Calculation

$$\begin{aligned}
 \text{setup check} &= \text{current latch edge} - \text{closest previous launch edge} \\
 &= 20 \text{ ns} - 0 \text{ ns} \\
 &= 20 \text{ ns}
 \end{aligned}$$

The most restrictive hold relationship with an end multicycle setup assignment value of two is 20 ns.

The following shows the setup report for this example in the Timing Analyzer and highlights the launch and latch edges.

Figure 115. Setup Report with Setup and Hold Multicycle Exception

Path #1: Setup slack is 18.226					
Path Summary Statistics Data Path Waveform					
Data Arrival Path					
	Total	Incr	RF	Type	Element
1	0.000	0.000			launch edge time
2	0.000	0.000		borrow	time borrowed
3	2.603	2.603			clock path
1	2.603	2.603	R		clock network delay
2	2.603	0.000			src
4	3.770	1.167			data path
1	2.832	0.229	RR	uTco	src q
2	2.991	0.159	RR	CELL	src~la_lab/laboutb[16]
3	3.770	0.779	RR	IC	dst d
4	3.770	0.000	RR	CELL	dst
Data Required Path					
	Total	Incr	RF	Type	Element
1	20.000	20.000			latch edge time
2	20.000	0.000		borrow	time borrowed
3	21.983	1.983			clock path
1	21.983	1.983	R		clock network delay
4	21.953	-0.030			clock uncertainty
5	21.996	0.043		uTsu	dst

The following shows the timing diagram for the hold checks for this example. The hold checks are relative to the setup check.

Figure 116. Hold Timing Diagram

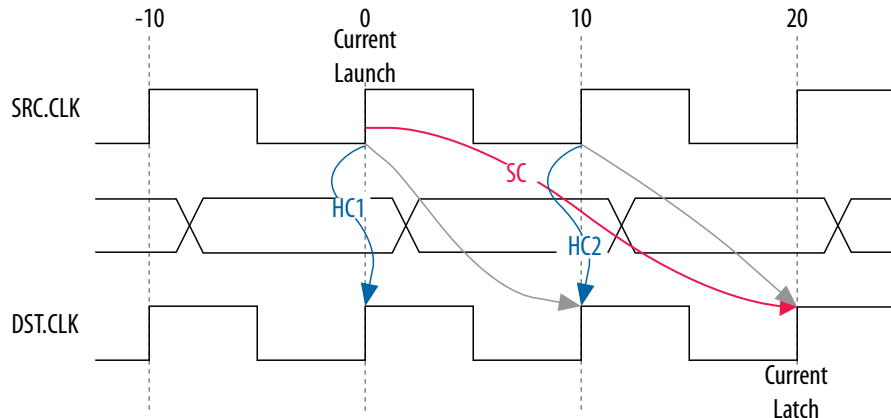


Figure 117. Hold Check Calculation

$$\begin{aligned}
 \text{hold check 1} &= \text{current launch edge} - \text{previous latch edge} \\
 &= 0 \text{ ns} - 0 \text{ ns} \\
 &= 0 \text{ ns} \\
 \\
 \text{hold check 2} &= \text{next launch edge} - \text{current latch edge} \\
 &= 10 \text{ ns} - 10 \text{ ns} \\
 &= 0 \text{ ns}
 \end{aligned}$$

The most restrictive hold relationship with an end multicycle setup assignment value of two and an end multicycle hold assignment value of one is 0 ns.

The following shows the hold report for this example in the Timing Analyzer and highlights the launch and latch edges.

Figure 118. Hold Report with Setup and Hold Multicycle Exception

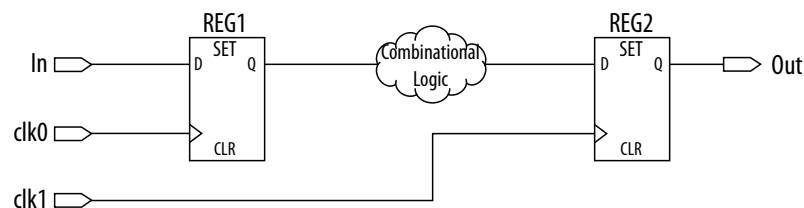
Path #1: Hold slack is 0.230					
Path Summary		Statistics	Data Path	Waveform	
Data Arrival Path					
	Total	Incr	RF	Type	Element
1	0.000	0.000			launch edge time
2	0.000	0.000		borrow	time borrowed
3	▼ 1.012	1.012			clock path
1	1.012	1.012	R		clock network delay
2	1.012	0.000			src
4	▼ 1.529	0.517			data path
1	1.113	0.101	RR	uTco	src[q]
2	1.175	0.062	RR	CELL	src~la_lab/laboutb[16]
3	1.529	0.354	RR	IC	dst[d]
4	1.529	0.000	RR	CELL	dst
Data Required Path					
	Total	Incr	RF	Type	Element
1	0.000	0.000			latch edge time
2	0.000	0.000		borrow	time borrowed
3	▼ 1.134	1.134			clock path
1	1.134	1.134	R		clock network delay
4	1.164	0.030			clock uncertainty
5	1.299	0.135		uTh	dst

2.4.4.5.4. Same Frequency Clocks with Destination Clock Offset

In this example, the source and destination clocks have the same frequency, but the destination clock is offset with a positive phase shift. Both the source and destination clocks have a period of 10 ns. The destination clock has a positive phase shift of 2 ns with respect to the source clock.

The following example shows a design with the same frequency clocks and a destination clock offset.

Figure 119. Same Frequency Clocks with Destination Clock Offset Diagram



The following timing diagram shows the default setup check analysis that the Timing Analyzer performs.

Figure 120. Setup Timing Diagram

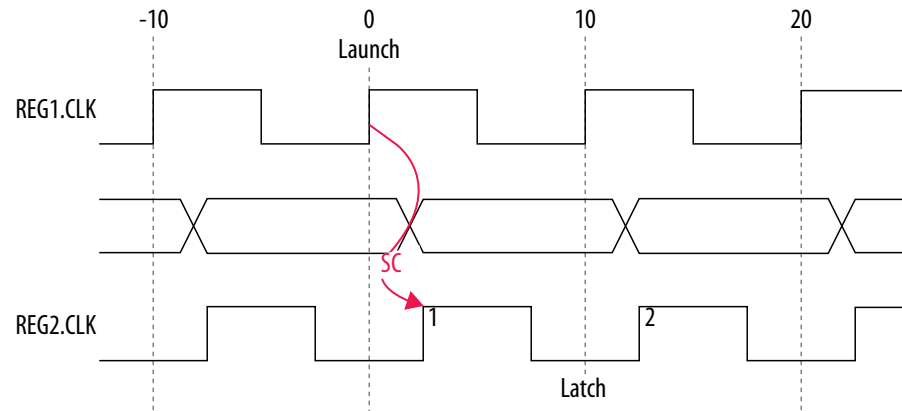


Figure 121. Setup Check Calculation

$$\begin{aligned} \text{setup check} &= \text{current latch edge} - \text{closest previous launch edge} \\ &= 2 \text{ ns} - 0 \text{ ns} \\ &= 2 \text{ ns} \end{aligned}$$

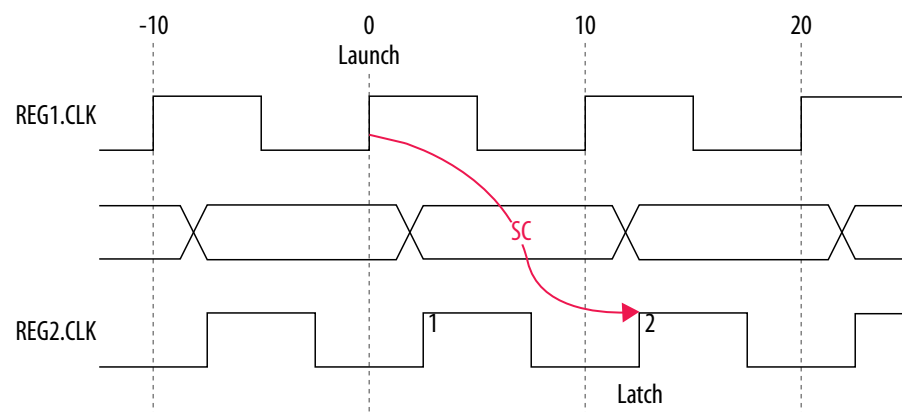
The setup relationship shown is too pessimistic and is not the setup relationship required for typical designs. To adjust the default analysis, you assign an end multicycle setup exception of two. The following shows a multicycle exception that adjusts the default analysis:

Multicycle Constraint

```
set_multicycle_path -from [get_clocks clk_src] -to [get_clocks clk_dst] \
-setup -end 2
```

The following timing diagram shows the preferred setup relationship for this example:

Figure 122. Preferred Setup Relationship



The following timing diagram shows the default hold check analysis that the Timing Analyzer performs with an end multicycle setup value of two.

Figure 123. Default Hold Check

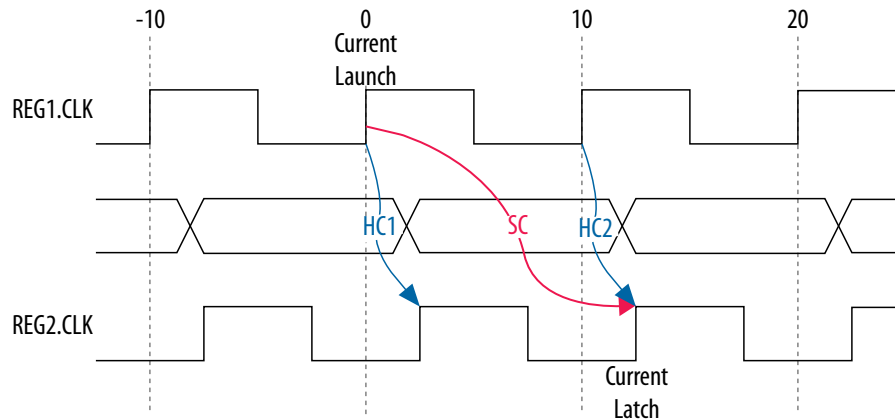


Figure 124. Hold Check Calculation

$$\begin{aligned} \text{hold check 1} &= \text{current launch edge} - \text{previous latch edge} \\ &= 0 \text{ ns} - 2 \text{ ns} \\ &= -2 \text{ ns} \end{aligned}$$

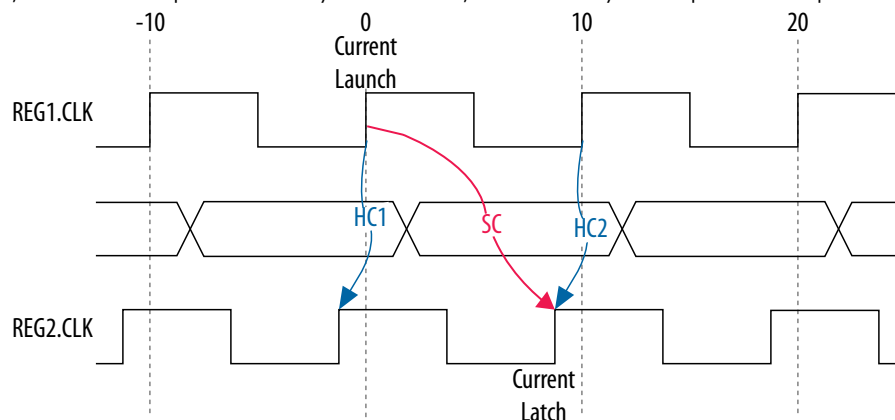
$$\begin{aligned} \text{hold check 2} &= \text{next launch edge} - \text{current latch edge} \\ &= 10 \text{ ns} - 12 \text{ ns} \\ &= -2 \text{ ns} \end{aligned}$$

In this example, the default hold analysis returns the preferred hold requirements and no multicycle hold exceptions are required.

The associated setup and hold analysis if the phase shift is -2 ns. In this example, the default hold analysis is correct for the negative phase shift of 2 ns, and no multicycle exceptions are required.

Figure 125. Negative Phase Shift

The figure shows an example of the setup and hold analysis for a negative phase shift of -2 ns. In this example, the default setup and hold analysis are correct, and no multicycle exceptions are required.

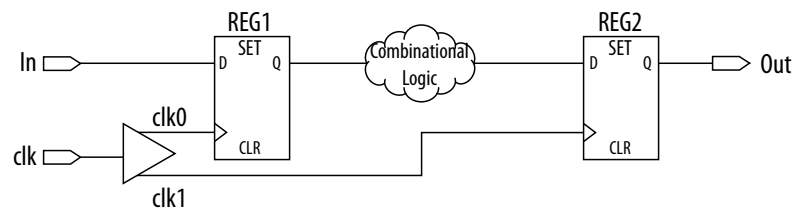


2.4.4.5.5. Destination Clock Frequency is a Multiple of the Source Clock Frequency

In this example, the destination clock frequency value of 5 ns is an integer multiple of the source clock frequency of 10 ns. The destination clock frequency can be an integer multiple of the source clock frequency when a PLL generates both clocks with a phase shift on the destination clock.

The following example shows a design in which the destination clock frequency is a multiple of the source clock frequency.

Figure 126. Destination Clock is Multiple of Source Clock



The following timing diagram shows the default setup check analysis that the Timing Analyzer performs:

Figure 127. Setup Timing Diagram

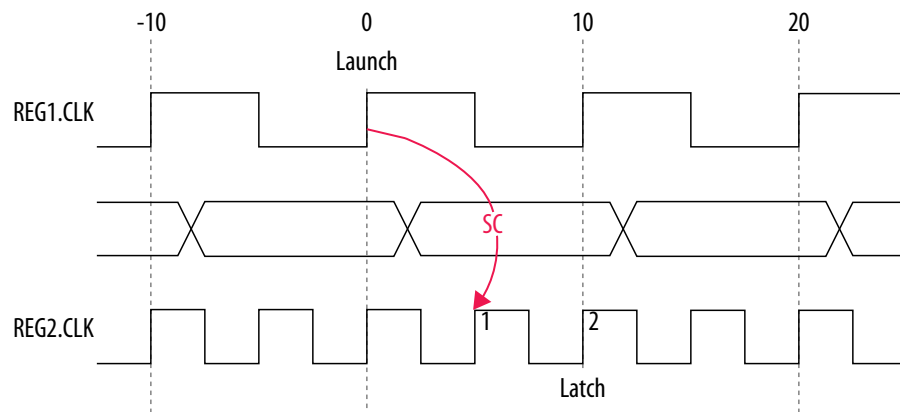


Figure 128. Setup Check Calculation

$$\begin{aligned}
 \text{setup check} &= \text{current latch edge} - \text{closest previous launch edge} \\
 &= 5 \text{ ns} - 0 \text{ ns} \\
 &= 5 \text{ ns}
 \end{aligned}$$

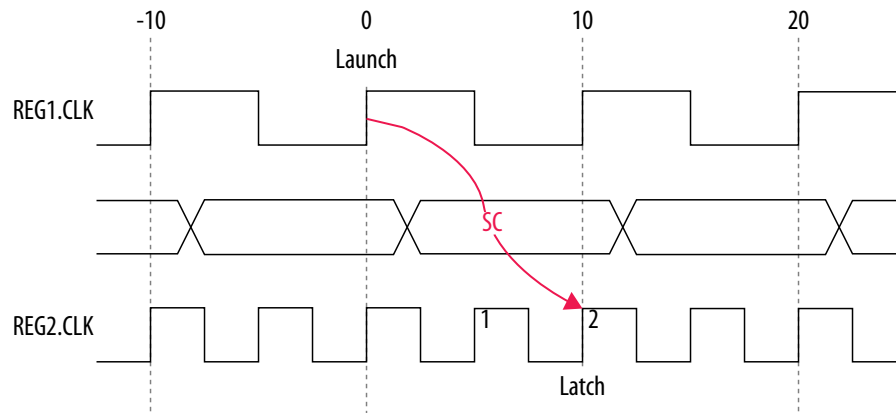
The setup relationship demonstrates that the data requires capture at edge two; therefore, you can relax the setup requirement. To correct the default analysis, you shift the latch edge by one clock period with an end multicycle setup exception of two. The following multicycle exception assignment adjusts the default analysis in this example:

Multicycle Constraint

```
set_multicycle_path -from [get_clocks clk_src] -to [get_clocks clk_dst] \
    -setup -end 2
```

The following timing diagram shows the preferred setup relationship for this example:

Figure 129. Preferred Setup Analysis



The following timing diagram shows the default hold check analysis the Timing Analyzer performs with an end multicycle setup value of two.

Figure 130. Default Hold Check

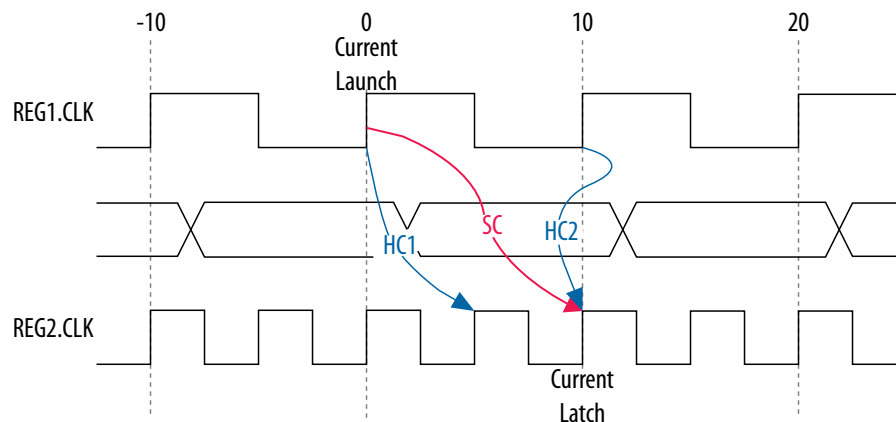


Figure 131. Hold Check Calculation

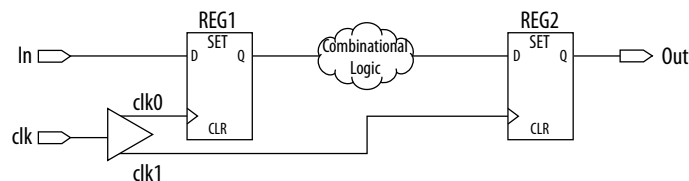
hold check 1	= current launch edge – previous latch edge
	= 0 ns – 5 ns
	= –5 ns
hold check 2	= next launch edge – current latch edge
	= 10 ns – 10 ns
	= 0 ns

In this example, hold check one is too restrictive. The data is launched by the edge at 0 ns and must check against the data captured by the previous latch edge at 0 ns, which does not occur in hold check one. To correct the default analysis, you must use an end multicyle hold exception of one.

2.4.4.5.6. Destination Clock Frequency is a Multiple of the Source Clock Frequency with an Offset

This example is a combination of the previous two examples. The destination clock frequency is an integer multiple of the source clock frequency, and the destination clock has a positive phase shift. The destination clock frequency is 5 ns, and the source clock frequency is 10 ns. The destination clock also has a positive offset of 2 ns with respect to the source clock. The destination clock frequency can be an integer multiple of the source clock frequency. The destination clock frequency can be with an offset when a PLL generates both clocks with a phase shift on the destination clock.

The following example shows a design in which the destination clock frequency is a multiple of the source clock frequency with an offset.

Figure 132. Destination Clock is Multiple of Source Clock with Offset

The timing diagram for the default setup check analysis the Timing Analyzer performs.

Figure 133. Setup Timing Diagram

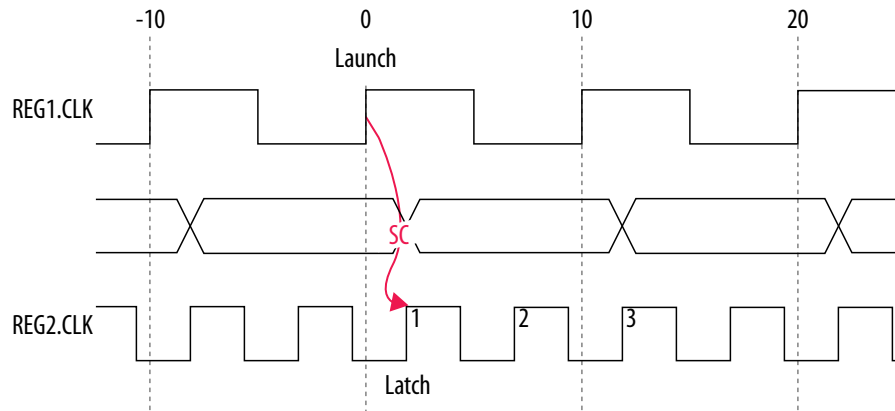


Figure 134. Setup Check Calculation

$$\begin{aligned} \text{setup check} &= \text{current latch edge} - \text{closest previous launch edge} \\ &= 2 \text{ ns} - 0 \text{ ns} \\ &= 2 \text{ ns} \end{aligned}$$

The setup relationship in this example demonstrates that the data does not require capture at edge one, but rather requires capture at edge three; therefore, you can relax the setup requirement. To adjust the default analysis, you shift the latch edge by two clock periods, and specify an end multicycle setup exception of three.

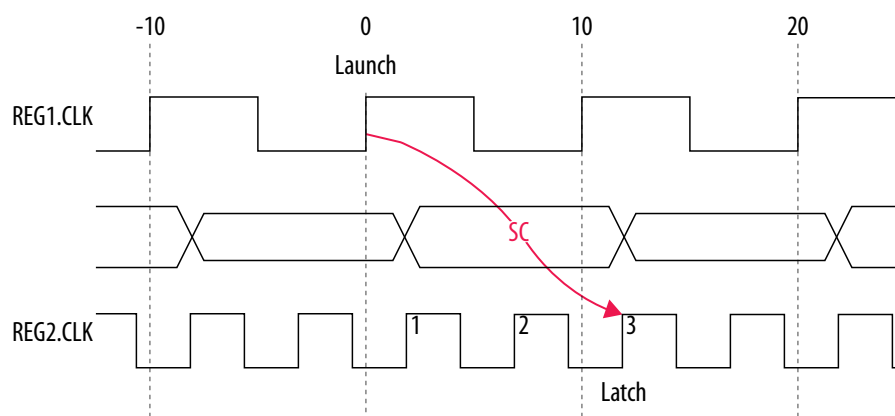
The multicycle exception adjusts the default analysis in this example:

Multicycle Constraint

```
set_multicycle_path -from [get_clocks clk_src] -to [get_clocks clk_dst] \
-setup -end 3
```

The timing diagram for the preferred setup relationship for this example.

Figure 135. Preferred Setup Analysis



The following timing diagram shows the default hold check analysis that the Timing Analyzer performs with an end multicycle setup value of three:

Figure 136. Default Hold Check

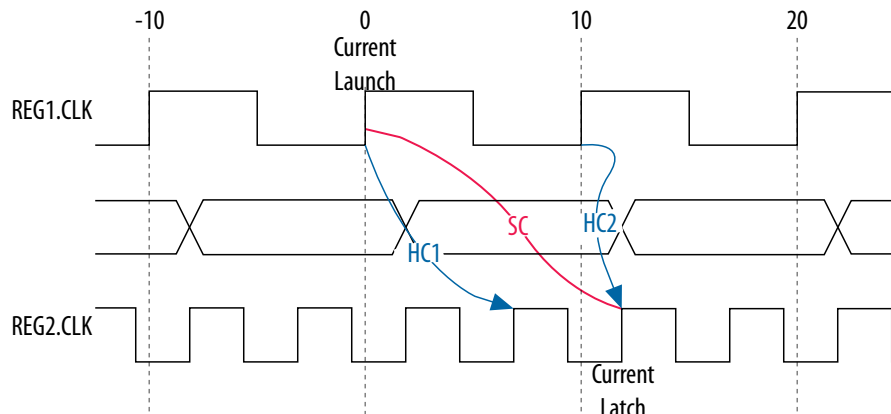


Figure 137. Hold Check Calculation

$$\begin{aligned} \text{hold check 1} &= \text{current launch edge} - \text{previous latch edge} \\ &= 0 \text{ ns} - 7 \text{ ns} \\ &= -7 \text{ ns} \\ \\ \text{hold check 2} &= \text{next launch edge} - \text{current latch edge} \\ &= 10 \text{ ns} - 12 \text{ ns} \\ &= -2 \text{ ns} \end{aligned}$$

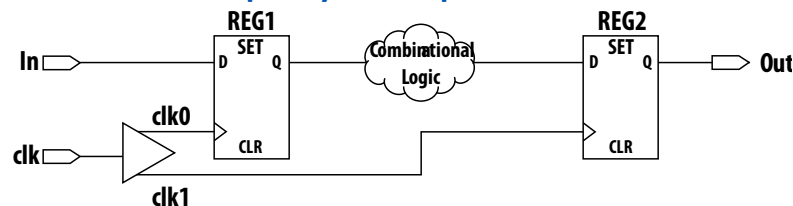
In this example, the hold check one is too restrictive. The data is launched by the edge at 0 ns, and must check against the data that the previous latch edge at 2ns captures. You can use the multicycle hold assignment of 1 to correct this.

2.4.4.5.7. Source Clock Frequency is a Multiple of the Destination Clock Frequency

In this example, the source clock frequency value of 5 ns is an integer multiple of the destination clock frequency of 10 ns. The source clock frequency can be an integer multiple of the destination clock frequency when a PLL generates both clocks and use different multiplication and division factors.

In the following example the source clock frequency is a multiple of the destination clock frequency:

Figure 138. Source Clock Frequency is Multiple of Destination Clock Frequency:



The following timing diagram shows the default setup check analysis the Timing Analyzer performs:

Figure 139. Default Setup Check Analysis

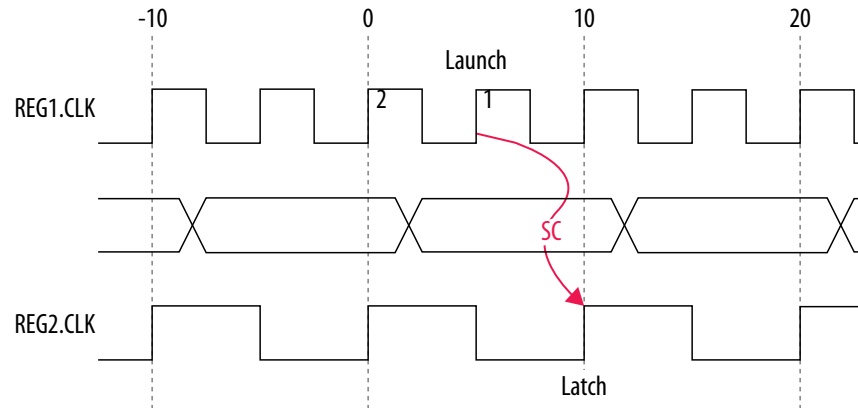


Figure 140. Setup Check Calculation

$$\begin{aligned} \text{setup check} &= \text{current latch edge} - \text{closest previous launch edge} \\ &= 10 \text{ ns} - 5 \text{ ns} \\ &= 5 \text{ ns} \end{aligned}$$

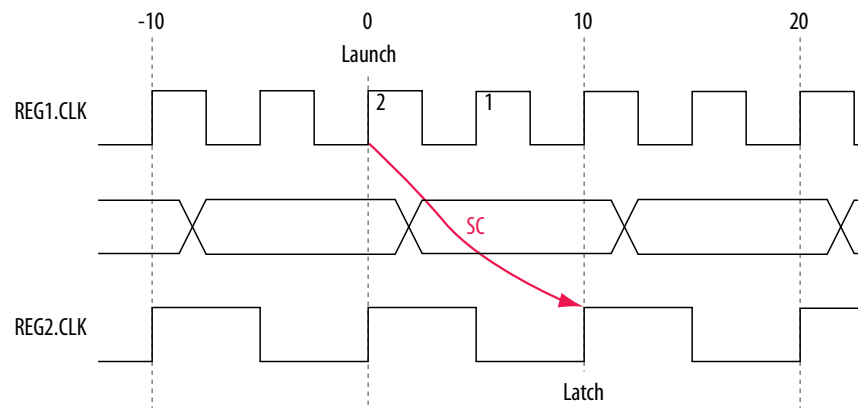
The setup relationship demonstrates that the data launched at edge one does not require capture, and the data launched at edge two requires capture; therefore, you can relax the setup requirement. To correct the default analysis, you shift the launch edge by one clock period with a start multicycle setup exception of two. The following multicycle exception adjusts the default analysis in this example:

Multicycle Constraint

```
set_multicycle_path -from [get_clocks clk_src] -to [get_clocks clk_dst] \
-setup -start 2
```

The following timing diagram shows the preferred setup relationship for this example:

Figure 141. Preferred Setup Check Analysis



The following timing diagram shows the default hold check analysis the Timing Analyzer performs for a start multicycle setup value of two:

Figure 142. Default Hold Check

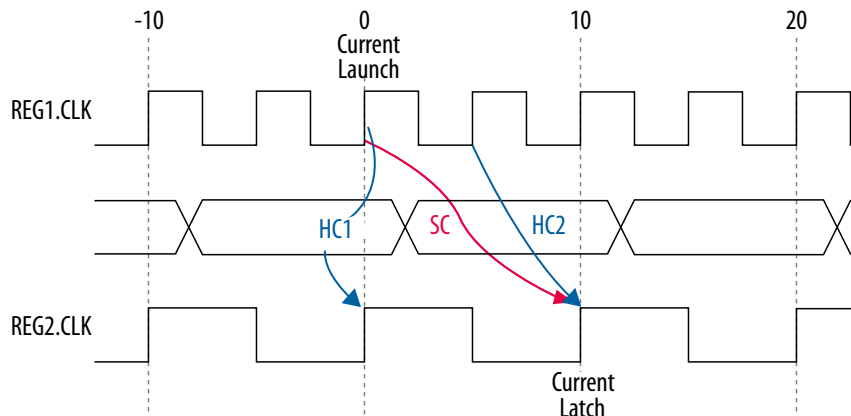


Figure 143. Hold Check Calculation

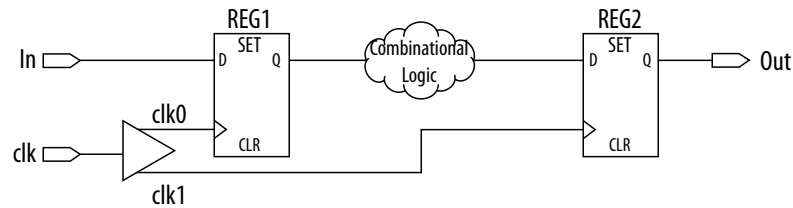
$$\begin{aligned}
 \text{hold check 1} &= \text{current launch edge} - \text{previous latch edge} \\
 &= 0 \text{ ns} - 0 \text{ ns} \\
 &= 0 \text{ ns} \\
 \\
 \text{hold check 2} &= \text{next launch edge} - \text{current latch edge} \\
 &= 5 \text{ ns} - 10 \text{ ns} \\
 &= -5 \text{ ns}
 \end{aligned}$$

In this example, the hold check two is too restrictive. The data is launched next by the edge at 10 ns and must check against the data captured by the current latch edge at 10 ns, which does not occur in hold check two. To correct the default analysis, you use a start multicycle hold exception of one.

2.4.4.5.8. Source Clock Frequency is a Multiple of the Destination Clock Frequency with an Offset

In this example, the source clock frequency is an integer multiple of the destination clock frequency and the destination clock has a positive phase offset. The source clock frequency is 5 ns and destination clock frequency is 10 ns. The destination clock also has a positive offset of 2 ns with respect to the source clock. The source clock frequency can be an integer multiple of the destination clock frequency with an offset when a PLL generates both clocks with different multiplication.

Figure 144. Source Clock Frequency is Multiple of Destination Clock Frequency with Offset



The following timing diagram shows the default setup check analysis the Timing Analyzer performs:

Figure 145. Setup Timing Diagram

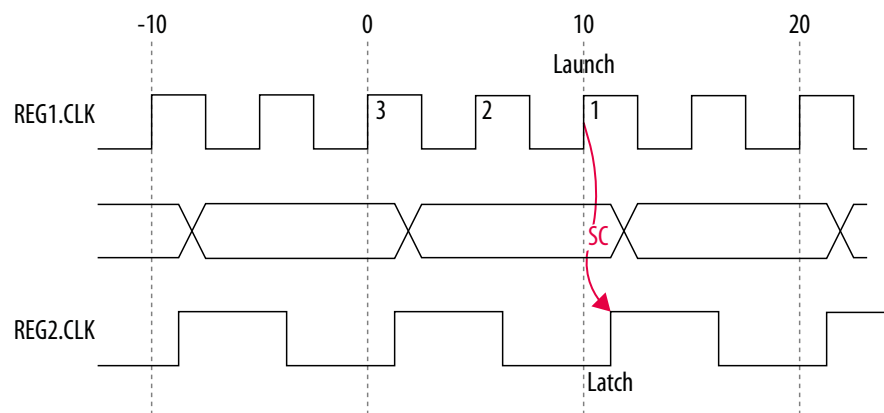


Figure 146. Setup Check Calculation

$$\begin{aligned} \text{setup check} &= \text{current latch edge} - \text{closest previous launch edge} \\ &= 12 \text{ ns} - 10 \text{ ns} \\ &= 2 \text{ ns} \end{aligned}$$

The setup relationship in this example demonstrates that the data is not launched at edge one, and the data that is launched at edge three must be captured; therefore, you can relax the setup requirement. To correct the default analysis, you shift the launch edge by two clock periods with a start multicycle setup exception of three.

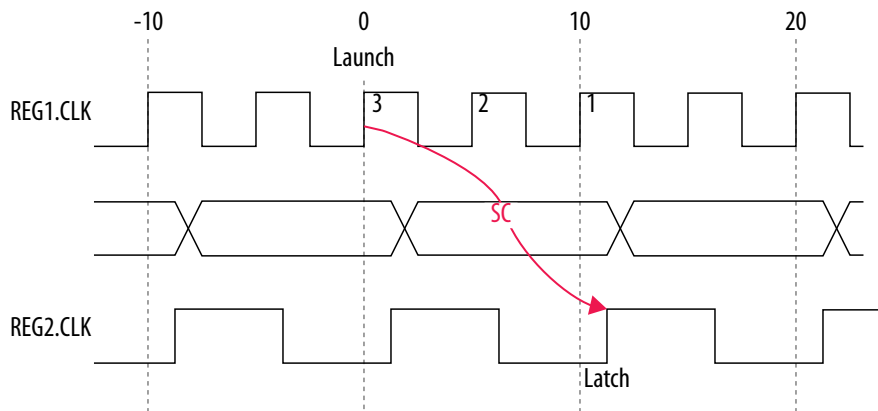
The following multicycle exception adjusts the default analysis in this example:

Multicycle Constraint

```
set_multicycle_path -from [get_clocks clk_src] -to [get_clocks clk_dst] \
-setup -start 3
```

The following timing diagram shows the preferred setup relationship for this example:

Figure 147. Preferred Setup Check Analysis



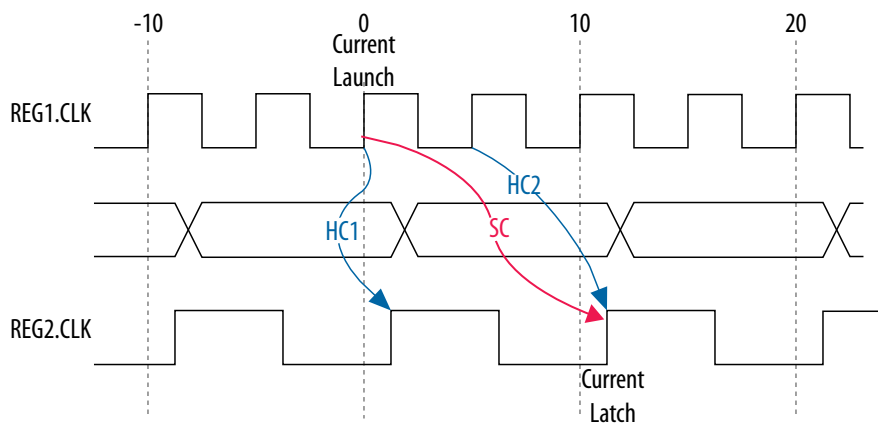
The Timing Analyzer performs the following calculation to determine the hold check:

Figure 148. Hold Check Calculation

$$\begin{aligned}
 \text{hold check 1} &= \text{current launch edge} - \text{previous latch edge} \\
 &= 0 \text{ ns} - 2 \text{ ns} \\
 &= -2 \text{ ns} \\
 \\
 \text{hold check 2} &= \text{next launch edge} - \text{current latch edge} \\
 &= 5 \text{ ns} - 12 \text{ ns} \\
 &= -7 \text{ ns}
 \end{aligned}$$

The following timing diagram shows the default hold check analysis the Timing Analyzer performs for a start multicycle setup value of three:

Figure 149. Default Hold Check Analysis



In this example, the hold check two is too restrictive. The data is launched next by the edge at 10 ns and must check against the data captured by the current latch edge at 12 ns, which does not occur in hold check two. To correct the default analysis, you must specify a multicycle hold exception of one.

2.4.4.6. Creating Efficient Timing Exceptions

When you create efficient timing constraints, the Timing Analyzer takes less time to process the constraints, resulting in shorter compile times. Refer to the following tips to creating efficient timing constraints.

Avoid `-from *` or `-to *` in path-based constraints. If you do not specify `-from` or `-to`, the Timing Analyzer assumes all fan-ins or all fan-outs.

Consider an example where you want to cut all paths to the input of a `sync_reg` synchronizer. The most efficient way is to specify the following:

```
set_false_path -to [get_registers sync_reg]
```

Because the `-from` option is not specified, the Timing Analyzer applies the false path to all paths that fan-in to the register. If you specify the following, the Timing Analyzer gets all registers in the design and checks whether a path exists between each register and `sync_reg`, incurring unnecessary processing time

```
set_false_path -from [get_registers *] -to [get_registers sync_reg]
```

Combine multiple constraint targets as a list argument to a collection filter. All collection commands, such as `get_registers`, support a list of names or wildcards. Instead of creating a separate constraint for each name or wildcard, create fewer constraints by grouping multiple names or wildcards in a list.

Consider an example of a bus synchronizer with a `data[*]` data bus and a `data_valid` data valid signal. If you want to constrain the data delay from both of those elements, you could write two separate data delay constraints:

```
set_data_delay -from [get_registers data[*]] 5
set_data_delay -from [get_registers data_valid] 5
```

However, it is more efficient to combine both signals as a single list argument:

```
set_data_delay -from [get_registers { data[*] data_valid }] 5
```

For small quantities of wildcards or affected names in a design, using separate or combined constraints does not significantly affect the speed at which the constraints are processed. However, if separate filters match hundreds or thousands of names, this requires additional time to combine names.

When you consider whether to specify combined constraint targets, the correctness of the constraints should always be your first priority. When you specify multiple names or wildcards, the constraint applies to all paths between permutations of the names. For example, if the following paths exist in your design (registers `reg_a` and `reg_b` drive both `reg_c` and `reg_d`).

```
reg_a -> reg_c
reg_a -> reg_d
reg_b -> reg_c
reg_b -> reg_d
```

Then the following sets of constraints have different functionality:

Example 1

```
set_multicycle_path -from [get_registers reg_a] -to [get_registers reg_c]  
set_multicycle_path -from [get_registers reg_b] -to [get_registers reg_d]
```

The two constraints in Example 1 affect only two paths:

- reg_a -> reg_c
- reg_b -> reg_d

Example 2

```
set_multicycle_path -from [get_registers { reg_a reg_b }] -to [get_registers  
{ reg_c reg_d }]
```

The one constraint in Example 2 affects four paths:

- reg_a -> reg_c
- reg_a -> reg_d
- reg_b -> reg_c
- reg_b -> reg_d

2.4.5. Delay Annotation

To modify the default delay values used during timing analysis, use the `set_annotated_delay` and `set_timing_derate` commands. You must update the timing netlist to apply these commands.

To specify different operating conditions in a single `.sdc` file, rather than having multiple `.sdc` files that specify different operating conditions, use the `set_annotated_delay -operating_conditions` command.

Related Information

- [set_timing_derate Command, Quartus Prime Help](#)
- [set_annotated_delay Command, Quartus Prime Help](#)

2.5. Timing Report Descriptions

The Timing Analyzer generates only a subset of all available reports by default, including the Setup Summary and Timing Analyzer Summary reports. However, you can generate dozens of other detailed reports in the Timing Analyzer GUI, or with command-line commands to help pin-point timing issues. You can customize the display of information in the reports.

You can automatically generate reports that are useful to you, to display after opening a project in the stand-alone Timing Analyzer GUI, as [The quartus_staw executable](#) describes.

The following section describes a partial list of all the timing analysis reports that you can generate:

- [Report Fmax Summary](#) on page 133
- [Report Timing](#) on page 134
- [Report Timing By Source Files](#) on page 140
- [Report Data Delay](#) on page 141
- [Report Net Delay](#) on page 141
- [Report Clocks and Clock Network](#) on page 141
- [Report Clock Transfers](#) on page 144
- [Report Metastability](#) on page 145
- [Report CDC Viewer](#) on page 146
- [Report Asynchronous CDC](#) on page 148
- [Report Logic Depth](#) on page 152
- [Report Neighbor Paths](#) on page 154
- [Report Register Spread](#) on page 155
- [Report Route Net of Interest](#) on page 160
- [Report Retiming Restrictions](#) on page 162
- [Report Register Statistics](#) on page 162
- [Report Pipelining Information](#) on page 164
- [Report Time Borrowing Data](#) on page 167
- [Report Exceptions and Exceptions Reachability](#) on page 168
- [Report Bottlenecks](#) on page 169
- [Check Timing](#) on page 170
- [Report SDC](#) on page 173
- [Design Closure Summary](#) on page 173

Related Information

- [Step 4: Analyze Timing Reports](#) on page 40

2.5.1. Report Fmax Summary

The Timing Analyzer **Reports > Datasheet > Report Fmax Summary** command generates a report panel showing the potential maximum frequency for every clock in your design. The **Fmax** column reports the fastest frequency that your clock can run, and still pass `report_timing -setup -intra_clock` with a slack of 0. The equivalent console command is `report_clock_fmax_summary`.

Figure 150. Fmax Summary Report

	Fmax	Restricted Fmax	Clock Name	Note	Worst-Case Operating Conditions
1	180.08 MHz	180.08 MHz	clk		Slow 900mV 100C Model

Note: The related `get_clock_fmax_info` command returns a Tcl list, which is useful for scripting and parsing. Refer to `get_clock_fmax_info (::quartus::sta)` in *Quartus Prime Pro Edition User Guide Scripting*.

Timing analysis only computes f_{MAX} for paths where the source and destination registers or ports are driven by the same clock. Timing analysis ignore paths of different clocks, including generated clocks. For paths between a clock and its inversion, the Timing Analyzer computes f_{MAX} as if the rising and falling edges of the clock scale along with f_{MAX} , such that duty cycle (by percentage) is maintained.

However, the **Fmax** report does not indicate whether your design meets timing for recovery, removal, nor setup or hold without the `intra_clock` option. For these reasons, always make sure to view the Setup, Hold, Recovery, Removal, and Min Pulse Width slack summaries to determine whether your design meets timing.

The **Restricted Fmax** column reports the lesser of the following values:

- The fastest frequency that your clock can run, and still pass `report_timing -hold -intra_clock` with a slack of 0 or `report_min_pulse_width` with a slack of 0.
- The **Fmax** column value.⁽⁵⁾

Restricted Fmax considers hold timing in addition to setup timing, as well as minimum pulse and minimum period restrictions. Similar to unrestricted f_{MAX} , the analysis computes the restricted f_{MAX} as if the rising and falling edges of the clock scale along with f_{MAX} , such that the duty cycle (in terms of a percentage) is maintained.

The **Restricted Fmax** may display text indicating any of the following limiting factors:

- Limit due to hold check
- Limit due to minimum pulse width restriction
- Limit due to high minimum pulse width restriction
- Limit due to low minimum pulse width restriction

⁽⁵⁾ The **Restricted Fmax** column never reports a value higher than the **Fmax** column.

Typically, hold checks do not limit the maximum frequency (f_{MAX}) because the checks are for same-edge relationships, and therefore independent of clock frequency. For example, when launch equals zero and latch equals zero. However, with an inverted clock transfer, or a multicycle transfer, the hold relationship is not a same-edge transfer and changes with the clock frequency.

Refer to the timing reports, such as those that you can generate using `report_timing`, or using minimum pulse width reports via the `report_min_pulse_width` command for details of specific paths, registers, or ports.

Related Information

[Quartus Prime Pro Edition User Guide: Scripting](#)

2.5.2. Report Timing

The Timing Analyzer's **Reports > Timing Slack > Report Timing...** command allows you to specify settings to report the timing of any path or clock domain in the design. The equivalent scripting command is `report_timing`.

Figure 151. Report Timing Report

Report Timing						
Command Info		Summary of Paths				
	Slack ▾	From Node	To Node	Launch Clock	Latch Clock	Relationship
1	39.103	addr...g[2]	my_...[2]	clk	clk	40.000
2	39.144	addr...g[2]	my_...[2]	clk	clk	40.000
3	39.153	addr...g[2]	my_...[2]	clk	clk	40.000
◀						
Path #1: Setup slack is 39.103						
Path Summary		Statistics	Data Path	Waveform		
	Property			Value		
1	From Node			addressr_reg[2]		
2	To Node			my_mlab_inst0 radd_reg_b[2]		
3	Launch Clock			clk		
4	Latch Clock			clk		
5	Data Arrival Time			3.055		
6	Data Required Time			42.158		
7	Slack			39.103		
8	Worst-Case Operating Conditions			Slow 900mV 100C Model		

You can specify diverse options to customize the reporting. You can specify the **Clocks** and **Targets** that the report displays, the **Analysis Type** to run, whether to display **Extra Info** in the report, and the **Output** options for the report. For example, you can increase the number of paths to report, add a **Target** filter, and add a **From Clock**.

Figure 152. Report Timing Dialog Box (Top Section)

Report Timing

Clocks

From clock:

To clock:

Targets

From: ...

Through: ...

To: ...

Analysis type

☒ Setup

☐ Hold

☐ Recovery

☐ Removal

Paths

Report number of paths:

Maximum number of paths per endpoint:

Maximum slack limit: ns

☐ Pairs only

Figure 153. Report Timing Dialog Box (Bottom Section)

Extra Info

☒ None ☐ Basic ☐ All

Output

Detail level:

☐ Show routing

☒ Report panel name:

☐ Split the report by operating conditions

☐ File name:

File options

☒ Overwrite ☐ Append

☐ Console

Tcl command:

Table 24. Report Timing Settings

Option	Description
Clocks	From Clock and To Clock filter paths in the report to show only the launching or latching clocks you specify.
Targets	Specifies the target node for From Clock and To Clock to report paths with only those endpoints. Specify an I/O or register name or I/O port for this option. The field also supports wildcard characters. For example, to report only paths within a specific hierarchy: <pre>report_timing -from * egress:egress_inst * \ -to * egress:egress_inst * -(other options)</pre> When the From, To, or Through boxes are empty, the Timing Analyzer assumes all possible targets in the device. The Through option limits the report to paths that pass through combinatorial logic, or a particular pin on a cell.
Analysis type	The Analysis type options are Setup , Hold , Recovery , or Removal . The Timing Analyzer reports the results for the type of analysis you select.
Paths	Specifies the number of paths to display by endpoint and slack level. The default value for Report number of paths is 10, otherwise, the report can be very long. Enable Pairs only to list only one path for each pair of source and destination. Limit further with Maximum number of paths per endpoints . You can also filter paths by entering a value in the Maximum slack limit field.
Extra Info	Provides additional data that is relevant for diagnosing timing failure root cause, such as setup slack breakdown, and unexpected routing detours caused by congestion and hold time fix-up. Specify whether to include None, Basic, or All extra information in the report. The Extra Info tab data can help you identify potential, unnecessary routing detours, as well as placement or circuit issues that restrict the path f_{MAX} performance. Refer to Setup Slack Breakdown On the Extra Info Tab on page 137. All—report includes Extra Info tab that reports extra information for source timing endpoints that pass through the unregistered output of a RAM or DSP block, or for destination timing endpoints that pass through the unregistered input of a DSP block. The Data Path tab includes Estimated Delay Added for Hold and Route Stage Congestion Impact data. Basic—report includes the Extra Info tab but no extra information on the Data Path tab. None—report includes no Extra Info tab or other extra information on the Data Path tab.
Output	Specify the path types the analysis includes in output for Detail level: Summary—level includes basic summary reports. Review the Clock Skew column in the Summary report. If the skew is less than +/-150ps, the clock tree is well balanced between source and destination. Path only—displays all the detailed information, except the Data Path tab displays the clock tree as one line item. Path and Clock—displays the same as Path only with respect to the clock. Full path—when higher clock skew is present, enable the Full path option. This option breaks the clock tree into greater detail, showing every cell, including the input buffer, PLL, global buffer (called CLKCTRL), and any logic. Review this data to determine the cause of clock skew in your design. Use the Full path option for I/O analysis because only the source clock or destination clock is inside the FPGA, and therefore the delay is a critical factor to meet timing.
Show routing	Shows routing data in the report.
Split the report by operating conditions	For the operating condition timing corners, subdivides the data by each operating condition.
Report panel name	Specifies the name of the report panel. You can optionally enable File name to write the information to a file. If you append .htm or .html as a suffix, the Timing Analyzer produces the report as HTML. If you enable File name , you can Overwrite or Append the file with latest data.
Tcl command	Displays the Tcl syntax that corresponds with the GUI options you select. You can copy the command from the Console into a Tcl file.

Figure 154. Extra Info Tab

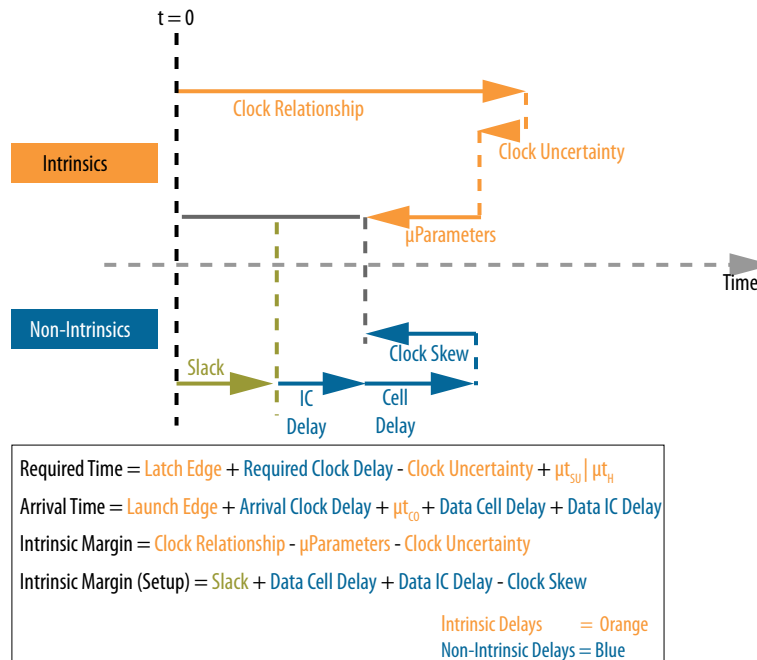
Path Summary	Statistics	Data Path	Extra Info	W
	Property		Value	
1	▾ Setup Slack Breakdown [=A+B-C-D]		9.259	
1	▾ [A] Intrinsic Margin [=a+b-c-d]		9.852	
1	▸ [a] Clock Edge...nship [=aa-bb]		10.000	
2	[b] Clock Uncertainty		-0.030	
3	[c] uTco		0.212	
4	[d] uTsu		-0.094	
2	[B] Clock Skew		-0.001	
3	[C] Cell Delay		0.589	
4	[D] Interconnect Delay		0.003	
2	▾ From Node Info			
1	Type		ALM Register	
2	Retiming Restriction		--	
3	Power-Up "Don't Care"		Yes	
3	▸ To Node Info			
4	▾ Interconnect Info			
1	Max Fanout		4	
2	Route Stage Congestion Impact		--	
3	Estimated Delay Added for Hold		0.000	
4	Sufficient Setup Margin for Hold		Yes	
5	▾ Bounding Box Info			
1	Source/Destination Bounding Box		--	
2	Source/Destination Area Covered		0	
3	Source/Destination Relative Area		0.000	
4	Cell Bounding Box		--	
5	Cell Area Covered		0	
6	Cell Relative Area		0.000	

Setup Slack Breakdown On the Extra Info Tab

The **Extra Info** tab contains other timing metrics to help you diagnose timing closure issues, including **Setup Slack Breakdown** for the path.

The slack of a path specifies the margin by which the path meets its timing requirement. The setup slack breakdown is a numeric value that the Timing Analyzer calculates from the following timing requirements and path element delays:

Figure 155. Setup Slack Breakdown Calculations



A path can fail timing requirements for many varied reasons. For example, the clock relationship can be impossibly tight, or there can be excessive routing delays that alone cause failure for the timing path. Calculating the intrinsic margin of a timing path, and then comparing that margin to other delays of the path, can help identify the specific reasons why a path fails its timing requirement.

The **Extra Info** tab can help you identify potential significant or unexpected routing detours caused by congestion and hold time fix-up. The **Extra Info** tab can also report extra information for source timing endpoints that pass through the unregistered output of a RAM or DSP block, or for destination timing endpoints that pass through the unregistered input of a DSP block.

You can review the **Extra Info** data and **Locate Path** or **Locate Chip Area** in Chip Planner, Technology Map Viewer, or Resource Property Viewer to determine whether to make changes to improve placement and routing.

Some delay elements are more sensitive to a path's placement and routing than others. Intrinsic delays that are part of **Setup Slack Breakdown** are less sensitive to placement and routing, and are inherent in the RTL and timing requirements. Non-intrinsic delays are the other delays that are sensitive to placement and routing.

Table 25. Extra Info Tab Data

Extra Info Data	Description
Intrinsic Margin	Reports the intrinsic and non-intrinsic timing elements that comprise the timing path slack value. Intrinsic margin is a numeric value that the Timing Analyzer calculates from the timing requirements and path element delays. The Timing Analyzer also derives the slack of the path from the same requirements and delays, but with a different calculation. Intrinsic delays are less sensitive to placement and routing, and are inherent in the RTL and timing requirements. Non-intrinsic delays are the other delays that are sensitive to placement and routing.
From Node Info	Specifies the node Type, any Retiming Restriction, and any Power-Up "Don't Care" attributes for the From Node. Consider removing the retiming restriction to allow retiming and improve performance for timing closure.
To Node Info	Specifies the node Type, any Retiming Restriction, and any Power-Up "Don't Care" attributes for the To Node. Consider removing the retiming restriction to allow retiming and improve performance for timing closure.
Max Fanout	Reports the maximum fan-out of register and combinational nodes in the path.
Route Stage Congestion Impact	Reports whether routing has a Low , Medium , or High impact on congestion. A Low value suggests timing issues are not congestion related. A High value suggests competition for scarce routing resources plays a role in poor timing.
Estimated Delay Added for Hold	Reports the estimated amount of delay added on to the fastest delay route to satisfy hold. This value can help you determine whether delays are routing congestion or Hold related.
Sufficient Setup Margin for Hold	Reports whether the setup margin is suitable for the hold timing. Yes , indicates that the setup margin is sufficient. No indicates that the setup margin is insufficient for hold timing.
Source/Destination Bounding Box	Reports the lower-left and upper-right coordinates for the boundary box enclosing the source and destination registers. In an ideal case, the Source/Destination Bounding Box , Cell Bounding Box , and Interconnect Bounding Box values are roughly the same, and the relative areas are approximately 1.0. If the cell bounding box size grows relative to the Source/Destination Bounding Box , that can indicate a potential unnecessary routing detour on the path.
Source/Destination Area Covered	Reports the total area covered in terms of LABs.
Source/Destination Relative Area	Reports the area for the source and destination, relative to the Source/Destination Bounding Box . The value is always 1.0, which equals the same size.
Cell Bounding Box	Reports the lower-left and upper-right coordinates for the boundary box enclosing the source and destination registers, and any cells in the path.
Cell Area Covered	Reports the area for the cell, relative to the Source/Destination Bounding Box . A value of 1.0 equals the same size. A value greater than 1.0 can indicate a path has a cell outside of the space between the registers in the path.

The following describe the interpretation of timing conditions indicated by the **Setup Slack Breakdown**:

- **When the Setup Slack Breakdown is less than 0**—the path has such a tight timing relationship, a significant difference in microparameters, or such significant clock source uncertainty, that the path fails before the addition of any delay. Review the SDC constraints to verify that the timing relationship is correct. An incorrect relationship can exist between unrelated clocks that lack the proper timing cut. Ensure that parameterizable hard blocks (such as 20K RAM and DSP blocks) are fully registered. Investigate clock sources to verify that the clocks use global signals for routing.
- **When the clock skew exceeds the Setup Slack Breakdown**—address the clock transfer to meet timing on the path. You may need to create clock region assignments. You might also need to redesign cross-clock transfers to switch from synchronous to asynchronous implementation, such as with a FIFO or other handshake.
- **When the cell delay is greater than its intrinsic margin**—reduce the cell delay, as the path would fail timing even if the clocks are perfect and use no routing wires. Rewrite RTL to reduce the logic depth, restructure logic to allow the Compiler to use faster LUT inputs, or unblock retiming optimizations. The Compiler can automatically retime registers to reduce logic depth, but only in ways that maintain functionality and that the device architecture supports. To unblock the Hyper-Rtimer, remove asynchronous resets and initial conditions.
- **When the interconnect delay is greater than its intrinsic margin**—the path would fail timing even if the clocks are perfect, and there is no logic. This occurs if registers are too far apart, or a timing path detours around a congested chip area. Review the fan-in and fan-out of registers that are far apart. Apply Logic Lock regions so the Fitter places the registers closer together. Use Logic Lock regions only after determining why placement is initially poor.

Related Information

[Hyperflex Architecture High-Performance Design Handbook](#)

2.5.3. Report Timing By Source Files

The Timing Analyzer's **Reports > Timing Slack > Report Timing By Source Files** command allows you to specify settings to report the timing of any path or clock domain in the design, similar to [Report Timing](#) on page 134. However, **Report Timing By Source Files** groups the timing paths by the containing entity, and groups the entities by the source file that defines the entity.

This report allows you to attribute timing paths to exactly one instance in the design. The Path TNS column shows the sum of all negative slacks within a file or entity. The equivalent scripting command is `report_timing_by_source_files`.

From the report in the Timing Analyzer GUI, you can right-click on any source file to open it in a text editor to easily investigate timing paths within a particular source file.

2.5.4. Report Data Delay

You can run the Timing Analyzer's **Reports > Other Slack Analyses > Report Data Delay...** command to generate a custom report showing the worst-case slack for the datapath delay exception for a given path. The report shows only paths covered by data delay (`set_data_delay`) constraints, including paths whose non-datapath analysis is cut by a false path.

Note: Exclusive clock groups (set with `set_clock_groups -exclusive`) override `set_data_delays` constraints.

Related Information

[Timing Exception Precedence](#) on page 102

2.5.5. Report Net Delay

The Timing Analyzer's **Reports > Other Slack Analyses > Report Net Delay...** command to configure and display a customized report that details the results of net delay analysis, as defined by timing constraints and exceptions.

Each `set_net_delay` command is treated as a separate analysis and `report_net_delay` reports the results of all `set_net_delay` commands in a single report. The report contains each `set_net_delay` command with the worst case slack result followed by the results of each edge matching the criteria set by that `set_net_delay` command. These results are ordered based on the slack value.

2.5.6. Report Clocks and Clock Network

The Timing Analyzer's **Reports > Clocks > Report Clocks** command reports all clock signals in the design. The equivalent scripting command is `report_clocks`.

Report Clocks generates the Clock Summary report that lists details about all of the signals with clock setting constraints in the design.

Figure 156. Clock Summary Report Shows Properties of Clock Signals in Design

Clocks Summary											
Show:	Visible ▾	Hide	Q <<Filter>>								
	Clock Name	Type	Period	Frequency	Rise	Fall	Duty Cycle	Divide by	Multiply by	Phase	Offset
1	clk	Base	40.000	25.0 MHz	0.000	20.000					

Similarly, you can click the **Reports > Clocks > Report Clock Network...** command to generate a custom report that helps you identify and evaluate advanced clock structures, such as clock muxes, clock gates, and clock dividers.

Figure 157. Report Clock Network Dialog Box

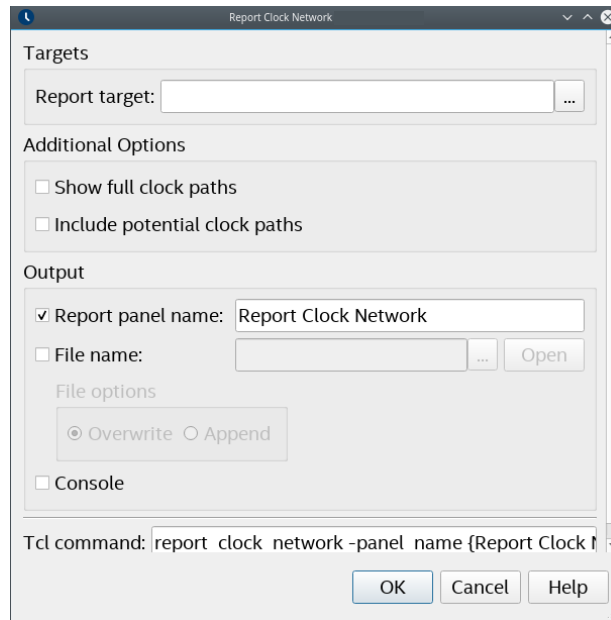
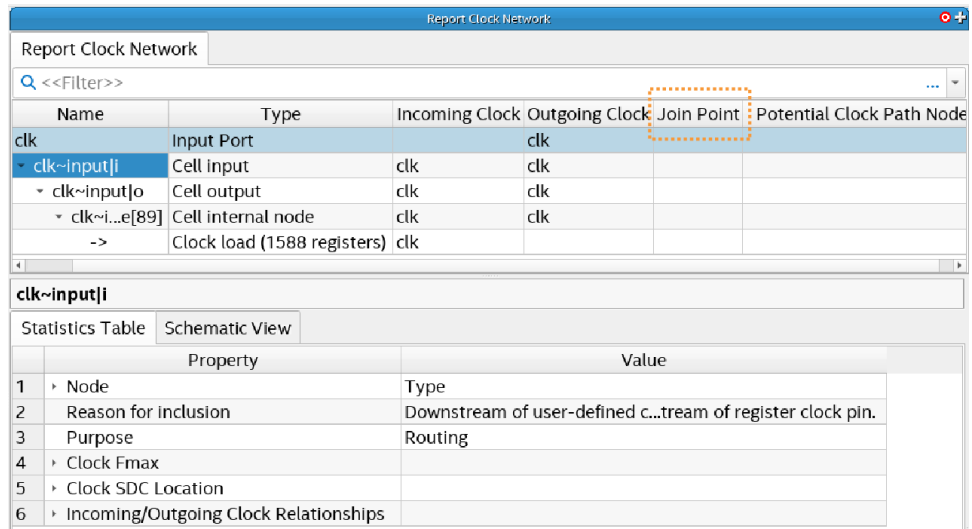


Table 26. Report Clock Network Dialog Box Settings

Option	Description
Report target	Specifies the collection of clocks and nodes that you want to analyze and report.
Expand clock path	Displays all subordinate nodes in expanded view. The default display collapses trivial nodes in the report.
Include potential clock paths	Includes nodes in the report that are not on a clock path, but are upstream of a register clock port.
Report panel name	Specifies the name that appears in the report panel title bar.
File name	Specifies the name of an optional output file to contain report data.
Tcl command	Displays the Tcl syntax that corresponds with the GUI options you select. You can copy the command from the Console into a Tcl file.

This report shows the nodes on the clock network hierarchically, starting from the input clock ports, followed by any other nodes that transform or route the clocks to the clock loads. The **Join Points** indicate whether the clock network has convergence, such as with clock muxes. The **Statistics Table** provides more details about the signals that you select in the report, such as the relationships between the incoming and outgoing clocks of this node.

Figure 158. Report Clock Network Report



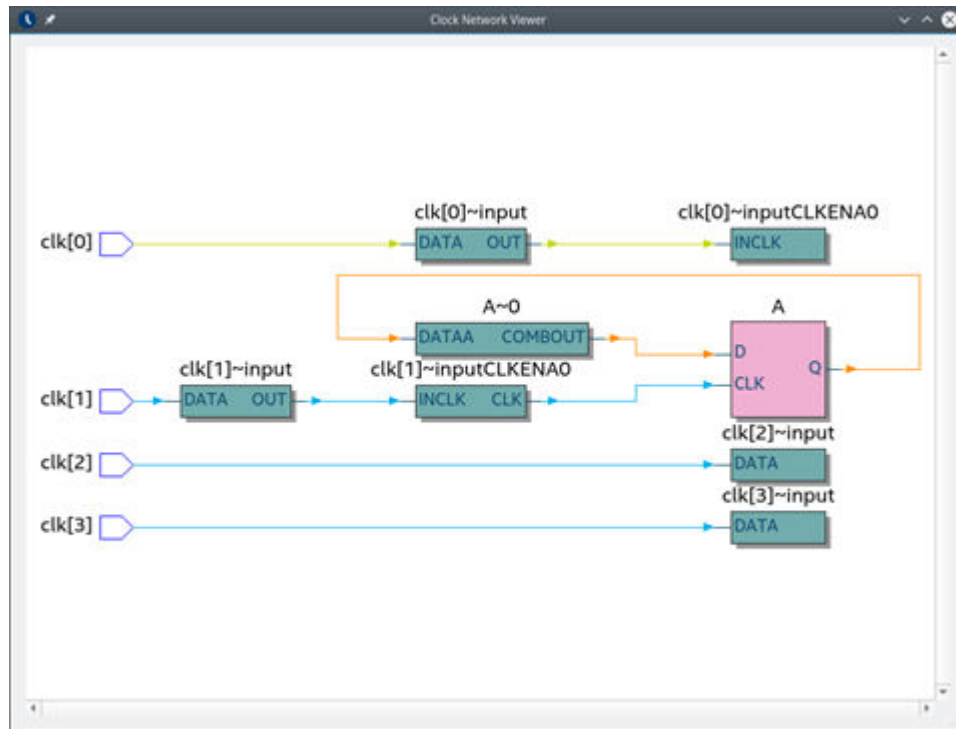
Name	Type	Incoming Clock	Outgoing Clock	Join Point	Potential Clock Path Node
clk	Input Port		clk		
clk~input i	Cell Input	clk	clk		
clk~input o	Cell output	clk	clk		
clk~i...e[89]	Cell internal node	clk	clk		
->	Clock load (1588 registers)	clk			

Property		Value
1	Node	Type
2	Reason for inclusion	Downstream of user-defined c...ream of register clock pin.
3	Purpose	Routing
4	Clock Fmax	
5	Clock SDC Location	
6	Incoming/Outgoing Clock Relationships	

The **Schematic View** tab displays a graphical representation of the clock domains. Use this graphical view to determine which clocks drive portions of the design. The color for each clock connection edge indicates the clock types. To identify the clock types for each color, right click on any signal to display the **Color Legend**.

The display shows a truncated signal name by default. Hover the mouse over the clock signals to display the full signal name. Click the **Zoom** controls to view more detail. You can export the schematic view as a PDF from the right-click menu.

Figure 159. Clock Network Viewer



You can also right-click and choose **Report Paths from Node**, **Report Paths Thru Node**, **Report Paths To Node**, or **Focus On Node** to rerun the report on the selected node.

2.5.7. Report Clock Transfers

The Timing Analyzer's **Reports > Clock Domain Crossings > Report Clock Transfers** command reports all clock-to-clock transfers in the design. The equivalent scripting command is `report_clock_transfers`.

Report Clock Transfers generates the Setup Transfers report and the Hold Transfers report that display data about the clock-to-clock transfers.

Figure 160. Setup Transfers Report Shows Clock-to-Clock Transfers

Setup Transfers								
Show:	Visible	Hide	Q <<Filter>>					
	From Clock	To Clock	RR Paths	FR Paths	RF Paths	FF Paths	Clock Pair Classification	Worst-Case Slack
1	clock	clock	34	0	0	0	Intra-Clock (Timed Safe)	3.319
							Worst-Case Operating Conditions	
							Slow vid2 100C Model	

The Setup Transfers report and Hold Transfers report display all possible transfers, including rising clock edge to rising clock edge (RR), falling clock edge to rising clock edge (FR), rising clock edge to falling clock edge (RF), and falling clock edge to falling clock edge (FF) paths.

- If a path exists in the design, the report column cell is white and lists the number of paths.
- If a path is a false path, the report column cell is light gray and contains the text "false path."
- If a path does not exist in the design, then the report column cell is dark gray.

The Setup Transfers report and Hold Transfers report also lists the Worst-Case Slack for setup, the Worst-Case Operating Conditions, and the Clock Pair Classification for each clock path. The Clock Pair Classification includes the following:

Table 27. Clock Pair Classifications

Clock Pair Classification	Definition
Intra-Clock (Timed Safe)	• From Clock and To Clock are the same. • No timing constraint required.
Inter-Clock Synchronous (Timed Safe)	• From Clock and To Clock relate synchronously, and have a known phase and frequency relationship. • Multicycle path constraint may or may not exist.
Asynchronous (Timed Unsafe)	• From Clock and To Clock are asynchronous. • Timing constraints (false path, clock groups, set_max_skew) do not exist.
Ignored (Not Timed)	• From Clock and To Clock are asynchronous. • Timing constraints (false path, clock groups, set_max_skew) exist and setup and hold slack are not applicable.

2.5.8. Report Metastability

The Timing Analyzer's **Reports > Clock Domain Crossings > Report Metastability...** command generates a list of synchronization register chains found in the design, and can provide estimates of the Mean Time Between Failures (MTBF) of each chain. The equivalent scripting command is `report_metastability`.

Metastable registers have outputs hovering at a voltage between high and low for a length of time beyond the normal t_{CO} for the register, which may cause subsequent registers that use this metastable signal to latch different values. Synchronize register chains when transferring data between unrelated clock domains to reduce the probability of the captured data signal becoming metastable.

A synchronization register chain is a sequence of registers with the same clock, that is driven by a pin, or logic from an unrelated clock domain. All but the last register in the chain must connect only to the next register, but may do so through logic.

The Metastability Report displays the following for each synchronization chain the analysis discovers:

- Typical Mean Time Between Failures (MTBF) for the chain
- MTBF equation and method of synchronizer identification
- Available settling time of the synchronization register chain
- Number of synchronization registers in the chain
- Names of synchronization registers in the chain
- name of asynchronous source registers

- Data toggle rate used in the MTBF estimation
- Source clock domain names
- Synchronization clock domain names

To help you analyze synchronizer chains in more detail, you can report the neighbor paths for synchronization registers in a chain. From the list of synchronization registers in the report in the Timing Analyzer GUI, you can right-click any of the synchronization register names and click **Report Neighbor Paths** in the context menu.

To report the timing for the synchronization registers, you can right-click and then click **Report Timing** for the synchronization registers. For the asynchronous source registers, right-click and click **Report Path**.

When you do not specify a data toggle rate, the Timing Analyzer uses the value of 12.5% of the frequency of the clock domain that contains the synchronizer to calculate MTBF.

Related Information

- [Metastability Analysis](#) on page 16
- [Step 1: Specify General Timing Analyzer Settings](#) on page 29

2.5.9. Report CDC Viewer

The Timing Analyzer's **Reports > Clock Domain Crossings > Report CDC Viewer...** command allows you to configure and display a custom clock domain crossing report and the Clock Domain Crossing (CDC) Viewer. The CDC Viewer graphically displays the setup, hold, recovery, or removal analysis of all clock transfers in your design. The equivalent scripting command is `report_cdc_viewer`.

Table 28. Report Clock Domain Crossing Viewer Settings

Option	Description
Clocks	From Clock and To Clock filter paths in the report to show only the launching or latching clocks you specify.
Analysis type	Options are Setup , Hold , Recovery , or Removal . The Timing Analyzer reports the results for the type of analysis you select.
Transfers	Specifies the type of clock transfers to include or exclude from the report, including Timed transfers , Fully cut transfers , Clock groups , Inactive clocks , and Non-crossing transfers . You can specify the Maximum slack limit and Grid options for the report.
Detail level	Full shows all details of the report and Summary filters the details and shows summary data.
Report panel name	Specifies the name of the report panel. You can optionally enable File name to write the information to a file. If you append <code>.htm</code> or <code>.html</code> as a suffix, the Timing Analyzer produces the report as HTML. If you enable File name , you can Overwrite or Append the file with latest data, and specify Grid or List format. <i>Note:</i> In grid format reports, clocks with non-crossing transfers always appear if they have transfers between other clocks.
Tcl command	Displays the Tcl syntax that corresponds with the GUI options you select. You can copy the command from the Console into a Tcl file.

You can specify the following options to customize CDC Viewer reporting:

Table 29. CDC Viewer Report Controls

Control	Description
From Clock: and To Clock:	Filters the display according to the clock names you specify. Click From Clock: or To Clock: to search for specific clock names.
Legend	Defines the status colors. A color coded grid displays the clock transfer status. The clock headers list each clock with transfers in the design. The GUI truncates long clock names, but you can view the full name in a tool tip or by resizing the clock header cell. The GUI represents the generated clocks as children of the parent clock. A '+' icon next to a clock name indicates the presence of generated clocks. Clicking on the clock header displays the generated clocks associated with that clock.
Toggle Data	The text in each transfer cell contains data specific to each transfer. Turn on or off display of the following types of data: • Number of timed endpoints between clocks— the number of timed, endpoint-unique paths in the transfer. A path being "timed" means that analysis occurs on that path. Only paths with unique endpoints count towards this total. • Number of cut endpoints between clocks— the number of cut endpoint-unique paths, instead of timed paths. These paths are cut by either a false path or clock group assignment. Timing analysis skips such paths. • Worst-case slack between clocks— the worst-case slack among all endpoint-unique paths in the transfer. • Total negative slack between clocks— the sum of all negative slacks among all endpoint-unique paths in this transfer. • Tightest relationship between clocks— the lowest-value setup, hold, recovery, or removal relationship between the two clocks in this transfer.
Show Filters and Show Legend	Turns on or off Filters and Legend .

Each block in the grid is a transfer cell. Each transfer cell uses color and text to display important details of the paths in the transfer. The color coding represents the following states:

Table 30. Transfer Cell Content

Cell Color	Color Legend
Black	Indicates no transfers. There are no paths crossing between the source and destination clock of this cell.
Green	Indicates passing timing. All timing paths in this transfer, that have not been cut, meet their timing requirements.
Red	Indicates failing timing. One or more of the timing paths in the transfer do not meet their timing requirements. If the transfer is between unrelated clocks, the paths likely require a synchronizer chain.
Blue	Indicates clock groups. The source and destination clocks of these transfers are cut by means of asynchronous clock groups.
Gray	Indicates a cut transfer. All paths in this transfer are cut by false paths. Therefore, timing analysis does not consider these paths.
Orange	Indicates inactive clocks. One of the clocks in the transfer is an inactive clock (with the <code>set_active_clocks</code> command). The Timing Analyzer ignores such transfers.

Right-click menus allow you to perform operations on transfer cells and clock headers. When the operation is a Timing Analyzer report or SDC command, a dialog box opens containing the contents of the transfer cell.

Table 31. Transfer Cell Right-Click Menus

Command	Description
Copy	Copies the contents of the transfer cell or clock header to the clipboard.
Report Timing	Reports timing. Not available for transfer cells with no valid paths (gray or black cells).
Report Endpoints	Reports endpoints. Not available for transfer cells with no cut paths (gray or black cells).
Report False Path	Reports false paths. Not available for transfer cells with no valid paths (black cells).
Report Exceptions	Reports exceptions. Only available for clock group transfers (blue cells).
Report Exceptions (with clock groups)	Reports exceptions with clock groups. Only available for clock group transfers (blue cells).
Set False Path	Sets a false path constraint.
Set Multicycle Path	Sets a multicycle path exception.
Set Min Delay	Sets a min delay constraint.
Set Max Delay	Sets a max delay constraint.
Set Clock Uncertainty	Sets a clock uncertainty constraint.

Table 32. Clock Header Right-Click Menus

Command	Description
Copy (include children)	Copies the name of the clock header, and the names of each of its derived clocks. This option only appears for clock headers with generated clocks.
Expand/Collapse All Rows/Columns	Shows or hides all derived clocks in the grid.
Create Slack Histogram	Generates a slack histogram report for the clock you select.
Report Timing From/To Clock	Generates a timing report for the clock you select. If you do not expand the clock to display derived clocks, the timing report includes all clocks that derive from the clock. To prevent this, expand the clock before right-clicking it.
Remove Clock(s)	Removes the clock you select from the design. If you do not expand the clock, timing analysis removes all clocks that derive from the clock.

You can view CDC Viewer output in any of the following formats:

- A report panel in the Timing Analyzer
- Output in the Timing Analyzer Tcl console
- A plain-text file
- An HTML file you can view in a web browser.

Related Information

- [Report Asynchronous CDC](#) on page 148
- [Constraining CDC Paths](#) on page 93

2.5.10. Report Asynchronous CDC

The Timing Analyzer's **Reports > Clock Domain Crossings > Report Asynchronous CDC** command allows you to classify and report all asynchronous clock-domain-crossing (CDC) transfers in your design. Asynchronous CDCs include

single-bit transfers, multibit transfers, and asynchronous reset CDCs. Designs often contain unintended CDCs or transfers. Use this report to ensure that the Timing Analyzer correctly detects all CDCs.

Table 33. Report Asynchronous CDC Settings

Settings	Description
Clocks	Filters the report to only show CDCs that originate from From clock and terminate at To clock .
Targets	Filters the report to only show CDCs that originate from From register and terminate at To register. Both the From and To must be registers. This option is optional with the Clocks option.
Entries	Limits the number of entries that are reported per CDC category
Detail	Chooses whether to show a summary of all CDC's or give detail on each individual CDC
CDC Categories	Specifies the CDC categories to be reported, including Single-bit CDC , Multi-bit CDC , and Reset CDC options.
Report panel name	Specifies the name of the report panel. You can optionally enable File name to write the information to a file. If you append .htm or .html as a suffix, the Timing Analyzer produces the report as HTML. If you enable File name , you can Overwrite or Append the file with latest data.
Tcl command	Displays the Tcl syntax that corresponds with the GUI options you select. You can copy the command from the Console into a Tcl file.

Figure 161. Report Asynchronous CDC Summary View

Asynchronous CDC Summary			
Q <<Filter>>			
	CDC Type	CDC Count	Total Transfer Width
1	▼ Single-bit CDC		
1	Unsynchronized Transfer	5	5
2	Unconstrained Synchronizer	1	1
3	Synchronizer Drive Clock Domains	2	2
4	Intra-Clock False Path Synchronizer	2	2
5	Synchronizer Not Protected for MTBF	1	1
6	Compliant Synchronizer	11	11
7	Transfer Precedence Combinational Logic	0	0
2	▼ Multi-bit CDC		
1	Unconstrained CE-type CDC Bus	2	5
2	Compliant CE-type CDC Bus	2	5
3	Unconstrained MUX-type CDC Bus	2	8
4	Compliant MUX-type CDC Bus	2	8
5	Unsynchronized Synchronizer Bus	2	8
6	Unconstrained Synchronizer Bus	1	2
7	Compliant Synchronizer Bus	1	4
8	Synchronizer Bus Uneven Lengths	0	0
9	Synchronizer after Control Signal	0	0
3	▼ Asynchronous Reset CDC		
1	Unsynchronized Reset Synchronizer	1	1
2	Unconstrained Reset Synchronizer	1	1
3	Reset Synchronizer Multiple Sources	1	1
4	Inactive Reset Synchronizer	1	1

The summary view of the Asynchronous CDC Report provides an overview of the number of CDC's that fall into each category in your design. CDC Count gives the number of topologies detected for a category. **Total Transfer Width** shows the total number of CDC crossings for a category. This value is different than **CDC Count** for multibit CDC's because each topology consists of multiple crossings.

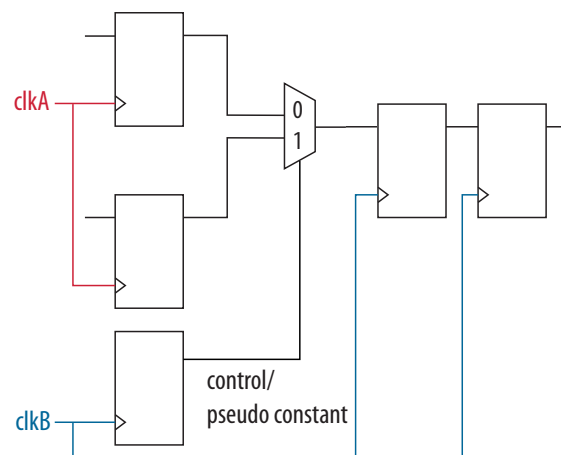
The full view of the Asynchronous CDC report shows the source and destination registers and clocks for each detected CDC topology in your design. You may click on any row of this report. Clicking on a row that contains the name of a CDC category brings up the description for that category and associated Design Assistant rules that check for such topologies.

Figure 162. Report Asynchronous CDC Full View

Asynchronous CDC Full Report			
	CDC Type	Source Nodes	Destination Nodes
1	▼ Single-bit CDC		
1	▼ Unsynchronized Transfer (5)		
1	--	unsynch_chain src_reg	unsynch_c...ynch_reg1
2	--	chain_with_...out src_reg	chain_with...synch_reg1
3	--	logic_within...hain src_reg	logic_withi... synch_reg1
4	--	logic_within...hain src_reg	logic_withi... synch_reg1
5	--	synch_reg_o...ain src_reg	synch_reg_...synch_reg1
2	▼ Unconstrained Synchronizer (1)		
1	--	unconstrain...ain src_reg	unconstrai...synch_reg1
3	▼ Synchronizer Drive... Clock Domains (2)		
1	--	multi_clock_...ain src_reg1	multi_cloc...synch_reg1
2	--	comb_logic_...ain src_reg	comb_logic...synch_reg1
4	▼ Intra-Clock False Path Synchronizer (2)		
1	--	intra_clock_...t_chain rst1	intra_clock..._chain rst2
2	--	intra_clock_...hain src_reg	intra_clock... synch_reg1

In the **Asynchronous CDC Full Report** tab, the **All Source Clocks (Synch/Asynch)** column displays all clocks driving all source nodes of a synchronizer chain, regardless of whether the transfers are synchronous or asynchronous. There can be asynchronous and synchronous transfers into the head of a synchronizer chain if the paths go through combinational logic before the head of the synchronizer chain. For example, Figure xx shows a circuit with both asynchronous and synchronous transfers into a synchronizer chain. For this example, the Asynchronous CDC Full Report lists both `clkA` and `clkB` as source clocks of the synchronizer chain, even though `clkB` is a synchronous transfer.

Figure 163. Circuit with Asynchronous and Synchronous Transfers to a Synchronizer Chain



Clicking on a row that contains a CDC displays detailed information on that CDC in the **CDC Statistics**, **SDC Statistics**, and **Schematic View** tabs. These tabs help you to more easily identify the root cause of inefficient CDC design by showing both the relevant topology and related SDCs together in one view.

Figure 164. Statistics Table Showing Detailed Information on Each CDC

Unsynchronized Transfer #1		
CDC Statistics SDC Statistics Schematic View		
	Property	Value
1	Number of Source Registers	1
2	▼ Source Registers	
1	altera_reserved_tdi	
3	Number of Discovered Synchronization Registers	1
4	▼ Discovered Synchronization Register	Synchronizer Stage Number
1	jtag_uart_0 jtag_uart_0 qsys_top_jtag_...wbag3pq_alt_jtag_atlantic td_shift[0]	1
5	Method of Synchronizer Identification	Automatic
6	▼ Chain Length Requested by User	3
1	User-requested chain length is respecte...adjacent registers in the synchronizer.	
7	▼ The destination register is the head of a data synchronizer because:	
1	Its d-pin is fed by a signal from an asynchronous clock domain.	
8	▼ The synchronizer is terminated at the last register because:	
1	▼ The last register in the synchronizer fans out to the following multiple registers:	
1	jtag_uart_0 jtag_uart_0 qsys_top_jta...pq_alt_jtag_atlantic user_saw_rvalid	
2	jtag_uart_0 jtag_uart_0 qsys_top_jta...bag3pq_alt_jtag_atlantic adapted_tdo	
9	Chain Length Protected	1
10	▼ This transfer does not appear in the metastability report because:	
1	▼ The following register has AUTO synchr...ion and forms a length-1 synchronizer:	
1	jtag_uart_0 jtag_uart_0 qsys_top_jtag...wbag3pq_alt_jtag_atlantic td_shift[0]	

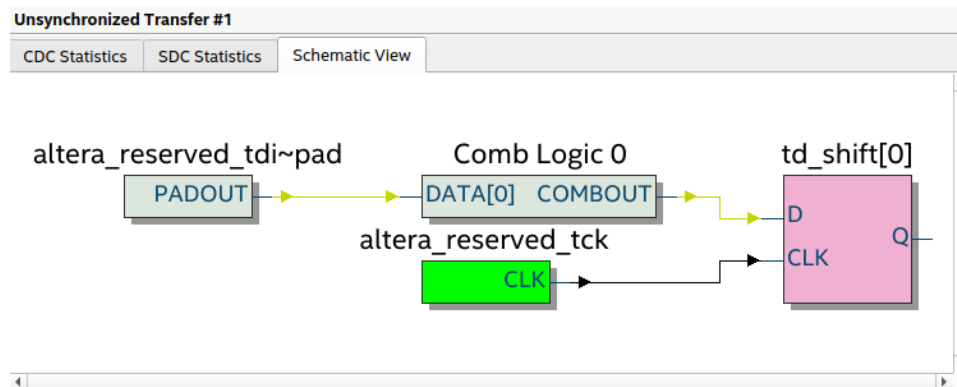
The **SDC Statistics** tab lists all the SDCs that are related to the CDC that you select from the report.

SDC Statistics Tab

Unsynchronized Transfer #1		
CDC Statistics SDC Statistics Schematic View		
Exception Statistics		
	Source Node	Destination Node
1	▼ Driver Nodes SDC	--
1	altera_reserved_tdi	jtag_uart_0 jtag_uart_0 qsys_top_jtag_uart...924_wbag3pq_alt_jtag_atlantic td_shift
Clock Group Constraint Statistics		
	Clock Group Constraint	Type
1	▼ Clock Group Assignment #1	-asynchronous
1	Group #1	--
2	▼ Clock Group Assignment #2	-asynchronous
1	Group #1	--

The **Schematic View** tab allows you to visualize the structure of the CDC. You can readily identify the head of the synchronizer chains, the synchronizer registers, and the clocks of each register.

Figure 165. Schematic View Tab



Related Information

- [Report CDC Viewer](#) on page 146
- [Constraining CDC Paths](#) on page 93
- [report_metastability](#)
- [Troubleshooting Clock Domain Crossing \(CDC\) Timing Issues](#) on page 187
- [Constraining CDC Paths](#) on page 93

2.5.11. Report Logic Depth

The Timing Analyzer's **Reports > Design Metrics > Report Logic Depth...** command allows you to report the number of logic levels within a clock domain. This value typically corresponds to the number of look-up tables (LUTs) that a path passes through.

The equivalent scripting command is `report_design_metrics -logic_depth`.

Report Logic Depth shows the distribution of logic depth among the critical paths, allowing you to identify areas where you can reduce logic levels in your RTL.

Figure 166. Report Logic Depth (Histogram)

Report Logic Depth						
Show: Visible Hide <<Filter>>						
	Clock Name	Relationship	Depth 1	Depth 2	Depth 3	
1	clock	10.000	82	45	33	

Figure 167. Report Paths of Depth 3

Call report logic depth by topology for each clock, intraclock only.

Show:	Visible	Hide	Q <<Filter>>					
	Clock Name	Relationship	Depth 0	Depth 1	Depth 2	Depth 3	Depth 4	
1	clk	2.600	100	480	287	132		
2	clk2	3.333	27	554	0	419		

Copy Ctrl+C
Select All Ctrl+A
Freeze First Column
Undo Sort
Collapse All
Expand All
Report Paths of Depth 3
Report Logic Depth...

Figure 168. Summary of Paths

Close timing with accurate histogram cross probing.

Paths of Depth 3 (Setup for Clock Domain clk)										
Slack	Logic Depth	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay	Worst-Case Operating Condi	
1 0.296	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_36	clk	clk	2.600	-0.027	2.324	1 Slow vid1 OC Model	
2 0.298	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_36	clk	clk	2.600	-0.027	2.322	1 Slow vid1 OC Model	
3 0.305	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_34	clk	clk	2.600	-0.027	2.315	1 Slow vid1 OC Model	
4 0.309	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_34	clk	clk	2.600	-0.027	2.311	1 Slow vid1 OC Model	
5 0.322	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_36	clk	clk	2.600	-0.027	2.298	1 Slow vid1 OC Model	
6 0.327	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_30	clk	clk	2.600	-0.027	2.293	1 Slow vid1 OC Model	
7 0.328	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_32	clk	clk	2.600	-0.027	2.293	1 Slow vid1 OC Model	
8 0.330	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_36	clk	clk	2.600	-0.027	2.290	1 Slow vid1 OC Model	
9 0.331	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_30	clk	clk	2.600	-0.027	2.289	1 Slow vid1 OC Model	
10 0.332	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_36	clk	clk	2.600	-0.027	2.288	1 Slow vid1 OC Model	
11 0.333	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_34	clk	clk	2.600	-0.027	2.287	1 Slow vid1 OC Model	
12 0.333	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_36	clk	clk	2.600	-0.027	2.287	1 Slow vid1 OC Model	
13 0.337	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_32	clk	clk	2.600	-0.027	2.284	1 Slow vid1 OC Model	
14 0.339	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_34	clk	clk	2.600	-0.027	2.281	1 Slow vid1 OC Model	
15 0.343	3	u_top_clk1jadd_5-181_LAB_RE_X138_Y194_NO_I0_0ff	sum(32)-regOPTHM_34	clk	clk	2.600	-0.027	2.277	1 Slow vid1 OC Model	

You can specify various options to customize the reporting.

Table 34. Report Logic Depth Settings

Option	Description
Clocks	From Clock and To Clock filter paths in the report to show only the launching or latching clocks you specify.
Targets	Specifies the target node for From Clock and To Clock to report logic depth with only those endpoints. Specify an I/O or register name or I/O port for this option. The field also supports wildcard characters. When the From , To , or Through boxes are empty, the Timing Analyzer assumes all possible targets in the device. The Through option limits the report for paths that pass through combinatorial logic, or a particular pin on a cell.
Analysis type	The Setup , Hold , Recovery , and Removal analyses report the logic depths of the top <i>X</i> paths by slack. Topology analysis reports the logic depths of the top <i>X</i> paths by logic depth.
Paths	Specifies the number of paths to display by endpoint and slack level. The default value for Report number of paths is 10, otherwise, the report can be very long. Enable Pairs only to list only one path for each pair of source and destination. Limit further with Maximum number of paths per endpoints . You can also filter paths by entering a value in the Maximum slack limit field.
Detail	Specify whether to display on Histogram or full Path level of detail.
Report panel name	Specifies the name of the report panel. You can optionally enable File name to write the information to a file. If you append .htm or .html as a suffix, the Timing Analyzer produces the report as HTML. If you enable File name , you can Overwrite or Append the file with latest data.
Tcl command	Displays the Tcl syntax that corresponds with the GUI options you select. You can copy the command from the Console into a Tcl file.

2.5.12. Report Neighbor Paths

The Timing Analyzer's **Reports > Design Metrics > Report Neighbor Paths...** command helps you to determine the root cause of critical paths (for example, high logic level, retiming limitation, sub-optimal placement, I/O column crossing, hold fix-up, time borrowing, or others). The equivalent scripting command is `report_design_metrics -neighbor_paths`.

Figure 169. Report Neighbor Paths Report

Report Neighbor Paths				
Command Info		Summary of Paths		
	Type	Slack	From Node	To Node
1	Path	-0.390	[2].L	sub[data_in_to_shifter[1217]~_Duplicate_225
1	Path Before	0.435	[2].L	sub[extinfo_r..._generated]altera_syncram_impl1[ram_block
2	Path After	0.363	[2].L	sub[shifter]lef...[1]dwshifter_slice[shift_left_0~121_ERTM1
Copy Select All Freeze First Column Undo Sort Collapse All Expand All Report Worst-Case Path Report Worst-Case Neighbour Paths Report Timing... Report Neighbour Paths... Export... Locate Path Locate Node				
Path #1 Path Summary Path Before 1 From Node 2 To Node 3 Launch Clock 4 Latch Clock 5 Setup Slack [2].LDPC[decoder_main decoder_core pcub]llr_gen[152].compute_llr_qv_out_s[1] [2].LDPC[decoder_main decoder_core]s...era_syncram_impl1[ram_block2a17~reg0 UPLL jiopll_o_outclk0 UPLL jiopll_o_outclk0 0.435 [2].LDPC[decoder_main decoder_core] [2].LDPC[decoder_main decoder_core] UPLL jiopll_o_outclk0 UPLL jiopll_o_outclk0 -0.390				

Report Neighbor Paths reports the most timing-critical paths in the design, including associated slack, additional path summary information, and path bounding boxes. **Report Neighbor Paths** shows the most timing-critical **Path Before** and **Path After** each critical **Path**. You can optionally view multiple before and after paths. Retiming or logic balancing of the **Path** can simplify timing closure if there is negative slack on the **Path**, but positive slack on the **Path Before** or **Path After**.

Table 35. Report Neighbor Path Dialog Box Settings

Option	Description
Clocks	From Clock and To Clock filter paths in the report to show only the launching or latching clocks you specify.
Targets	Specifies the target node for From Clock and To Clock to report neighbor paths with only those endpoints. Specify an I/O or register name or I/O port for this option. The field also supports wildcard characters. When the From , To , or Through boxes are empty, the Timing Analyzer assumes all possible targets in the device. The Through option limits the report for paths that pass through combinatorial logic, or a particular pin on a cell.
Analysis type	The Analysis type options are Setup , Hold , Recovery , or Removal . The Timing Analyzer reports the results for the type of analysis you select.
Paths	Specifies the number of paths to display by endpoint and slack level. The default value for Report number of paths is 10, otherwise, the report can be very long. Enable Pairs only to list only one path for each pair of source and destination. Limit further with Maximum number of paths per endpoints . You can also filter paths by entering a value in the Maximum slack limit field.

continued...

Option	Description
Report Number of Neighbor Paths	Specifies the number of neighbor paths to report, allowing you to view a number of the top adjacent paths entering the critical path, and the top paths exiting the critical path.
Report panel name	Specifies the name of the report panel. You can optionally enable File name to write the information to a file. If you append <code>.htm</code> or <code>.html</code> as a suffix, the Timing Analyzer produces the report as HTML. If you enable File name , you can Overwrite or Append the file with latest data.
Tcl command	Displays the Tcl syntax that corresponds with the GUI options you select. You can copy the command from the Console into a Tcl file.
Extra Info	Specifies extra info.

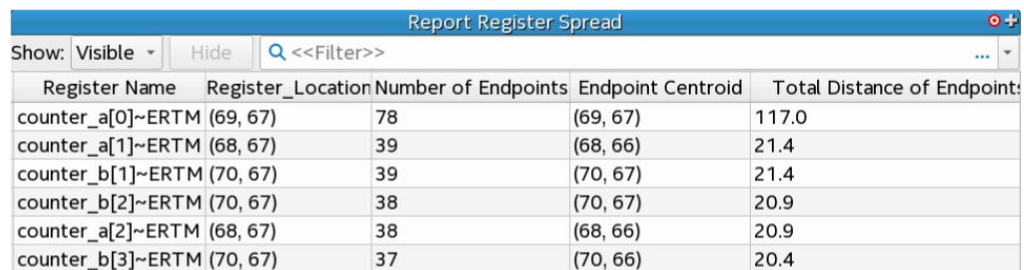
2.5.13. Report Register Spread

The Timing Analyzer's **Reports > Design Metrics > Report Register Spread** command analyzes the final placement to identify registers with sinks pulling them in various directions. These registers are potential candidates for duplication. The equivalent scripting command is `report_register_spread`.

Registers that drive in opposite directions and connect to high fan-out can have placement-warping effects on the floorplan that impact f_{MAX} . The placement-warping may not cause timing failures. Therefore, you can view this report to identify such registers. Taking steps to address the registers listed in the report can make placement of the design easier and improve f_{MAX} performance.

You can automate duplication of registers with the `DUPLICATE_REGISTER` and `DUPLICATE_HIERARCHY_DEPTH` .qsf assignments, or you can manually modify RTL to duplicate registers or refactor logic. Refer to "Automatic Register Duplication: Hierarchical Proximity" in *Quartus Prime Pro Edition User Guide: Design Optimization*.

Figure 170. Report Register Spread Report



Register Name	Register_Location	Number of Endpoints	Endpoint Centroid	Total Distance of Endpoints
counter_a[0]~ERTM	(69, 67)	78	(69, 67)	117.0
counter_a[1]~ERTM	(68, 67)	39	(68, 66)	21.4
counter_b[1]~ERTM	(70, 67)	39	(70, 67)	21.4
counter_b[2]~ERTM	(70, 67)	38	(70, 67)	20.9
counter_a[2]~ERTM	(68, 67)	38	(68, 66)	20.9
counter_b[3]~ERTM	(70, 67)	37	(70, 66)	20.4

You can specify various options to customize the report.

Table 36. Report Register Spread Settings

Option	Available Settings
Spread Type	Specifies the type of spread data in the report:

continued...

Option	Available Settings
	Tension—reports the sum over each sink of the distance from it to the centroid of all the sinks. Angle—reports how far around the source register the fan-outs wrap, expressed from 0 to 360 degrees. This value corresponds to 360 minus the maximum angle between any two angularly adjacent sinks. This metric complements Tension by identifying registers which are surrounded by their sinks in all directions, and not those registers only being pulled in a few directions. Span—reports the maximum 1-dimensional delta between the left bottom-most sink and the right top-most sink. Area—reports the coverage of the sinks by number of LABs on the FPGA device. This option multiplies the span of the sinks in both X- and Y- dimensions. This metric complements Span by incorporating both dimensional spans of the sinks, and not only the maximum sink. Count—reports registers with the largest sink counts.
Sink Type	Specifies the type of sink in the report: Endpoint—the nodes (usually registers) that terminate timing paths from a register. Immediate Fanout—the immediately connected nodes of the register. For example, lookup tables, other registers, RAM, or DSP blocks.
From Clock	Filters paths in the report to show only the launching clocks you specify.
To Clock	Filters paths in the report to show only the latching clocks you specify, allowing you to debug one clock at a time.
Report number of registers	Specifies the number of registers to display in the report. The default value for Report number of registers is 10.
Report panel name	Specifies the name of the report panel. You can optionally enable File name to write the information to a file. If you append .htm or .html as a suffix, the Timing Analyzer produces the report as HTML. If you enable File name , you can Overwrite or Append the file with latest data.
Tcl command	Displays the Tcl syntax that corresponds with the GUI options you select. You can copy the command from the Console into a Tcl file.

Figure 171. Report Register Spread Types

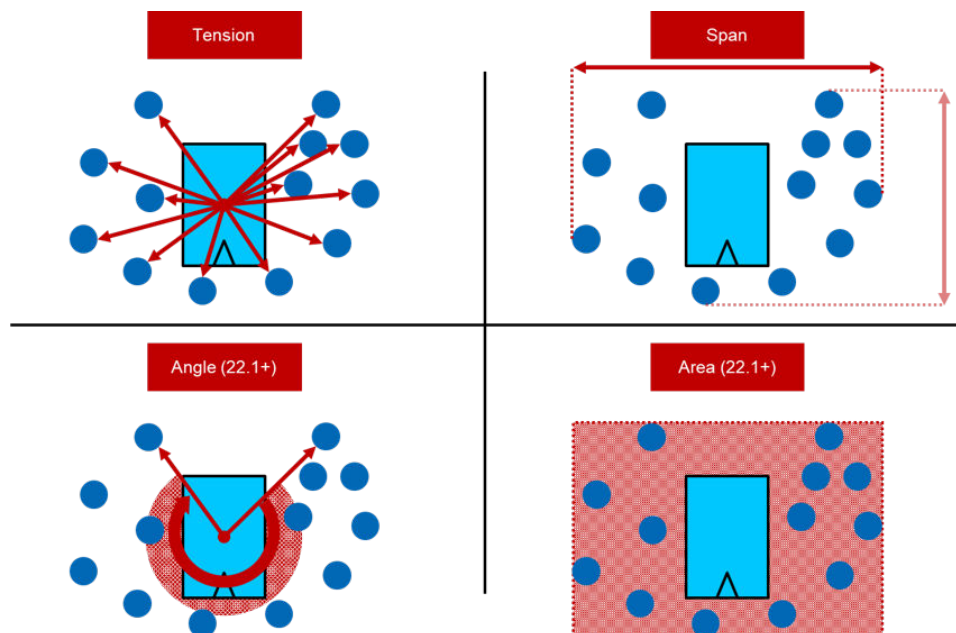


Figure 172. Report Register Spread Dialog Box

Report Register Spread

Spread Type

☒ Tension ☐ Span ☐ Area ☐ Angle ☐ Count

Sink Type

☒ Endpoint ☐ Immediate Fanout

Clocks

From clock:

To clock:

Report Number of Registers:

Minimum Number of Sinks per Register:

Report Panel Name:

Output

☐ File name: ...

File options

☒ Overwrite ☐ Append

☐ Console

Tcl command:

Related Information

Automatic Register Duplication: Hierarchical Proximity, Quartus Prime Pro Edition

2.5.13.1. Understanding Report Register Spread Data

It is helpful to understand the concept of tension in utilizing Report Register Spread data. Tension is the sum over each sink of the distance from it to the centroid of all the sinks. The tension value is therefore dependent on the number of sinks.

What Is a Good Tension Value?

There is no absolute threshold for a good or bad tension value. A register's tension value depends on the number of fan-outs, their distance, and the degree to which the fan-outs go in opposite directions. When tension is high, duplicating the registers with high tension can be the easiest way to reduce the tension value.

Figure 173. Register Tension Values in Report Register Spread

Report Register Spread(endpoint tension)		
de Q <<Filter>> (use !<string> to invert filter)		
Endpoint Centroid	Total Distance of Endpoints to Centroid (Tension)	Average Distance
(94, 86)	13785	12
(94, 86)	13782	12
(94, 86)	13456	12
(94, 86)	13453	12
(94, 86)	13440	12
(94, 86)	13404	12
(94, 86)	13404	12
(94, 86)	13404	12
(94, 86)	13404	12
(94, 86)	13404	12
(94, 86)	13404	12

Duplicate a Register to Reduce Tension Value

You can use Report Register Spread in combination with other design information to make effective changes. For example, in a design suffering from routing congestion, congested logic is sometimes pulled together by registers that are also near the top of the tension list.

You can use Report Register Spread to cross-reference the registers listed near the top of the tension data with design hierarchies affected by congestion. You can then duplicate a register near the top of the tension list to allow the congested logic to spread farther apart, thereby reducing congestion.

Selecting Registers for Duplication

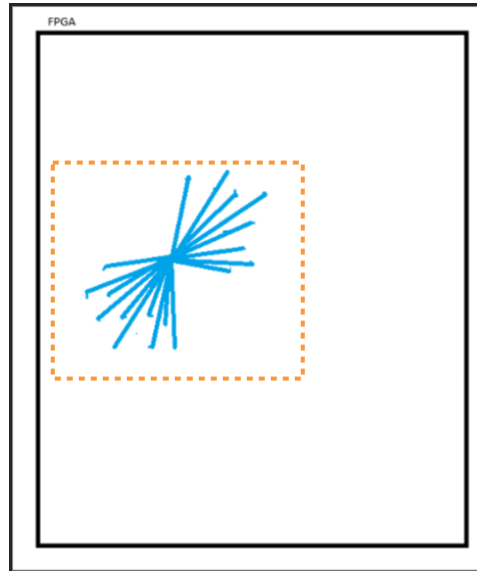
The best candidate registers for duplication are those with a single data fan-in because such nodes are unlikely to have much additional congestion to feed other register copies. Registers that have complex support logic (high numbers of fan-in signals) are poor candidates for duplication because all the fan-in sources then have to feed other copies of the register.

Consider the following guidelines in selecting registers for duplication:

- Review the way the fan-outs are distributed physically and logically. For the best performance improvement, fan-outs that connect to a new duplicate should be in a similar physical location on the chip.
- If a register with high tension fans out to two separate instances, on opposite sides of the chip, you should make a duplicate for each instance.
- If a register with high tension fans out to a cloud of related registers (like an enable signal to a pipeline stage in a bus), it can be difficult to segment the fan-outs in a way that correlates to their physical placement. The individual bits of a pipeline stage of a bus might not have any attraction or inherent connectivity to a specific physical part of the chip.

Consider an example register from the tension report with the following fan-out pattern in Chip Planner:

Figure 174. Example Fan-Out Pattern in Chip Planner



The best case scenario is for the fan-outs in the upper-right quadrant of the Chip Planner to be logically separate from the fan-outs in the lower-left quadrant. You can achieve this by creating a duplicate of the register and connecting the fan-outs in the upper-right quadrant to one register, and the fan-outs in the lower-left quadrant to the other register.

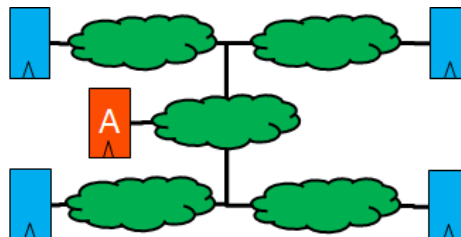
Related Information

[Design Floorplan Analysis in Chip Planner](#)

2.5.13.2. Registers with High Timing Path Endpoint Tension

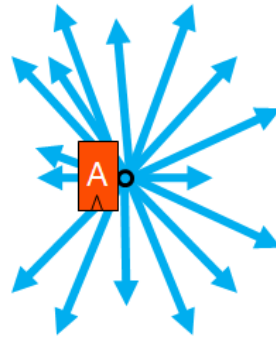
Timing path endpoints are the nodes (usually registers) that terminate timing paths from a register. The Timing path endpoint is equivalent to the nodes that the `get_fanouts` command returns, or the overall set of nodes that appear as a "From Node" after running the `report_timing` command. Register duplication is necessary, but not always sufficient, in helping to distribute the signal more efficiently. In addition, you may need to duplicate or restructure any intermediate logic before duplicating the register.

Figure 175. Register A has 4 Timing Path Endpoints



Tension is the sum over each sink of the distance from the sink to the centroid of all the sinks. The value of tension is therefore dependent on the number of sinks. Register duplication can help to break up these clouds, since they may be the result of the placement solution getting "warped" by the presence of the register.

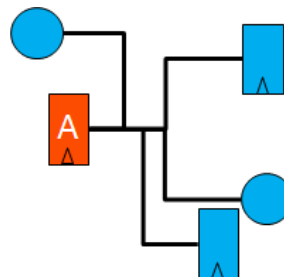
Figure 176. Register A has High Tension



2.5.13.3. Registers with High Immediate Fan-Out Tension

There are two **Sink Type** options: **Endpoint** and **Immediate Fanout**. The immediate fan-outs are the immediately connected nodes (lookup tables, other registers, RAM or DSP blocks, and others) of the register. This fan-out is equivalent to fan-outs that the Chip Planner displays, and in various high fan-out reports. Register duplication directly distributes the immediate fan-outs of a register among the duplicates.

Figure 177. Register A has High Immediate Fan-Out



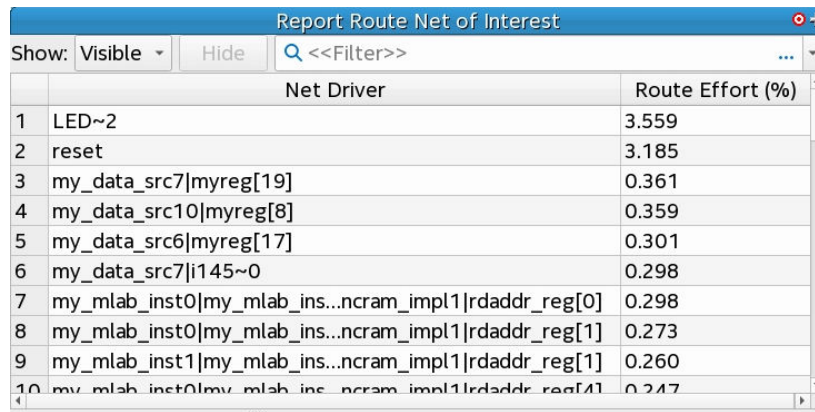
2.5.14. Report Route Net of Interest

The Timing Analyzer's **Reports > Design Metrics > Report Route Net of Interest...** command allows you to report the nets that require the most effort from the router. The report shows the percentage of total router effort for the nets reported. The equivalent scripting command is `report_route_net_of_interest`.

This report allows you to identify nets that should not require significant router effort. For example, you might expect that low speed management interface nets are not timing critical, and therefore not require much router effort. However, if **Report Route Net of Interest** reports that some nets in the low speed management interface require significant effort from the router, you can investigate that further.

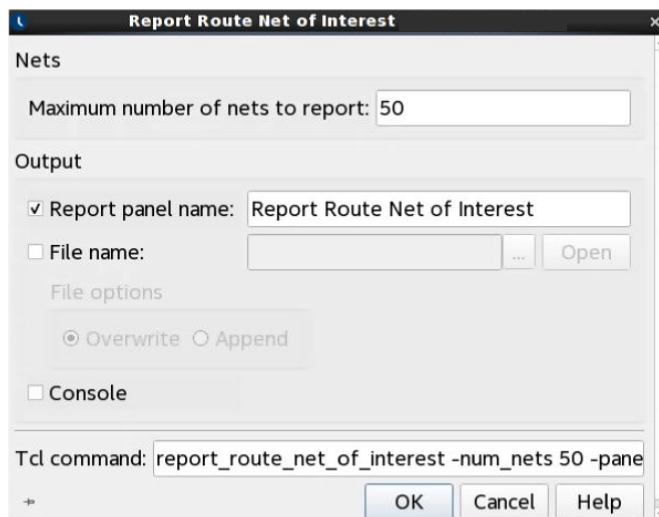
The investigation can determine whether the timing constraints are correct, whether the fan-out is significant and can reduce through driver duplication, or whether the net passes through congested areas.

Figure 178. Report Route Net of Interest Report



	Net Driver	Route Effort (%)
1	LED~2	3.559
2	reset	3.185
3	my_data_src7 myreg[19]	0.361
4	my_data_src10 myreg[8]	0.359
5	my_data_src6 myreg[17]	0.301
6	my_data_src7 i145~0	0.298
7	my_mlab_inst0 my_mlab_ins...ncram_impl1 rdaddr_reg[0]	0.298
8	my_mlab_inst0 my_mlab_ins...ncram_impl1 rdaddr_reg[1]	0.273
9	my_mlab_inst1 my_mlab_ins...ncram_impl1 rdaddr_reg[1]	0.260
10	my_mlab_inst0 my_mlab_ins...ncram_impl1 rdaddr_reg[1]	0.247

Figure 179. Report Route Net of Interest Dialog Box



Report Route Net of Interest

Nets

Maximum number of nets to report: 50

Output

☒ Report panel name: Report Route Net of Interest

☐ File name: [] [Open]

File options

☒ Overwrite ☐ Append

☐ Console

Tcl command: report_route_net_of_interest -num_nets 50 -pane

[OK] [Cancel] [Help]

From the Route Net of Interest Report in the Timing Analyzer GUI, you can right-click on any net and run Report Timing for more details about the net, its slack, and any of the net's paths.

Table 37. Report Route Net of Interest Settings

Option	Available Settings
Nets	Specifies the Maximum number of nets to report . The default value is 50.
Report panel name	Specifies the name of the report panel. You can optionally enable File name to write the information to a file. If you append <code>.htm</code> or <code>.html</code> as a suffix, the Timing Analyzer produces the report as HTML. If you enable File name , you can Overwrite or Append the file with latest data.
Tcl command	Displays the Tcl syntax that corresponds with the GUI options you select. You can copy the command from the Console into a Tcl file.

2.5.15. Report Retiming Restrictions

The Timing Analyzer's **Reports > Design Metrics > Report Retiming Restrictions...** command allows you to report the occurrences of design conditions that restrict Hyper-Retiming, such as Power-up "Care" restrictions, and don't touch or preserve attributes for each port. You can refer to this report to improve the circuit and remove retiming restrictions that limit circuit performance. `report_retiming_restrictions` is the equivalent scripting command.

Figure 180. Report Retiming Restrictions Report

Report Retiming Restrictions			
Q <<Filter>>			
	Compilation Hierarchy Node	Register is part of a synchronization chain ^	Preserve assignm
1		114 (0)	292 (50)
1	▾ my_data_src7	21 (21)	0 (0)
1	myreg[*]	20 (20)	0 (0)
2	enable_reg	1 (1)	0 (0)
2	▾ my_data_src8	21 (21)	0 (0)
1	myreg[*]	20 (20)	0 (0)
2	enable_reg	1 (1)	0 (0)
3	▾ my_data_src5	19 (19)	0 (0)
1	myreg[*]	18 (18)	0 (0)
2	enable_reg	1 (1)	0 (0)
4	▾ my_data_src6	19 (19)	0 (0)

Note: For table entries with two numbers listed, the numbers in parentheses indicate the number of retiming restrictions in the specific entity alone. The numbers listed outside of parentheses indicate the number of retiming restrictions in the specific entity and all of its sub-entities in the hierarchy.

For table entries with two number values, the number in parentheses indicates the number of retiming restrictions in the specific entity alone. The number listed outside of parentheses indicates the number of retiming restrictions in the specific entity and all of its sub-entities in the hierarchy.

Related Information

[Retiming Restrictions and Workarounds, Hyperflex Architecture High-Performance Design Handbook](#)

2.5.16. Report Register Statistics

The Timing Analyzer's **Reports > Design Metrics > Report Register Statistics** command allows you to report the number of synchronous and asynchronous resets, hyper registers, and registers with clock enables in the design. You can use this information, combined with timing slack, congestion, and other analysis reports, to identify timing-critical parts of your design that can have resets removed or control schemes changed to meet timing requirements more efficiently.

Figure 181. Report Register Statistics (Report Truncated)

Report Register Statistics						
Q <<Filter>> (use !<string> to invert filter)						
Compilation Hierarchy Node	Register Count	Without a Clock	Unique Clocks	Hyper-Register Count	Synchronous Reset	Asynchronous Reset
1	3645 (0)	0 (0)	2 (0)	0 (0)	311 (0)	777 (0)
1 auto_fab_0	88 (0)	0 (0)	1 (0)	0 (0)	7 (0)	38 (0)
1 alt_sid_fab_0	88 (0)	0 (0)	1 (0)	0 (0)	7 (0)	38 (0)
2 intel_niosv_m_0	2789 (0)	0 (0)	2 (0)	0 (0)	235 (0)	658 (0)
1 intel_niosv_m_0	2789 (0)	0 (0)	2 (0)	0 (0)	235 (0)	658 (0)

Truncated Report Columns:

Clock Enable Synchronous Load	Without a Control Signal	Full Hierarchy Name
2472 (0)	0 (0)	726 (0)
63 (0)	0 (0)	12 (0)
63 (0)	0 (0)	12 (0)
1961 (0)	0 (0)	454 (0)
1061 (0)	0 (0)	454 (0)

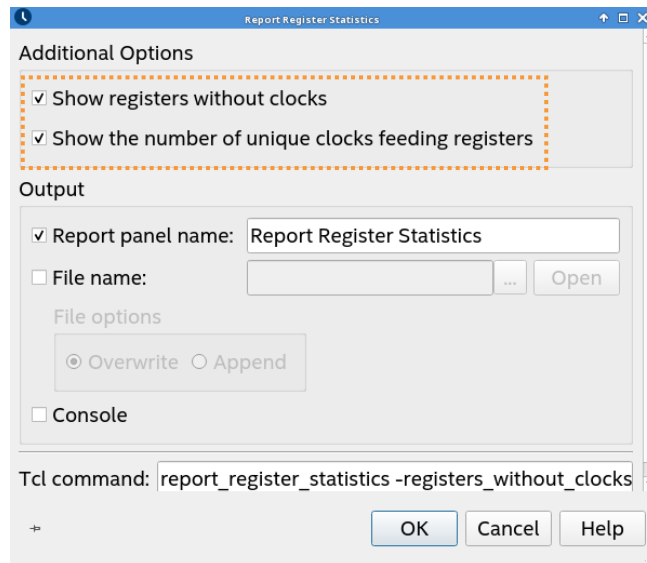
Note:

- This report works similarly in both post-synthesis Early Timing Analysis and post-fit timing analysis. However, the report's **Without a Clock** column is more helpful for Early Timing Analysis because you typically do not apply conventional SDCs for Early Timing Analysis.
- Clocks generated from `derive_clocks` commands do not count as user clocks.
- The report's **Without a Control Signal** column identifies registers that have no corresponding control signal.
- The report's **Synchronous Load** column identifies any synchronous load that can apply to Arria 10 devices only.

The **Without a Clock** column informs you of the number of registers where no defined clock feeds the registers in the hierarchy shown in the **Register Count** column. A value of 0 in this column suggests that your design has SDC-defined clocks feeding registers in the design. The **Unique Clocks** column indicates the number of unique SDC-defined clocks feeding registers in the hierarchy identified by the

Register Count. To view these columns, enable **Show registers without clocks** and **Show the number of unique clocks feeding registers** additional options in the dialog that displays when you run the report, as shown in the following image:

Figure 182. Report Register Statistics Additional Options Dialog



2.5.17. Report Pipelining Information

The Timing Analyzer's **Reports > Design Metrics > Report Pipelining Information...** command allows you to generate a report that can help you to identify potential areas of over-pipelining in your design. Excessive pipelining unnecessarily consumes area. The equivalent scripting command is `report_pipelining_info`.

Report Pipelining Information... does not perform any functional analysis in making the recommended pipeline stage adjustment. You must be aware of any potential functional changes from removing pipeline stages. There may be circumstances when all the stages in a register pipeline are necessary for functional reasons. The report helps to identify location with more registers than necessary for covering distance.

Figure 183. Report Pipelining Information Report

Report Pipelining Information					
Show:	Visible	Hide	Q <<Filter>>		
	Full Hierarchy Name	Clock Name	Recommended Pipeline Stage Adjustment Across Bus	Minimum Total Slack of One Bit Across Bus	Minimum Av
1	GE100_2 alt...dm0 din_d_r0	GE10... ch1	-4	13.035	1.629
2	GE100_0 alt...dm0 din_d_r0	GE10... ch1	-4	13.348	1.668
3	l8_tx_data_d[2]	GE10... ch1	-2	9.651	1.608
4	l8_tx_data_d[1]	GE10... ch1	-2	8.736	1.456
5	l8_tx_data_d[0]	GE10... ch1	-2	10.001	1.666
6	GE100_1 alt...dm0 din_d_r0	GE10... ch1	-3	12.683	1.585
7	[2]LDPC deco... mins_valid_s	UPLL... tclk0	-2	8.673	1.734
8	[0]LDPC deco... mins_valid_s	UPLL... tclk0	-2	8.922	1.784
9	GE100_0 alt_e... rx_data_in_r	GE10... ch1	-2	8.420	1.684
10	GE100_2 alt_e... rx_data_in_r	GE10... ch1	-2	8.735	1.747
11	[1]LDPC deco... mins_valid_s	UPLL... tclk0	-1	8.093	1.618
12	l8_tx_data_d	GE10... ch1	-2	9.537	1.589
13	l8_tx_data_d	GE10... ch1	-2	10.200	1.700
14	GE100_1 alt_e... rx_data_in_r	GE10... ch1	0	3.529	0.705
15	DDR4_1 jemif...readdata_0	DDR4... clk	0	4.216	1.405
16	[0]LDPC deco...self_data_out	UPLL... tclk0	0	2.988	0.996
17	DDR4_0 jemif...readdata_0	DDR4... clk	0	4.717	1.572
18	[2]LDPC deco...self_data_out	UPLL... tclk0	0	4.277	1.425
19	GE100_1 alt_e... tx4l_d_2fifo	GE10... ch1	0	2.280	1.140
20	[1]LDPC deco...self_data_out	UPLL... tclk0	0	4.492	1.497
21	GE100_0 alt_e... tx4l_d_2fifo	GE10... ch1	0	2.480	1.240
22	GE100_2 alt_e... tx4l_d_2fifo	GE10... ch1	0	3.076	1.538

Figure 184. Report Detailed Pipelining

Report Pipelining Inform			
Show:	Visible	Hide	Q <<Filter>>
	Full Hierarchy Name	Clock Name	Recommended Pipeline Stage Adjustment Across Bus
1	u_top_clk2_final[o_final	clk2	-15
2	u_top_clk2[dat	Copy	0
3	u_final_hp GEN_REG_INPUT.R_dat	Select All	0
Freeze First Column Undo Sort Collapse All Expand All Report Timing... Report Detailed Pipelining... Locate Node			

The detailed report shows every register in a tree structure. Over- or under-pipelining recommendations are in the main report. The following shows every single register inside the bus chain in a tree structure:

Figure 185. Detailed Pipelining Result

Q <<Filter>>				
	Full Hierarchy Name	Slack	Manhattan Distance	Chain Depth
1	u_top_clk2_final o_final[0]	58.579	46	21
1	u_final_dat_hp GEN_REG_INPUT.R_data[0][0]	2.538	8	1
2	u_final_dat_hp GEN_REG_INPUT.R_data[1][0]	2.632	7	1
3	u_final_dat_hp GEN_REG_INPUT.R_data[2][0]	2.433	14	1
4	u_final_dat_hp GEN_REG_INPUT.R_data[3][0]	2.878	0	1
5	u_final_dat_hp GEN_REG_INPUT.R_data[4][0]	2.901	0	1
6	u_final_dat_hp GEN_REG_INPUT.R_data[5][0]	2.872	0	1
7	u_final_dat_hp GEN_REG_INPUT.R_data[6][0]	2.864	0	1
8	u_final_dat_hp GEN_REG_INPUT.R_data[7][0]	2.867	0	1
9	u_final_dat_hp GEN_REG_INPUT.R_data[8][0]	2.387	12	1
10	u_final_dat_hp GEN_REG_INPUT.R_data[9][0]	2.735	3	1
11	u_final_dat_hp GEN_REG_INPUT.R_data[10][0]	2.923	0	1
12	u_final_dat_hp GEN_REG_INPUT.R_data[11][0]	2.919	0	1
13	u_final_dat_hp GEN_REG_INPUT.R_data[12][0]	2.908	0	1
14	u_final_dat_hp GEN_REG_INPUT.R_data[13][0]	2.887	0	1
15	u_final_dat_hp GEN_REG_INPUT.R_data[14][0]	2.894	0	1
16	u_final_dat_hp GEN_REG_INPUT.R_data[15][0]	2.832	0	1
17	u_final_dat_hp GEN_REG_INPUT.R_data[16][0]	2.837	0	1
18	u_final_dat_hp GEN_REG_INPUT.R_data[17][0]	2.809	0	1
19	u_final_dat_hp GEN_REG_INPUT.R_data[18][0]	2.865	0	1
20	u_final_dat_hp GEN_REG_INPUT.R_data[19][0]	2.743	2	1
21	final_dat[0]-reg0	2.855	0	1

To help identify potential over-pipelining, **Report Pipelining Information** reports:

- The recommended pipeline stage adjustment across bus
- The minimum total slack of one bit across bus
- The minimum average slack of one bit across bus
- The distance between the registers
- The width of buses in your design
- The number of sequential registers
- The number of registers on the bus

The Recommended Pipeline Stage Adjustment Across Bus reports the number of registers that you can remove from the bus for each bit. The Average Distance Per Stage, Max Distance Per Stage, and Min Distance Per Stage columns report the Manhattan distance measured in logic array blocks (LABs). The Bus Average Depth, Bus Max Depth, and Bus Min Depth columns report the number of sequential, single fan-out registers. For registers that have more than one clock source, the report lists the fastest one.

The **1+** sign under Recommended Pipeline Stage Adjustment Across Bus column means that the bus might need to add more registers to meet timing requirement. Refer to the Fast Forward Timing Closure Recommendations report.

If the report identifies a large register chain with multiple sequential registers, and the distance between registers is low, that condition can suggest over-pipelining. You may be able to remove some registers to recover some of the device area and reduce congestion.

The following options are available for this report:

Figure 186. Report Pipelining Information Dialog Box

Table 38. Report Pipelining Information Settings

Option	Available Settings
Pipeline	Specifies the thresholds for reporting a register pipeline. You can define the Minimum average bus depth , the Minimum bus width , and the Maximum number of rows that the report includes.
Report panel name	Specifies the name of the report panel. You can optionally enable File name to write the information to a file. If you append <code>.htm</code> or <code>.html</code> as a suffix, the Timing Analyzer produces the report as HTML. If you enable File name , you can Overwrite or Append the file with latest data.
Tcl command	Displays the Tcl syntax that corresponds with the GUI options you select. You can copy the command from the Console into a Tcl file.

2.5.18. Report Time Borrowing Data

You can run the Timing Analyzer's **Reports > Timing Slack > Report Timing...** command to generate a report showing time borrowing data. The equivalent scripting command is `report_timing` (with specific arguments).

The Timing Analyzer reports time borrowing data for the **Data Arrival Path** or **Data Required Path**, according to whether borrowing occurs at the destination or the source.

Figure 187. Report for Time Borrowing At Destination

The following report shows 100ps borrowed on the Data Required path. The setup slack improves on incoming paths, at the expense of worse setup slack for outgoing paths.

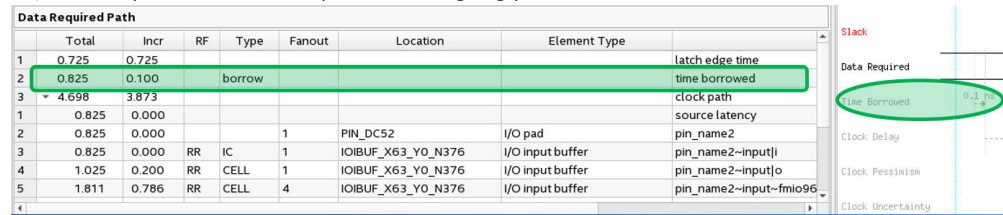
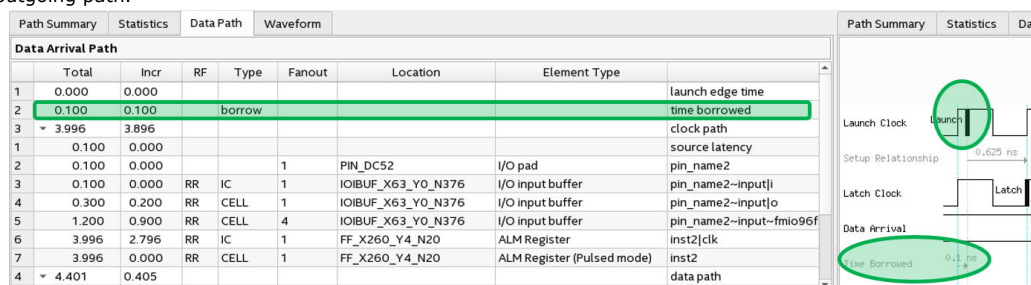


Figure 188. Report for Time Borrowing At Source

The following report shows 100ps borrowed on the Data Arrival Path, worsening the setup relationship for the outgoing path.



Related Information

Time Borrowing on page 20

2.5.19. Report Exceptions and Exceptions Reachability

The Timing Analyzer's **Reports > Constraint Diagnostics > Report Exceptions...** command allows you to report all exceptions to default timing analysis conditions, as specified by the **Set False Path**, **Set Multicycle Path**, **Set Minimum Delay**, or **Set Maximum Delay** commands (and the corresponding Tcl commands: `set_false_path`, `set_multicycle_path`, `set_min_delay`, and `set_max_delay`.)

Figure 189. Report Exceptions Reachability Report

Report Exceptions Reachability					
Show: Visible Hide <<Filter>>					
	Status	Setup Reachability	Hold Reachability	Recovery Reachability	Removal Reachability
46	Invalid	n/a	n/a	n/a	n/a
47	Invalid	n/a	n/a	n/a	n/a
48	Invalid	n/a	n/a	n/a	n/a
49	Invalid	n/a	n/a	n/a	n/a
50	Invalid	n/a	n/a	n/a	n/a
51	Fully overridden	0.0%	n/a	0.0%	n/a
52	Fully overridden	n/a	0.0%	n/a	0.0%
53	Complete	n/a	0.0%	n/a	100.0%
54	Complete	n/a	0.4%	n/a	0.0%
55	Complete	0.0%	n/a	100.0%	n/a
56	Complete	n/a	0.0%	n/a	100.0%
57	Complete	0.0%	n/a	100.0%	n/a
58	Complete	n/a	0.0%	n/a	100.0%
59	Complete	0.0%	n/a	100.0%	n/a
60	Complete	0.0%	n/a	100.0%	n/a
61	Complete	85.7%	85.7%	100.0%	100.0%
62	Complete	0.0%	n/a	100.0%	n/a
63	Complete	0.0%	n/a	100.0%	n/a

Reachability Metrics	
Property	Value
1 Status	Exception is complete (not overridden)
2 Number of possible "-to-" targets	7
3 Number of unique "-to-" targets satisfied by the exception	
1 Setup analysis	n/a
2 Hold analysis	0
3 Recovery analysis	n/a
4 Removal analysis	7

Similarly, you can click on **Reports > Constraint Diagnostics > Report Exceptions Reachability...** to report the scope of exception constraints in your project. This report allows you to determine whether constraints apply fully or partially, are overridden, and whether the source and destination node are reachable for the constraint by providing a "reachability" percentage. Reachability is the percentage of paths to which the constraint applies. Low reachability indicates a constraint that may be too broad, potentially covering many unrelated items. High reachability indicates that the exception is very targeted.

2.5.20. Report Bottlenecks

You can run the Timing Analyzer's **Reports > Design Metrics > Report Bottlenecks...** command to list all nodes in a design ranked by specified criteria. The equivalent scripting command is `report_bottleneck`.

The following ranking criteria are pre-defined:

- `num_fpaths`—the number of paths that fail timing through a node.
- `num_fanins`—the number of fan-in edges from a node.
- `num_fanouts`—the number of fan-out edges from a node.
- `num_paths`—the number of paths through a node.
- `tns`—the total negative slack of all the paths through a node.

When using scripting, you can specify the paths for analysis by passing the result of any `get_timing_paths` call as the last argument to `report_bottleneck`. When using the GUI, the **Report Bottlenecks** dialog box handles this argument automatically. If you specify no paths, `report_bottleneck` analyzes the worst 1000 setup paths in the design by default.

You can direct the report output to the Tcl console (`-stdout`), the Timing Analyzer GUI (`-panel`), or to a combination of console and GUI.

Figure 190. Report Bottlenecks Rated on Number of Failing Paths Through a Node

	Rating	Node
1	2	bidir
2	2	dff2
3	1	tmpin combout
4	1	tmpin
5	1	dff3~feeder dataf
6	1	dff3~feeder combout
7	1	dff3 datain
8	1	dff3
9	0	clk combout
10	0	clk
11	0	bidir datain
12	0	dff1

2.5.20.1. Specifying Custom Bottleneck Criteria

You can optionally specify your own custom criteria for evaluating nodes based on the combination of the number of fan-outs, fan-ins, failing paths, and total paths.

To specify custom bottleneck criteria, follow these steps:

1. Create a Tcl procedure that takes one argument. For example, `arg`.
2. Use `upvar $arg metric` in the procedure.
3. Calculate the rating based on `$metric(tns)`, `$metric(num_fanouts)`, `$metric(num_fanins)`, and `$metric(num_fpaths)`.
4. Return the rating with `return $rating`.
5. Pass the name of your custom criteria procedure to `report_bottleneck` using the `-cmetric` option

2.5.21. Check Timing

The Timing Analyzer's **Reports > Constraint Diagnostics > Check Timing** command (`check_timing`) checks your design and constraint files for problems with design constraints.

Check Timing can perform a series of different checks based on the variables and options that you specify for the command. When using scripted methods, use the `-include` option to specify which checks to perform. You must run **Update Timing Netlist** (`update_timing_netlist`) before running **Check Timing** (`check_timing`).

Figure 191. Check Timing Report and No Output Delay Subreport

The screenshot displays two overlapping windows from the Quartus Prime Timing Analyzer. The background window is titled 'Summary' and shows a table of timing checks. The foreground window is titled 'No Output Delay' and shows a detailed report for the 'no_output_delay' check.

Summary	
Check	Number of Issues Found
1 no_clock	0
2 multiple_clock	0
3 pos_neg_clock_domain	0
4 generated_clock	0
5 virtual_clock	0
6 no_input_delay	0
7 no_output_delay	4
8 partial_input_delay	0
9 partial_output_delay	0
10 io_min_max_delay_consistency	0
11 reference_pin	0
12 internal_io_delay	0
13 latency_override	0
14 partial_multicycle	0
15 multicycle_consistency	0
16 loops	0
17 unsupported_latches	0
18 pll_cross_check	0
19 uncertainty	0
20 partial_min_max_delay	0
21 clock_assignments_on_output_ports	0
22 input_delay_assigned_to_clock	0
23 synchronous_data_delay	0

No Output Delay		
	Port	Reason
1	led_zero_on	No output delay was set on output port.
2	led_one_on	No output delay was set on output port.
3	led_two_on	No output delay was set on output port.
4	led_three_on	No output delay was set on output port.

Check Timing can report the following data:

Table 39. Check Timing Report Data

Check Timing Data	Description
no_clock	Reports the registers that do not have at least one clock assignment at their clock pin, including any PLLs without a clock assignment.
multiple_clock	Reports the registers that have more than one clock at their clock pin. If multiple clocks reach a register clock pin, you must define which clock is used for analysis.
generated_clock	Reports the generated clocks that are invalid. Generated clocks must have a source that is triggered by a valid clock.
no_input_delay	Reports the input ports that are not clocks that have no input delay constraint.
no_output_delay	Reports the output ports that have no output delay constraint.
continued...	

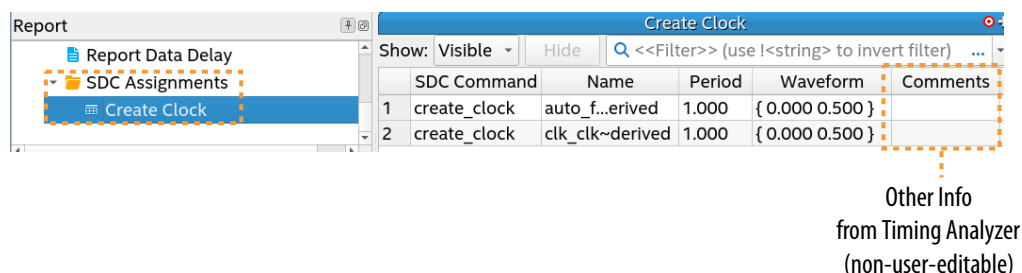
Check Timing Data	Description
partial_input_delay	Reports the input delays that lack a <code>rise-min</code> , <code>fall-min</code> , <code>rise-max</code> , and <code>fall-max</code> constraint set.
partial_output_delay	Reports the output delays that lack a <code>rise-min</code> , <code>fall-min</code> , <code>rise-max</code> , and <code>fall-max</code> constraint set.
io_min_max_delay_consistency	Reports the minimum delay values that you specify in <code>set_input_delay</code> or <code>set_output_delay</code> constraints that are not less than maximum delay values.
reference_pin	Reports the reference pins that you specify with <code>set_input_delay</code> and <code>set_output_delay</code> using the <code>-reference_pin</code> that are invalid. A <code>reference_pin</code> is only valid if the <code>-clock</code> option in the same <code>set_input_delay</code> or <code>set_output_delay</code> command matches the clock that is in the direct fan-in of the <code>reference_pin</code> . Being in the direct fan-in of the <code>reference_pin</code> means that there must be no keepers between the clock and the <code>reference_pin</code> .
latency_override	Reports the instances where the clock latency that you set on a port or pin overrides the more generic clock latency set on a clock. You can set clock latency on a clock, where the latency applies to all keepers clocked by the clock. You can also set clock latency on a port or pin, where the latency applies to registers in the fan-out of the port or pin.
loops	Reports the instances where there are strongly connected components in the netlist. These loops prevent a design from properly analysis. The loops check also reports whether loops exist but are marked so that they are not traversed by timing analysis.
latches	Reports the instances where latches are present in the design and warns that latches may not be analyzed properly. For best results, change your design to remove latches whenever possible.
pos_neg_clock_domain	Reports the instances where any register is clocked by both the rising and falling edges of the same clock. If this scenario is necessary, such as in a clock multiplexer, create two separate clocks that have similar settings and are assigned to the same node.
pll_cross_check	Reports the instances where clocks that are assigned to a PLL do not correspond properly with the PLL settings you define in design files. The subreport specifies the inconsistent settings, or an unmatched number of clocks associated with the PLL.
uncertainty	Reports the clock-to-clock transfers that do not have a clock uncertainty assignment set between the two clocks. If the target device family has <code>derive_clock_uncertainty</code> support, this report also includes the number of user-defined <code>set_clock_uncertainty</code> assignments that have less than recommended clock uncertainty value.
virtual_clock	Reports the unreferenced virtual clocks without constraint.
partial_multicycle	Reports the setup multicycle assignments without a corresponding hold multicycle assignment, and whether each hold multicycle assignment has a corresponding setup multicycle assignment.
multicycle_consistency	Reports the multicycle instances where a setup multicycle does not equal one more than the hold multicycle. Appropriate Hold multicycle assignments are usually one cycle less than setup multicycle assignments.
partial_min_max_delay	Reports the minimum delay assignments without a corresponding maximum delay assignment, and vice versa.
clock_assignments_on_output_ports	Reports the output ports that have clock assignments.
input_delay_assigned_to_clock	Reports the clocks with input delay values set. The Timing Analyzer ignores input delays set on clock ports because clock-as-data analysis takes precedence.
internal_io_delay	Reports the I/O delay constraints that have no specification for <code>-reference_pin</code> and <code>-source_latency_included</code> , and where <code>-clock</code> is a clock that is not assigned to a top level input or output port.

2.5.22. Report SDC

The Timing Analyzer's **Reports** ► **Constraint Diagnostics** ► **Report SDC** command generates the **SDC Assignments** report folder.

The **SDC Assignments** folder contains separately named reports for any SDC constraints found in the current project. For example, the Create Clock report appears in the **SDC Assignments** folder if the project includes any `create_clock` SDC constraint.

Figure 192. Create Clock Report Lists Clock Constraints in Current Project



	SDC Command	Name	Period	Waveform	Comments
1	create_clock	auto_f...erived	1.000	{ 0.000 0.500 }	
2	create_clock	clk_clk~derived	1.000	{ 0.000 0.500 }	

Other Info
from Timing Analyzer
(non-user-editable)

In **SDC Assignment** reports, the **Comments** column may contain non-user-editable information reported by the Timing Analyzer.

Note:

The **Comments** column in the **SDC Assignment** report has no relation to the SDC-on-RTL `-comment` argument that allows you to add comments to `.sdc` constraints. The SDC-on-RTL `-comment` does not appear in **SDC Assignment** reports.

The equivalent scripting command for **SDC Assignments** is `report_sdc`, for example:

```
report_sdc -panel_name sdc_report_panel
```

You can use the `-ignored` option to report any SDC constraints that the Timing Analyzer is ignoring and the reason for ignoring the constraint.

2.5.23. Design Closure Summary

After running the Compiler's Timing Analysis (Signoff) stage, the Timing Analyzer automatically generates the **Design Closure Summary** report. This report summarizes the results of all design closure timing metrics, and provides a summary design closure **Result Flag** of Pass or Fail based on the timing summaries or any individual critical timing failure.

In addition to the various Timing Analyzer summaries, the **Design Closure Summary** also considers and reports the status of timing related Design Assistant Rule checks.

Figure 193. Design Closure Summary

Design Closure Summary		
Q <<Filter>> (use !<string> to invert filter)		
	Panel Name	Result Flag
1	Design Closure Summary	Fail
1	Timing Closure	Pass
1	Setup Summary	Pass
2	Hold Summary	Pass
3	Recovery Summary	Pass
4	Removal Summary	Pass
5	Setup Data Delay Summary	Not Found
6	Recovery Data Delay Summary	Not Found
7	Minimum Pulse Width Summary	Pass
8	Max Skew Summary	Pass
9	Max Clock Skew Summary	Not Found
10	Net Delay Summary	Pass
11	Metastability Summary	Pass
12	Double Data Rate (DDR) Summary	Not Found
13	Transmitter Channel-to-...nel Skew (TCCS) Summary	Not Found
14	Receiver Input Skew Margin (RSKM) Summary	Not Found
2	Design Assistant Summary	High Severity Violations
3	Unconstrained Paths	Fail

2.6. Scripting Timing Analysis

You can optionally use Tcl commands from the Quartus Prime software Tcl Application Programming Interface (API) to constrain, analyze, and collect timing information for your design. This section describes running the Timing Analyzer and setting constraints using Tcl commands. You can alternatively perform these same functions in the Timing Analyzer GUI. Tcl `.sdc` extensions provide additional methods for controlling timing analysis and reporting. The following Tcl packages support the Tcl timing analysis commands this chapter describes:

- `::quartus::sta`
- `::quartus::sdc`
- `::quartus::sdc_ext`

The Quartus Prime software provides the following two executables you can use for timing analysis and reporting:

- The `quartus_sta` executable—can perform timing analysis of your design without running the full Quartus Prime software GUI. If you use separate executables for various stages of design compilation (for example, `quartus_syn`, `quartus_fit`) you must use then use the `quartus_sta` executable to analyze timing for your placed and routed design. The `quartus_sta` executable also supports the interactive command-shell text-based timing reporting or scripted report generation.
- The `quartus_staw` executable—opens a GUI and supports interactive, graphical timing analysis and reporting.

Related Information

- `::quartus::sta`

- `::quartus::staw`
- `::quartus::sdc`
- `::quartus::sdc_ext`

2.6.1. The quartus_sta Executable

The `quartus_sta` executable allows you to run timing analysis without running the full Quartus Prime software GUI. The following methods are available:

- To run the Timing Analyzer in interactive command-shell mode, type the following at the command prompt:

```
quartus_sta -s
```

- To run timing analysis from a system command prompt, type the following command:

```
quartus_sta <options><project_name>
```

You can optionally use command line options available to perform iterative timing analysis on large designs. You can perform a less intensive analysis with `quartus_sta --mode=implement`. In this mode, the Quartus Prime software performs a reduced-corner timing analysis. When you achieve the desired result, you can use `quartus_sta --mode=finalize` to perform final Fitter optimizations and a full multi-corner timing analysis under all operating conditions.

Table 40. quartus_sta Command-Line Options

Command-Line Option	Description
<code>-h --help</code>	Provides help information on <code>quartus_sta</code> .
<code>-t <script file> --script=<script file></code>	Sources the <code><script file></code> .
<code>-s --shell</code>	Enters shell mode.
<code>--tcl_eval <tcl command></code>	Evaluates the Tcl command <code><tcl command></code> .
<code>--do_report_timing</code>	For all clocks in the design, run the following commands: <pre>report_timing -npaths 1 -to_clock \$clock report_timing -setup -npaths 1 -to_clock \$clock report_timing -hold -npaths 1 -to_clock \$clock report_timing -recovery -npaths 1 -to_clock \$clock report_timing -removal -npaths 1 -to_clock \$clock</pre>
<code>--force_dat</code>	Forces an update of the project database with new delay information.
<code>--lower_priority</code>	Lowest the computing priority of the <code>quartus_sta</code> process.
<code>--post_map</code>	Uses the post-map database results.
<code>--sdc=<SDC file></code>	Specifies the <code>.sdc</code> file to use.
<code>--report_script=<custom script></code>	Specifies a custom report script to call.
<code>--speed=<value></code>	Specifies the device speed grade used for timing analysis.
<code>-f <argument file></code>	Specifies a file containing additional command-line arguments.
continued...	

Command-Line Option	Description
<code>-c <revision name> --rev=<revision_name></code>	Specifies which revision and its associated Quartus Prime Settings File (.qsf) to use.
<code>--multicorner</code>	Specifies that the Timing Analyzer generates all slack summary reports for both slow- and fast-corners.
<code>--multicorner[=on off]</code>	Turns off multicorner timing analysis.
<code>--voltage=<value_in_mV></code>	Specifies the device voltage, in mV used for timing analysis.
<code>--temperature=<value_in_C></code>	Specifies the device temperature in degrees Celsius, used for timing analysis.
<code>--parallel [= <num_processors>]</code>	Specifies the number of computer processors to use on a multiprocessor system.
<code>--mode=implement finalize</code>	Regulates whether Timing Analyzer performs a reduced-corner analysis for intermediate operations (implement), or a four-corner analysis for final Fitter optimization and placement (finalize).

2.6.2. The quartus_staw Executable

The `quartus_staw` executable opens a graphical interface to perform interactive timing reporting and analysis of your Quartus Prime project. The following methods are available:

- To run the Timing Analyzer as a stand-alone GUI application, type the following at the command prompt:

```
quartus_staw
```

`quartus_staw` can automatically open a project in the Timing Analyzer GUI if you specify the project name as an argument, as the following example shows. Specifying a project name as an argument saves you time because you do not have to browse to your project file after the GUI opens.

```
quartus_staw <project name>
```

You can specify a Tcl script that contains timing reporting and analysis commands to automatically run each time you open a project in the Timing Analyzer GUI. This feature allows you to easily define a uniform set of reports that the Timing Analyzer GUI always generates, saving you time when you open a new project for interactive timing reporting. Use the `--report_script` option and specify a path to your report script:

```
quartus_staw --report_script=<custom script>
```

You must include only reporting and analysis commands in your report script. Do not include Tcl commands that open or close projects, create the timing netlist, read .sdc files, or update the timing netlist. The report script runs after the Timing Analyzer completely prepares the timing netlist for analysis.

You can optionally specify the directory and file names for the output of the reporting and analysis commands. The output of the reporting commands in the script are visible in the Timing Analyzer GUI, in the folder names you specify, when the reporting script completes. At that point, you can perform other reporting or analysis, as necessary.

2.6.3. Collection Commands

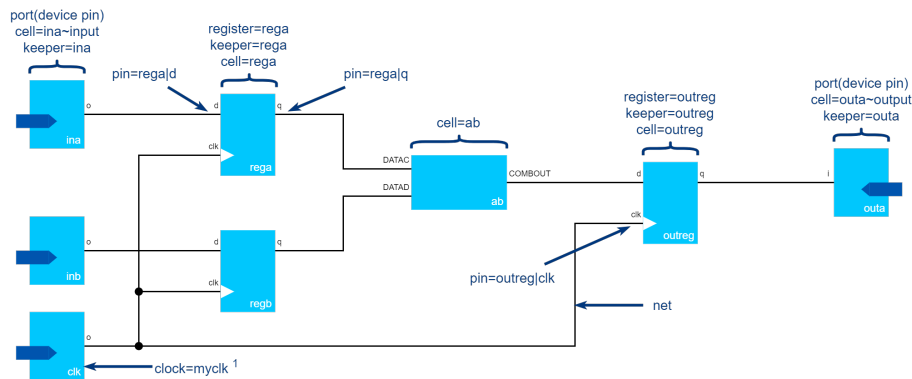
The Timing Analyzer supports functions that return data sets similar to Tcl lists. These data structures, called collections, provide easy access to ports, pins, cells, or nodes from the timing netlist. You can use collection commands with any valid constraints or Tcl commands supported by the Timing Analyzer. Table 1 describes the collection commands that the Timing Analyzer supports. Figure 1 illustrates the elements retrieved by collection commands in a flat design.

Using collection commands instead of raw strings is advisable to ensure precise targeting and minimize ambiguity. Employing the correct collection commands helps ensure that constraints function as intended, and are compatible across different tools. For more information on collections that access the timing netlist, refer to the *Using the Quartus Prime Timing Analyzer* and the *TCL Commands and Packages* section in the *Quartus Prime Pro Edition User Guide: Scripting*.

Table 41. Collection Commands

Command	Collection Returned
all_clocks	Returns a collection of all previously defined clocks in the design.
all_inputs	Returns a collection of all input ports in the design.
all_outputs	Returns a collection of all output ports in the design.
all_registers	Returns a collection of all registers in the design.
get_cells	Returns a collection of cells in the design. All cell names in the collection match the specified pattern. Use wildcards to select multiple cells at once.
get_clocks	Returns a collection of clocks currently defined in the design. All clock names in the collection match the specified pattern. Use wildcards to select multiple clocks at once.
get_nets	Returns a collection of nets in the design. All net names in the collection match the specified pattern. Use wildcards to select multiple nets at once.
get_pins	Returns a collection of pins in the design. All pin names in the collection match the specified pattern. Use wildcards to select multiple pins at once.
get_ports	Returns a collection of ports (design inputs and outputs) in the design. All ports in the collection match the specified pattern.
get_keepers	Returns a collection of non-combinational or "keeper" nodes in the design. Keepers are I/O ports, registers, or clock targets. All keepers in the collection match the specified pattern.
get_nodes	Returns a collection of nodes in the design. All nodes in the collection match the specified pattern.
get_registers	Returns a collection of registers in the design. All registers in the collection match the specified pattern.

Figure 194. Elements Retrieved by Collection Commands in a Flat Design



¹ Before utilizing the `get_clocks` command, it's essential to first define the clock using the `create_clock` command, e.g. `create_clock -period 10 -name myclk [get_ports clk]`

The following example shows various uses of some Tcl commands with collection commands:

```
# Create a simple ten ns clock with a 60 % duty cycle targeting the clk
port(device pin)
create_clock -period 10 -waveform {0 6} -name clk [get_ports clk]
# The following multicycle applies to all paths ending at registers clocked by
clk
set_multicycle_path -to [get_clocks clk] 2
# Create a false path the between clk pin of rega register and the d pin of the
regb register
set_false_path -from [get_pins rega|clk] -to [get_pins regb|d]
```

2.6.3.1. Wildcard Characters

To apply constraints to many nodes in a design, use the “*” and “?” wildcard characters. The “*” wildcard character matches any string; the “?” wildcard character matches any single character.

If you apply a constraint to node `reg*`, the Timing Analyzer searches for and applies the constraint to all design nodes that match the prefix `reg` with any number of following characters, such as `reg`, `reg1`, `reg[2]`, `regbank`, and `reg12bank`.

If you apply a constraint to a node specified as `reg?`, the Timing Analyzer searches and applies the constraint to all design nodes that match the prefix `reg` and any single character following; for example, `reg1`, `rega`, and `reg4`.

2.6.3.2. Adding and Removing Collection Items

Wildcards that you use with collection commands define collection items that the command identifies. For example, if a design contains registers with the name `src0`, `src1`, `src2`, and `dst0`, the collection command `[get_registers src*]` identifies registers `src0`, `src1`, and `src2`, but not register `dst0`. To identify register `dst0`, you must use an additional command, `[get_registers dst*]`. To include `dst0`, you can also specify a collection command `[get_registers {src* dst*}]`.

To modify collections, use the `add_to_collection` and `remove_from_collection` commands. The `add_to_collection` command allows you to add additional items to an existing collection.

add_to_collection Command

`add_to_collection <first collection> <second collection>`

Note: The `add_to_collection` command creates a new collection that is the union of the two collections you specify.

The `remove_from_collection` command allows you to remove items from an existing collection.

remove_from_collection Command

`remove_from_collection <first collection> <second collection>`

The following example shows use of `add_to_collection` to add items to a collection.

Adding Items to a Collection

```
#Setting up initial collection of registers
set regs1 [get_registers a*]
#Setting up initial collection of keepers
set kprs1 [get_keepers b*]
#Creating a new set of registers of $regs1 and $kprs1
set regs_union [add_to_collection $kprs1 $regs1]
#OR
#Creating a new set of registers of $regs1 and b*
#Note that the new collection appends only registers with name b*
# not all keepers
set regs_union [add_to_collection $regs1 b*]
```

In the Quartus Prime software, keepers are I/O ports or registers. An `.sdc` file that includes `get_keepers` is incompatible with third-party timing analysis flows.

Related Information

- [add_to_collection Command, Quartus Prime Help](#)
- [remove_from_collection Command, Quartus Prime Help](#)

2.6.3.3. Query of Collections

You can display the contents of a collection with the `query_collection` command. Use the `-report_format` option to return the contents in a format of one element per line. The `-list_format` option returns the contents in a Tcl list.

```
query_collection -report_format -all $regs_union
```

Use the `get_collection_size` command to return the number of items the collection contains. If your collection is in a variable with the name `col`, use `set num_items [get_collection_size $col]` rather than `set num_items [llength [query_collection -list_format $col]]` for more efficiency.

2.6.3.4. Using the get_pins Command

The `get_pins` command supports options that control the matching behavior of the wildcard character (*). Depending on the combination of options you use, you can make the wildcard character (*) respect or ignore individual levels of hierarchy. The pipe character (|) indicates levels of hierarchy. By default, the wildcard character (*) matches only a single level of hierarchy.

These examples filter the following node and pin names to illustrate function:

- `lvl` (a hierarchy level with the name `lvl`)
- `lvl|dataa` (an input pin in the instance `lvl`)
- `lvl|datab` (an input pin in the instance `lvl`)
- `lvl|cnod` (a combinational node with the name `cnod` in the `lvl` instance)
- `lvl|cnod|datac` (an input pin to the combinational node with the name `cnod`)
- `lvl|cnod|datad` (an input pin to the combinational node `cnod`)

Table 42. Sample Search Strings and Search Results

Search String	Search Result
<code>get_pins * dataa</code>	<code>lvl dataa</code>
<code>get_pins * datac</code>	<code><empty></code> ⁽⁶⁾
<code>get_pins * * datac</code>	<code>lvl cnod datac</code>
<code>get_pins lvl *</code>	<code>lvl dataa, lvl datab</code>
<code>get_pins -hierarchical * * datac</code>	<code><empty></code> ⁽⁶⁾
<code>get_pins -hierarchical lvl *</code>	<code>lvl dataa, lvl datab</code>
<code>get_pins -hierarchical * datac</code>	<code>lvl cnod datac</code>
<code>get_pins -hierarchical lvl * datac</code>	<code><empty></code> ⁽⁶⁾
<code>get_pins -compatibility_mode * datac</code>	<code>lvl cnod datac</code> ⁽⁷⁾
<code>get_pins -compatibility_mode * * datac</code>	<code>lvl cnod datac</code>

The default method separates hierarchy levels of instances from nodes and pins with the pipe character (|). A match occurs when the levels of hierarchy match, and the string values including wildcards match the instance or pin names. For example, the command `get_pins <instance_name>|*|datac` returns all the `datac` pins for registers in a given instance. However, the command `get_pins *|datac` returns an empty collection because the levels of hierarchy do not match.

Use the `-hierarchical` matching scheme to return a collection of cells or pins in all hierarchies of your design.

⁽⁶⁾ The search result is `<empty>` because the wildcard character (*) does not match more than one hierarchy level, that a pipe character (|) indicates, by default. This command matches any pin with the name `datac` in instances at the top level of the design.

⁽⁷⁾ When you use `-compatibility_mode`, the Timing Analyzer does not treat pipe characters (|) as special characters when you use the characters with wildcards.

For example, the command `get_pins -hierarchical *|datac` returns all the `datac` pins for all registers in your design. However, the command `get_pins -hierarchical *|*|datac` returns an empty collection because more than one pipe character (|) is not supported.

The `-compatibility_mode` option returns collections matching wildcard strings through any number of hierarchy levels. For example, an asterisk can match a pipe character when using `-compatibility_mode`.

2.7. Using the Quartus Prime Timing Analyzer Document Revision History

Document Version	Quartus Prime Version	Changes
2025.12.04	25.3	- Updated the <i>Report Clocks and Clock Network</i> topic to remove descriptions about Open Clock Network Viewer option and added description about the Schematic View tab. - Updated the <i>Report Clock Network Report</i> figure.
2025.09.29	25.3	- Added reference to report search filter bar to <i>Step 4: Analyzer Timing Reports</i> topic. - Applied Altera rebranding throughout.
2025.04.18	25.1	Added note to <i>Cross-Probing with Design Assistant</i> topic.
2025.03.31	25.1	- Added new <i>Design Closure Summary</i> topic. - Added new <i>Understanding Report Register Spread Data</i> topic.
2024.11.15	24.3	Added links to new <i>Troubleshooting Common Timing Issues</i> chapter.
2024.09.30	24.3	- Revised <i>Applying Timing Constraints</i> topic for SDC correctness. - Revised <i>Using Fitter Overconstraints</i> topic for all supported device families. - Revised <i>Clock Constraints</i> topic and linked to <i>Exclusive Clock Groups</i> topic. - Added new <i>Data Delay (set_data_delay)</i> topic. - Revised <i>Using Entity-based SDC-on-RTL Constraints</i> topic for greater clarity. - Revised <i>Targeting Constraints to Module Inputs and Outputs</i> topic. - Revised <i>Entity Based SDC-on-RTL Constraint Scope</i> topic for <code>get_ports</code>. - Updated <i>Constraining Design Partition Ports</i> topic for deprecation of the <code>-blackbox</code> argument. - Added new <i>Deprecation of the -blackbox Argument</i> topic. - Revised <i>Input Constraints</i> topic for deprecation of the <code>-blackbox</code> argument. - Revised <i>Output Constraints</i> topic for deprecation of the <code>-blackbox</code> argument. - Added new <i>Creating Efficient Timing Exceptions</i> topic. - Updated <i>Report Asynchronous CDC</i> topic for new SDC Statistics and SDC Schematic tabs. - Updated <i>Collection Commands</i> topic for latest available commands.
2024.04.01	24.1	- Added new <i>Using Timing Constraints throughout the Design Flow</i> topic. - Revised the <i>Step 1: Specify General Timing Analyzer Settings</i> topic to add link to SDC-on-RTL info. - Revised <i>Step 3: Run the Timing Analyzer</i> topic for Early Timing Analysis and SDC-on-RTL. - Revised <i>Applying Timing Constraints</i> section for Early Timing Analysis and SDC-on-RTL.
continued...		

Document Version	Quartus Prime Version	Changes
		- Revised <i>SDC File Precedence</i> topic for SDC-on-RTL. - Revised <i>Iteratively Modifying Constraints</i> topic for SDC-on-RTL. - Added new <i>Using Entity-based SDC-on-RTL Constraints</i> section. - Revised <i>Using Entity-bound SDC Files</i> section for SDC-on-RTL. - Renamed <i>Generate Timing Reports</i> topic to <i>Timing Report Descriptions</i> and relocated. - Revised the <i>Timing Report Descriptions</i> topic to reference quartus_staw method. - Revised <i>Report Timing By Source File</i> topic to mention ability to open the source file in a text editor from the report. - Revised <i>Report Metastability</i> topic to mention ability to run Report Neighbor Paths, default toggle rate, and more report details. - Revised <i>Report Route Net of Interest</i> topic to mention ability to run report_timing from the report. - Revised <i>Timing Analyzer Scripting</i> topic to reference quartus_sta and quartus_staw executables. - Added more details about derived_clock naming to <i>Automatic Clock Detection and Constraint Creation</i> topic. - Added new <i>The quartus_staw Executable</i> topic. - Enhanced <i>Using Fitter Overconstraints</i> topic for details specific to the Hyperflex architecture, new examples, and <i>Other Overconstraint Combinations</i> table of examples. - Added new <i>Report SDC</i> topic explaining report and Comments column. - Revised constraint precedence in <i>Maximum Skew</i> topic.
2023.12.04	23.4	- Revised the <i>Applying Timing Constraints</i> topic to add introductory information and context about typical constraints. - Added new <i>Check Timing</i> topic describing Check Timing command and report. - Added new cross-referencing links throughout <i>Applying Timing Constraints</i> section.
2023.10.02	23.3	- Updated compilation dashboard image in <i>Step 3: Run the Timing Analyzer</i>. - Revised the following topics entirely: <i>Using Entity-bound SDC Files</i> <i>Entity-bound Constraint Scope</i> <i>Entity-bound Constraint Examples</i>
2023.08.03	23.1	- Corrected typo in command example in <i>Report Data Delay</i> topic. - Corrected typo in command example in <i>Constraining CDC Paths</i> topic. - Corrected typo in command example in <i>Maximum Skew</i> topic. - Added <i>Promoting Critical Warnings to Errors</i> topic.
2023.04.03	23.1	Revised updated image and description in <i>Report Register Description</i> for new Without a Control Signal and Synchronous Load columns and data.
2023.01.31	22.4	Revised outdated timing model descriptions in <i>Setting the Operating Conditions for Timing Analysis</i> topic.
2022.09.26	22.3	- Renamed the report title "Report Reset Statistics" as "Report Register Statistics." - Revised the <i>Report Register Statistics</i> topic to describe some new features of the report. - Revised <i>Report Fmax Summary</i> topic to refer to get_clock_fmax_info command and provide more detail. - Revised <i>Example SDC Constraints for External Clock Mux</i> to replace logically_exclusive with physically_exclusive.
continued...		

Document Version	Quartus Prime Version	Changes
2022.03.28	22.1	- Described new Clock Network Viewer in <i>Report Clocks and Clock Networks</i> topic. - Updated <i>Report Register Spread</i> topic for new angle and area spread types and <code>-to_clock</code> filtering. - Added new <i>Report Timing By Source Files</i> topic. - Added new <i>Report Metastability</i> topic. - Added new <i>Report Bottlenecks</i> topic. - Added new <i>Specifying Custom Bottleneck Criteria</i> topic. - Revised Basic .sdc Constraints Example in <i>Example Circuit and SDC File</i> topic. - Added more detailed constraint example and diagram to <i>Set Clock Groups</i> topic. - Revised scripting examples and screenshots in <i>Correlating Constraints to the Timing Report</i> topic. - Revised scripting example in <i>Creating Base Clocks</i> topic. - Revised scripting example in <i>Clock Divider Example</i> topic. - Revised <i>Constraining CDC Paths</i> topic. - Added note about referenced SDCs within IP to <i>SDC File Precedence</i> topic.
2021.09.27	21.3	- Updated name of Report Hierarchical Retiming Restrictions command and report to Report Retiming Restrictions. - Added <i>Constraining CDC Paths</i> topic and linked to related topics. - Updated <i>Setting the Operating Conditions</i> with details about operating condition nomenclature. - Replaced Custom Reports, Device Specific, Diagnostic, and Slack report folder names throughout. - Added <i>Report Exceptions and Exceptions Reachability</i> topic describing new report. - Added <i>Report Clocks and Clock Networks</i> topic describing new report. - Added <i>Report Data Delay</i> topic describing report. - Mentioned option to view multiple before and after paths in <i>Report Neighbor Paths</i> topic. - Added <code>set_clock_groups</code> to <i>Timing Exception Precedence</i> - Updated content of Extra Info tab in <i>Report Timing</i> topic. - Corrected the <code>set_clock_groups -group A -group B</code> table in the <i>Creating Clock Groups</i> topic. - Removed <i>Report Custom CDC Viewer Command</i> topic. - Revised assignment examples in <i>Exclusive Clock Groups</i> topic.
2021.04.05	21.1	- Added: "Report Reset Statistics" "Report Asynchronous CDC" - Two new fields to "Report Pipelining Information". - New screenshots to "Report Logic Depth" and "Report Neighbor Paths" <code>get_registers</code> and <code>get_keepers</code> to "Collection Commands". - Removed <code>-include</code> and <code>-exclude</code> options from "Maximum Skew"
2021.02.22	20.3	Added extra <code>SDC_ENTITY_FILE</code> info to "Using Entity-bound SDC Files"
2020.09.28	20.3	- Added "Cross Probing with Design Assistant" section. - Updated "Step 3: Run the Timing Analyzer" for multiple methods. - Updated "Step 1: Specify Timing Analyzer Settings for new tabbed dialog box and options. - Added new "Report Register Spread," "Report Route Net of Interest," "Report Hierarchical Retiming Restrictions," and "Report Pipelining Information" topics.
continued...		

Document Version	Quartus Prime Version	Changes
		- Updated "Report Clock Transfers" topic for new data columns. - Updated "Report Timing" topic for Extra Info tab data. - Updated "Report Fmax Summary," "Report Logic Depth," "Report Neighbor Paths," "Report CDC Viewer," and "Report Custom CDC Viewer" topics for latest GUI and consistency.
2020.04.13	20.1	- Added details and Agilex 7 device examples to "Setting the Operating Conditions" topic. - Added "Report Logic Level Depth" topic. - Added "Report Neighbor Paths" topic. - Added "Enabling Time Borrowing Optimization" topic. - Added "Report Time Borrowing Data" topic.
2019.07.15	19.2	- Updated "Setting Operating Conditions" for SmartVID timing models. - Added step for setting operating conditions to "Step 4: Run Timing Analysis." - Added details about exclusive paths to "Maximum Skew" topic. - Added GUI steps for creating entity-bound SDC files to "Using Entity-bound SDC Files" topic.
2019.04.15	19.1	- Corrected typo in "Timing Constraint Precedence" topic. - Corrected typo in "Maximum Skew" topic. - Updated "Viewing Design Assistant Recommendations" for latest GUI changes.
2018.11.07	18.1	Improved description and diagram for "Exclusive Clock Groups" topic.
2018.09.24	18.1	- Added "Using Entity-bound SDC Files" topic. - Added "Scoping Entity-bound Constraints" topic. - Added "Entity-bound Constraint Examples" topic. - Revised "Basic Timing Analysis Flow" section to add sequential step organization, update steps, and add supporting screenshots. - Added Timing Analyzer screenshot to "Using the Timing Analyzer" topic. - Removed "Creating a Constraint File from Templates with the Text Editor" topic due to limitations of this feature in this version of the software. - Retitled "SDC Constraint Creation Summary" to "Dual Clock SDC Example." - Retitled "Default Settings" to "Default Multicycle Analysis." - Retitled "SDC (Clock and Exception) Assignments on Blackbox Ports" to "Constraining Design Partition Ports." - Added "Viewing Design Assistant Recommendations" topic.
2018.05.07	18.0	First release as part of the stand-alone <i>Timing Analyzer User Guide</i>
2017.11.27	17.1.0	- Removed outdated figure: Design Flow with the Timing Analyzer. - Updated Performing an Initial Analysis and Synthesis topic with Quartus Prime Pro Edition commands.
2017.11.06	17.1	Updated <i>Using Fitter Overconstraints</i> topic for Stratix 10 support.
2017.05.08	17.0	- Added <i>Using Fitter Overconstraints</i> topic. - Added <i>Clock Domain Crossing report</i> topics
2016.10.31	16.1	- Implemented Intel rebranding. - Added support for <code>-blackbox</code> option with <code>set_input_delay</code>, <code>set_output_delay</code>, <code>remove_input_delay</code>, <code>remove_output_delay</code>.
2016.05.03	16.0	Added new topic: SCDS (Clock and Exception) Assignments on Blackbox Ports
continued...		

Document Version	Quartus Prime Version	Changes
2015.11.02	15.1.0	- Changed instances of <i>Quartus II</i> to <i>Quartus Prime</i>. - Added a description of running three- and four-corner analysis with <code>--mode=implement finalize</code>. - Added description for new <code>set_operating_conditions</code> UI.
2015.05.04	15.0.0	Added and updated contents in support of new timing algorithms for Arria 10: - Enhanced Timing Analysis for Arria 10 - Maximum Skew (<code>set_max_skew</code> command) - Net Delay (<code>set_net_delay</code> command) - Create Generated Clocks (clock-as-data example)
2014.12.15	14.1	Major reorganization. Revised and added content to the following topic areas: - Timing Constraints - Create Clocks and Clock Constraints - Creating Generated Clocks - Creating Clock Groups - Clock Uncertainty - Running the Timing Analyzer - Generating Timing Reports - Understanding Results - Constraining and Analyzing with Tcl Commands
August 2014	14.0a10.0	Added command line compilation requirements for Arria 10 devices.
June 2014	14.0	- Minor updates. - Updated format.
November 2013	13.1	Removed HardCopy device information.
June 2012	12.0	- Reorganized chapter. - Added "Creating a Constraint File from Quartus Prime Templates with the Quartus Prime Text Editor" section on creating an SDC constraints file with the Insert Template dialog box. - Added "Identifying the Quartus Prime Software Executable from the SDC File" section. - Revised multicyle exceptions section.
November 2011	11.1	- Consolidated content from the Best Practices for the Quartus Prime Timing Analyzer chapter. - Changed to new document template.
May 2011	11.0	Updated to improve flow. Minor editorial updates.
December 2010	10.1	- Changed to new document template. - Revised and reorganized entire chapter. - Linked to Quartus Prime Help.
July 2010	10.0	Updated to link to content on SDC commands and the Timing Analyzer GUI in Quartus Prime Help.
November 2009	9.1	Updated for the Quartus Prime software version 9.1, including: - Added information about commands for adding and removing items from collections - Added information about the <code>set_timing_derate</code> and <code>report_skew</code> commands - Added information about worst-case timing reporting - Minor editorial updates
November 2008	8.1	Updated for the Quartus Prime software version 8.1, including:

Document Version	Quartus Prime Version	Changes
		- Added the following sections: "set_net_delay" on page 7-42 "Annotated Delay" on page 7-49 "report_net_delay" on page 7-66 - Updated the descriptions of the <code>-append</code> and <code>-file <name></code> options in tables throughout the chapter - Updated entire chapter using 8½" × 11" chapter template - Minor editorial updates

2.8. Quartus Prime Pro Edition User Guide: Timing Analyzer Archive

For the latest and previous versions of this user guide, refer to [Quartus Prime Pro Edition User Guide: Timing Analyzer](#). If an IP or software version is not listed, the user guide for the previous IP or software version applies.



3. Troubleshooting Common Timing Issues

This chapter uses simple examples to demonstrate how to identify and troubleshoot common timing issues using the Quartus Prime Timing Analyzer.

3.1. Troubleshooting Asynchronous Clock Domain Crossing (CDC) Timing Issues

Clock-domain crossings (CDCs) are essential elements of digital design. This section describes how to use the Quartus Prime Timing Analyzer and Design Assistant to manage and troubleshoot CDC timing issues.

As digital designs become more complex, it is crucial to ensure proper transitions between different clock domains. This section emphasizes the importance of precisely defining CDCs and timing constraints, as essential steps in preventing metastability and achieving timing closure.

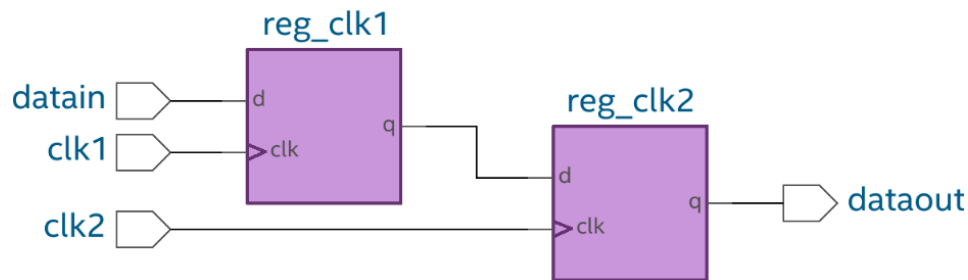
3.1.1. CDC Timing Overview

Digital clocking provides the pulses that synchronize the operations of digital circuits. These pulses coordinate the timing of data movement and processing within a circuit. A clock domain is a specific region of a digital system where the same clock signal governs all the sequential elements, such as flipflops and registers.

Complex electronic systems typically include multiple clock domains to meet the diverse operational demands of sophisticated systems. As system complexity increases, the system often requires different sections to function at distinct frequencies or phases. Independent clocks are often necessary when interfacing to different I/Os. For example, when data coming in from two ethernet cables is clocked completely independently of each other. Using this approach allows for specialized processing blocks to run at their optimal speed, enhancing performance and efficiency.

The use of multiple clock domains within a single design enables fine-tuned control over each subsystem, but also introduces the challenge of managing the interactions between these asynchronous environments. These challenges are particularly evident when you must safely transfer data from one domain to another.

Figure 195. Single CDC Design without Synchronizers



In the [Single CDC Design without Synchronizers](#) example, `clk1` triggers the flipflop `reg_clk1`. `clk2` triggers the flipflop `reg_clk2`. There is a data transfer between `reg_clk1` and `reg_clk2`.

Clock domain crossings (CDCs) occur when a signal or data transfers from one clock domain to another within a digital system. Each clock domain operates under its clock signal. When a signal crosses into a domain that a different clock governs, there is a risk that the receiving domain's flipflops do not properly latch the incoming data. This risk is present because of the potential misalignment of the source and destination clock edges, which can lead to setup and hold time violations.

Metastability is a condition that can arise when a flipflop receives an input signal that changes too close to the clock edge, and not within the required setup or the hold time. In such cases, the flipflop can fail to resolve the input to a definitive logical '0' or '1' in the time expected. Instead, the output can enter an unstable, mid-value voltage that requires excessive time (beyond t_{CO}) to enter a stable 0 or 1 state. This metastability state can propagate through the circuit, leading to unpredictable behavior, data corruption, and system errors.

In the [Single CDC Design without Synchronizers](#) diagram, `reg_clk1` data crosses clock domains when `reg_clk2` samples this data on the rising edge of `clk2`. If you do not manage CDCs correctly, the probability of metastability increases, especially in systems with high clock frequencies. To mitigate the risks associated with CDC, you must use synchronization techniques, such as using chains of flipflops as synchronizers, to allow enough time for the signal to stabilize before the receiving clock domain uses the signal.

Related Information

- [Metastability Analysis](#) on page 16
- [Constraining CDC Paths](#) on page 93

3.1.2. Identifying CDC Timing Issues Using Design Assistant

The Quartus Prime software includes various tools to help you identify and resolve potential CDC timing issues. For example, the Design Assistant automatically identify violations against predefined design guidelines during compilation stages. With respect to CDC, each single-bit asynchronous data transfer unfollowed by a synchronizer chain violates the CDC-50001 rule. Design Assistant provides a description of the rule, a summary of each bit crossing between multiple clock domains, and a recommendation to solve the issue in your design.

For the [Single CDC without Synchronizers](#) example, the Design Assistant reports the asynchronous transfer between `reg_clk1` and `reg_clk2` because a synchronizer chain does not follow the respective CDC.

Figure 196. Design Assistant Reporting CDC-50001 Violation

	Rule	Severity	Tags	Violations
1	CDC-50001 - 1-Bit Asynchronous Transfer Not Synchronized	High	synchronizer	1
2	CDC-50002 - 1-Bit Asynchronous Transfer with Insufficient Constraints	High	synchronizer	0
3	CDC-50003 - CE-Type CDC Transfer with Insufficient Constraints	High	cdc-bus	0
4	CDC-50004 - MUX-type CDC Transfer with Insufficient Constraints	High	cdc-bus	0
5	CDC-50005 - CDC Bus Construction: Chains of Different Lengths	High	synchronizer, cdc-bus	0

Depending on the number of reports, following all the paths reported in the Design Assistant window may be complicated, especially since there is a predefined limit on how many paths are reported in the Design Assistant window. To investigate CDC violations further, you can review specific timing reports, as the next section describes.

Related Information

[Resolve Violation: 1-Bit Asynchronous Transfer Not Synchronized](#) on page 200

3.1.3. Identifying CDC Timing Issues Using Timing Reports

The Timing Analyzer generates dedicated reports for CDC analysis that report detailed information for each path crossing between clock domains. The Timing Analyzer can report on all paths and has no predefined limit on the number of paths reported.

The following topics describe the Timing Analyzer reports to help identify and troubleshoot CDC timing issues.

3.1.3.1. Using Report Clock Transfers to Identify CDC Issues

The Timing Analyzer's **Report Clock Transfers** report provides additional detail on each path in clock transfers (CDCs). Open this report by clicking **Reports > Clock Domain Crossings > Report Clock Transfers** in the Timing Analyzer.

Report Clock Transfers generates the **Setup Transfers** report and the **Hold Transfers** report displaying all possible clock-to-clock transfers. This data includes the rising clock edge to rising clock edge (RR), falling clock edge to rising clock edge (FR), rising clock edge to falling clock edge (RF), and falling clock edge to falling clock edge (FF) paths.

For every clock-to-clock transfer in the design, the **Setup Transfers** report and the **Hold Transfers** report list all the possible clock-to-clock crossing combinations. The report uses the following color coding by default:

- White report cell—indicates the number of paths that exist for a given transfer.
- Light-gray report cell with 'false path'—indicates the crossing combination includes false paths.
- Dark gray report cell—indicates the crossing combination lacks existing paths in the design.

In addition, the **Clock Pair Classification** column provides the following relevant information to resolve CDC issues:

- If the **From Clock** and **To Clock** are the same or synchronously related, the paths in this category classify as **Intra-Clock (Timed Safe)** or **Inter-Clock Synchronous (Timed Safe)**, and timing constraints may or may not exist.
- When the **From Clock** and **To Clock** do not share a common parent clock, they are asynchronous. Asynchronous clocks lack a predictable phase relationship, which can lead to setup and hold violations if signals directly transfer between them. Without constraints to guide the Fitter, the Compiler cannot account for the timing challenges inherent in CDCs.
- Paths crossing between unrelated clock domains lacking synchronizers, or without appropriate timing constraints, classify as **Asynchronous (Timed Unsafe)**.

For the [Single CDC without Synchronizers](#) example, the Timing Analyzer reports the path between `reg_clk1` and `reg_clk2` as **Asynchronous (Timed Unsafe)**.

Figure 197. Combined Setup Transfers and Hold Transfers Reports Showing Asynchronous (Timed Unsafe) Transfers

Setup Transfers							
Show:	Visible	Hide	Q <<Filter>> (use !<string> to invert filter)				
	From Clock	To Clock	RR Paths	FR Paths	RF Paths	FF Paths	Clock Pair Classification
1	clk1	clk2	1	0	0	0	Asynchronous (Timed Unsafe)
2	clk1	clk1	1	0	0	0	Intra-Clock (Timed Safe)
3	clk2	clk2	1	0	0	0	Intra-Clock (Timed Safe)
An "Asynchronous (Timed Unsafe)" transfer occurs when the From & To clocks are asynchronous and no timing constraints exist. Entries labeled "false path" only account for clock-to-clock false paths. Path-based false path counts are shown in rows with "Ignored (Not Timed)" classifications							
Hold Transfers							
Show:	Visible	Hide	Q <<Filter>> (use !<string> to invert filter)				
	From Clock	To Clock	RR Paths	FR Paths	RF Paths	FF Paths	Clock Pair Classification
1	clk1	clk2	1	0	0	0	Asynchronous (Timed Unsafe)
2	clk1	clk1	1	0	0	0	Intra-Clock (Timed Safe)
3	clk2	clk2	1	0	0	0	Intra-Clock (Timed Safe)
An "Asynchronous (Timed Unsafe)" transfer occurs when the From & To clocks are asynchronous and no timing constraints exist. Entries labeled "false path" only account for clock-to-clock false paths. Path-based false path counts are shown in rows with "Ignored (Not Timed)" classifications							

3.1.3.2. Using Report Asynchronous CDC to Identify CDC Issues

Synchronizers are a fundamental solution employed to manage CDCs in digital circuits. When signals pass between different clock domains, synchronizers act as a buffer to ensure that these signals are correctly captured and stabilized, regardless of any timing discrepancies between the asynchronous clock domains.

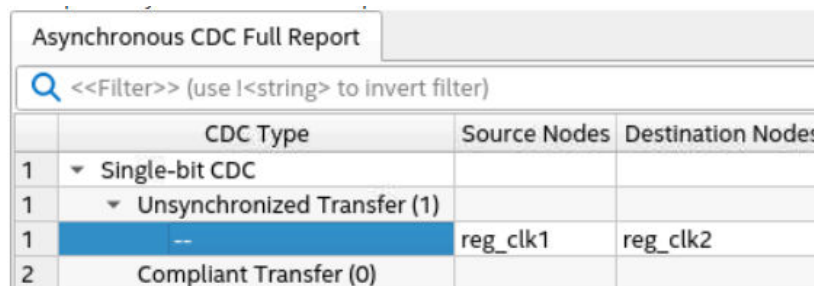
Typically, a synchronizer consists of a series of D flipflops arranged in a chain. The first flipflop captures the incoming signal from the source domain, while subsequent flipflops help to filter out the metastability that might occur due to the signal arriving too close to the clock edge.

The Timing Analyzer's **Report Asynchronous CDC** report provides additional detail on asynchronous CDCs in the design. Open this report by clicking **Reports > Clock Domain Crossings > Report Asynchronous CDC** in the Timing Analyzer. Use this report to ensure the Timing Analyzer correctly detects all unintended CDCs.

Report Asynchronous CDC provides an overview of different topologies detected to understand the reasons why a CDC is unsafe, allowing you to classify the asynchronous clock-domain-crossing transfers in your design. The data is organized according to the following categories:

- **Single-bit CDC**
- **Multi-bit CDC**
- **Asynchronous Reset CDC**

Figure 198. Report Asynchronous CDC Report



	CDC Type	Source Nodes	Destination Nodes
1	▼ Single-bit CDC		
1	▼ Unsynchronized Transfer (1)		
1	---	reg_clk1	reg_clk2
2	Compliant Transfer (0)		

Clicking on a row that contains a CDC displays detailed information on that CDC in the **CDC Statistics** table. The information available varies depending on the CDC's topology.

Figure 199. CDC Statistic Tab Shows Unconstrained CDC Path

CDC Statistics		SDC Statistics	Schematic View
	Property	Value	
2	▼ Source Registers		
1	reg_clk1		
3	Number of Discovered Synchronization Registers	1	
4	▼ Discovered Synchronization Register	Synchronizer Stage Number	
1	reg_clk2	1	
5	Method of Synchronizer Identification	Automatic	
6	▼ Chain Length Requested by User	3	
1	User-requested chain l...s in the synchronizer.		
7	▼ The destination register...ta synchronizer because:		
1	Its d-pin is fed by a s...nchronous clock domain.		
8	▼ The synchronizer is termi...he last register because:		
1	▼ The following nodes ca...ynchronizer registers:		
4			

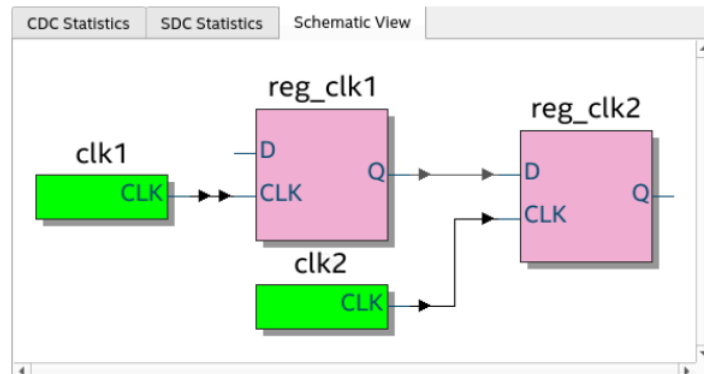
The **SDC Statistics** tab shows any SDC constraints associated with the CDC path.

Figure 200. SDC Statistics Tab

CDC Statistics		SDC Statistics	Schematic View
Exception Statistics			
	Source Node	Destination Node	SDC Constraint
1	No SDC constraints related to the chain	--	--
Clock Group Constraint Statistics			
	Clock Group Constraint	Type	SDC Constraint
1	No clock group constraints related to the chain	--	--

The **Schematic View** tab shows a diagram of the identified CDC topology.

Figure 201. Schematic View Tab



For the [Single CDC without Synchronizers](#) example, the CDC between reg_clk1 and reg_clk2 is reported under the **Single-bit CDC** category.

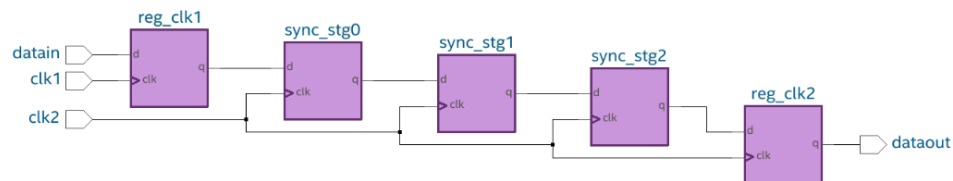
3.1.3.3. Identifying Synchronizer CDC Issues

The use of synchronizers is common in digital design to enable reliable communication and data integrity across different clock domains.

However, improper synchronizer implementation is a common cause of CDC issues. For correct synchronizer implementation, you must ensure the following:

- The destination of an asynchronous transfer forms a chain of two or more registers, with each register in the same clock domain as the destination of the transfer.
- No combinational logic is present between any of the registers in the chain.
- No combinational logic is present on the path to the first register in the chain.

Figure 202. Synchronizer Example



The following Verilog HDL and VHDL examples show how to define a three-stage synchronizer. These synchronizers take an asynchronous input signal from one clock domain, and then produce a synchronized output signal for use in another clock domain.

Both examples implement the synchronizer as a chain of three flipflops (`sync_ff`). The asynchronous input signal (`async_in`) is captured by the first flipflop on the rising edge of the clock signal (`clk`) of the destination domain. The signal then propagates through the second and third flipflops, with each stage providing additional time for the signal to stabilize. The output of the third flipflop (`sync_out`) is then used within the destination clock domain, significantly reducing the risk of metastability.

Three-Stage Synchronizer Example (Verilog HDL)

```
module three_stage_synchronizer (
    input wire clk,           // Clock signal of the destination domain
    input wire async_in,     // Asynchronous input signal from the source domain
    output reg sync_out      // Synchronized output signal for the destination
    domain
);

reg [2:0] sync_ff; // 3 flip-flops for the synchronizer chain

always @(posedge clk) begin
    sync_ff[0] <= async_in;    // First stage captures the asynchronous input
    sync_ff[1] <= sync_ff[0];  // Second stage
    sync_ff[2] <= sync_ff[1];  // Third stage
end
assign sync_out = sync_ff[2]; // Output from the third stage

endmodule
```

Three-Stage Synchronizer Example (VHDL)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity three_stage_synchronizer is
    Port (
        clk : in STD_LOGIC;           -- Clock signal of the destination domain
        async_in : in STD_LOGIC;      -- Asynchronous input signal from the
        source domain
        sync_out : out STD_LOGIC      -- Synchronized output signal for the
        destination domain
    );
end three_stage_synchronizer;
architecture Behavioral of three_stage_synchronizer is
    signal sync_ff : STD_LOGIC_VECTOR(2 downto 0); -- 3 flip-flops for the
    synchronizer chain
begin

    process(clk)
    begin
        if rising_edge(clk) then
            sync_ff(0) <= async_in;    -- First stage captures the
            asynchronous input
            sync_ff(1) <= sync_ff(0);  -- Second stage
            sync_ff(2) <= sync_ff(1);  -- Third stage
        end if;
    end process;

    sync_out <= sync_ff(2);           -- Output from the third stage

end Behavioral;
```

The following topics describe how to identify and constrain CDCs in synchronizers.

3.1.3.3.1. Reviewing Reports about Synchronizers

If you implement synchronizers in your design, the **Report Asynchronous CDC** report identifies the topology that you use. For example, the following figure shows an unconstrained transfer in the **Asynchronous CDC Full Report** tab.

Figure 203. Unconstrained Transfer in the Asynchronous CDC Full Report Tab

Asynchronous CDC Full Report			
Q <<Filter>> (use !<string> to invert filter)			
	CDC Type	Source Nodes	Destination Nodes
1	Single-bit CDC		
1	Unconstrained Transfer (1)		
1	--	reg_clk1	my_sync[...nc_ff[0]
2	Compliant Transfer (0)		

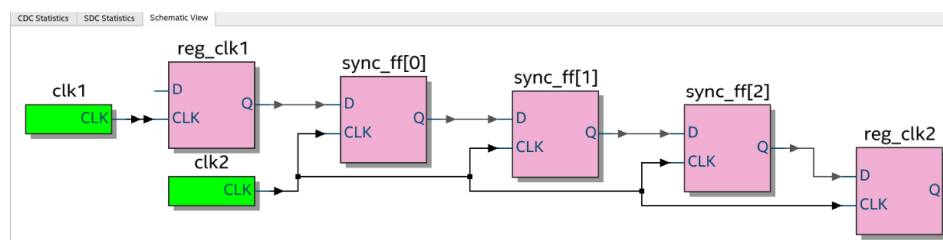
In addition, the **CDC Statistics** tab presents basic information about the synchronizer, such as the source register and the registers that form the synchronizer.

Figure 204. CDC Statistics Tab Shows Synchronizer Data

CDC Statistics		SDC Statistics	Schematic View
	Property		
1	Number of Source Registers		
2	Source Registers		
1	reg_clk1		
3	Number of Discovered Synchronization Registers		
4	Discovered Synchronization Register		
1	my_sync[sync_ff[0]		
2	my_sync[sync_ff[1]		
3	my_sync[sync_ff[2]		
4	reg_clk2		

The **Schematic View** tab shows a representation of the synchronizer identified in the crossing.

Figure 205. Schematic View Shows Synchronizer Schematic



3.1.3.3.2. Constraining Synchronizers in CDCs

Using synchronizers in cross-domain crossings requires that you specify timing constraints to guide Analysis & Synthesis and the Fitter to ensure the synchronizer operates within safe timing margins to prevent metastability.

You can identify any unconstrained synchronizers in the **Report Asynchronous CDC** report under **Unconstrained Transfer**. In addition, the Design Assistant reports unconstrained synchronizers under the **CDC-50002 - 1-Bit Asynchronous Transfer with Insufficient Constraints** rule.

Figure 206. Design Assistant Reports Unconstrained Synchronizers

	Rule	Severity	Tags	Violations
1	CDC-50002 - 1-Bit Asynchronous Transfer with Insufficient Constraints	High	synchronizer	1
2	CDC-50001 - 1-Bit Asynchronous Transfer Not Synchronized	High	synchronizer	0
3	CDC-50003 - CE-Type CDC Transfer with Insufficient Constraints	High	cdc-bus	0
4	CDC-50004 - MUX-type CDC Transfer with Insufficient Constraints	High	cdc-bus	0
5	CDC-50005 - CDC Bus Construction: Chains of Different Lengths	High	synchronizer-c-bus	0

In the Quartus Prime Timing Analyzer, you specify timing constraints using Synopsys Design Constraints (SDC) files. With respect to clock domain crossings, SDC timing constraints allow you to identify paths that cross clock domains. For signals that you can safely synchronize using a CDC synchronizer, use SDC constraints to define the path to the first register in the synchronizer chain as false path. This false path constraint informs the Timing Analyzer not to consider these paths for timing analysis because the synchronizer handles them.

SDC also allows you to define clock groups that you can use to specify asynchronous clock relationships. Specifying such a relationship is important for CDC analysis because it indicates to the Timing Analyzer whether clocks do not have a phase or frequency relationship and may require special consideration for data transfer.

The following example shows basic SDC constraints for a synchronizer. The two clock domain definitions are first. Next, the `set_clock_groups` command clarifies that both clocks are asynchronous and are unrelated. Alternatively, `set_false_path` excludes the path that crosses between clock domains from timing analysis.

```
# Define the clocks (assuming clock names are clk1 and clk2 for the two domains)
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 20 [get_ports clk2]

# Group the clocks to specify that they are asynchronous to each other
set_clock_groups -asynchronous -group [get_clocks clk1] -group [get_clocks clk2]
# Specify false path for the CDC path between the two clock domains
set_false_path -from [get_registers reg_clk1] -to [get_registers sync_ff[0]]
```

When you appropriately define and constrain a CDC, the **Report Asynchronous CDC** report indicates that the CDC is time-compliant, and provides additional details about SDC constraints on the **SDC Statistics** tab. Use the **SDC Statistics** tab to corroborate the correct application of the false path between the register in the transmitter clock domain, and the first stage of the synchronizer in the receiver clock domain.

Figure 207. Asynchronous CDC Full Report Indicates CDC Complies with Constraints

Asynchronous CDC Full Report			
Q <<Filter>> (use !<string> to invert filter)			
	CDC Type	Source Nodes	Destination Nodes
1	Single-bit CDC		
1	Compliant Transfer (1)		
1	--	reg_clk1	my_sync sync_ff[0]
2	Synchronizer ... for MTBF (0)		

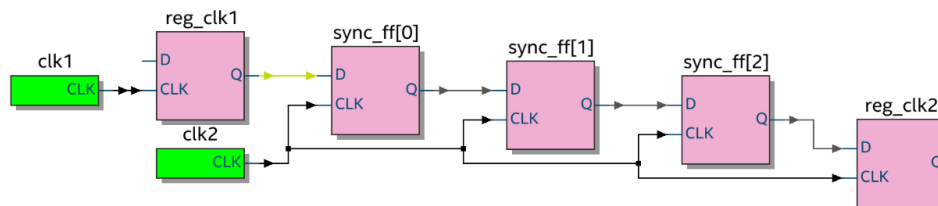
The **SDC Statistics** then presents basic information such as the source register and the registers that form the synchronizer.

Figure 208. SDC Statistics Tab Shows Basic Synchronizer Information

CDC Statistics SDC Statistics Schematic View			
Exception Statistics			
	Source Node	Destination Node	SDC Constraint
1	Driver Nodes SDC	--	--
1	reg_clk1	my_sync sync_ff[0]	set_false_path -from [get_...ters {my_sync sync_ff[0]}]

The **Schematic View** provides a representation of the synchronizer. The light green line between `reg_clk1` and `sync_ff[0]` registers indicates that there is a false path exception on the path. Right-click on the background of the **Schematic View** tab and choose **Color Legend** to display the color legend.

Figure 209. Schematic View Shows False Path Exception



The **Setup Transfers** and **Hold Transfers** reports indicate that timing analysis ignores the path crossing from `clk1` to `clk2` because it is a false path.

Figure 210. Setup Transfers and Hold Transfers Report

Setup Transfers							
Show:	Visible	Hide	Q <<Filter>> (use !<string> to invert filter)				
	From Clock	To Clock	RR Paths	FR Paths	RF Paths	FF Paths	Clock Pair Classification
1	clk1	clk2	1	0	0	0	Ignored (Not Timed)
2	clk1	clk1	1	0	0	0	Intra-Clock (Timed Safe)
3	clk2	clk2	4	0	0	0	Intra-Clock (Timed Safe)
An "Asynchronous (Timed Unsafe)" transfer occurs when the From & To clocks are asynchronous and no timing constraints exist. Entries labeled "false path" only account for clock-to-clock false paths. Path-based false path counts are shown in rows with "Ignored (Not Timed)" classifications							
Hold Transfers							
Show:	Visible	Hide	Q <<Filter>> (use !<string> to invert filter)				
	From Clock	To Clock	RR Paths	FR Paths	RF Paths	FF Paths	Clock Pair Classification
1	clk1	clk2	1	0	0	0	Ignored (Not Timed)
2	clk1	clk1	1	0	0	0	Intra-Clock (Timed Safe)
3	clk2	clk2	4	0	0	0	Intra-Clock (Timed Safe)
An "Asynchronous (Timed Unsafe)" transfer occurs when the From & To clocks are asynchronous and no timing constraints exist. Entries labeled "false path" only account for clock-to-clock false paths. Path-based false path counts are shown in rows with "Ignored (Not Timed)" classifications							

Correct RTL coding and accurate constraint definition for synchronizers in clock domain crossings are essential for robust digital designs. Defining the RTL structure of synchronizers improves the reliability of data transfers across asynchronous clock domains and ensures signal integrity, thereby minimizing the risk of metastability. Complemented with exact timing constraints, the Quartus Prime software can correctly interpret and implement the synchronizers during the Fitter. Such a method simplifies timing analysis, guarantees that all CDC paths satisfy their timing requirements, and enhances the design's overall performance.

3.1.4. Debug CDC Example 1—Incorrect SDC Definition

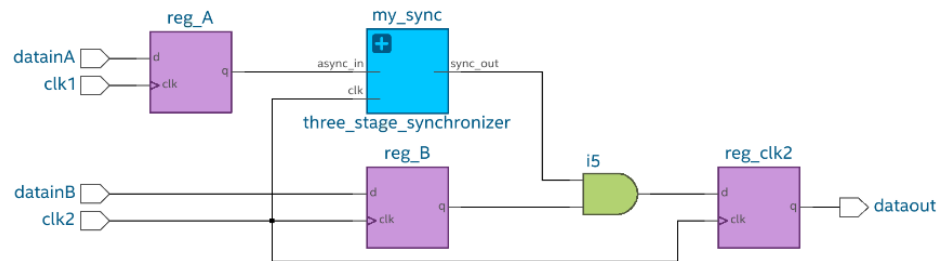
Clock Domain Crossing (CDC) issues are a common challenge in FPGA design. Often CDC issues lead to metastability, data corruption, and timing violations if you do not resolve them. This section provides a series of debug examples to help you identify and resolve common CDC issues. The examples cover various scenarios, including incorrect SDC (Synopsys Design Constraints) definitions, additional logic in the crossing path, and CDC dependencies on two simultaneous clock domains.

CDC Example 1 demonstrates how improper SDC constraints can lead to false paths and timing analysis errors. This example shows how to debug a CDC synchronizer chain with an incorrect SDC exception to ensure accurate timing verification.

CDC Example 1 Description

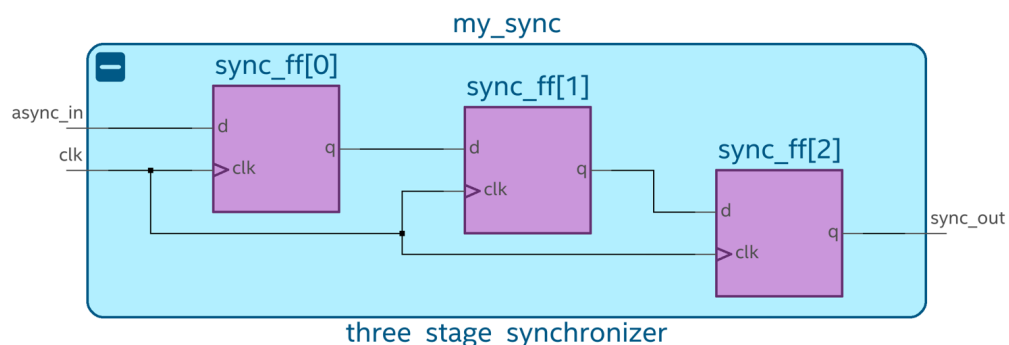
CDC Example 1 is a three-stage CDC synchronizer instance named `my_sync` in the following circuit:

Figure 211. CDC Example 1 Circuit



The three-stage synchronizer comprises a chain of three flipflops.

Figure 212. CDC Example 1 Three-Stage Synchronizer



CDC Example 1 has the following incorrect SDC exception that applies to the synchronizer:

```
set_false_path -to [get_registers my_sync|sync_ff[*]]
```

CDC Example 1 Debug Overview

The CDC Example 1 false path is incorrect because it cuts paths between each of the registers in the synchronizer chain, in addition to the path into the synchronizer chain. This constraint represents a common mistake that can cause intermittent failures on hardware. Cutting the paths between the registers in the synchronizer chain causes the Compiler to detect improperly synchronized paths.

The target of the false path should be the first register in the synchronizer chain `my_sync|sync_ff[0]` instead of `my_sync|sync_ff[*]`.

The following topics use CDC Example 1 to describe how you can identify and resolve similar CDC issues by analyzing the multiple CDC rule violations that Design Assistant reports, and then making necessary corrections to resolve the violations.

3.1.4.1. Resolve Violation: 1-Bit Asynchronous Transfer Not Synchronized

The following steps describe how to resolve the single CDC-50001 1-Bit Asynchronous Transfer Not Synchronized DRC violation for [CDC Example 1](#).

Figure 213. Design Rule Violations In Design Assistant

	Rule	Severity	Tags
1	CDC-50001 - 1-Bit Asynchronous Transfer Not Synchronized	High	synchronizer
2	CDC-50006 - CDC Bus Construction with Unsynchronized Registers	High	synchronizer, cdc-bus
3	CDC-50101 - Intra-Clock False Path Synchronizer	Low	synchroni...chronizer
4	CDC-50002 - 1-Bit Asynchronous Transfer with Insufficient Constraints	High	synchronizer
5	CDC-50003 - CE-Type CDC Transfer with Insufficient Constraints	High	cdc-bus

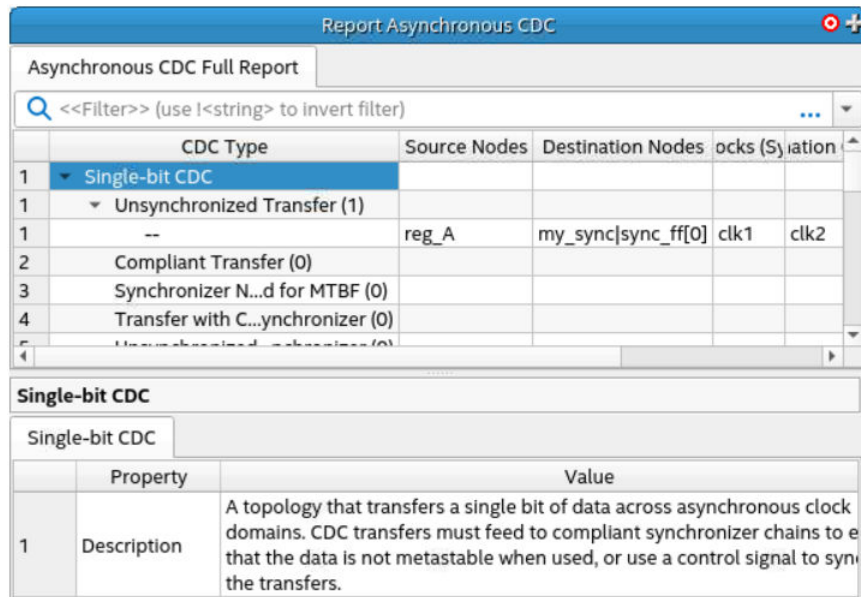
1. In the Design Assistant, select the **CDC-50001 1-Bit Asynchronous Transfer Not Synchronized** row. The right pane displays each violation with the same topology. Each row in this pane includes the associated source and destination registers and clock domains.

Figure 214. Design Assistant Right Pane Shows Source, Destination, and Clock Domains

CDC-50001 - 1-Bit Asynchronous Transfer Not Synchronized						
Status: PASS						
Severity: High						
Number of violations: 0						
Rule Parameters: max_violations = 500						
Show: Visible Hide Q <<Filter>> (use !<string> to invert filter)						
	From	To	From Clock	To Clock	Reason	Waived
1	reg_A	my_sync sync_ff[0]	clk1	clk2	Asynchronous transfer	
						Copy Ctrl+C
						Select All Ctrl+A
						Freeze First Column
						Undo Sort
						Report Timing To Chain
						Report Asynchronous CDC

2. Right-click the **CDC-50001** violation, then click **Report Asynchronous CDC**. The **Report Asynchronous CDC** reports group transfers according to CDC type. Click the row of a transfer to get additional information about it, such as the registers and why the chain terminates at the last register.

Figure 215. Transfers Grouped According to CDC Type

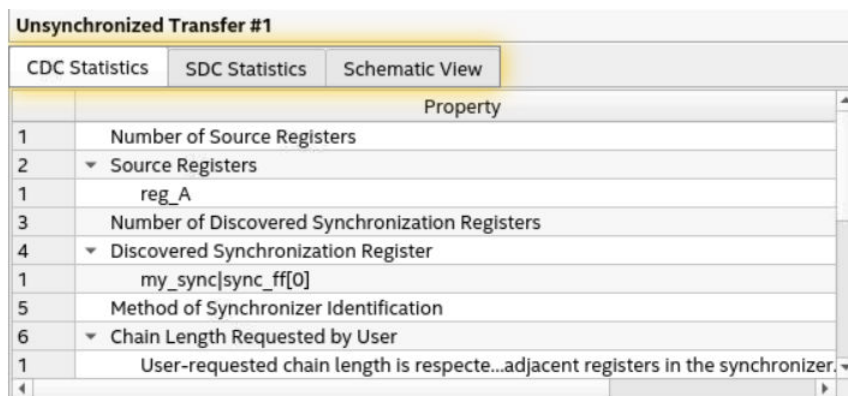


Report Asynchronous CDC					
Asynchronous CDC Full Report					
Q <<Filter>> (use !<string> to invert filter) ...					
	CDC Type	Source Nodes	Destination Nodes	clocks (S)	clocks (D)
1	Single-bit CDC				
1	Unsyncronized Transfer (1)				
1	--	reg_A	my_sync sync_ff[0]	clk1	clk2
2	Compliant Transfer (0)				
3	Synchronizer N...d for MTBF (0)				
4	Transfer with C...ynchronizer (0)				
5	Unsyncronized Transfer (0)				

Single-bit CDC		
Single-bit CDC		
	Property	Value
1	Description	A topology that transfers a single bit of data across asynchronous clock domains. CDC transfers must feed to compliant synchronizer chains to ensure that the data is not metastable when used, or use a control signal to synchronize the transfers.

- Click the **CDC Statistics** tab and confirm that the Compiler identifies the synchronizer properly, with two registers acting as sources for the synchronizer. Using the extra details, you can debug the DRC violations.

Figure 216. CDC Statistics Tab Identifies Synchronizer Source Registers



Unsyncronized Transfer #1	
CDC Statistics SDC Statistics Schematic View	
	Property
1	Number of Source Registers
2	Source Registers
1	reg_A
3	Number of Discovered Synchronization Registers
4	Discovered Synchronization Register
1	my_sync sync_ff[0]
5	Method of Synchronizer Identification
6	Chain Length Requested by User
1	User-requested chain length is respected...adjacent registers in the synchronizer.

- In the **CDC Statistics** tab, review the information about the last discovered synchronization register that includes only one register, `my_sync|sync_ff[0]`. The actual design intent is for the `my_sync|sync_ff[1]` and `my_sync|sync_ff[2]` registers to also be part of the synchronizer. The **CDC Statistics** tab also reports that the last synchronization register feeds into another CDC with the destination node `my_sync|sync_ff[1]`, which is why the chain terminates.

Figure 217. Last Synchronization Register Feeds into Another CDC

CDC Statistics		SDC Statistics	Schematic View
Property			
1	my_sync sync_ff[0]		
5	Method of Synchronizer Identification		
6	Chain Length Requested by User		
1	User-requested chain length is respecte...adjacent registers in the synchronizer.		
7	The destination register is the head of a data synchronizer because:		
1	Its d-pin is fed by a signal from an asynchronous clock domain.		
8	The synchronizer is terminated at the last register because:		
1	The last synchronization register feeds into another CDC with destination node:		
1	my_sync sync_ff[1]		
9	Chain Length Protected		
10	This transfer does not appear in the metastability report because:		
1	The following register has AUTO synchr...ion and forms a length-1 synchronizer:		

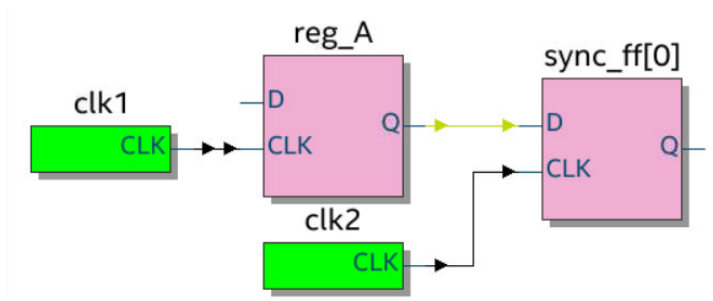
- Click the **SDC Statistics** tab to view all the SDC exceptions that affect the CDC path flagged by the violation. In this case, you see the false path exception with a target of `my_sync|sync_ff[*]`. You might recognize that the target of the exception was too broad. If not, you could also review the two other DRC violations, described next, which help confirm that the false path is the cause of the DRC violation.

Figure 218. SDC Statistics Shows False Path Exception

CDC Statistics		SDC Statistics	Schematic View
Exception Statistics			
	Source Node	Destination Node	SDC Constraint
1	Driver Nodes SDC	--	--
1	reg_A	my_sync sync_ff[0]	set_false_path -to [get_registers {my_sync sync_ff[*]]}

- Click the **Schematic View** tab to view a schematic of the circuit the **Asynchronous CDC Report** describes.

Figure 219. Schematic View



- Right-click on the background of the **Schematic View** tab and choose **Color Legend** to display the color legend. The light green line between the `reg_A` and `sync_ff[0]` registers indicates that there is a false path exception affecting the path.

3.1.4.2. Resolve Violation: CDC Bus Constructed with Unsynchronized Registers

Alternatively, you can start resolving the CDC issue in [CDC Example 1](#) by reviewing the CDC-50006 Bus Constructed with Unsynchronized Registers rule violation. The following steps describe how to resolve this DRC violation for [CDC Example 1](#).

1. In the Design Assistant, right-click the **CDC-50006** violation, then click **Report Asynchronous CDC**.

Figure 220. CDC-50006 Violation

CDC-50006 - CDC Bus Constructed with Unsynchronized Registers					
Status: PASS					
Severity: High					
Number of violations: 0					
Rule Parameters: max_violations = 500					
Show: Visible Hide Q <<Filter>> (use !<string> to invert filter)					
	Source Nodes	Synchronization Nodes	From Clock	To Clock	Chain Length
1	my_sync sync_ff[*]	my_sync sync_ff[*]	clk2	clk2	1

The Quartus Prime software classifies this as a CDC bus based on the name pattern of the transfers that are flagged as CDC transfers. That is, one transfer is from sync_ff[1] to sync_ff[2]. The other transfer is from sync_ff[0] to sync_ff[1]. In this particular circuit, the topology is a chain, not a bus. However, the naming pattern that includes bit indices 0, 1, and 2, is similar to naming patterns associated with a bus.

Figure 221. Asynchronous CDC Full Report Shows Unsynchronized Synchronizer Bus

Report Asynchronous CDC				
Asynchronous CDC Full Report				
Q <<Filter>> (use !<string> to invert filter)				
	CDC Type	Source Nodes	Destination Nodes	clock
2	Multi-bit CDC			
1	Unsynchronized Synchronizer Bus (1)			
1	--	my_sync sync_ff[*]	my_sync sync_ff[*]	clk2
2	Compliant Synchronizer Bus (0)			
3	Unconstrained Synchronizer Bus (0)			
4	Synchronizer Bus ...even Lengths (0)			
5	Synchronizer after...Control Signal (0)			

2. Click the **CDC Statistics** tab to view details about the CDC transfers. The tab reports each synchronized path, from index 0 to 1, and from index 1 to 2, and the false path exceptions that affect them.

Figure 222. CDC Statistics Tab

CDC Statistics		SDC Statistics	Schematic View
	Property	Value	
1	Bus Width	2	
2	Control Signal	N/A	
3	Source Register	Destination Register	Protected Chain Length
1	my_sync sync_ff[1]	my_sync sync_ff[2]	1
2	my_sync sync_ff[0]	my_sync sync_ff[1]	1

- To view how the false path exceptions cut these paths, click the **SDC Statistics** tab.

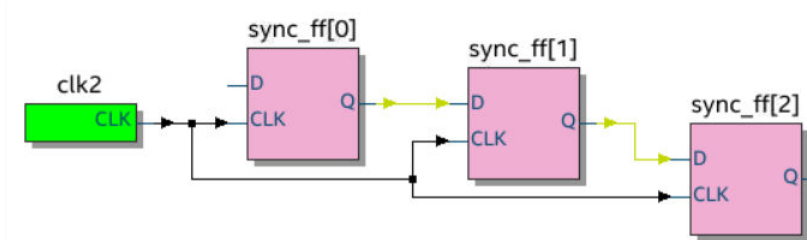
Figure 223. SDC Statistics Tab Showing Cut Paths

CDC Statistics		SDC Statistics	Schematic View
Exception Statistics			
	Source Node	Destination Node	SDC Constraint
1	Driver Nodes SDC	--	--
1	my_sync sync_ff[1]	my_sync sync_ff[2]	set_false_path -to [get_registers {my_sync sync_ff[*]}]
2	my_sync sync_ff[0]	my_sync sync_ff[1]	set_false_path -to [get_registers {my_sync sync_ff[*]}]

The tab shows how the overly broad false path exception is the cause of the DRC violation. Changing the false path target from `my_sync|sync_ff[*]` to `my_sync|sync_ff[0]` constrains the synchronizer correctly and avoids these violations.

- Click the **Schematic View** tab to display the synchronizer bus paths. The light green paths between index 0 and index 1, and between index 1 and index 2, indicate false path exceptions apply to those paths.

Figure 224. Schematic View



3.1.4.3. Resolve Violation: Intra-Clock False Path Synchronizer

You could also start resolving CDC issues for [CDC Example 1](#) by reviewing the CDC-50101 Intra-Clock False Path Synchronizer DRC violation. The following steps describe how to resolve this DRC violation for [CDC Example 1](#).

For the CDC scenario in [CDC Example 1](#), the CDC-50101 violation directly points to the root cause of the problem: the incorrect false path exception. The incorrect false path exception actually causes multiple, related DRC violations for [CDC Example 1](#). This same condition can occur for your design unless you take care in specifying an appropriately narrow false path exception.

Figure 225. CDC-50101 Violations

CDC-50101 - Intra-Clock False Path Synchronizer			
Status: PASS			
Severity: Low			
Number of violations: 0			
Rule Parameters: max_violations = 500			
Show:	Visible ▾	Hide	🔍 <<Filter>> (use !<string> to invert filter)
	From	To	From/To Clock
1	my_sync sync_ff[0]	my_sync sync_ff[1]	clk2
2	my_sync sync_ff[1]	my_sync sync_ff[2]	clk2

1. In the Design Assistant, right-click the **CDC-50101** violation, then click **Report Asynchronous CDC**. The **Report Asynchronous CDC** report opens showing the intra-clock false path synchronizers listed in the DRC violation.

Figure 226. Report Asynchronous CDC Report Showing Intra-Clock False Path Synchronizers

Report Asynchronous CDC			
Asynchronous CDC Full Report			
🔍 <<Filter>> (use !<string> to invert filter) ... ▾			
	CDC Type	Source Nodes	Destination Nodes
1	▼ Single-bit CDC		
1	▼ Intra-Clock False Path Synchronizer (2)		
1	--	my_sync sync_ff[1]	my_sync sync_ff[2]
2	--	my_sync sync_ff[0]	my_sync sync_ff[1]
2	Compliant Transfer (0)		
3	Synchronizer Not Protected for MTBF (0)		
4	Transfer with Contr...nd Synchronizer (0)		
5	Unsynchronized Intr...ed Synchronizer (0)		
6	Synchronizer Driven...e Clock Domains (0)		

2. Click the **CDC Statistics** tab to obtain details about the registers in each chain, such as reasons why the synchronizer chain starts and ends at specific points.

Figure 227. CDC Statistics Tab Showing Reasons for Synchronizer Chain Interruption

CDC Statistics		SDC Statistics	Schematic View
		Property	
1	Number of Source Registers		
2	▼ Source Registers		
1	my_sync sync_ff[0]		
3	Number of Discovered Synchronization Registers		
4	▼ Discovered Synchronization Register		
1	my_sync sync_ff[1]		
5	Method of Synchronizer Identification		
6	▼ Chain Length Requested by User		
1	User-requested chain length is respected during Fitter optimizations; however, the		
7	▼ The destination register is the head of a data synchronizer because:		
1	Its d-pin is the destination of a false path or relaxing max delay exception.		
8	▼ The synchronizer is terminated at the last register because:		
1	▼ The last synchronization register feeds into another CDC with destination node:		
1	my_sync sync_ff[2]		
9	Chain Length Protected		
10	▼ This transfer does not appear in the metastability report because:		
1	▼ The following register has AUTO synchronizer identification and forms a length-1 s		
1	my_sync sync_ff[1]		

The **CDC Statistics** tab indicates that the destination register `my_sync|sync_ff[2]` is the head of a data synchronizer. This identification is made because the D pin is the destination of a false path or a relaxing max delay exception.

- To view the incorrect false path SDC constraint and the corresponding source and destination nodes, click the **SDC Statistics** tab. This tab clearly indicates that the incorrect false path exception causes the DRC violation.

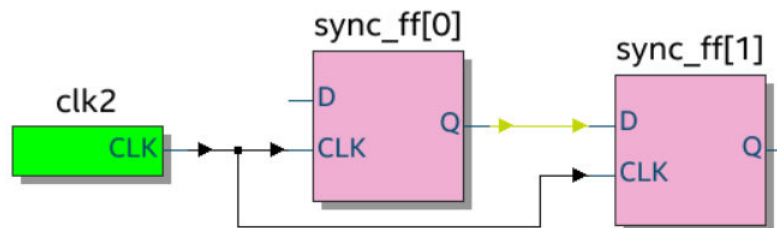
Figure 228. SDC Statistics Tab Showing Incorrect False Path Exception

CDC Statistics		SDC Statistics	Schematic View
Exception Statistics			
Source Node	Destination Node	SDC Constraint	
iver Nodes SDC	--	--	
my_sync sync_ff[0]	my_sync sync_ff[1]	set_false_path -to [get_registers {my_sync sync_ff[*]}]	

The incorrect false path exception cuts the timing paths for all the internal register-to-register paths in the synchronizer, causing the Compiler to detect these as intra-clock synchronizers.

- To view a diagram of the synchronization chain, click the **Schematic View** tab. The light green line indicates a false path.

Figure 229. Schematic View Tab Showing False Path



3.1.4.4. Example 1 CDC Debug Summary

In the case of [CDC Example 1](#), one incorrect false path exception causes three different (but related) DRC violations. [CDC Example 1](#) shows how you can use the Design Assistant rule violations and **Report Asynchronous CDC** reports to debug these CDC violations.

You can refer to the **CDC Statistics** tab for details about each synchronization chain, including chain members, reasons for detection, and reasons for chain termination. Similarly, refer to the **SDC Statistics** tab to view the timing exceptions affecting the CDC path. Finally, refer to the **Schematic View** tab to view a circuit diagram with color-coded paths indicating what types of timing exceptions apply to each part of the circuit. This data allows you to identify and eliminate certain Design Assistant violations about synchronizer chains.

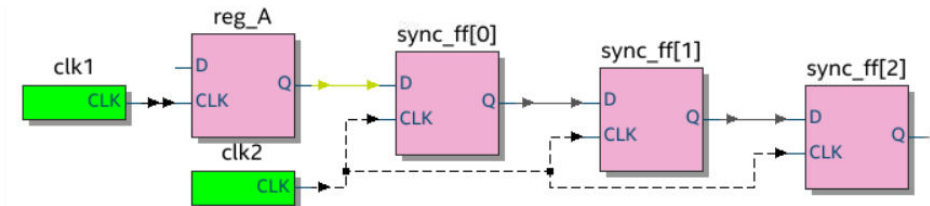
Once you properly define the CDC, the **Report Asynchronous CDC** report classifies this as a **Compliant Transfer**.

Figure 230. Report Asynchronous CDC Report Showing Compliant Transfer

	CDC Type	Source Nodes	Destination Nodes	cks (S	ation
1	▼ Single-bit CDC				
1	▼ Compliant Transfer (1)				
1	--	reg_A	my_sync sync_ff[0]	clk1	clk2

Verify the CDC topology using the **CDC Statistics**, **SDC Statistics**, and **Schematic View** tabs to corroborate that the synchronizer is properly defined.

Figure 231. Schematic View Showing CDC Topology



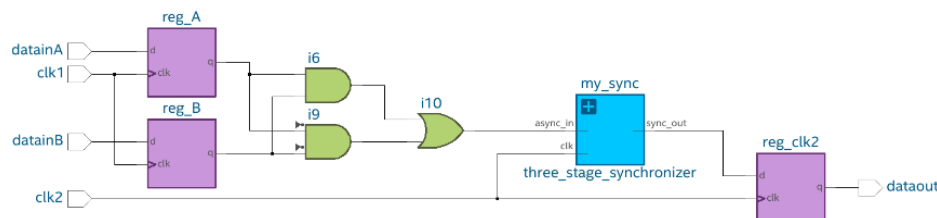
3.1.5. Debug CDC Example 2—Additional Logic in the Crossing

CDC Example 2 highlights the impact of introducing combinational logic between registers in a synchronizer chain. This example shows how to identify a CDC synchronizer that includes additional logic before the synchronizer, and provides steps to resolve the CDC timing issue.

CDC Example 2 Description

CDC Example 2 is a three-stage synchronizer (same as CDC Example 1) with the name `my_sync` in the following circuit:

Figure 232. CDC Example 2 Circuit (Incorrect Topology)



The three-stage synchronizer comprises a chain of three flipflops. The synchronizer is preceded by a combinational function that relies on `reg_A` and `reg_B` registers in the `clk1` clock domain.

CDC Example 2 has the following SDC exceptions defined in the design:

```
set_false_path -from [get_registers reg_A] -to \
[get_registers my_sync|sync_ff[0]]

set_false_path -from [get_registers reg_B] -to \
[get_registers my_sync|sync_ff[0]]
```

CDC Example 2 Debug Overview

CDC Example 2 topology is incorrect because incorporating logic between the registers in a synchronizer can undermine its function of mitigating the risks of metastability when transferring signals across clock domains. The synchronizer's flipflops are timed to capture and stabilize an asynchronous signal. Introducing combinational logic can add unpredictable delays, complicate timing analysis, and lead to setup and hold violations. For example, combinational logic might glitch. While such a glitch is not a problem for synchronous logic, for asynchronous crossings this can cause the synchronizer to capture a value that is not real.

The following topics use CDC Example 2 to describe how you can identify and resolve similar circuits by analyzing the CDC-50011 Combinational Logic Before Synchronizer Chain rule violations that Design Assistant reports, and then relocating logic and adding flipflops to resolve the violations.

3.1.5.1. Resolve Violation: Combinational Logic Before Synchronizer Chain

The following steps describe how to resolve the CDC-50011 Combinational Logic Before Synchronizer Chain DRC violation that occurs for [CDC Example 2](#):

Figure 233. Design Rule Violations In Design Assistant

ID	Rule	Severity	Tags
1	CDC-50011 - Combinational Logic Before Synchronizer Chain	High	synchronizer
2	CDC-50001 - 1-Bit Asynchronous Transfer Not Synchronized	High	synchronizer
3	CDC-50002 - 1-Bit Asynchronous...with Insufficient Constraints	High	synchronizer
4	CDC-50003 - CE-Type CDC Tran...ith Insufficient Constraints	High	cdc-bus
5	CDC-50004 - MUX-type CDC Tra...ith Insufficient Constraints	High	cdc-bus

1. In the Design Assistant, click the **CDC-50011** violation. The right pane displays each violation with the same topology. Each row includes the associated source and destination registers and clock domains.

Figure 234. CDC-50011 Details

CDC-50011 - Combinational Logic Before Synchronizer Chain					
Status: PASS					
Severity: High					
Number of violations: 0					
Rule Parameters: max_violations = 500					
Show: Visible Hide <<Filter>> (use <string> to invert filter)					
	From	To	From Clock	To Clock	Reason
1	reg_B reg_A	my_sync sync_ff[0]	clk1	clk2	Asynchronous transfer
Copy Ctrl+C Select All Ctrl+A Freeze First Column Undo Sort Report Timing To Chain Report Asynchronous CDC					

2. Right-click the **CDC-50011** violation, then click **Report Asynchronous CDC**. The **Report Asynchronous CDC** report shows the topology under the **Transfer Preceded by Combinational Logic** category.

Figure 235. CDC Topology Under Transfer Preceded by Combinational Logic

Asynchronous CDC Full Report			
<<Filter>> (use <string> to invert filter)			
	CDC Type	Source Nodes	Destination Nodes
1	Single-bit CDC		
1	Transfer Preceded by Combinational Logic (1)		
1	--	reg_B reg_A	my_sync sync_ff[0]
2	Compliant Transfer (0)		

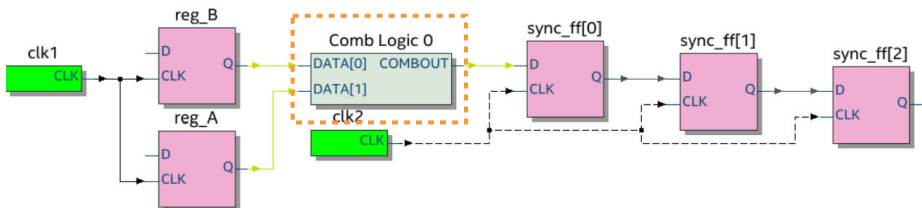
3. Click the **CDC Statistics** tab to confirm that the Compiler properly identifies the synchronizer and the two registers as sources for the synchronizer.

Figure 236. CDC Statistics Tab Identifies Synchronizer and Register Sources

CDC Statistics		SDC Statistics	Schematic View
	Property		
1	Number of Source Registers		
2	▼ Source Registers		
1	reg_B		
2	reg_A		
3	Number of Discovered Synchronization Registers		
4	▼ Discovered Synchronization Register		
1	my_sync sync_ff[0]		
2	my_sync sync_ff[1]		
3	my_sync sync_ff[2]		
4	reg_clk2		
5	Method of Synchronizer Identification		
6	▼ Chain Length Requested by User		
1	User-requested chain length is respo...acent registers in the synchronizer.		
7	▼ The destination register is the head of a data synchronizer because:		
1	Its d-pin is fed by a signal from an asynchronous clock domain.		
8	▼ The synchronizer is terminated at the last register because:		
1	▼ The following nodes cannot be used as synchronizer registers:		
1	dataout		

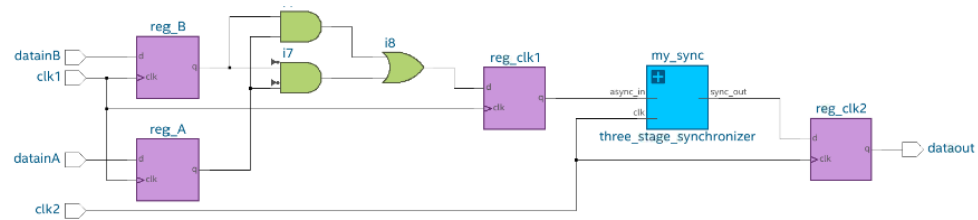
- Click the **Schematic View** tab to identify the additional Comb Logic 0 logic block between the source registers and the first register of the synchronizer. The light green color indicates a false path.

Figure 237. Schematic View Showing Additional Logic and False Path



- Take corrective action to resolve the logic interfering with the CDC chain:
 - Relocate any necessary logic upstream from the synchronizer.
 - Add an additional register before the synchronizer. This extra register serves as a staging register to ensure that the data is properly synchronized and stable before entering the synchronizer. This prevents variable delays that could compromise the synchronizer's ability to mitigate metastability.

Figure 238. Schematic View Showing Relocation of Logic and Additional Register



6. Modify the false path definition in the .sdc to reflect the addition of the new reg_clk1 register. Define the false path between reg_clk1 and the first register inside the synchronizer, as follows:

```
set_false_path -from [get_registers reg_clk1] -to \
[get_registers my_sync|sync_ff[0]]
```

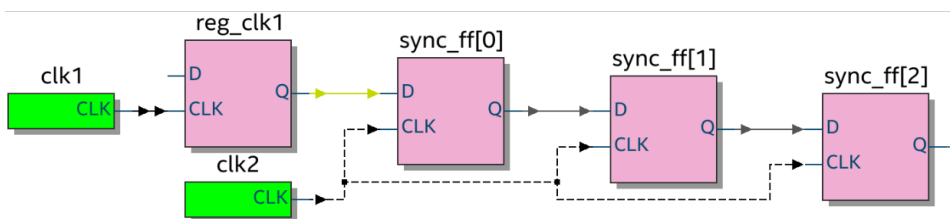
7. After modifying the false path, **Report Asynchronous CDC** reports this as a **Compliant Transfer**.

Figure 239. Compliant Transfer in Report Asynchronous CDC

Asynchronous CDC Full Report					
Q <<Filter>> (use !<string> to invert filter)					
	CDC Type	Source Nodes	Destination Nodes	cks (S	ation
1	▼ Single-bit CDC				
1	▼ Compliant Transfer (1)				
1	--	reg_clk1	my_sync sync_ff[0]	clk1	clk2

8. Verify the correct CDC topology in the **CDC Statistics**, **SDC Statistics**, and **Schematic View** tabs.

Figure 240. Schematic View Showing Correct Topology



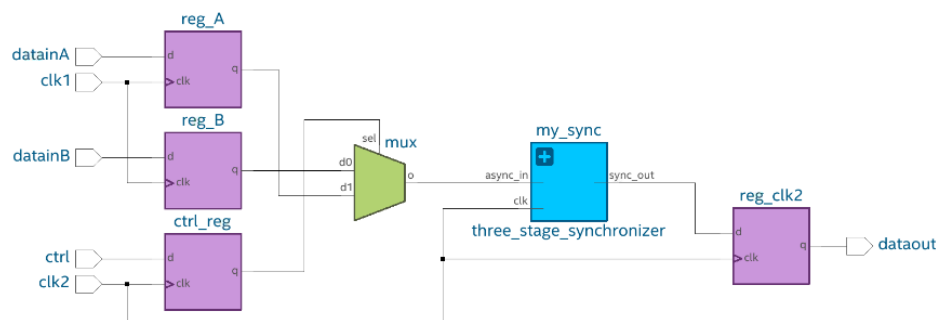
3.1.6. Debug CDC Example 3—CDC Depending on Two Simultaneous Clock Domains

CDC Example 3 addresses the complexities of a CDC synchronizer fed by signals dependent on two different clock domains. CDC Example 3 demonstrates the potential for instability of this condition, and shows how to debug and resolve such issues.

CDC Example 3 Description

CDC Example 3 includes the same three-stage CDC synchronizer instance from previous examples named `my_sync`. In CDC Example 3, the inputs of a multiplexer are under the `clk1` clock domain, while the control signal depends on the `clk2` clock domain, as the following circuit shows:

Figure 241. CDC Example 3 Circuit (Incorrect Topology)



CDC Example 3 has the following clock relationship derived from this `set_clock_groups` constraint:

```
set_clock_groups -asynchronous -group [get_clocks clk1] \
-group [get_clocks clk2]
```

CDC Example 3 Debug Overview

CDC Example 3 topology is incorrect because the multiplexer control signal is asynchronous to the `reg_A` and `reg_B` registers. This topology is incorrect, even though the multiplexer control signal is synchronous to the destination clock domain. CDC Example 3 topology results in a control signal that can change at a critical time, causing instability at the multiplexer output.

The following topics use CDC Example 3 to describe how you can identify and resolve similar CDC issues by analyzing the CDC-50012 Multiple Clock Domains Driving a Synchronizer Chain rule violation that Design Assistant reports, and then synchronizing each input to the multiplexer with the destination clock domain to resolve the violations.

3.1.6.1. Resolve Violation: Multiple Clock Domains Driving a Synchronizer Chain

The following steps describe how to resolve the CDC-50012 Multiple Clock Domains Driving a Synchronizer Chain DRC violation that occurs for [CDC Example 3](#):

Figure 242. Design Rule Violations In Design Assistant

	Rule	Severity	Tags
1	CDC-50012 - Multiple Clock D...Driving a Synchronizer Chain	High	synchronizer
2	CDC-50001 - 1-Bit Asynchronous Transfer Not Synchronized	High	synchronizer
3	CDC-50002 - 1-Bit Asynchronou...with Insufficient Constraints	High	synchronizer
4	CDC-50003 - CE-Type CDC Tran...ith Insufficient Constraints	High	cdc-bus
5	CDC-50004 - MUX-type CDC Tra...ith Insufficient Constraints	High	cdc-bus

1. In the Design Assistant, click the **CDC-50012** violation. The right pane displays each topology that represents a violation. Each row includes the associated source and destination registers and clock domains.

Figure 243. CDC-50012 Details

CDC-50012 - Multiple Clock Domains Driving a Synchronizer Chain						
Status: FAIL						
Severity: High						
Number of violations: 1						
Rule Parameters: max_violations = 500						
Show: Visible Hide Q <<Filter>> (use !<string> to invert filter)						
	From	To	From Clock	To Clock	Reason	
1	ctrl_reg reg_A reg_B	my_sync[sync_ff[0]	clk1	clk2	Asynchronous transfer	

2. In the Design Assistant, right-click the **CDC-50012** violation, then click **Report Asynchronous CDC**. The report shows the topology under the **Synchronizer Driven by Multiple Clock Domains** category.

Figure 244. Report Shows Synchronizer Driven by Multiple Clock Domains

Asynchronous CDC Full Report			
Q <<Filter>> (use !<string> to invert filter)			
	CDC Type	Source Nodes	Destination Nodes
1	Single-bit CDC		
1	Synchronizer Driven by Multiple Clock Domains (1)		
1	--	reg_A reg_B	my_sync[sync_ff[0]
2	Compliant Transfer (0)		

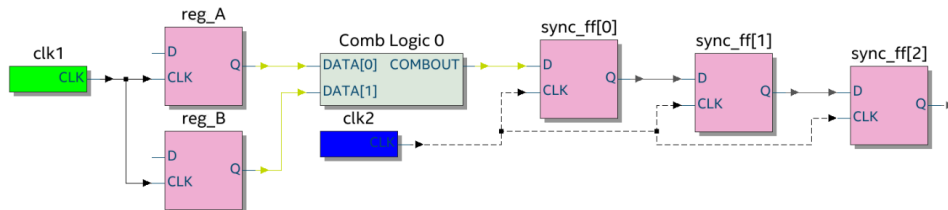
3. Click the **CDC Statistics** tab to confirm that the Compiler properly identifies the synchronizer and the two registers as sources for the synchronizer.

Figure 245. CDC Statistics Tab Identifies Synchronizer and Register Sources

CDC Statistics			SDC Statistics		Schematic View	
			Property		Value	
1	Number of Source Registers				2	
2	Source Registers					
1	reg_A					
2	reg_B					
3	Number of Discovered Synchronization Registers				4	
4	Discovered Synchronization Register				Synchronizer Stage Number	
1	my_sync[sync_ff[0]]				1	
2	my_sync[sync_ff[1]]				2	
3	my_sync[sync_ff[2]]				3	
4	reg_clk2				4	

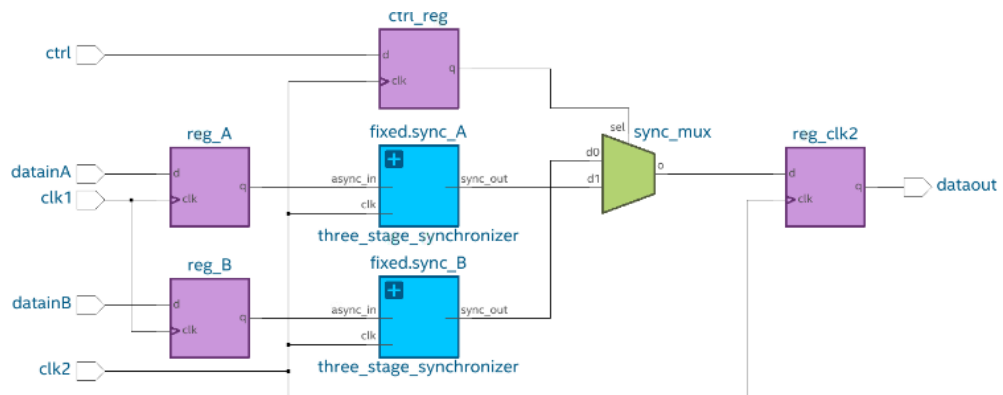
- In the **Schematic View** tab, you can see a similar topology to [CDC Example 2](#). In this case, there is a combinational block between the source registers and the first register of the synchronizer. The light green color indicates a false path.

Figure 246. Schematic View Showing Additional Logic and False Path



- Take corrective action to resolve the logic interfering with the CDC chain by synchronizing each input to the multiplexer with the destination clock domain. This ensures that both the inputs and the control signal of the multiplexer are synchronized, thereby avoiding potential instability at the multiplexer output and allowing subsequent registers sufficient time to capture the data properly.

Figure 247. Each Input to the Multiplexer Synchronized with Destination Clock Domain



6. Modify the false path definition in the .sdc to reflect the new topology that includes two synchronizers. The following exceptions can replace the clock groups defined before:

```
set_false_path -from [get_registers reg_A] -to \
[get_registers fixed.sync_A|sync_ff[0]]
set_false_path -from [get_registers reg_B] -to \
[get_registers fixed.sync_B|sync_ff[0]]
```

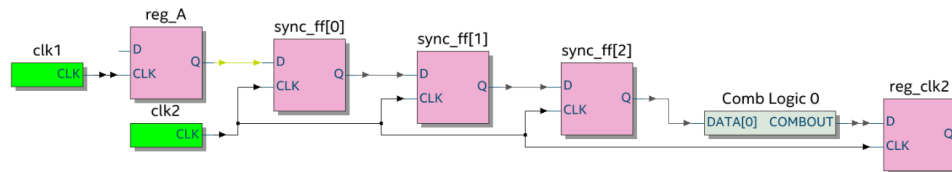
7. After modifying the false path, **Report Asynchronous CDC** reports this as a **Compliant Transfer**.

Figure 248. Compliant Transfer in Report Asynchronous CDC

	CDC Type	Source Nodes	Destination Nodes	cks (Sation	
1	▼ Single-bit CDC				
1	▼ Compliant Transfer (2)				
1	--	reg_A	sync_A sync_ff[0]	clk1	clk2
2	--	reg_B	sync_B sync_ff[0]	clk1	clk2

8. Verify the CDC topology in the **CDC Statistics**, **SDC Statistics**, and **Schematic View** tabs.

Figure 249. Schematic View Showing Correct Topology



3.2. Troubleshooting Reset Domain Crossing (RDC) Timing Issues

This section describes how to use the Quartus Prime Timing Analyzer and Design Assistant to manage and troubleshoot reset domain crossing (RDC) timing issues. Along with CDCs, RDCs are essential elements of digital design.

As digital designs become more complex, it is crucial to ensure proper transitions between different reset domains. This section emphasizes the importance of precisely defining RDCs and timing constraints, as essential steps in preventing metastability and achieving timing closure.

3.2.1. RDC Timing Overview

In digital design, a reset domain crossing (RDC) occurs when a reset signal originates in one clock domain, and then resets registers or logic in a different clock domain. Similar to clock domain crossings, RDCs require careful handling because the reset signal is asynchronous to the clock of the destination domain. This condition can lead to any of the following issues if you do not manage the RDC properly.

- **Metastability**—similar to CDC, if you do not properly synchronize a reset signal, the flipflops can enter a metastable state. This state occurs if the reset signal deasserts near the clock edge of the destination domain, leading to uncertainty in the flipflop output.
- **Partial reset**—without proper synchronization, only some portions of the circuit may exit reset, leading to a partial reset condition. This condition can cause inconsistent states within the system and unpredictable behavior.
- **Glitches**—asynchronous reset signals can cause glitches during the assertion or deassertion of the reset. These glitches can propagate through the system and cause logic to reset unexpectedly, leading to errors and instability. To prevent glitches, drive asynchronous reset signals with a single flipflop (that is, no combinational logic). A reset synchronizer does not prevent asynchronous reset glitching. Only adherence to the no combinational logic in between guideline can prevent glitching.

For a topology with an asynchronous reset signal originating from an external source or a different clock domain, you must synchronize the reset to avoid metastability. Similar to signals that cross domain crossings, the reset synchronizer typically requires a series of flipflops clocked by the destination clock domain. These flipflops sequentially capture and stabilize the asynchronous reset signal, filtering out potential metastability. You then use the output of the synchronizer to reset the sequential logic elements within the destination clock domain. This approach ensures all the elements within the same clock domain are correctly reset, maintaining the integrity of the system's behavior.

The following section describes how to identify and resolve a common RDC error that Design Assistant reports.

Related Information

- [Metastability Analysis](#) on page 16
- [Constraining CDC Paths](#) on page 93

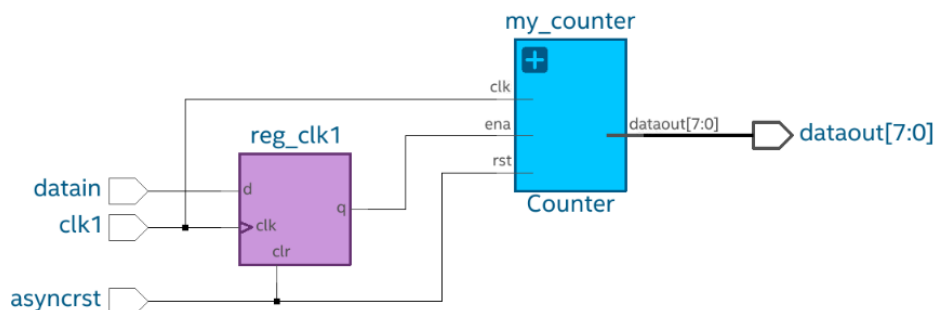
3.2.2. Debug RDC Example—Asynchronous Reset Is Not Synchronized

This RDC Example demonstrates how to use Design Assistant to identify improper synchronization of the reset signal, and how to resolve this issue by implementing reset synchronizers.

RDC Example Description

In this RDC Example, the asynchronous reset signal is driven by the `asynrst` port that is received from an external source unrelated in phase with the clock domain `clk1`. This unsynchronized reset signal feeds register `reg_clk1` and the `my_counter` module.

Figure 250. RDC Example - Asynchronous Reset (Incorrect Topology)



RDC Example Debug Overview

The RDC Example is incorrect topology because the reset signal must be synchronized with the destination register. The following topic uses the RDC Example to describe how you can identify and resolve similar RDC issues by analyzing the RES-50001 Asynchronous Reset Is Not Synchronized rule violation that Design Assistant reports, and then implementing reset synchronizers to resolve the violations.

3.2.2.1. Resolve Violation: Asynchronous Reset Is Not Synchronized

The following steps describe how to resolve the RES-50001 Asynchronous Reset Is Not Synchronized DRC violation that occurs for the RDC Example:

Figure 251. Design Assistant Identifies Asynchronous Reset

Design Assistant (final) Re		
Show:	Visible ▾	Hide
	Q <<Filter>> (use !<string> to invert filter) ... ▾	
	Rule	
1	RES-50001 - Asynchronous Reset Is Not Synchronized	Hi
2	TMC-20011 - Missing Input Delay Constraint	Hi
3	CDC-50001 - 1-Bit Asynchronous Transfer Not Synchronized	Hi
4	CDC-50002 - 1-Bit Asynchronous Transfer with Insufficient Constraints	Hi
5	CDC-50003 - CE-Type CDC Transfer with Insufficient Constraints	Hi
6	CDC-50004 - MUX-type CDC Transfer with Insufficient Constraints	Hi
7	CDC-50005 - CDC Bus Constructed with Mul...Synchronizer Chains of Different Lengths	Hi

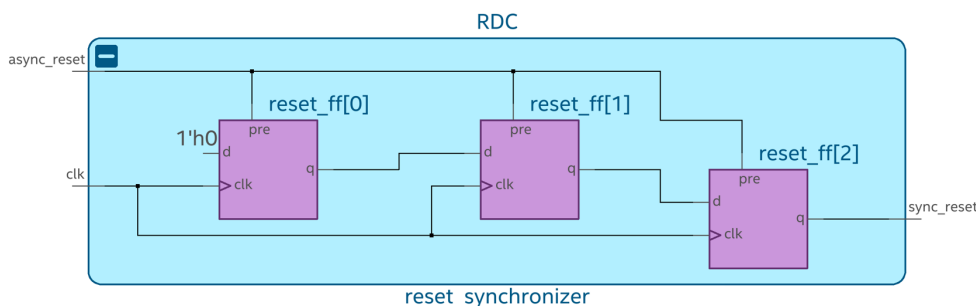
1. In the Design Assistant, right-click the **RES-50001** violation, then click **Report Asynchronous CDC**. The report shows the topology of RDCs, including improperly synchronized destination nodes grouped under the **Register Reset by Unsynchronized Signal** category.

Figure 252. Report Asynchronous CDC - Register Reset by Unsynchronized Signal

Asynchronous CDC Full Report			
Q <<Filter>> (use !<string> to invert filter)			
	CDC Type	Source Nodes	Destination Nodes
2	▼ Asynchronous Reset CDC		
1	▼ Register Reset by Unsynchronized Signal (13)		
1	--	asyncrst	my_counter count_a[0]
2	--	asyncrst	my_counter count_a[1]
3	--	asyncrst	my_counter count_a[2]
4	--	asyncrst	my_counter count_a[3]
5	--	asyncrst	my_counter count_a[4]
6	--	asyncrst	my_counter count_a[5]
7	--	asyncrst	my_counter count_a[6]
8	--	asyncrst	my_counter count_a[7]
9	--	asyncrst	reg_clk1
10	--	asyncrst	my_counter count_a[0]~DUPL

- Take corrective action to implement reset synchronizers. The topology of a reset synchronizer typically consists of a series of D flipflops set by the asynchronous reset, as the following diagram shows:

Figure 253. Reset Synchronizers



The following examples define a three-stage synchronizer for a reset signal in Verilog and VHDL:

Three-Stage Synchronizer for Reset (Verilog HDL)

```

module reset_synchronizer (
    input wire clk,           // Clock signal of the destination domain
    input wire async_reset,   // Asynchronous reset signal from the source
                                domain
    output reg sync_reset     // Synchronized reset signal for the
                                destination domain
);

reg [2:0] reset_ff; // 3 flip-flops for the synchronizer chain

always @(posedge clk or posedge async_reset) begin
    if (async_reset) begin
        reset_ff <= 3'b111; // Assert the synchronizer flip-flops
    end else begin
        reset_ff[0] <= 1'b0; // First stage
        reset_ff[1] <= reset_ff[0]; // Second stage
        reset_ff[2] <= reset_ff[1]; // Third stage
    end
    sync_reset <= reset_ff[2]; // Output from the third stage
end

endmodule

```

Three-Stage Synchronizer for Reset (VHDL)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity reset_synchronizer is
    Port (
        clk : in STD_LOGIC;           -- Clock signal of the destination
    domain
        async_reset : in STD_LOGIC;   -- Asynchronous reset signal from the
    source domain
        sync_reset : out STD_LOGIC    -- Synchronized reset signal for the
    destination domain
    );
end reset_synchronizer;

architecture Behavioral of reset_synchronizer is
    signal reset_ff : STD_LOGIC_VECTOR(2 downto 0); -- 3 flip-flops for the
    synchronizer chain
begin

    process(clk, async_reset)
    begin
        if async_reset = '1' then
            reset_ff <= (others => '1'); -- Assert the synchronizer flip-
        fops
        elsif rising_edge(clk) then
            reset_ff(0) <= '0';         -- First stage
            reset_ff(1) <= reset_ff(0); -- Second stage
            reset_ff(2) <= reset_ff(1); -- Third stage
        end if;
        sync_reset <= reset_ff(2);      -- Output from the third stage
    end process;

end Behavioral;

```

3. After properly synchronizing the reset signal, Design Assistant and **Report Asynchronous CDC** identify the implemented topology under the **Unconstrained Reset Synchronizer** category.

Figure 254. Design Assistant Results

RES-50003 - Asynchronous Reset with Insufficient Constraints			
Status: FAIL			
Severity: High			
Number of violations: 1			
Rule Parameters: max_violations = 500			
Show: Visible Hide Q <<Filter>> (use !<string> to invert filter) ...			
	Unconstrained Chain Registers	Leading Reset Source	Reset
1	RDC reset_ff[0] RDC reset_ff[1] RDC reset_ff[2]	asynchrst	RDC rese

Figure 255. Unconstrained Reset Synchronizer in Report Asynchronous CDC

Asynchronous CDC Full Report			
Q <<Filter>> (use <string> to invert filter)			
	CDC Type	Source Nodes	Destination Nodes
2	▼ Asynchronous Reset CDC		
1	▼ Unconstrained Reset Synchronizer (1)		
1	Unconstrained Reset Synchronizer (1)	asynrst	RDC reset_ff[0]
2	Compliant Reset Synchronizer (0)		
3	Reset Synchronizer Not Protected for MTBF (0)		
4	Intra-Clock False Path Reset Synchronizer (0)		
5	Inactive Reset Synchronizer (0)		
6	Reset Synchronizer Reset by Multiple Sources (0)		
7	Unsynchronized Reset Synchronizer (0)		
8	Register Reset by Unsynchronized Signal (0)		

4. Modify the .sdc timing constraints to ensure the synchronizer flipflops have sufficient time to stabilize the asynchronous reset signal and to ensure the setup and hold times are met for the flipflops within the synchronizer chain. The following is an example of the necessary timing constraints in SDC format:

```
# Define the clock (assuming the clock name is clk for the destination domain)
create_clock -name clk -period 10 [get_ports {clk_port_name}] # Adjust
period according to actual clock period

# Define the asynchronous reset (assuming the reset name is async_reset)
set_false_path -from [get_ports {async_reset_port_name}] -to [get_registers
RDC|reset_ff[*]]
```

5. After modifying the RDC constraints, **Report Asynchronous CDC** reports this as a **Compliant Reset Synchronizer**.

Figure 256. Compliant Reset Synchronizer

Asynchronous CDC Full Report			
Q <<Filter>> (use <string> to invert filter)			
	CDC Type	Source Nodes	Destination Nodes
1	▼ Asynchronous Reset CDC		
1	▼ Compliant Reset Synchronizer (1)		
1	Compliant Reset Synchronizer (1)	asynrst	RDC reset_ff[0]
2	Reset Synchronizer Not Protected for MTBF (0)		
3	Intra-Clock False Path Reset Synchronizer (0)		
4	Inactive Reset Synchronizer (0)		
5	Reset Synchronizer Reset by Multiple Sources (0)		
6	Unconstrained Reset Synchronizer (0)		
7	Unsynchronized Reset Synchronizer (0)		
8	Register Reset by Unsynchronized Signal (0)		

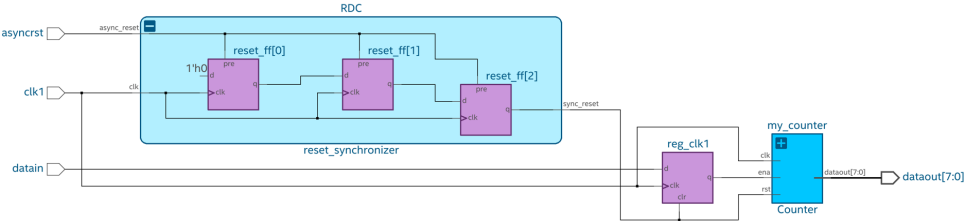
6. Verify the RDC topology in the **CDC Statistic** tab.

Figure 257. CDC Statistic Tab

CDC Statistics		
	Property	Value
1	Number of Discovered Synchronization Registers	3
2	Reset Source	Discover...Register
1	asyncrst	RDC reset_ff[0]
2	asyncrst	RDC reset_ff[1]
3	asyncrst	RDC reset_ff[2]
3	Method of Synchronizer Identification	Automatic
4	Chain Length Requested by User	3
1	User-requested chain l...s in the synchronizer.	
5	The destination registe...t synchronizer because:	
1	It is connected to an a...has a constant d input.	

The following diagram shows how the design appears following these changes:

Figure 258. Design After Violation Resolution



3.3. Troubleshooting Common Timing Issues Revision History

Document Version	Quartus Prime Version	Changes
2024.11.26	24.3	Added new <i>Troubleshooting Common Timing Issues</i> chapter, including CDC and RDC troubleshooting sections.



A. Quartus Prime Pro Edition User Guides

Refer to the following user guides for comprehensive information on all phases of the Quartus Prime Pro Edition FPGA design flow.

Related Information

- [Quartus Prime Pro Edition User Guide: Getting Started](#)
Introduces the basic features, files, and design flow of the Quartus Prime Pro Edition software, including managing Quartus Prime Pro Edition projects and IP, initial design planning considerations, and project migration from previous software versions.
- [Quartus Prime Pro Edition User Guide: Platform Designer](#)
Describes creating and optimizing systems using Platform Designer, a system integration tool that simplifies integrating customized IP cores in your project. Platform Designer automatically generates interconnect logic to connect intellectual property (IP) functions and subsystems.
- [Quartus Prime Pro Edition User Guide: Design Recommendations](#)
Describes best design practices for designing FPGAs with the Quartus Prime Pro Edition software. HDL coding styles and synchronous design practices can significantly impact design performance. Following recommended HDL coding styles ensures that Quartus Prime Pro Edition synthesis optimally implements your design in hardware.
- [Quartus Prime Pro Edition User Guide: Design Compilation](#)
Describes set up, running, and optimization for all stages of the Quartus Prime Pro Edition Compiler. The Compiler synthesizes, places, and routes your design before generating a device programming file.
- [Quartus Prime Pro Edition User Guide: Design Optimization](#)
Describes Quartus Prime Pro Edition settings, tools, and techniques that you can use to achieve the highest design performance in Altera® FPGAs. Techniques include optimizing the design netlist, addressing critical chains that limit retiming and timing closure, optimizing device resource usage, device floorplanning, and implementing engineering change orders (ECOs).
- [Quartus Prime Pro Edition User Guide: Programmer](#)
Describes operation of the Quartus Prime Pro Edition Programmer, which allows you to configure Altera® FPGA devices, and program CPLD and configuration devices, via connection with an Altera® FPGA download cable.
- [Quartus Prime Pro Edition User Guide: Block-Based Design](#)
Describes block-based design flows, also known as modular or hierarchical design flows. These advanced flows enable preservation of design blocks (or logic that comprises a hierarchical design instance) within a project, and reuse of design blocks in other projects.

- [Quartus Prime Pro Edition User Guide: Partial Reconfiguration](#)
Describes Partial Reconfiguration, an advanced design flow that allows you to reconfigure a portion of the FPGA dynamically, while the remaining FPGA design continues to function. Define multiple personas for a particular design region, without impacting operation in other areas.
- [Quartus Prime Pro Edition User Guide: Third-party Simulation](#)
Describes RTL- and gate-level design simulation support for third-party simulation tools by Aldec*, Cadence*, Siemens EDA, and Synopsys* that allow you to verify design behavior before device programming. Includes simulator support, simulation flows, and simulating IP.
- [Quartus Prime Pro Edition User Guide: Third-party Synthesis](#)
Describes support for optional synthesis of your design in third-party synthesis tools by Siemens EDA, and Synopsys*. Includes design flow steps, generated file descriptions, and synthesis guidelines.
- [Quartus Prime Pro Edition User Guide: Third-party Logic Equivalence Checking Tools](#)
Describes support for optional logic equivalence checking (LEC) of your design in third-party LEC tools by OneSpin*.
- [Quartus Prime Pro Edition User Guide: Debug Tools](#)
Describes a portfolio of Quartus Prime Pro Edition in-system design debugging tools for real-time verification of your design. These tools provide visibility by routing (or “tapping”) signals in your design to debugging logic. These tools include System Console, Signal Tap logic analyzer, system debugging toolkits, In-System Memory Content Editor, and In-System Sources and Probes Editor.
- [Quartus Prime Pro Edition User Guide: Timing Analyzer](#)
Explains basic static timing analysis principals and use of the Quartus Prime Pro Edition Timing Analyzer, a powerful ASIC-style timing analysis tool that validates the timing performance of all logic in your design using an industry-standard constraint, analysis, and reporting methodology.
- [Quartus Prime Pro Edition User Guide: Power Analysis and Optimization](#)
Describes the Quartus Prime Pro Edition Power Analysis tools that allow accurate estimation of device power consumption. Estimate the power consumption of a device to develop power budgets and design power supplies, voltage regulators, heat sink, and cooling systems.
- [Quartus Prime Pro Edition User Guide: Design Constraints](#)
Describes timing and logic constraints that influence how the Compiler implements your design, such as pin assignments, device options, logic options, and timing constraints. Use the Interface Planner to prototype interface implementations, plan clocks, and quickly define a legal device floorplan. Use the Pin Planner to visualize, modify, and validate all I/O assignments in a graphical representation of the target device.
- [Quartus Prime Pro Edition User Guide: PCB Design Tools](#)
Describes support for optional third-party PCB design tools by Siemens EDA and Cadence*. Also includes information about signal integrity analysis and simulations with HSPICE and IBIS Models.
- [Quartus Prime Pro Edition User Guide: Scripting](#)
Describes use of Tcl and command line scripts to control the Quartus Prime Pro Edition software and to perform a wide range of functions, such as managing projects, specifying constraints, running compilation or timing analysis, or generating reports.